Tina's Random Number Generator Library

Version 4.29

Heiko Bauke

November 8, 2026

"The state of the art for generating uniform deviates has advanced considerably in the last decade and now begins to resemble a mature field."

Press et al. [66]

Contents

1 TRNG in a nutshell **4**
- 1.1 Introduction . . . 4
- 1.2 History . . . 5

2 Pseudo-random numbers for parallel Monte Carlo simulations **9**
- 2.1 Pseudo-random numbers . . . 9
- 2.2 General parallelization techniques for PRNGs . . . 10
- 2.3 Playing fair . . . 11
- 2.4 Linear recurrences . . . 12
  - 2.4.1 Linear congruential generators . . . 12
  - 2.4.2 Linear feedback shift register sequences . . . 13
  - 2.4.3 Matrix linear congruential generators . . . 17
- 2.5 Non-linear transformations and YARN sequences . . . 18

3 Basic concepts **22**
- 3.1 Random number engines . . . 22
- 3.2 Random number distributions . . . 24

4 TRNG classes **28**
- 4.1 Random number engines . . . 28
  - 4.1.1 Linear congruential generators and variants . . . 30
  - 4.1.2 Counter-based generators . . . 34
  - 4.1.3 Multiple recursive generators . . . 37
  - 4.1.4 YARN generators . . . 43
  - 4.1.5 Lagged Fibonacci generators . . . 51
  - 4.1.6 Xoshiro type generator . . . 56
  - 4.1.7 Scrambled multiply-with-carry generators . . . 57
  - 4.1.8 Minimal standard generator . . . 60
  - 4.1.9 Mersenne twister generators . . . 61
- 4.2 Random number distributions . . . 63
  - 4.2.1 Uniform distributions . . . 64
  - 4.2.2 Exponential distribution . . . 68
  - 4.2.3 Two-sided exponential distribution . . . 69
  - 4.2.4 Normal distributions . . . 70
  - 4.2.5 Truncated normal distribution . . . 75
  - 4.2.6 Maxwell distribution . . . 77
  - 4.2.7 Cauchy distribution . . . 78

4.2.8 Logistic distribution . . . 80
4.2.9 Lognormal distribution . . . 81
4.2.10 Pareto distribution . . . 83
4.2.11 Power-law distribution . . . 84
4.2.12 Tent distribution . . . 86
4.2.13 Weibull distribution . . . 87
4.2.14 Extreme value distribution . . . 89
4.2.15 Γ-distribution . . . 91
4.2.16 B-distribution . . . 92
4.2.17 χ^2-distribution . . . 94
4.2.18 Student-*t* distribution . . . 95
4.2.19 Snedecor-*F* distribution . . . 97
4.2.20 Rayleigh distribution . . . 98
4.2.21 Bernoulli distribution . . . 100
4.2.22 Binomial distribution . . . 102
4.2.23 Negative binomial distribution . . . 103
4.2.24 Hypergeometric distribution . . . 105
4.2.25 Geometric distribution . . . 107
4.2.26 Poisson distribution . . . 108
4.2.27 Zero-truncated Poisson distribution . . . 109
4.2.28 Discrete distribution . . . 110
4.3 Function template `generate_canonical` . . . 113
4.4 CUDA and HIP support . . . 114

5 Installation **116**
5.1 Prerequisites . . . 116
5.2 Compilation . . . 116
5.3 Running unit tests . . . 118

6 Examples **119**
6.1 Hello world! . . . 119
6.2 Hello parallel world! . . . 121
6.2.1 Block splitting . . . 121
6.2.2 Leapfrog . . . 125
6.2.3 Block splitting or leapfrog? . . . 127
6.3 Using TRNG with STL and Boost . . . 131
6.4 Using TRNG with C++ standard library random number facility . . . 134
6.5 CMake package support . . . 134

7 Implementation details and efficiency **136**
7.1 Efficient modular reduction . . . 136
7.2 Fast delinearization . . . 138
7.3 Performance . . . 139

8 Quality and statistical tests **141**

9 Frequently asked questions **182**

License **184**

Bibliography **185**

Index **190**

1 TRNG in a nutshell

1.1 Introduction

The Monte Carlo method is a widely used and commonly accepted simulation technique in physics, operations research, artificial intelligence, and other fields, and pseudo-random numbers (PRNs) are its key resource. All Monte Carlo simulations include some sort of averaging of independent samples, a calculation that is embarrassingly parallel. Hence, it is no surprise that more and more large scale simulations are run on parallel systems like networked workstations, clusters, multicore systems or high-performance graphics cards. For each Monte Carlo simulation the quality of the PRN generator (PRNG) is a crucial factor. In a parallel environment the quality of a PRNG is even more important than in a non-parallel environment to some extent because feasible sample sizes are easily $10 \ldots 10^4$ times as large as on a sequential machine. The main problem, however, is the parallelization of the PRNG itself.

Application programmers and scientists need not grapple with all the technical details of pseudo-random number generation if a PRNG library is used. The following requirements are frequently demanded from a library for (parallel) pseudo-random number generation:

- The library should provide a set of different interchangeable algorithms for pseudo-random number generation.
- For each algorithm different well tested parameter sets should be provided that guarantee a long period and good statistical properties.
- The internal state of a PRNG can be saved for later use and restored. This makes it possible to stop a simulation and to carry on later.
- PRNGs have to support block splitting and leapfrog, see section 2.1.
- The library should provide methods for generating random variables with various distributions, uniform and non-uniform.
- The library should be implemented in a portable, speed-optimized fashion.

If these are also your requirements for a PRNG library, you should go with Tina's Random Number Generator Library.

Tina's Random Number Generator Library (TRNG) is a state-of-the-art C++ pseudo-random number generator library for sequential and parallel Monte Carlo simulations. Its design principles are based on the extensible random number generator facility that was introduced in the C++11 standard [29, 30]. The TRNG library features an object oriented design, is easy to use and has been speed optimized. Its implementation does not depend on any communication library or hardware architecture. TRNG is suited for shared memory as- well as for distributed memory computers and may be used

in any parallel programming environment, e. g., Message Passing Interface Standard or OpenMP. All generators that are implemented by TRNG have been subjected to thorough statistical tests in sequential and parallel setups, see also section 8.

This reference is organized as follows. In chapter 2 we present some basic techniques for parallel random number generation, chapter 3 introduces the basic concepts of TRNG, whereas chapter 4 describes all classes of TRNG in detail. In chapter 5 we give installation instructions, and chapter 6 presents some example programs that demonstrate the usage of TRNG in sequential as well as in parallel Monte Carlo applications. Chapter 7 deals with some implementation details and performance issues. We complete the TRNG reference with the presentation of some statistical tests of the PRNGs of TRNG in chapter 8 and answer some FAQs in chapter 9.

This manual can be read in several ways. You might read this manual chapter by chapter from the beginning to its end. Impatient readers should read at least chapter 2 to familiarize themselves with some basic terms that are used in this text before they jump to chapter 5 and chapter 6. These chapters deal with the installation and give some example code. Chapters 3 and 4 are mainly for reference and the reader will come back to them again and again.

The TRNG manual is not written as an introduction to the Monte Carlo method. It is assumed that the reader already knows the basic concepts of Monte Carlo. Novices in the Monte Carlo business find further information in various textbooks on this topic [22, 69, 59, 37, 36, 54].

1.2 History

TRNG started in 2000 as a student research project. Its implementation as well as its technical design has changed several times. Starting with version 4.0 we adopted the interface proposed by [12] and finally adopted by the C++11 standard [29, 30].

Version 4.0 Initial release of TRNG that implements the interface proposed by [12].

Version 4.1 Additive and exclusive-or lagged Fibonacci generators with two and four feedback taps have been added to the set of PRNGs. Lagged Fibonacci generators do not provide any splitting facilities. TRNG implements the template function `generate_canonical` introduced by [12].

Version 4.2 Documentation has been revised. Minor bug-fixes to lagged Fibonacci generators.

Version 4.3 Rayleigh distribution and class for correlated normal distributed random numbers added. Changed default parameter sets for generators `mrg3s`, `mrg5s`, `yarn3s`, and `yarn5s`. The new parameter sets perform better in the spectral test.

Version 4.4 Class for discrete distributions rewritten to allow efficient change of relative probabilities after initialization. New random number engine `lcg64_shift` introduced.

Version 4.5 Minor improvements and bug fixes. Utility functions `uniformcc`, `uniformco`, `uniformoc`, and `uniformoo` (declared in `trng/uniformxx.hpp`) had been reimplemented as suggested by Bruce Carneal. The new implementation of these functions is slightly faster and generates random numbers that are distributed more evenly in the intervals $[0,1]$, $[0,1)$, $(0,1]$, and $(0,1)$ respectively. Added support for Snedecor-*F*- and Student-*t*-distribution and the class `fast_discrete_dist` for faster generation of discrete random numbers withe arbitrary distribution.

Version 4.6 Reimplementation of `generate_canonical`, added sequential random number engines `mt19937` and `mt19937_64` (Mersenne twister generators). All classes for continuous random number distributions had been reimplemented as template classes. The template parameter determines the `result_type`. Permissible types are `float`, `double` or `long double` and `double` is the default template parameter. Bugfixes for several continuous random number distributions.

Version 4.7 In order to prevent name clashes macros in header file `trng/config.hpp` have been put into its own namespace `TRNG`. (This header no longer exists; its role has been taken over by the header `trng_export.hpp` that is generated by CMake.) Section 6 has been extended to demonstrate how to write parallel Monte Carlo applications using TRNG and Intel Threading Building Blocks.

Version 4.8 Performance improvements for `split` methods of the classes `mrg`*n*, `mrg`*n*`s`, `yarn`*n*, and `yarn`*n*`s`. The computational complexity has been reduced from linear (in the number of sub-streams) to logarithmic scaling.

Version 4.9 A new random number distribution class `hypergeometric_dist` has been implemented. Performance improvements for `split` methods of the classes `lcg64` and `lcg64_shift`. The computational complexity has been reduced from linear (in the number of sub-streams) to logarithmic scaling. Applied various corrections[1] and clarifications to the TRNG documentation. TRNG compiles now with Sun Studio compiler. Starting from version 4.9, the TRNG library is distributed under the terms of a BSD style license (3-clause license).

Version 4.10 Two additional random number distribution classes `twosided_exponential_dist` and `truncated_normal_dist` have been implemented.

Version 4.11 TRNG starts to support parallel processing on graphics cards via the CUDA architecture. Various minor improvements.

Version 4.12 Bug fixes and various minor improvements.

Version 4.13 Bug-fix and service release.

Version 4.14 Some minor changes of the class interfaces, bugfix for class `binomial_dist`. Starting with version 4.14 we move from the class interface as proposed by [12] to

[1] Many thanks to Rodney Sparapani.

the class interface of the C++11 standard [29, 30]. These interfaces differ in some details only. Adopting the C++11 interface for TRNG allows to mix TRNG classes and classes from the C++11 random number library, see section 6.4 for details.

Version 4.15 Bug-fix and service release. Improvements mainly related to the build system. The additional random number distribution classes `maxwell_dist` and `beta_dist` have been implemented. New e-mail address trng@mail.de.

Version 4.16 Bug-fix and service release. Some bug fixes for the classes `discrete_distribution` and `beta_dist` have been applied. (One of the corresponding bugs appeared in the class `discrete_distribution` if the number of weights was a power of 2. The other bugs were syntactical errors preventing TRNG to compile.) TRNG 4.16 features the new random number distribution class `negative_binomial_dist`.

Version 4.17 Bug-fix and service release.

Version 4.18 The additional random number distribution class `zero_truncated_poisson_dist` has been implemented.

Version 4.19 Random number engines use internally integer types of exactly 32 bits or 64 bits, respectively, instead of `(unsigned) long int` and `(unsigned) long long int`. New typedefs for lagged Fibonacci generators have been introduced. The old ones (ending with `_ul` or `_ull`) are architecture dependent and should be considered as depreciated. This and later versions will not compile on exotic platforms where none of the integer types `int`, `long int`, and `long long int` has exactly 32 or 64 bits. This version beaks ABI compatibility to earlier versions but retains source code compatibility.

Version 4.20 Bug-fix and service release.

Version 4.21 Bug-fix and service release. Fixes numerical convergence problems in the inverse of the incomplete Beta function.

Version 4.22 This maintenance release removes old code for supporting C++ language standards older than C++11. Many minor code enhancements and bug fixes have been applied. The autotools-based build system has been replaced by CMake to modernize the build process and enhance portability, see installation instructions. The negative binomial distribution has been generalized to real-valued parameters.

Version 4.23 This is primarily a maintenance release focusing on code quality. Starting with this release TRNG employs systematic unit testing on the basis of the Boost unit test frame work. The numerical accuracy of several special mathematical functions (e. g., cumulative probability density of the normal distribution) have been enhanced. The `discard` method of the lagged Fibonacci generators has been re-implemented using an algorithm with logarithmic asymptotic complexity.

Version 4.24 The two new random number engines, called `xoshiro256plus` and `lcg64_count_shift`, have been implemented. New unit tests have been introduced to extend test coverage. Special-functions unit tests use reference values with improved numerical accuracy now. The numerical accuracy of various special functions has been improved to reach machine precision also in 128-bit floating point number arithmetic, e. g., the inverse cumulative probability distribution of the normal distribution, incomplete gamma functions and the Beta function. An uninitialized memory read access has been fixed. (Many thanks to Mirai Solutions [72] for reporting this issue.) The documentation has been improved and extended. The chapter on quality and statistical tests has been rewritten based on results of the Dieharder test suite.

Version 4.25 All unit tests have been converted to Catch2 unit test framework. TRNG can be consumed as a third-party component in CMake-based projects supporting CMake's `find_package`. TRNG supports building static *or* shared libraries depending on the `BUILD_SHARED_LIBS` CMake variable. Cuda support has been revised to work with Cuda 12.2. Experimental support for AMD's Heterogeneous-compute Interface for Portability (HIP). This release contains also several minor fixes and improvements.

Version 4.26 Improved numerical accuracy and performance in the calculation of the incomplete gamma function and its inverse by incorporating methods from [23]. The new random number engine `count128_lcg_shift` has been implemented.

Version 4.27 Bug-fix release eliminating some undefined behaviour in some unit tests.

Version 4.28 Maintenance release for improving compatibility with C++20 and updating the unit test framework and build tools.

Version 4.29 Four scrambled multiply-with-carry random number engines, `mwc64x32`, `mwc128x32`, `mwc128x`, and `mwc192x`, have been added. Reworked `xoshiro256plus` jump/discard calculations to speed up this jumping ahead in the random number stream.

2 Pseudo-random numbers for parallel Monte Carlo simulations

2.1 Pseudo-random numbers

Monte Carlo methods are a class of computational algorithms for simulating the behavior of various physical and mathematical systems by a stochastic process. While simulating such a stochastic process on a computer, large amounts of random numbers are consumed. Actually, a computer as a deterministic machine is not able to generate random digits. John von Neumann, pioneer in Monte Carlo simulation, summarized this problem in his famous quote:

> "Anyone who considers arithmetical methods of producing random digits is, of course, in a state of sin."

For computer simulations we have to content ourselves with something weaker than random numbers, namely *pseudo-random* numbers. We define a stream of PRNs r_i in the following in an informal manner:

- PRNs are generated by a deterministic rule.
- A stream of PRNs r_i cannot be distinguished from a true random sequence by means of practicable methods applying a *finite* set of statistical tests on *finite* samples.

Almost all PRNGs produce a sequence $r_0, r_1, r_2, \ldots$ of PRNs by a recurrence

$$r_i = f(r_{i-1}, r_{i-2}, \ldots, r_{i-k}) \,, \tag{2.1}$$

and the art of random number generation lies in the design of the function $f(\cdot)$. The objective in PRNG design is to find a transition algorithm $f(\cdot)$ that yields a PRNG with a long period and good statistical properties within the stream of PRNs. Statistical properties of a PRNG may be investigated by theoretical or empirical means, see [35]. But experience shows, there is nothing like an ideal PRNG. A PRNG may behave like a perfect source of randomness in one kind of Monte Carlo simulation, whereas it may suffer from significant statistical correlations if it is used in another context, which makes the particular Monte Carlo simulation unreliable.

Numerous recipes for $f(\cdot)$ in (2.1) have been discussed in the literature, see [35, 42] and references therein. We will present some popular schemes and review some of theirs mathematical properties in sections 2.4 and 2.5. Readers how do not want to bother with mathematical details might skip these sections and may come back later if necessary. However, the next two sections on the parallelization of PRN sequences and on playing fair present important concepts of the TRNG library.

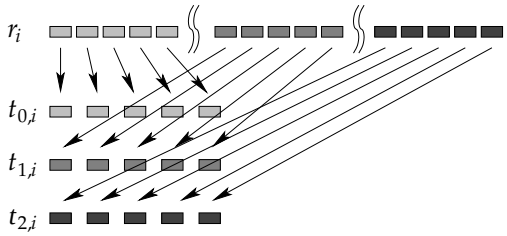


Figure 2.1: Parallelization by block splitting.

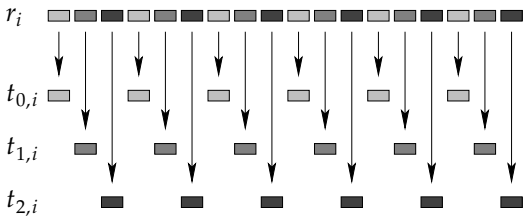


Figure 2.2: Parallelization by leapfrogging.

2.2 General parallelization techniques for PRNGs

In parallel applications, we need to generate streams $t_{j,i}$ of random numbers [7, 55, 60]. Streams are numbered by $j = 0, 1, \ldots, p-1$, where p is the number of processes. We require statistical independence of the $t_{j,i}$ within each stream and between streams as well. Four different parallelization techniques are used in practice:

Random seeding: All processes use the same PRNG but a different "random" seed. The hope is that they will generate non-overlapping and uncorrelated subsequences of the original PRNG. This hope, however, has no theoretical foundation. Random seeding is a violation of Donald Knuth's advice "Random number generators should not be chosen at random" [35].

Parameterization: All processes use the same type of generator but with different parameters for each processor. Example: linear congruential generators with additive constant b_j for the jth stream [65]:

$$t_{j,i} = a \cdot t_{j,i-1} + b_j \bmod 2^e\,, \tag{2.2}$$

where b_j is the $(j+2)$th prime number. Another variant uses different multipliers a for different streams [48]. The theoretical foundation of these methods is weak, and empirical tests have revealed serious correlations between streams [52]. On massive parallel system you may need thousands of parallel streams, and it is not trivial to find a type of PRNG with thousands of "well tested" parameter sets.

Block splitting: Let M be the maximum number of calls to a PRNG by each processor, and let p be the number of processes. Then we can split the sequence r_i of a sequential PRNG into consecutive blocks of length M such that

$$\begin{aligned} t_{0,i} &= r_i \\ t_{1,i} &= r_{i+M} \\ &\cdots \\ t_{p-1,i} &= r_{i+M(p-1)}\,. \end{aligned} \tag{2.3}$$

This method works only if we know M in advance or can at least safely estimate an upper bound for M. To apply block splitting it is necessary to jump from the

ith random number to the $(i + M)$th number without calculating all the numbers in between, which cannot be done efficiently for many PRNGs. A potential disadvantage of this method is that long range correlations, usually not observed in sequential simulations, may become short range correlations between sub-streams [53, 19]. Block splitting is illustrated in Figure 2.1.

Leapfrog: The leapfrog method distributes a sequence r_i of random numbers over p processes by decimating this base sequence such that

$$\begin{aligned} t_{0,i} &= r_{pi} \\ t_{1,i} &= r_{pi+1} \\ &\dots \\ t_{p-1,i} &= r_{pi+(p-1)} \,. \end{aligned} \tag{2.4}$$

Leapfrogging is illustrated in Figure 2.2. It is the most versatile and robust method for parallelization, and it does not require an a priori estimate of how many random numbers will be consumed by each processor. An efficient implementation requires a PRNG that can be modified to generate directly only every pth element of the original sequence. Again this excludes many popular PRNGs.

At first glance block splitting and leapfrog seem to be quite different approaches. But in fact, these are closely related to each other. Because if leapfrog is applied to any *finite* base sequence the leapfrog sequences are cyclic shifts of each other. Consider an arbitrary sequence r_i with period T. If $\gcd(T, p) = 1$, all leapfrog sequences $t_{1,i}, t_{2,i}, \dots, t_{p,i})$ are cyclic shifts of each other, i. e., for every pair of leapfrog sequences $t_{j_1,i}$ and $t_{j_2,i}$ of a common base sequence r_i with period T there is a constant s, such that $t_{j_1,i} = t_{j_2,i+s}$ for all i, and s is at least $\lfloor T/p \rfloor$. Furthermore, if $\gcd(T, p) = d > 1$, the period of each leapfrog sequence equals T/d and there are d classes of leapfrog sequences. Within a class of leapfrog sequences there are p/d sequences, each sequence is just a cyclic shift of another and the size of the shift is at least $\lfloor T/p \rfloor$.

The first two methods, random seeding and parameterization, have little or no theoretical backup, but their weakest point is yet another. The results of a simulation should not depend on the number of processors it runs on. Leapfrog and block splitting do allow to organize simulations such that the same random numbers are used independently of the number of processors. With parameterization or random seeding the results will always depend on the parallelization, see section 6.2 for details. PRNGs that do not support leapfrog and block splitting should not be used in parallel simulations.

2.3 Playing fair

We say that a parallel Monte Carlo simulation *plays fair*, if its outcome is strictly independent of the underlying hardware. Fair play implies the use of the same PRNs in the same context, independently of the number of parallel processes. It is mandatory for debugging, especially in parallel environments where the number of parallel processes

varies from run to run, but another benefit of playing fair is even more important: Fair play guarantees that the quality of a PRNG with respect to an application does not depend on the degree of parallelization.

Obviously the use of parameterization or random seeding prevent a simulation from playing fair. Leapfrog and block splitting, on the other hand, do allow the use of the same PRNs within the same context independently of the number of parallel streams.

Consider the site percolation problem. A site in a lattice of size N is occupied with some probability, and the occupancy is determined by a PRN. M random configurations are generated. A naive parallel simulation on p processes could split a base sequence into p leapfrog streams and having each process generate $\approx M/p$ lattice configurations, independently of the other processes. Obviously this parallel simulation is not equivalent to its sequential version that consumes PRNs from the base sequence to generate one lattice configuration after another. The effective shape of the resulting lattice configurations depends on the number of processes. This parallel algorithm does not play fair.

We can turn the site percolation simulation into a fair playing algorithm by leapfrogging on the level of lattice configurations. Here each process consumes distinct contiguous blocks of PRNs form the sequence r_i, and the workload is spread over p processors in such a way that each process analyzes each pth lattice. If we number the processes by their rank i from 0 to $p-1$ and the lattices form 0 to $M-1$, each process starts with a lattice whose number equals its own rank. That means process i has to skip $i \cdot N$ PRNs from the sequence r_i before the first lattice configuration is generated. Thereafter, each process can skip $p-1$ lattices, i. e., $(p-1) \cdot N$ PRNs and continue with the next lattice. In section 6.2 we investigate this approach in more detail and will give further examples of fair playing Monte Carlo algorithms and their implementation.

Organizing simulation algorithms such that they play fair is not always as easy as in the above example, but with a little effort one can achieve fair play in more complicated situations, too. This may require the combination of block splitting and the leapfrog method, or iterated leapfrogging. Sometimes it is also necessary to use more than one stream of PRNs per process, e. g. in the Swendsen Wang cluster algorithm [74, 59] one may use one PRNG to construct the bond percolation clusters and another PRNG to decide if a cluster has to be flipped.

2.4 Linear recurrences

The majority of the PRNG algorithms that are implemented by TRNG are based on linear recurrences in prime fields. Thus, we review some of theirs mathematical properties in this section.

2.4.1 Linear congruential generators

Linear recurrences where introduced as PRNGs by Lehmer [44], who proposed the linear congruential generator (LCG) with the recurrence

$$r_i = a \cdot r_{i-1} + b \bmod m\,, \tag{2.5}$$

with $a = 23$, $b = 0$, and $m = 10^8 + 1$. Obviously, the period of such a generator cannot exceed m. If $b = 0$ then period will be at most $m - 1$, because $r_i = 0$ is a fixed point. In fact, the original Lehmer generator has a period of only 5 882 352.

The period of a LCG depends on the choice of its parameter. There are two important kinds of moduli m that allow for a maximal period, namely moduli that are a power of 2 and prime moduli. For prime moduli, a has to be a generating element of the multiplicative group modulo m and $b = 0$. While for power of 2 moduli, a and b must be odd and $a - 1$ has to be a multiple of four. These and more theoretical properties of LCGs are presented in [35]

Parallelization

One may show by complete induction that the M-fold successive iteration of (2.5) is given by

$$r_i = a^M r_{i-M} + b \sum_{j=0}^{M-1} a^j \bmod m\,. \tag{2.6}$$

Note that $\sum_{j=0}^{M-1} a^j$ may be computed efficiently if M is a power of 2, say $M = 2^e$, by employing

$$\sum_{j=0}^{2^e-1} a^j \bmod m = \prod_{j=0}^{e-1} \left(1 + a^{2^j}\right) \bmod m\,. \tag{2.7}$$

If M is not a power of two, we can use the more general relation

$$\sum_{j=0}^{M-1} a^j \bmod m = \prod_{j=0}^{e-1} \left(1 + a^{2^j}\right) + a^{2^e} \sum_{j=0}^{M-2^e-1} a^j \bmod m \tag{2.8}$$

instead, where e denotes the largest integer such that $M \leq 2^e$. The left side as well as the right side of (2.8) include terms of the form $\sum_j a^j \bmod m$, but on the rigth hand side the number of terms in the sum is much smaller. Applying of (2.8) recursively allows an efficient computation of $\sum_{j=0}^{M-1} a^j \bmod m$ and, therefore, an efficient implementation of block splitting and leapfrogging.

2.4.2 Linear feedback shift register sequences

The majority of the PRNG algorithms that are implemented by TRNG are based on so-called linear feedback shift register sequences. Therefore, we review some of theirs mathematical properties in this section. Readers how do not want to bother with mathematical details might skip this as well as the next section on YARN generators and may come back later if necessary.

Knuth [34] proposed a generalization of Lehmer's method known as multiple recurrence generator (MRG) that obeys the recurrence

$$r_i = a_1 r_{i-1} + a_2 r_{i-2} + \ldots + a_n r_{i-n} \bmod m \tag{2.9}$$

with prime modulus m. In the theory of finite fields, a sequence of type (2.9) is called *linear feedback shift register sequence*, or LFSR sequence for short. Note that a LFSR sequence is fully determined by specifying n coefficients $(a_1, a_2, \ldots, a_n)$ plus n initial values $(r_1, r_2, \ldots, r_n)$. There is a wealth of rigorous results on LFSR sequences that can (and should) be used to construct a good PRNG. Here we only discuss a few but important facts without proofs. A detailed presentation of LFSR sequences including theorems and proofs can be found in [24, 31, 45, 46, 21, 78].

Since the all zero tuple $(0, 0, \ldots, 0)$ is a fixed-point of (2.9), the maximum period of a LFSR sequence cannot exceed $m^n - 1$. The following theorem tells us precisely how to choose the coefficients $(a_1, a_2, \ldots, a_n)$ to achieve this period [35]:

Theorem 1 The LFSR sequence (2.9) over $\mathbb{F}_m$ has period $m^n - 1$, if and only if the characteristic polynomial

$$f(x) = x^n - a_1 x^{n-1} - a_2 x^{n-2} - \ldots - a_n \tag{2.10}$$

is *primitive* modulo m.

A monic polynomial $f(x)$ of degree n over $\mathbb{F}_m$ is primitive modulo m, if and only if it is irreducible (i. e., cannot be factorized over $\mathbb{F}_m$), and if it has a primitive element of the extension field $\mathbb{F}_{m^n}$ as one of its roots. The number of primitive polynomials of degree n modulo m is equal to $\phi(m^n - 1)/n = \mathcal{O}\left(m^n/(n \ln(n \ln m))\right)$ [77], where $\phi(x)$ denotes Euler's totient function. As a consequence a random polynomial of degree n is primitive modulo m with probability $\simeq 1/(n \ln(n \ln m))$, and finding primitive polynomials reduces to testing whether a given polynomial is primitive. The latter can be done efficiently, if the factorization of $m^n - 1$ is known [31], and most computer algebra systems offer a procedure for this test.

Theorem 2 Let r_i be an LFSR sequence (2.9) with a primitive characteristic polynomial. Then each k-tuple $(r_{i+1}, \ldots, r_{i+k})$ occurs m^{n-k} times per period for $k \leq n$ (except the all zero tuple for $k = n$).

From this theorem it follows that, if a k-tuple of consecutive numbers with $k \leq n$ is chosen randomly from a LFSR sequence, the outcome is uniformly distributed over all possible k-tuples in $\mathbb{F}_m$. This is exactly what one would expect from a truly random sequence. In terms of Compagner's ensemble theory tuples of size less than or equal to n drawn from a LFSR sequence with primitive characteristic polynomial are indistinguishable from truly random tuples [15, 16].

Theorem 3 Let r_i be an LFSR sequence (2.9) with period $T = m^n - 1$ and let α be a complex mth root of unity and $\overline{\alpha}$ its complex conjugated. Then

$$C(h) := \sum_{i=1}^{T} \alpha^{r_i} \cdot \overline{\alpha}^{r_{i+h}} = \begin{cases} T & \text{if } h = 0 \bmod T \\ -1 & \text{if } h \neq 0 \bmod T \end{cases}. \tag{2.11}$$

$C(h)$ can be interpreted as autocorrelation function of the sequence, and Theorem 3 tells us that LFSR sequences with maximum period have autocorrelations that are very similar to the autocorrelations of a random sequence with period T. Together with the nice equidistribution properties (Theorem 2) this qualifies LFSR sequences with maximum period as *pseudo-noise sequences*, a term originally coined by Golomb for binary sequences [24, 31].

Parallelization

As a matter of fact, LFSR sequences do support leapfrog and block splitting very well. Block splitting means basically jumping ahead in a PRN sequence. In the case of LFSR sequences this can be done quite efficiently. Note, that by introducing a companion matrix $\boldsymbol{A}$, the linear recurrence (2.9) can be written as a vector matrix product.

$$\begin{pmatrix} r_{i-(n-1)} \\ \vdots \\ r_{i-1} \\ r_i \end{pmatrix} = \underbrace{\begin{pmatrix} 0 & 1 & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & 1 \\ a_n & a_{n-1} & \dots & a_1 \end{pmatrix}}_{\boldsymbol{A}} \begin{pmatrix} r_{i-n} \\ \vdots \\ r_{i-2} \\ r_{i-1} \end{pmatrix} \bmod m \tag{2.12}$$

From this formula it follows immediately that the M-fold successive iteration of (2.9) may be written as

$$\begin{pmatrix} r_{i-(n-1)} \\ \vdots \\ r_{i-1} \\ r_i \end{pmatrix} = \boldsymbol{A}^M \begin{pmatrix} r_{i-M-(n-1)} \\ \vdots \\ r_{i-M-1} \\ r_{i-M} \end{pmatrix} \bmod m\,. \tag{2.13}$$

Matrix exponentiation can be accomplished in $\mathcal{O}\left(n^3 \ln M\right)$ steps via binary exponentiation (also known as exponentiation by squaring).

Implementing leapfrogging efficiently is less straightforward. Calculating $t_{j,i} = r_{pi+j}$ via

$$\begin{pmatrix} r_{pi+j-(n-1)} \\ \vdots \\ r_{pi+j-1} \\ r_{pi+j} \end{pmatrix} = \boldsymbol{A}^p \begin{pmatrix} r_{p(i-1)+j-(n-1)} \\ \vdots \\ r_{p(i-1)+j-1} \\ r_{p(i-1)+j} \end{pmatrix} \bmod m \tag{2.14}$$

is no option, because $\boldsymbol{A}^p$ is usually a dense matrix, in which case calculating a new element from the leapfrog sequence requires $\mathcal{O}\left(n^2\right)$ operations instead of $\mathcal{O}\left(n\right)$ operations in the base sequence.

The following theorem assures that the leapfrog subsequences of LFSR sequences are again LFSR sequences [31]. This will provide us with a very efficient way to generate leapfrog sequences.

Theorem 4 Let r_i be a LFSR sequence based on a primitive polynomial of degree n with period $m^n - 1$ (pseudo-noise sequence) over $\mathbb{F}_m$, and let (t) be the decimated sequence with lag $p > 0$ and offset j, e. g.

$$t_{j,i} = r_{pi+j}\,. \tag{2.15}$$

Then $t_{j,i}$ is a LFSR sequence based on a primitive polynomial of degree n, too, if and only if p and $m^n - 1$ are coprime, e. g. $\gcd(m^n - 1, p) = 1$. In addition, r_i and $t_{j,i}$ are not just cyclic shifts of each other, except when

$$p = m^h \bmod (m^n - 1) \tag{2.16}$$

for some $0 \leq h < n$. If $\gcd(m^n - 1, p) > 1$ the sequence $t_{j,i}$ is still a LFSR sequence, but not a pseudo-noise sequence.

It is not hard to find prime numbers m such that $m^n - 1$ has very few (and large) prime factors. For such numbers, the leapfrog method yields pseudo-noise sequences for any reasonable number of parallel streams [7]. While Theorem 4 ensures that leapfrog sequences are not just cyclic shifts of the base sequence (unless (2.16) holds), the leapfrog sequences are cyclic shifts of each other, see section 2.2.

Theorem 4 tells us that all leapfrog sequences of a LFSR sequence of degree n can be generated by another LFSR of degree n or less. The following theorem gives us a recipe to calculate the coefficients $(b_1, b_2, \ldots, b_n)$ of the corresponding leapfrog feedback polynomial.

Theorem 5 Let t_i be a (periodic) LFSR sequence over the field $\mathbb{F}_m$ and $f(x)$ its characteristic polynomial of degree n. Then the coefficients $(b_1, b_2, \ldots, b_n)$ of $f(x)$ can be computed from $2n$ successive elements of t_i by solving the linear system

$$\begin{pmatrix} t_{i+n} \\ t_{i+n+1} \\ \vdots \\ t_{i+2n-1} \end{pmatrix} = \begin{pmatrix} t_{i+n-1} & \cdots & t_{i+1} & t_i \\ t_{i+n} & \cdots & t_{i+2} & t_{i+1} \\ \vdots & \ddots & \vdots & \vdots \\ t_{i+2n-2} & \cdots & t_{i+n} & t_{i+n-1} \end{pmatrix} \begin{pmatrix} b_1 \\ b_2 \\ \vdots \\ b_n \end{pmatrix} \bmod m \tag{2.17}$$

over $\mathbb{F}_m$.

Starting from the base sequence we determine $2n$ values of the sequence t_i by applying the leapfrog rule. Then we solve (2.17) by Gaussian elimination to get the characteristic polynomial for a new LFSR generator that yields the elements of the leapfrog sequence directly with each call. If the matrix in (2.17) is singular, the linear system has more than one solution, and it is sufficient to pick one of them. In this case it is always possible to generate the leapfrog sequence by a LFSR of degree less than the degree of the original sequence.

Choice of modulus

LFSR sequences can be defined over any prime field. In particular LFSR sequences over $\mathbb{F}_2$ with sparse feedback polynomials are popular sources of PRNs [33, 79, 35] and generators of this type can be found in various software libraries. This is due to the fact that multiplication in $\mathbb{F}_2$ is trivial, addition reduces to exclusive-or and the modulo operation comes for free. As a result, generators that operate in $\mathbb{F}_2$ are extremely fast. Unfortunately, these generators suffer from serious statistical defects [20, 25, 71, 79] that can be blamed to the small size of the underlying field [5]. In parallel applications we have the additional drawback that, if the leapfrog method is applied to a LFSR sequence with sparse characteristic polynomial, the new sequence will have a dense polynomial. The computational complexity of generating values of the LFSR sequence grows from $\mathcal{O}(1)$ to $\mathcal{O}(n)$. Remember that for generators in $\mathbb{F}_2$, n is typically of order 1000 or even larger to get a long period $2^n - 1$ and reasonable statistical properties.

The theorems and parallelization techniques we have presented so far do apply to LFSR sequences over any finite field $\mathbb{F}_m$. Therefore, we are free to choose the prime modulus m. In order to get maximum entropy on the macrostate level [56] m should be as large as possible. A good choice is to set m to a value that is of the order of the largest representable integer of the computer. If the computer deals with e-bit registers, we may write the modulus as $m = 2^e - k$, with k reasonably small. In fact if $k(k+2) \leq m$ modular reduction can be done reasonably fast by a few bit-shifts, additions and multiplications, see chapter 7. Furthermore, a large modulus allows us to restrict the degree of the LFSR to rather small values, e. g. $n \approx 4$, while the PRNG has a large period and good statistical properties.

In accordance with Theorem 4, a leapfrog sequence of a pseudo-noise sequence is a pseudo-noise sequence, too, if and only if its period $m^n - 1$ and the lag p are coprime. For that reason $m^n - 1$ should have a small number of prime factors. It can be shown that $m^n - 1$ has at least three prime factors and if the number of prime factors does not exceed three, then m is necessarily a Sophie-Germain Prime and n a prime larger than two [7].

To sum up, the modulus m of a LFSR sequence should be a Sophie-Germain Prime, such that $m^n - 1$ has not more than three prime factors and such that $m = 2^e - k$ and $k(k+2) \leq m$ for some integers e and k.

2.4.3 Matrix linear congruential generators

It has been shown before that multiple recurrence generators can be written as a matrix equation with a companion matrix. Matrix linear congruential generators are based on generalized recurrence of the form [26, 17]

$$\boldsymbol{r}_i = \boldsymbol{A}\boldsymbol{r}_{i-1} \mod m\,, \tag{2.18}$$

where m is a prime number, $\boldsymbol{r}_i$ denotes vector of n elements and $\boldsymbol{A}$ represents an $n \times n$ invertible matrix over the filed $\mathbb{F}_m$. The elements of $\boldsymbol{r}_i$ and $\boldsymbol{A}$ are integers $\in 0, 1, \ldots, m-1$, i. e., the elements of the field $\mathbb{F}_m$. The state $\boldsymbol{r}_i$ of such a generator can take m^n different

values. The state $\boldsymbol{r}_i = (0, 0, \ldots, 0)$ is a fixed point of the recurrence (2.18). Therefore, the period of a matrix linear congruential generator cannot exceed $m^n - 1$. This maximal period is attained if the matrix $\boldsymbol{A}$ is chosen appropriately, i. e., the matrix $\boldsymbol{A}$ is such that its rank equals $m^n - 1$.

Typical parameters that are employed for matrix linear congruential generators are $m = 2$ or m equal to a large prime that is close to the largest integer that can be represented by a machine register. The parameter n must be relatively large, e.g., $n \geq 64$, in the former case to reach a sufficient period, whereas in the latter case $n = 2$ or $n = 3$ may be sufficient depending on the size of m. The matrix $\boldsymbol{A}$ is often designed to allow an efficient implementation of the matrix-vector multiplication $\boldsymbol{A}\boldsymbol{r}_{i-1} \mod m$ while ensuring that the generator reaches the maximal period.

The parallelization of matrix linear congruential generators via block splitting and leapfrogging is straight forward. The M-fold successive iteration of (2.18) is given by

$$\boldsymbol{r}_i = \boldsymbol{A}^M \boldsymbol{r}_{i-M} \bmod m\,. \tag{2.19}$$

Block splitting can be directly implemented by the application of (2.19). Leapfrogging can be realized by replacing the matrix $\boldsymbol{A}$ by $\boldsymbol{A}^p$, where p denotes the number of independent streams. It should be noted, however, that if $\boldsymbol{A}$ has been chosen to be sparse to allow an efficient implementation of the matrix-vector product $\boldsymbol{A}\boldsymbol{r}_{i-1}$ then $\boldsymbol{A}^p$ is no longer sparse, which may render leapfrogging impractical.

2.5 Non-linear transformations and YARN sequences

LFSR sequences over prime fields with a large prime modulus seem to be ideally suited as PRNGs. They have, however, a well known weakness. When used to sample coordinates in d-dimensional space, pseudo-noise sequences cover every point for $d < n$, and every point except $(0, 0, \ldots, 0)$ for $d = n$. For $d > n$ the set of positions generated is obviously sparse, and the linearity of the production rule (2.9) leads to the concentration of the sampling points on n-dimensional hyper-planes [27, 39], see also Figure 2.3. This phenomenon, first noticed by Marsaglia in 1968 [47], constitutes one of the well known tests of randomness applied to PRNGs, the so-called spectral test [35]. The spectral test checks the behavior of a generator when its outputs are used to form d-tuples. Closely related to this mechanism are the observed correlations in other empirical tests like the birthday spacings test and the collision test [41, 43]. Non-linear generators do quite well in all these tests, but compared to LFSR sequences they have much less nice and *provable* properties, and they are not suited for fair playing parallelization.

To get the best of both worlds we propose to apply an output transformation that delinearizes the LFSR sequence but preserves all the nice properties of linear pseudo-noise sequences. That means each element of a linear pseudo-noise sequence $q_i \in \mathbb{F}_m$ is transformed to another element in $\mathbb{F}_m$ by a non-linear bijective mapping. If m is prime, such a bijective mapping is given by an exponentiation.

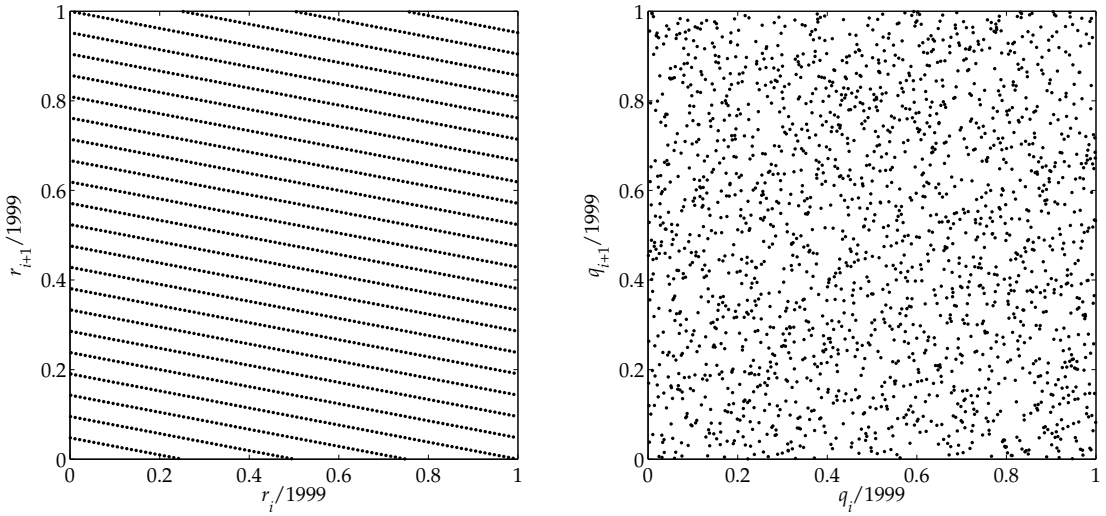


Figure 2.3: Exponentiation of a generating element in a prime field is an effective way to destroy the linear structures of LFSR sequences. Both pictures show the full period of the generator. Left: $r_i = 95 \cdot r_{i-i} \bmod 1999$. Right: $q_i = 1099^{r_i} \bmod 1999$ with $r_i = 95 \cdot r_{i-i} \bmod 1999$.

Theorem 6 Let r_i be a pseudo-noise sequence in $\mathbb{F}_m$, and let g be a generating element of the multiplicative group $\mathbb{F}_m^*$. Then the sequence q_i with

$$q_i = \begin{cases} g^{r_i} \bmod m & \text{if } r_i > 0 \\ 0 & \text{if } r_i = 0 \end{cases} \tag{2.20}$$

is a pseudo-noise sequence, too.

The proof of this theorem is trivial: since g is a generator of $\mathbb{F}_m^*$, the map (2.20) is bijective. We call delinearized generators based on Theorem 6 YARN generators (yet another random number).

The linearity is completely destroyed by the map (2.20), see Figure 2.3. Let $L_{(r)}(l)$ denote the linear complexity of the subsequence $(r_1, r_2, \ldots, r_l)$. This function is known as the linear complexity profile of r_i. For a truly random sequence it grows on average like $l/2$. Figure 2.4 shows the linear complexity profile $L_{(r)}(l)$ of a typical YARN sequence. It shows the same growth rate as a truly random sequence up to the point where more than 99 % of the period have been considered. Sharing the linear complexity profile with a truly random sequence, we may say that the YARN generator is as non-linear as it can get.

The non-linear transform by exponentiation in Theorem 6 has to be carried out in a prime field $\mathbb{F}_m$. If the underlying generator produces integers in some range $[0, m)$, where m is not prime (i. e. a power of two), another kind of non-linear transformation has to be applied to improve the underlying generator. For $m = 2^e$ Press et al. [66]

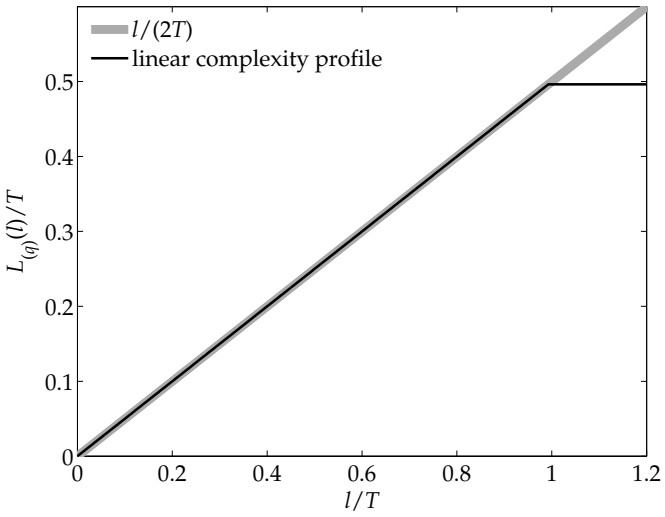


Figure 2.4: Linear complexity profile $L_{(q)}(l)$ of a YARN sequence, produced by the recurrence $r_i = 173 \cdot r_{i-1} + 219 \cdot r_{i-2} \bmod 317$ and $q_i = 151^{r_i} \bmod 317$. The period of this sequence equals $T = 317^2 - 1$.

suggest to transform the output r_i of a base generator by

$$\begin{aligned} t_{i,0} &= r_i \\ t_{i,1} &= t_{i,0} \oplus (t_{i,0} \gg s_0) \\ t_{i,2} &= t_{i,1} \oplus (t_{i,1} \ll s_1) \\ t_{i,3} &= t_{i,2} \oplus (t_{i,2} \gg s_2) \\ q_i &= t_{i,3} \end{aligned} \tag{2.21}$$

where $\oplus$ denotes binary addition (exclusive-or), $x \gg n$ bit-shift of x to the right of size n and $x \ll n$ bit-shift of x to the left of size n, respectively. The shift parameters s_0, s_1 and s_2 have to be chosen suitable to make (2.21) a bijective mapping from r_i to q_i, see [66]. Figure 2.5 shows how the mapping (2.21) effectively destroys the lattice structures of linear congruential generators modulo a power of two.

The concept of applying an output transformation to a pseudo-random sequence to improve its statistical properties is very general. Alternative mappings to the ones proposed here have been suggested in [62].

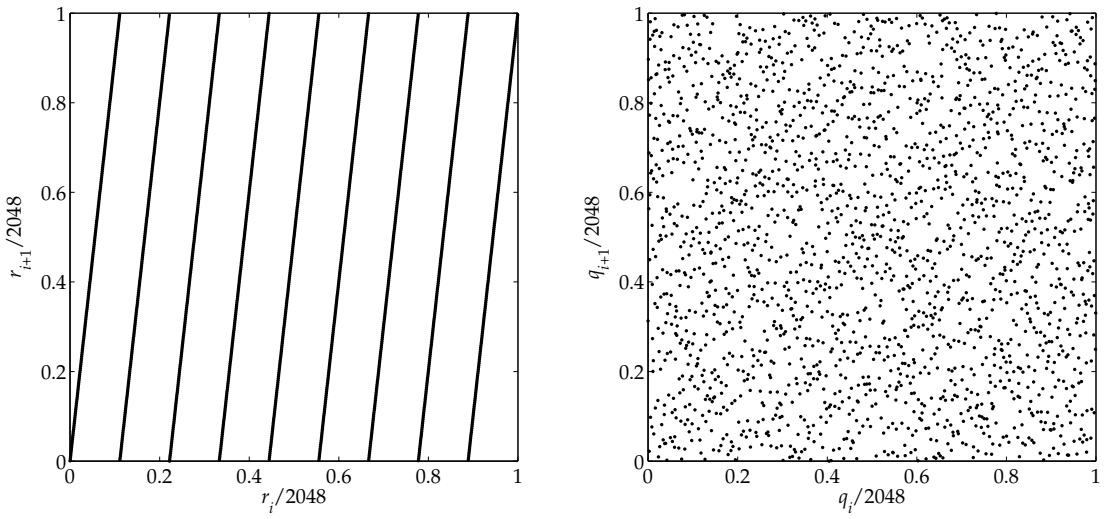


Figure 2.5: The non-linear mapping (2.21) destroys the lattice structures of linear congruential generators. Both pictures show the full period of the generator. Left: $r_i = 9 \cdot r_{i-i} + 1 \bmod 2048$. Right: q_i given by (2.21) with $s_0 = 5$, $s_1 = 9$, $s_2 = 2$ and $r_i = 9 \cdot r_{i-i} + 1 \bmod 2048$.

3 Basic concepts

The TRNG library consists of a loose bunch of classes. These classes can be divided into two kinds of classes, *random number engines* and *random number distributions*.

Random number engines are the workhorses of TRNG. Each random number engine implements some algorithm that is used to produce pseudo-random numbers. The notion of a random number engine as it is used by TRNG was introduced by [12] and it is a very general concept. For example, the random number engine concept does not specify what kind of pseudo-random numbers (integers, floating point numbers or just bits) are generated. All random number engine classes of TRNG implement the concept of a random number engine that has been introduced in [12] and that was later included in the C++11 language standard [29]. As a library of *parallel* random number generators, however, TRNG extends the notion of a random number engine to a *parallel random number engine*. To fulfill the requirements of a parallel random number engine, a class has to fulfill all the requirements of a random number engine and in addition some further requirements that make them applicable for parallel Monte Carlo simulations. The random number engine concept and the parallel random number engine concept will be discussed in detail in section 3.1.

A random number engine is not very useful by itself. To write some real world Monte Carlo applications we need random number distribution classes, too. A random number distribution class converts the output of an arbitrary random number engine into a pseudo-random number drawn from some specific distribution. The general concept of a random number distribution is discussed in section 3.2.

Note that the design of TRNG was initially based on a proposal for the 2011 revision of the C++ language standard [12]. This proposal has eventually become part of the C++ 11 language standard with some minor modifications. This language standard (as well its successors C++ 13 and C++ 17) is supported by all mayor C++ compilers now. TRNG version 4.22 and later versions follow the conventions of the random number generator facility of the C++ standard library, no longer supporting the original proposal [12]. This means, TRNG requires a compiler that supports C++ 11 (or any later language standard) and TRNG classes can be used in combination with classes of the random number generator facility of the C++ standard library.

3.1 Random number engines

To be a random number engine, a class has to fulfill a set of requirements that we will summarize as follows, see [12] for details. A class `X` satisfies the requirements of a random number engine, if the expressions as shown in Table 3.1 are valid and have the

indicated semantics. In that table and throughout this section,

- `T` is the type named by `X`'s associated `result_type`;
- `t` is a value of `T`;
- `u` is a value of `X`, `v` is an lvalue of `X`, `x` and `y` are (possibly const) values of `X`;
- `s` is a value of integral type;
- `g` is an lvalue, of a type other than `X`, that defines a zero-argument function object returning values of type `unsigned long`;
- `os` is an lvalue of the type of some class template specialization `std::basic_ostream <charT, traits>`; and
- `is` is an lvalue of the type of some class template specialization `std::basic_istream <charT, traits>`.

A random number engine object `x` has at any given time a state x_i for some integer $i \geq 0$. Upon construction, a random number engine `x` has an initial state x_0. The state of an engine may be established by invoking its constructor, `seed` member function, `operator=`, or a suitable `operator>>`.

The specification of each random number engine defines the size of its state in multiples of the size of its `result_type`, given as an integral constant expression. The specification of each random number engine also defines

- the *transition algorithm* TA by which the state x_i of an engine is advanced to its *successor state* x_{i+1}, and
- the *generation algorithm* GA by which the state of an engine is mapped to a value of type `result_type`.

Furthermore, a random number engine shall fulfill the requirements of the concepts "CopyConstructible" and of "Assignable". That means roughly, random number engines support copy and assignment operations with the same semantic like build-in types as `int` or `double`. Copy construction and assignment shall each be of complexity $\mathcal{O}$ (size of state).

Random number engine requirements had been adopted from [12]. For parallel Monte Carlo applications we extend the concept of a random number engine to a parallel random number engine. Such an engine has to meet all the requirements of a parallel random number engine and additionally the requirements shown in Table 3.2. A parallel random number engine provides block splitting and leapfrog. It is demanded that leapfrog is implemented in such a way that the complexity of `operator()` will not depend on how many sub-streams a stream has been split into. That means, a valid implementation of leapfrog will not just calculate all random numbers of a stream and then throw away bunches of numbers to derive the random numbers of a leapfrog sub-stream. This rather strong requirement restricts the number of pseudo-random number generator algorithms that are proper for parallel random number engines. But LFSR sequences and YARN generators, which had been discussed in sections 2.4.2 and 6, meet these conditions easily. Note that the methods `discard` and `jump` have the same effect but `jump` has tighter time-complexity requirements.

3.2 Random number distributions

To model the concept of a random number distribution a class has to fulfill a set of requirements that we will summarize as follows, refer to [12] for details. A class `X` satisfies the requirements of a random number distribution if the expressions shown in Table 3.3 are valid and have the indicated semantics, and if `X` and its associated types also satisfies all other requirements of this section. In that table and throughout this section,

- `T` is the type named by `X`'s associated `result_type`;
- `P` is the type named by `X`'s associated `param_type`;
- `u` is a value of `X` and `x` is a (possibly const) value of `X`;
- `glb` and `lub` are values of `T` respectively corresponding to the greatest lower bound and the least upper bound on the values potentially returned by `u`'s `operator()`, as determined by the current values of `u`'s parameters;
- `p` is a value of `P`;
- `e` is an lvalue of an arbitrary type that satisfies the requirements of a uniform random number generator;
- `os` is an lvalue of the type of some class template specialization `basic_ostream<charT, traits>`; and
- `is` is an lvalue of the type of some class template specialization `basic_istream<charT, traits>`.

The specification of each random number distribution identifies an associated mathematical *probability density function* $p(z)$ or an associated discrete *probability function* $P(z_i)$. Such functions are typically expressed using certain externally supplied quantities known as the *parameters of the distribution*. Such distribution parameters are identified in this context by writing, for example, $p(z|a,b)$ or $P(z_i|a,b)$, to name specific parameters, or by writing, for example, $p(z|\{p\})$ or $P(z_i|\{p\})$, to denote the parameters `p` of a distribution taken as a whole.

Furthermore, a random number distribution shall fulfill the requirements of the concepts "CopyConstructible" and of "Assignable". That means roughly, random number distributions support copy and assignment operations with the same semantic as build-in types like `int` or `double`. Copy construction and assignment shall each be of complexity $\mathcal{O}$ (size of state).

For each of the constructors of `X` taking arguments corresponding to parameters of the distribution, `P` shall have a corresponding constructor subject to the same requirements and taking arguments identical in number, type, and default values. Moreover, for each of the member functions of `X` that return values corresponding to parameters of the distribution, `P` shall have a corresponding member function with the identical name, type, and semantics.

Table 3.1: Random number engine requirements.

expression	return type	pre/post-condition	complexity
`X::result_type`	`T`	`T` is an arithmetic type other than `bool`.	compile-time
`u()`	`T`	Sets the state to $u_{i+1} = \mathsf{TA}(u_i)$ and returns $\mathsf{GA}(u_i)$. If `X` is integral, returns a value in the closed interval $[$`X::min()`, `X::max()`$]$; otherwise, returns a value in the open interval $(0, 1)$.	amortized constant
`u.discard(s)`	`void`	pre: `s` is of type `unsigned long long`. post: Internal state of the random number engine is changed in such a way that the engine jumps `s` steps ahead.	$\mathcal{O}$ (`d`) or less
`X::min()`	`T`, if `X` is integral; otherwise `int`.	If `X` is integral, denotes the least value potentially returned by `operator()`; otherwise denotes 0.	compile-time
`X::max()`	`T`, if `X` is integral; otherwise `int`.	If `X` is integral, denotes the greatest value potentially returned by `operator()`; otherwise denotes 1.	compile-time
`X()`		Creates an engine with the same initial state as all other default-constructed engines of type `X`.	$\mathcal{O}$ (size of state)
`X(s)`		Creates an engine with initial state determined by `static_cast<unsigned long>(s)`.	$\mathcal{O}$ (size of state)
`X(g)`		Creates an engine with initial state determined by the results of successive invocations of `g`. Throws what and when `g` throws.	$\mathcal{O}$ (size of state)
`u.seed()`	`void`	post: `u==X()`	same as `X()`
`u.seed(s)`	`void`	post: `u==X(s)`	same as `X(s)`
`u.seed(g)`	`void`	post: If `g` does not throw, `u==v`, where the state of `v` is as if constructed by `X(g)`. Otherwise, the exception is re-thrown and the engine `s` state is deemed invalid. Thereafter, further use of `u` is undefined except for destruction or invoking a function that establishes a valid state.	same as `X(g)`
`x==y`	`bool`	With S_x and S_y as the infinite sequences of values that would be generated by repeated calls to `x()` and `y()`, respectively, returns `true` if $S_x = S_y$; returns `false` otherwise.	$\mathcal{O}$ (size of state)
`x!=y`	`bool`	`!(x==y)`	$\mathcal{O}$ (size of state)

Table 3.1: Random number engine requirements continued.

expression	return type	pre/post-condition	complexity
`os << x`	reference to the type of `os`	With `os`.*fmtflags* set to `std::ios_base::dec|std::ios_base::fixed|std::ios_base::left` and the fill character set to the space character, writes to `os` the textual representation of `x`'s current state. In the output, adjacent numbers are separated by one or more space characters. post: The `os`.*fmtflags* and fill character are unchanged.	$\mathcal{O}$ (size of state)
`is >> v`	reference to the type of `is`	Sets `v`'s state as determined by reading its textual representation from `is`. If bad input is encountered, ensures that `v`'s state is unchanged by the operation and calls `is.setstate(std::ios::failbit)` (which may throw `std::ios::failure`). pre: The textual representation was previously written using an `os` whose imbued locale and whose type's template specialization arguments `charT` and `traits` were the same as those of `is`. post: The `is`.*fmtflags* are unchanged.	$\mathcal{O}$ (size of state)

Table 3.2: Parallel random number engine requirements.

expression	return type	pre/post-condition	complexity
`split(p, s)`	`void`	pre: `s` and `p` are of type `unsigned int` with $s < p$. If $s \geq p$ an exception `std::invalid_argument` is thrown. post: Internal parameters of the random number engine are changed in such a way that future calls to `operator()` will generate the `s`th sub-stream of `p` sub-streams. Sub-streams are numbered from 0 to $p - 1$. The complexity of `operator()` will not change.	polynomial in size of state, (at most) linear in `p` and `s`
`jump2(s)`	`void`	pre: `s` is of type `unsigned int`. post: Internal state of the random number engine is changed in such a way that the engine jumps 2^s steps ahead.	polynomial in size of state and `s`
`jump(s)`	`void`	pre: `s` is of type `unsigned long long`. post: Internal state of the random number engine is changed in such a way that the engine jumps `s` steps ahead.	polynomial in size of state and the logarithm of `s`

Table 3.3: Random number distribution requirements.

expression	return type	pre/post-condition	complexity
`X::result_type`	`T`	`T` is an arithmetic type.	compile-time
`X::param_type`	`P`		compile-time
`X(p)`		Creates a distribution whose behavior is indistinguishable from that of a distribution newly constructed directly from the values used to construct `p`.	same as `p`'s construction
`u.reset()`	`void`	Subsequent uses of `u` do not depend on values produced by `e` prior to invoking `reset`.	constant
`x.param()`	`P`	Returns a value `p` such that `X(p).param()==p`.	no worse than the complexity of `X(p)`
`u.param(p)`	`void`	post: `u.param() == p`.	no worse than the complexity of `X(p)`
`u(e)`	`T`	With `p=u.param()`, the sequence of numbers returned by successive invocations with the same object `e` is randomly distributed according to the associated $p(z\|\{p\})$ or $P(z_i\|\{p\})$ function.	amortized constant number of invocations of `e`
`u(e,p)`	`T`	The sequence of numbers returned by successive invocations with the same objects `e` and `p` is randomly distributed according to the associated $p(z\|\{p\})$ or $P(z_i\|\{p\})$ function	
`x.min()`	`T`	Returns `glb`.	constant
`x.max()`	`T`	Returns `lub`.	constant
`os << x`	reference to the type of `os`	Writes to os a textual representation for the parameters and the additional internal data of `x`. post: The `os`.*fmtflags* and fill character are unchanged.	
`is >> u`	reference to the type of `is`	Restores from is the parameters and additional internal data of `u`. If bad input is encountered, ensures that `u`'s state is unchanged by the operation and calls `is.setstate(ios::failbit)` (which may throw `std::ios::failure`). pre: `is` provides a textual representation that was previously written using an `os` whose imbued locale and whose type's template specialization arguments `charT` and `traits` were the same as those of `is`. post: The `is`.*fmtflags* are unchanged.	

4 TRNG classes

In chapter 3 the abstract concepts of (parallel) random number engines and random number distributions had been introduced. Now we look at some actual realizations of these concepts. TRNG provides several (parallel) random number engines and random number distributions. Each engine and each distribution is implemented by its own class that resides in the name space `trng`.

4.1 Random number engines

In this section we give a detailed documentation of all random number engines. Each subsection describes the public interface of one random number engine and focuses on aspects that are specific for a particular random number engine. This includes extensions to the random number engine interface as well as algorithmic details. The part of the public interface, that is mandatory for each (parallel) random number engine, will not be discussed in detail. Read section 3.1 instead. Table 4.1 gives an overview over all random number engines of TRNG.

All classes that will be described in this section model either a random number engine or a parallel random number engine and therefore fulfill the requirements introduced in section 3.1. Note that `trng::xoshiro256plus` and the multiply-with-carry engines `trng::mwc64x32`, `trng::mwc128x32`, `trng::mwc128x` and `trng::mwc192x` are *not* parallel random number engines in the strict sense of section 3.1, because they do not provide the member function `split(p, s)` and hence no leapfrog method. They are random number engines that additionally support block splitting by means of the member functions `jump2(s)`, `jump(s)` and `discard(s)`. But for convenience their interface provides even more. For example all random number engines model a *random number generator* as well. The notion of a random number generator had been introduced by the C++ Standard Template Library. A random number generator is a class that provides an `operator()(long)` that returns a uniformly distributed random integer larger than or equal to zero but less than its argument. This interface was formerly used by the STL algorithm `std::random_shuffle`, which has been removed from the C++ standard library in C++17. Use `std::shuffle` together with a TRNG engine instead. Additionally, TRNG (parallel) random number engines provide a function `name()` that returns a string with the name of the random number engine.

Table 4.1: Random number engines of TRNG.

random number engine	description	concept
`trng::lcg64`	linear congruential generator with modulus 2^{64}	parallel random number engine
`trng::lcg64_shift`	linear congruential generator with modulus 2^{64} with a bit-shift output transformation	parallel random number engine
`trng::lcg64_count_shift`	linear congruential generator with modulus 2^{64} with combined with an additional linear congruence and a bit-shift output transformation	parallel random number engine
`trng::count128_lcg_shift`	counter-based generator with 128-bit counter and an additional output transformation using linear congruences and bit-shift operations	parallel random number engine
`trng::mrg`*n*	multiple recurrence generator based on a linear feedback shift register sequence over $\mathbb{F}_{2^{31}-1}$ of depth n	parallel random number engine
`trng::mrg`*n*`s`	multiple recurrence generator based on a linear feedback shift register sequence over $\mathbb{F}_m$ of depth n, with m being a Sophie-Germain Prime	parallel random number engine
`trng::yarn`*n*	YARN sequence based on a linear feedback shift register sequence over $\mathbb{F}_{2^{31}-1}$ of depth n	parallel random number engine
`trng::yarn`*n*`s`	YARN sequence based on a linear feedback shift register sequence over $\mathbb{F}_m$ of depth n, with m being a Sophie-Germain Prime	parallel random number engine
`trng::lagfib`*n*`xor`	lagged Fibonacci generator with n feedback taps and exclusive-or operation	random number engine
`trng::lagfib`*n*`plus`	lagged Fibonacci generator with n feedback taps and addition	random number engine
`trng::xoshiro256plus`	xoshiro (xor/shift/rotate)	random number engine
`trng::mwc64x32`	scrambled multiply-with-carry generator with a 64-bit state and 32-bit output	random number engine
`trng::mwc128x32`	scrambled multiply-with-carry generator with a 128-bit state and 32-bit output	random number engine
`trng::mwc128x`	scrambled multiply-with-carry generator with a 128-bit state and 64-bit output	random number engine
`trng::mwc192x`	scrambled multiply-with-carry generator with a 192-bit state and 64-bit output	random number engine
`trng::minstd`	Lehmer generator with modulus $2^{31}-1$ and multiplier 16807 (minimal standard generator)	random number engine
`trng::mt19937`	Mersenne twister generating 32 random bits	random number engine
`trng::mt19937_64`	Mersenne twister generating 64 random bits	random number engine

4.1.1 Linear congruential generators and variants

The classes `trng::lcg64` and `trng::lcg64_shift` implement linear congruential generators. Both generators are based on the transition algorithm [44, 35]

$$r_{i+1} = a \cdot r_i + b \bmod 2^{64}.$$

The state of this generator at time i is given by r_i. Its period equals 2^{64} if and only if b is odd and $a \bmod 4 = 1$ [35]. The statistical properties of linear congruential generators depend crucial on the choice of the multiplier a, which has to be chosen carefully.

This linear congruential generator `trng::lcg64` is the quick and dirty generator of TRNG. It's dammed fast, see section 7, but even for proper chosen parameters a and b the lower bits of r_i are less random than the higher order bits. The class `trng::lcg64` should be avoided whenever the randomness of lower bits have a significant impact to the simulation. In [38] L'Ecuyer warns about multiplicative linear congruential generators (in the following quotation denoted as MLCG) with $r_{i+1} = a \cdot r_i \bmod m$:

> "If $m = 2^e$ where e is the number of bits on the computer word, and if one can use unsigned integers without overflow checking, the products modulo m are easy to compute: just discard the overflow. This is quick and simple. For that reason, MLCGs with moduli of this form are used abundantly in practice, despite their serious drawbacks. Some nuclear physicists, for instance, perform simulations that use billions of random numbers on supercomputers and are quite reluctant to give up using them [...]. Usually, they also generate many substreams in parallel. In view of the above remarks, all this appears dangerous. Perhaps some people like playing with fire."

The same warning applies if $b \neq 0$. In spite of its weakness this generator is well suited for a large classes of generic Monte Carlo schemes, e. g. simulating a (biased) coin or cluster Monte Carlo [20].

But in some kinds of simulations linear congruential generators reveal their weakness, i. e. their lattice structure, see left part of Figure 2.5. There are two general approaches to improve linear congruential generators: output transformation and combination with other generators. Both approaches are employed in the classes `trng::lcg64_shift` and `trng::lcg64_count_shift`. Both classes are based on the linear recursion

$$r_{i+1} = a \cdot r_i + b \bmod 2^{64}.$$

The class `trng::lcg64_shift` destroys the lattice structure of r_i by the non-linear output transformation

$$\begin{aligned} t_{i,0} &= r_i \\ t_{i,1} &= t_{i,0} \oplus (t_{i,0} \gg 17) \\ t_{i,2} &= t_{i,1} \oplus (t_{i,1} \ll 31) \\ t_{i,3} &= t_{i,2} \oplus (t_{i,2} \gg 8) \\ q_i &= t_{i,3} \end{aligned}$$

that yields the pseudo-random number q_i from r_i. Here, $\oplus$ denotes binary addition (exclusive-or), $x \gg n$ bit-shift of x to the right of size n and $x \ll n$ bit-shift of x to the left of size n, respectively. Class `trng::lcg64_shift` is only slightly slower than `trng::lcg64` but the statistical quality is considerably increased by the non-linear transformation.

The class `trng::lcg64_count_shift` combines two linear congruences to construct a combined generator with a period that is larger than the periods of the two underlying generators. More precisely, it is based on the two recurrences

$$\begin{aligned} r_{i+1} &= a \cdot r_i + b \bmod 2^{64}\,, \\ r'_{i+1} &= r'_i + c \bmod 2^{61} - 1\,, \end{aligned}$$

with $c = 1\,425\,089\,352\,415\,399\,810$. The output transform for this generator is defined as

$$\begin{aligned} t_{i,0} &= r_i + r'_i \bmod 2^{64} \\ t_{i,1} &= t_{i,0} \oplus (t_{i,0} \gg 17) \\ t_{i,2} &= t_{i,1} \oplus (t_{i,1} \ll 31) \\ t_{i,3} &= t_{i,2} \oplus (t_{i,2} \gg 8) \\ q_i &= t_{i,3}\,. \end{aligned}$$

The modulus of the second recurrence $2^{61} - 1$ is a Mersenne prime. Thus, both moduli are coprime and the period of the combined generator is the product $2^{64}(2^{61} - 1) \approx 2^{125}$. The sequence r'_i is a counting sequence with non-unit increment, which is trivial to parallelize via block splitting and leap frogging. It is, however, a rather poor pseudo-random number sequence. In combination with the other linear congruence for r_i it merely serves to yield a large period of the combined generator and due to the output transform the statistical properties of the combined generator are much better than those of the base sequences r_i and r'_i.

The class `trng::lcg64` is declared in the header file `trng/lcg64.hpp` and its public interface is given as follows:

```
namespace trng {

  class lcg64 {
  public:
```

First the necessary type, static class constants, and the call operator are declared.

```
    using result_type = uint64_t;
    result_type operator()();
    static constexpr result_type min();
    static constexpr result_type max();
```

We also define some parameter and status classes that will be used internally and by the constructor.

```
    class parameter_type;
    class status_type;
```

TRNG provides four parameter sets for a and b, which are chosen to give good statistical properties. Three of these are taken from [40], the default parameter set had been found by the author of the TRNG library.

$$a = 18\,145\,460\,002\,477\,866\,997\,, \quad b = 1$$

```
static const parameter_type Default;
```

$$a = 2\,862\,933\,555\,777\,941\,757\,, \quad b = 1$$

```
static const parameter_type LEcuyer1;
```

$$a = 3\,202\,034\,522\,624\,059\,733\,, \quad b = 1$$

```
static const parameter_type LEcuyer2;
```

$$a = 3\,935\,559\,000\,370\,003\,845\,, \quad b = 1$$

```
static const parameter_type LEcuyer3;
```

An instance of class `trng::lcg64` can be instantiated by various constructors as specified for a random number engine. Additionally, a non-default parameter set may be given.

```
explicit lcg64(parameter_type = Default);
explicit lcg64(unsigned long, parameter_type = Default);
template<typename gen>
explicit lcg64(gen &, parameter_type P = Default);
```

The class `trng::lcg64` provides all necessary seeding functions (see Table 3.1) and an additional function that sets r_i.

```
void seed();
void seed(unsigned long);
template<typename gen>
void seed(gen &);
void seed(unsigned long long);
```

The following three methods are necessary for a parallel random number engine.

```
void split(unsigned int, unsigned int);
void jump2(unsigned int);
void jump(unsigned long long);
void discard(unsigned long long);
```

Furthermore, the class `trng::lcg64` provides a function that returns the string `lcg64` and an operator `operator()`.

```
  static const char * name();
  long operator()(long);
};
```

Random number engines are comparable and can be written to or read from a stream.

```
  bool operator==(const lcg64 &, const lcg64 &);
  bool operator!=(const lcg64 &, const lcg64 &);
  template<typename char_t, typename traits_t>
  std::basic_ostream<char_t, traits_t> &
  operator<<(std::basic_ostream<char_t, traits_t> &, const lcg64 &);
  template<typename char_t, typename traits_t>
  std::basic_istream<char_t, traits_t> &
  operator>>(std::basic_istream<char_t, traits_t> &, lcg64 &);
}
```

The class `trng::lcg64_shift` provides the same public interface as `trng::lcg64`.

```
namespace trng {

  class lcg64_shift {
  public:
    using result_type = uint64_t;
    result_type operator()();
    static constexpr result_type min();
    static constexpr result_type max();
    class parameter_type;
    class status_type;
    static const parameter_type Default;
    static const parameter_type LEcuyer1;
    static const parameter_type LEcuyer2;
    static const parameter_type LEcuyer3;
    explicit lcg64_shift(parameter_type = Default);
    explicit lcg64_shift(unsigned long, parameter_type = Default);
    template<typename gen>
    explicit lcg64_shift(gen &, parameter_type P = Default);
    void seed();
    void seed(unsigned long);
    template<typename gen>
    void seed(gen &);
    void seed(unsigned long long);
    void split(unsigned int, unsigned int);
    void jump2(unsigned int);
    void jump(unsigned long long);
    void discard(unsigned long long);
    static const char * name();
    long operator()(long);
  };

  bool operator==(const lcg64_shift &, const lcg64_shift &);
  bool operator!=(const lcg64_shift &, const lcg64_shift &);
  template<typename char_t, typename traits_t>
  std::basic_ostream<char_t, traits_t> &
  operator<<(std::basic_ostream<char_t, traits_t> &, const lcg64_shift &);
  template<typename char_t, typename traits_t>
  std::basic_istream<char_t, traits_t> &
  operator>>(std::basic_istream<char_t, traits_t> &, lcg64_shift &);
}
```

The class `trng::lcg64_count_shift` provides the same public interface as `trng::lcg64` and `trng::lcg64_shift`.

```
namespace trng {

  class lcg64_count_shift {
  public:
    using result_type = uint64_t;
    result_type operator()();
    static constexpr result_type min();
    static constexpr result_type max();
    class parameter_type;
    class status_type;
    static const parameter_type Default;
    static const parameter_type LEcuyer1;
    static const parameter_type LEcuyer2;
    static const parameter_type LEcuyer3;
    explicit lcg64_count_shift(parameter_type = Default);
    explicit lcg64_count_shift(unsigned long, parameter_type = Default);
    template<typename gen>
    explicit lcg64_shift(gen &, parameter_type P = Default);
    void seed();
    void seed(unsigned long);
    template<typename gen>
    void seed(gen &);
    void seed(unsigned long long);
    void split(unsigned int, unsigned int);
    void jump2(unsigned int);
    void jump(unsigned long long);
    void discard(unsigned long long);
    static const char * name();
    long operator()(long);
  };

  bool operator==(const lcg64_count_shift &, const lcg64_count_shift &);
  bool operator!=(const lcg64_count_shift &, const lcg64_count_shift &);
  template<typename char_t, typename traits_t>
  std::basic_ostream<char_t, traits_t> &
  operator<<(std::basic_ostream<char_t, traits_t> &, const lcg64_count_shift &);
  template<typename char_t, typename traits_t>
  std::basic_istream<char_t, traits_t> &
  operator>>(std::basic_istream<char_t, traits_t> &, lcg64_count_shift &);
}
```

4.1.2 Counter-based generators

A linear congruential generator with a multiplier equal to one reduces a counting generator of the form

$$r_{i+1} = r_i + c \bmod m\,.$$

A counting generator has the period m provided that the modulus m and the increment c are coprime. This kind of generator is fast and trivial to parallelize but has poor statistical

properties. Therefore, it is used in combination with a non-linear output transformation. TRNG implements a counter-based generator with $m = 2^{128}$ and c being a prime number of the order of $m/2$. The output transform is defined as follows:

$$
\begin{aligned}
t_{i,0} &= \left(\lfloor r_i / 2^{64} \rfloor \oplus (r_i \bmod 2^{64}) \right) \cdot a + b \mod 2^{64} \\
t_{i,1} &= t_{i,0} \oplus (t_{i,0} \gg s_0) \\
t_{i,2} &= t_{i,1} \oplus (t_{i,1} \ll s_1) \\
t_{i,3} &= t_{i,2} \oplus (t_{i,2} \gg s_2) \\
q_i &= t_{i,3}
\end{aligned}
$$

This output transform first combines the upper and lower half-parts of the 128-bit integer r_i by an bitwise exclusive-or operation and which is then transformed by a linear mapping and an additional series of bit-shift and exclusive-or operations.

The class `trng::count128_lcg_lcg64` implements a counter-based generato as described above. It is declared in the header file `trng/count128_lcg_lcg64.hpp` and its public interface is given as follows:

```
namespace trng {

  class count128_lcg_lcg64 {
  public:
```

First the necessary type, static class constants, and the call operator are declared.

```
    using result_type = uint64_t;
    result_type operator()();
    static constexpr result_type min();
    static constexpr result_type max();
```

We also define some parameter and status classes that will be used internally and by the constructor.

```
    class parameter_type;
    class status_type;
```

TRNG provides four parameter sets for the parameters a and b, which are chosen to give good statistical properties. Three of these are taken from [40], the default parameter set had been found by the author of the TRNG library.

$a = 18\,145\,460\,002\,477\,866\,997\,, \quad b = 1\,, \quad c = 337\,796\,325\,545\,380\,861\,827\,125\,810\,166\,389\,624\,843$

```
    static const parameter_type Default;
```

$a = 2\,862\,933\,555\,777\,941\,757\,, \quad b = 1\,, \quad c = 337\,796\,325\,545\,380\,861\,827\,125\,810\,166\,389\,624\,843$

```
    static const parameter_type LEcuyer1;
```

$a = 3\,202\,034\,522\,624\,059\,733\,, \quad b = 1\,, \quad c = 337\,796\,325\,545\,380\,861\,827\,125\,810\,166\,389\,624\,843$

```
    static const parameter_type LEcuyer2;
```

$a = 3\,935\,559\,000\,370\,003\,845\,, \quad b = 1\,, \quad c = 337\,796\,325\,545\,380\,861\,827\,125\,810\,166\,389\,624\,843$

```
    static const parameter_type LEcuyer3;
```

An instance of class `trng::count128_lcg_lcg64` can be instantiated by various constructors as specified for a random number engine. Additionally, a non-default parameter set may be given.

```
    explicit count128_lcg_lcg64(parameter_type = Default);
    explicit count128_lcg_lcg64(unsigned long, parameter_type = Default);
    template<typename gen>
    explicit count128_lcg_lcg64(gen &, parameter_type P = Default);
```

The class `trng::count128_lcg_lcg64` provides all necessary seeding functions (see Table 3.1) and an additional function that sets r_i.

```
    void seed();
    void seed(unsigned long);
    template<typename gen>
    void seed(gen &);
    void seed(unsigned long long);
```

The following three methods are necessary for a parallel random number engine.

```
    void split(unsigned int, unsigned int);
    void jump2(unsigned int);
    void jump(unsigned long long);
    void discard(unsigned long long);
```

Furthermore, the class `trng::count128_lcg_lcg64` provides a function that returns the string `count128_lcg_lcg64` and an operator `operator()`.

```
    static const char * name();
    long operator()(long);
  };
```

Random number engines are comparable and can be written to or read from a stream.

```
  bool operator==(const count128_lcg_lcg64 &, const count128_lcg_lcg64 &);
  bool operator!=(const count128_lcg_lcg64 &, const count128_lcg_lcg64 &);
  template<typename char_t, typename traits_t>
  std::basic_ostream<char_t, traits_t> &
  operator<<(std::basic_ostream<char_t, traits_t> &, const lcg64 &);
  template<typename char_t, typename traits_t>
  std::basic_istream<char_t, traits_t> &
  operator>>(std::basic_istream<char_t, traits_t> &, lcg64 &);
}
```

4.1.3 Multiple recursive generators

TRNG offers several multiple recursive generators based on LFSR sequences over prime fields $\mathbb{F}_m$ with different numbers of feedback taps. These are implemented by the classes `trng::mrg2`, `trng::mrg3`, `trng::mrg3s`, `trng::mrg4`, `trng::mrg5`, and `trng::mrg5s`. Table 4.2 summarizes the key features of these classes. The transition algorithm of a multiple recursive generator with n feedback taps reads

$$r_i = a_1 \cdot r_{i-1} + a_2 \cdot r_{i-2} + \ldots + a_n \cdot r_{n-2} \bmod m\,.$$

The state of this generator at time i is given by $(r_{i-1}, r_{i-2}, \ldots, r_{i-n})$. See section 2.4.2 for details on LFSR sequences.

The prime modulus m that characterizes the prime field $\mathbb{F}_m$ was either chosen as the Mersenne Prime (classes `trng::mrg`n) or a Sophie-Germain Prime such that $m^n - 1$ has as few prime factors as possible (classes `trng::mrg`n`s`). The former choice gives us some performance benefits, see section 7.1, whereas the second has some theoretical advantages, see section 2.4.2.

The classes `trng::mrg`n and `trng::mrg`n`s` implement the interface described in section 3.1. Each class defines some parameter and status classes that will be used internally and by the constructor. Furthermore for each generator several parameter sets are given, see Table 4.3. Most of the parameter sets are taken from [39] and chosen to give generators with good statistical properties.

An instance of a class `trng::mrg`n or `trng::mrg`n`s` can be instantiated by various constructors as specified for a random number engine. Additionally, a non-default parameter set may be chosen. The classes `trng::mrg`n and `trng::mrg`n`s` provide all necessary seeding functions (see Table 3.1) and additionally a function that sets the internal state $(r_{i-1}, r_{i-2}, \ldots, r_{i-n})$. This function should never be called with all arguments set to zero. The classes `trng::mrg`n and `trng::mrg`n`s` model the concept of a parallel random number engine and therefore the methods

```
void split(unsigned int, unsigned int);
void jump2(unsigned int);
void jump(unsigned long long);
void discard(unsigned long long);
```

are implemented. Furthermore the classes `trng::mrg`n or `trng::mrg`n`s` provide a function that returns a string with its name and an operator `operator()`. Random number engines are comparable and can be written to or read from a stream.

The detailed interface of the classes `trng::mrg`n or `trng::mrg`n`s` is given as follows:

```
namespace trng {

  class mrg2 {
  public:
    using result_type = int32_t;
    result_type operator()();
    static constexpr result_type min();
    static constexpr result_type max();
```

Table 4.2: Key features of multiple recursive generator classes.

class	header file	feedback taps n	prime field $\mathbb{F}_m$	period	return value of `name()`
`trng::mrg2`	`trng/mrg2.hpp`	2	$\mathbb{F}_{2^{31}-1}$	$m^2-1 \approx 2^{62} \approx 4.61 \cdot 10^{18}$	`mrg2`
`trng::mrg3`	`trng/mrg3.hpp`	3	$\mathbb{F}_{2^{31}-1}$	$m^3-1 \approx 2^{93} \approx 9.90 \cdot 10^{27}$	`mrg3`
`trng::mrg3s`	`trng/mrg3s.hpp`	3	$\mathbb{F}_{2^{31}-21\,069}$	$m^3-1 \approx 2^{93} \approx 9.90 \cdot 10^{27}$	`mrg3s`
`trng::mrg4`	`trng/mrg4.hpp`	4	$\mathbb{F}_{2^{31}-1}$	$m^4-1 \approx 2^{124} \approx 2.13 \cdot 10^{37}$	`mrg4`
`trng::mrg5`	`trng/mrg5.hpp`	5	$\mathbb{F}_{2^{31}-1}$	$m^5-1 \approx 2^{155} \approx 4.57 \cdot 10^{46}$	`mrg5`
`trng::mrg5s`	`trng/mrg5s.hpp`	5	$\mathbb{F}_{2^{31}-22\,641}$	$m^5-1 \approx 2^{155} \approx 4.57 \cdot 10^{46}$	`mrg5s`

Table 4.3: Parameter sets for multiple recursive generators.

parameter set	a_1	a_2	a_3	a_4	a_5
`trng::mrg2::LEcuyer1`	1 498 809 829	1 160 990 996			
`trng::mrg2::LEcuyer2`	46 325	1 084 587			
`trng::mrg3::LEcuyer1`	2 021 422 057	1 826 992 351	1 977 753 457		
`trng::mrg3::LEcuyer2`	1 476 728 729	0	1 155 643 113		
`trng::mrg3::LEcuyer3`	65 338	0	64 636		
`trng::mrg3s::trng0`	2 025 213 985	1 112 953 677	2 038 969 601		
`trng::mrg3s::trng1`	1 287 767 370	1 045 931 779	58 150 106		
`trng::mrg4::LEcuyer1`	2 001 982 722	1 412 284 257	1 155 380 217	1 668 339 922	
`trng::mrg4::LEcuyer2`	64 886	0	0	64 322	
`trng::mrg5::LEcuyer1`	107 374 182	0	0	0	104 480
`trng::mrg5s::trng0`	1 053 223 373	1 530 818 118	1 612 122 482	133 497 989	573 245 311
`trng::mrg5s::trng1`	2 068 619 238	2 138 332 912	671 754 166	1 442 240 992	1 526 958 817

```
    class parameter_type;
    class status_type;

    static const parameter_type LEcuyer1;
    static const parameter_type LEcuyer2;

    explicit mrg2(parameter_type = LEcuyer1);
    explicit mrg2(unsigned long, parameter_type = LEcuyer1);
    template<typename gen>
    explicit mrg2(gen &, parameter_type P = LEcuyer1);

    void seed();
    void seed(unsigned long);
    template<typename gen>
    void seed(gen &);
    void seed(result_type, result_type);

    void split(unsigned int, unsigned int);
    void jump2(unsigned int);
    void jump(unsigned long long);
    void discard(unsigned long long);

    static const char * name();
    long operator()(long);
  };

  bool operator==(const mrg2 &, const mrg2 &);
  bool operator!=(const mrg2 &, const mrg2 &);
  template<typename char_t, typename traits_t>
  std::basic_ostream<char_t, traits_t> &
  operator<<(std::basic_ostream<char_t, traits_t> &, const mrg2 &);
  template<typename char_t, typename traits_t>
  std::basic_istream<char_t, traits_t> &
  operator>>(std::basic_istream<char_t, traits_t> &, mrg2 &);
}
```

```
namespace trng {

  class mrg3 {
  public:
    using result_type = int32_t;
    result_type operator()();
    static constexpr result_type min();
    static constexpr result_type max();

    class parameter_type;
    class status_type;

    static const parameter_type LEcuyer1;
    static const parameter_type LEcuyer2;
    static const parameter_type LEcuyer3;

    explicit mrg3(parameter_type = LEcuyer1);
```

```
    explicit mrg3(unsigned long, parameter_type = LEcuyer1);
    template<typename gen>
    explicit mrg3(gen &, parameter_type P = LEcuyer1);

    void seed();
    void seed(unsigned long);
    template<typename gen>
    void seed(gen &);
    void seed(result_type, result_type, result_type);

    void split(unsigned int, unsigned int);
    void jump2(unsigned int);
    void jump(unsigned long long);
    void discard(unsigned long long);

    static const char * name();
    long operator()(long);
  };

  bool operator==(const mrg3 &, const mrg3 &);
  bool operator!=(const mrg3 &, const mrg3 &);
  template<typename char_t, typename traits_t>
  std::basic_ostream<char_t, traits_t> &
  operator<<(std::basic_ostream<char_t, traits_t> &, const mrg3 &);
  template<typename char_t, typename traits_t>
  std::basic_istream<char_t, traits_t> &
  operator>>(std::basic_istream<char_t, traits_t> &, mrg3 &);
}
```

```
namespace trng {

  class mrg3s {
  public:
    using result_type = int32_t;
    result_type operator()();
    static constexpr result_type min();
    static constexpr result_type max();

    class parameter_type;
    class status_type;

    static const parameter_type trng0;
    static const parameter_type trng1;

    explicit mrg3s(parameter_type=trng0);
    explicit mrg3s(unsigned long, parameter_type=trng0);
    template<typename gen>
    explicit mrg3s(gen &, parameter_type P=trng0);

    void seed();
    void seed(unsigned long);
    template<typename gen>
    void seed(gen &);
    void seed(result_type, result_type, result_type);
```

```
    void split(unsigned int, unsigned int);
    void jump2(unsigned int);
    void jump(unsigned long long);
    void discard(unsigned long long);

    static const char * name();
    long operator()(long);
  };

  bool operator==(const mrg3s &, const mrg3s &);
  bool operator!=(const mrg3s &, const mrg3s &);
  template<typename char_t, typename traits_t>
  std::basic_ostream<char_t, traits_t> &
  operator<<(std::basic_ostream<char_t, traits_t> &, const mrg3s &);
  template<typename char_t, typename traits_t>
  std::basic_istream<char_t, traits_t> &
  operator>>(std::basic_istream<char_t, traits_t> &, mrg3s &);
}
```

```
namespace trng {

  class mrg4 {
  public:
    using result_type = int32_t;
    result_type operator()();
    static constexpr result_type min();
    static constexpr result_type max();

    class parameter_type;
    class status_type;

    static const parameter_type LEcuyer1;
    static const parameter_type LEcuyer2;

    explicit mrg4(parameter_type = LEcuyer1);
    explicit mrg4(unsigned long, parameter_type = LEcuyer1);
    template<typename gen>
    explicit mrg4(gen &, parameter_type P = LEcuyer1);

    void seed();
    void seed(unsigned long);
    template<typename gen>
    void seed(gen &);
    void seed(result_type, result_type, result_type, result_type);

    void split(unsigned int, unsigned int);
    void jump2(unsigned int);
    void jump(unsigned long long);
    void discard(unsigned long long);

    static const char * name();
    long operator()(long);
  };
```

```
  bool operator==(const mrg4 &, const mrg4 &);
  bool operator!=(const mrg4 &, const mrg4 &);
  template<typename char_t, typename traits_t>
  std::basic_ostream<char_t, traits_t> &
  operator<<(std::basic_ostream<char_t, traits_t> &, const mrg4 &);
  template<typename char_t, typename traits_t>
  std::basic_istream<char_t, traits_t> &
  operator>>(std::basic_istream<char_t, traits_t> &, mrg4 &);
}
```

```
namespace trng {

  class mrg5 {
  public:
    using result_type = int32_t;
    result_type operator()();
    static constexpr result_type min();
    static constexpr result_type max();

    class parameter_type;
    class status_type;

    static const parameter_type LEcuyer1;

    explicit mrg5(parameter_type = LEcuyer1);
    explicit mrg5(unsigned long, parameter_type = LEcuyer1);
    template<typename gen>
    explicit mrg5(gen &, parameter_type P = LEcuyer1);

    void seed();
    void seed(unsigned long);
    template<typename gen>
    void seed(gen &);
    void seed(result_type, result_type, result_type, result_type, result_type);

    void split(unsigned int, unsigned int);
    void jump2(unsigned int);
    void jump(unsigned long long);
    void discard(unsigned long long);

    static const char * name();
    long operator()(long);
  };

  bool operator==(const mrg5 &, const mrg5 &);
  bool operator!=(const mrg5 &, const mrg5 &);
  template<typename char_t, typename traits_t>
  std::basic_ostream<char_t, traits_t> &
  operator<<(std::basic_ostream<char_t, traits_t> &, const mrg5 &);
  template<typename char_t, typename traits_t>
  std::basic_istream<char_t, traits_t> &
  operator>>(std::basic_istream<char_t, traits_t> &, mrg5 &);
}
```

```
namespace trng {

  class mrg5s {
  public:
    using result_type = int32_t;
    result_type operator()();
    static constexpr result_type min();
    static constexpr result_type max();

    class parameter_type;
    class status_type;

    static const parameter_type trng0;
    static const parameter_type trng1;

    explicit mrg5s(parameter_type=trng0);
    explicit mrg5s(unsigned long, parameter_type=trng0);
    template<typename gen>
    explicit mrg5s(gen &, parameter_type P=trng0);

    void seed();
    void seed(unsigned long);
    template<typename gen>
    void seed(gen &);
    void seed(result_type, result_type, result_type, result_type, result_type);

    void split(unsigned int, unsigned int);
    void jump2(unsigned int);
    void jump(unsigned long long);
    void discard(unsigned long long);

    static const char * name();
    long operator()(long);
  };

  bool operator==(const mrg5s &, const mrg5s &);
  bool operator!=(const mrg5s &, const mrg5s &);
  template<typename char_t, typename traits_t>
  std::basic_ostream<char_t, traits_t> &
  operator<<(std::basic_ostream<char_t, traits_t> &, const mrg5s &);
  template<typename char_t, typename traits_t>
  std::basic_istream<char_t, traits_t> &
  operator>>(std::basic_istream<char_t, traits_t> &, mrg5s &);
}
```

4.1.4 YARN generators

The classes `trng::yarn`*n* and `trng::yarn`*n*s implement so-called YARN generators (yet another random number generator). Table 4.4 summarizes the key features of these classes. Each of them is based on a multiple recursive generator with *n* feedback taps,

for which the transition algorithm reads

$$r_i = a_1 \cdot r_{i-1} + a_2 \cdot r_{i-2} + \ldots + a_n \cdot r_{i-n} \bmod m\,.$$

The state of this generator at time i is given by $(r_{i-1}, r_{i-2}, \ldots, r_{i-n})$. See section 2.4.2 for details on LFSR sequences.

The prime modulus m that characterizes the prime field $\mathbb{F}_m$ was either chosen as the Mersenne Prime (classes `trng::mrg`n) or a Sophie-Germain Prime such that $m^n - 1$ has as few prime factors as possible (classes `trng::mrg`n`s`). The former choice gives us some performance benefits, see section 7.1, whereas the second has some theoretical advantages, see section 2.4.2.

While pure multiple recursive generators return the r_i as pseudo-random numbers directly, a YARN generator "shuffles" the output of the underlying multiple recursive generator by a bijective mapping. In the case of a YARN generator with modulus m this mapping reads

$$q_i = \begin{cases} b^{r_i} \bmod m & \text{if } r_i > 0 \\ 0 & \text{if } r_i = 0 \end{cases},$$

where b is a generating element of the multiplicative group modulo m. This bijective mapping destroys the linear structures of the linear feedback shift register sequence. But on the other hand the new sequence q_i inherits all the nice features of the linear feedback shift register sequence r_i, e. g. its period. Block splitting and leapfrog methods can be implemented as easily as for multiple recursive generators, see section 2.4.2 and 2.5 for details.

The classes `trng::yarn`n and `trng::yarn`n`s` implement the interface described in section 3.1. Each class defines some parameter and status classes that will be used internally and by the constructor. Furthermore, for each generator several parameter sets are given, see Table 4.3. Most of the parameter sets are taken from [39] and chosen to give generators with good statistical properties.

An instance of a class `trng::yarn`n or `trng::yarn`n`s` can be instantiated by various constructors as specified for a random number engine. Additionally, a non-default parameter set may be chosen. The classes `trng::yarn`n and `trng::yarn`n`s` provide all necessary seeding functions (see Table 3.1) and additionally a function that sets the internal state $(r_{i-1}, r_{i-2}, \ldots, r_{i-n})$. This function should never be called with all arguments set to zero. The classes `trng::yarn`n and `trng::yarn`n`s` model the concept of a parallel random number engine and therefore the methods

```
void split(unsigned int, unsigned int);
void jump2(unsigned int);
void jump(unsigned long long);
void discard(unsigned long long);
```

are implemented. Furthermore, the classes `trng::yarn`n or `trng::yarn`n`s` provide a function that returns a string with its name and an operator `operator()`. Random number engines are comparable and can be written to or read from a stream.

The detailed interface of the classes `trng::mrg`n or `trng::mrg`n`s` is given as follows:

Table 4.4: Key features of YARN generator classes.

class	header file	feedback taps n	prime field $\mathbb{F}_m$	period	return value of `name()`
`trng::yarn2`	`trng/yarn2.hpp`	2	$\mathbb{F}_{2^{31}-1}$	$m^2-1 \approx 2^{62} \approx 4.61 \cdot 10^{18}$	`yarn2`
`trng::yarn3`	`trng/yarn3.hpp`	3	$\mathbb{F}_{2^{31}-1}$	$m^3-1 \approx 2^{93} \approx 9.90 \cdot 10^{27}$	`yarn3`
`trng::yarn3s`	`trng/yarn3s.hpp`	3	$\mathbb{F}_{2^{31}-21\,069}$	$m^3-1 \approx 2^{93} \approx 9.90 \cdot 10^{27}$	`yarn3s`
`trng::yarn4`	`trng/yarn4.hpp`	4	$\mathbb{F}_{2^{31}-1}$	$m^4-1 \approx 2^{124} \approx 2.13 \cdot 10^{37}$	`yarn4`
`trng::yarn5`	`trng/yarn5.hpp`	5	$\mathbb{F}_{2^{31}-1}$	$m^5-1 \approx 2^{155} \approx 4.57 \cdot 10^{46}$	`yarn5`
`trng::yarn5s`	`trng/yarn5s.hpp`	5	$\mathbb{F}_{2^{31}-22\,641}$	$m^5-1 \approx 2^{155} \approx 4.57 \cdot 10^{46}$	`yarn5s`

Table 4.5: Parameter sets for YARN generators.

parameter set	a_1	a_2	a_3	a_4	a_5	b
`trng::yarn2::LEcuyer1`	1 498 809 829	1 160 990 996				123 567 893
`trng::yarn2::LEcuyer2`	46 325	1 084 587				123 567 893
`trng::yarn3::LEcuyer1`	2 021 422 057	1 826 992 351	1 977 753 457			123 567 893
`trng::yarn3::LEcuyer2`	1 476 728 729	0	1 155 643 113			123 567 893
`trng::yarn3::LEcuyer3`	65 338	0	64 636			123 567 893
`trng::yarn3s::trng0`	2 025 213 985	1 112 953 677	2 038 969 601			1 616 076 847
`trng::yarn3s::trng1`	1 287 767 370	1 045 931 779	58 150 106			1 616 076 847
`trng::yarn4::LEcuyer1`	2 001 982 722	1 412 284 257	1 155 380 217	1 668 339 922		123 567 893
`trng::yarn4::LEcuyer2`	64 886	0	0	64 322		123 567 893
`trng::yarn5::LEcuyer1`	107 374 182	0	0	0	104 480	123 567 893
`trng::yarn5s::trng0`	1 053 223 373	1 530 818 118	1 612 122 482	133 497 989	573 245 311	889 744 251
`trng::yarn5s::trng1`	2 068 619 238	2 138 332 912	671 754 166	1 442 240 992	1 526 958 817	889 744 251

```
namespace trng {

  class yarn2 {
  public:
    using result_type = int32_t;
    result_type operator()();
    static constexpr result_type min();
    static constexpr result_type max();

    class parameter_type;
    class status_type;

    static const parameter_type LEcuyer1;
    static const parameter_type LEcuyer2;

    explicit yarn2(parameter_type = LEcuyer1);
    explicit yarn2(unsigned long, parameter_type = LEcuyer1);
    template<typename gen>
    explicit yarn2(gen &, parameter_type P = LEcuyer1);

    void seed();
    void seed(unsigned long);
    template<typename gen>
    void seed(gen &);
    void seed(result_type, result_type);

    void split(unsigned int, unsigned int);
    void jump2(unsigned int);
    void jump(unsigned long long);
    void discard(unsigned long long);

    static const char * name();
    long operator()(long);
  };

  bool operator==(const yarn2 &, const yarn2 &);
  bool operator!=(const yarn2 &, const yarn2 &);
  template<typename char_t, typename traits_t>
  std::basic_ostream<char_t, traits_t> &
  operator<<(std::basic_ostream<char_t, traits_t> &t, const yarn2 &);
  template<typename char_t, typename traits_t>
  std::basic_istream<char_t, traits_t> &
  operator>>(std::basic_istream<char_t, traits_t> &, yarn2 &);
}
```

```
namespace trng {

  class yarn3 {
  public:
    using result_type = int32_t;
    result_type operator()();
    static constexpr result_type min();
    static constexpr result_type max();
```

```
    class parameter_type;
    class status_type;

    static const parameter_type LEcuyer1;
    static const parameter_type LEcuyer2;

    static const parameter_type LEcuyer3;

    explicit yarn3(parameter_type = LEcuyer1);
    explicit yarn3(unsigned long, parameter_type = LEcuyer1);
    template<typename gen>
    explicit yarn3(gen &, parameter_type P = LEcuyer1);

    void seed();
    void seed(unsigned long);
    template<typename gen>
    void seed(gen &);
    void seed(result_type, result_type, result_type);

    void split(unsigned int, unsigned int);
    void jump2(unsigned int);
    void jump(unsigned long long);
    void discard(unsigned long long);

    static const char * name();
    long operator()(long);
  };

  bool operator==(const yarn3 &, const yarn3 &);
  bool operator!=(const yarn3 &, const yarn3 &);
  template<typename char_t, typename traits_t>
  std::basic_ostream<char_t, traits_t> &
  operator<<(std::basic_ostream<char_t, traits_t> &, const yarn3 &);
  template<typename char_t, typename traits_t>
  std::basic_istream<char_t, traits_t> &
  operator>>(std::basic_istream<char_t, traits_t> &, yarn3 &);
}
```

```
namespace trng {

  class yarn3s {
  public:
    using result_type = int32_t;
    result_type operator()();
    static constexpr result_type min();
    static constexpr result_type max();

    class parameter_type;
    class status_type;

    static const parameter_type trng0;
    static const parameter_type trng1;
```

```
    explicit yarn3s(parameter_type=trng0);
    explicit yarn3s(unsigned long, parameter_type=trng0);
    template<typename gen>
    explicit yarn3s(gen &, parameter_type P=trng0);

    void seed();
    void seed(unsigned long);
    template<typename gen>
    void seed(gen &);
    void seed(result_type, result_type, result_type);

    void split(unsigned int, unsigned int);
    void jump2(unsigned int);
    void jump(unsigned long long);
    void discard(unsigned long long);

    static const char * name();
    long operator()(long);
  };

  bool operator==(const yarn3s &, const yarn3s &);
  bool operator!=(const yarn3s &, const yarn3s &);
  template<typename char_t, typename traits_t>
  std::basic_ostream<char_t, traits_t> &
  operator<<(std::basic_ostream<char_t, traits_t> &, const yarn3s &);
  template<typename char_t, typename traits_t>
  std::basic_istream<char_t, traits_t> &
  operator>>(std::basic_istream<char_t, traits_t> &, yarn3s &);
}
```

```
namespace trng {

  class yarn4 {
  public:
    using result_type = int32_t;
    result_type operator()();
    static constexpr result_type min();
    static constexpr result_type max();

    class parameter_type;
    class status_type;

    static const parameter_type LEcuyer1;
    static const parameter_type LEcuyer2;

    explicit yarn4(parameter_type = LEcuyer1);
    explicit yarn4(unsigned long, parameter_type = LEcuyer1);
    template<typename gen>
    explicit yarn4(gen &, parameter_type P = LEcuyer1);

    void seed();
    void seed(unsigned long);
    template<typename gen>
    void seed(gen &);
```

```
    void seed(result_type, result_type, result_type, result_type);

    void split(unsigned int, unsigned int);
    void jump2(unsigned int);
    void jump(unsigned long long);
    void discard(unsigned long long);

    static const char * name();
    long operator()(long);
  };

  bool operator==(const yarn4 &, const yarn4 &);
  bool operator!=(const yarn4 &, const yarn4 &);
  template<typename char_t, typename traits_t>
  std::basic_ostream<char_t, traits_t> &
  operator<<(std::basic_ostream<char_t, traits_t> &, const yarn4 &);
  template<typename char_t, typename traits_t>
  std::basic_istream<char_t, traits_t> &
  operator>>(std::basic_istream<char_t, traits_t> &, yarn4 &);
}
```

```
namespace trng {

  class yarn5 {
  public:
    using result_type = int32_t;
    result_type operator()();
    static constexpr result_type min();
    static constexpr result_type max();

    class parameter_type;
    class status_type;

    static const parameter_type LEcuyer1;

    explicit yarn5(parameter_type = LEcuyer1);
    explicit yarn5(unsigned long, parameter_type = LEcuyer1);
    template<typename gen>
    explicit yarn5(gen &, parameter_type P = LEcuyer1);

    void seed();
    void seed(unsigned long);
    template<typename gen>
    void seed(gen &);
    void seed(result_type, result_type, result_type, result_type, result_type);

    void split(unsigned int, unsigned int);
    void jump2(unsigned int);
    void jump(unsigned long long);
    void discard(unsigned long long);

    static const char * name();
    long operator()(long);
  };
```

```
  bool operator==(const yarn5 &, const yarn5 &);
  bool operator!=(const yarn5 &, const yarn5 &);
  template<typename char_t, typename traits_t>
  std::basic_ostream<char_t, traits_t> &
  operator<<(std::basic_ostream<char_t, traits_t> &, const yarn5 &);
  template<typename char_t, typename traits_t>
  std::basic_istream<char_t, traits_t> &
  operator>>(std::basic_istream<char_t, traits_t> &, yarn5 &);
}
```

```
namespace trng {

  class yarn5s {
  public:
    using result_type = int32_t;
    result_type operator()();
    static constexpr result_type min();
    static constexpr result_type max();

    class parameter_type;
    class status_type;

    static const parameter_type trng0;
    static const parameter_type trng1;

    explicit yarn5s(parameter_type=trng0);
    explicit yarn5s(unsigned long, parameter_type=trng0);
    template<typename gen>
    explicit yarn5s(gen &, parameter_type P=trng0);

    void seed();
    void seed(unsigned long);
    template<typename gen>
    void seed(gen &);
    void seed(result_type, result_type, result_type, result_type, result_type);

    void split(unsigned int, unsigned int);
    void jump2(unsigned int);
    void jump(unsigned long long);
    void discard(unsigned long long);

    static const char * name();
    long operator()(long);
  };

  bool operator==(const yarn5s &, const yarn5s &);
  bool operator!=(const yarn5s &, const yarn5s &);
  template<typename char_t, typename traits_t>
  std::basic_ostream<char_t, traits_t> &
  operator<<(std::basic_ostream<char_t, traits_t> &, const yarn5s &);
  template<typename char_t, typename traits_t>
  std::basic_istream<char_t, traits_t> &
  operator>>(std::basic_istream<char_t, traits_t> &, yarn5s &);
```

```
}
```

4.1.5 Lagged Fibonacci generators

The template classes `trng::lagfib2xor`, `trng::lagfib4xor`, `trng::lagfib2plus`, `trng::lagfib4plus` model random number engines (no splitting facilities) and implement lagged Fibonacci generators with two or four feedback taps and exclusive-or or additive operation. The recursion relation of these types of generators read

$$
\begin{aligned}
r_i &= r_{i-A} \oplus r_{i-B} \\
r_i &= r_{i-A} \oplus r_{i-B} \oplus r_{i-C} \oplus r_{i-D} \\
r_i &= r_{i-A} + r_{i-B} \bmod 2^l \\
r_i &= r_{i-A} + r_{i-B} + r_{i-C} + r_{i-D} \bmod 2^l .
\end{aligned}
$$

These template classes are parameterized by an unsigned integer type, e. g. `unsigned int` or `unsigned long long`, and the position of the feedback taps with $A < B < C < D$. For properly chosen feedback taps the period of an exclusive-or generator is $2^B - 1$ or $2^D - 1$ respectively, and the period of an plus generator is $(2^B - 1)2^{l-1}$ or $(2^D - 1)2^{l-1}$ respectively, where l denotes the number of significant bits of the integer type given as a template argument. Template classes are declared in the header files `trng/lagfib2xor.hpp`, `trng/lagfib4xor.hpp`, `trng/lagfib2plus.hpp`, and `trng/lagfib4plus.hpp`. For convenience TRNG provides some typedefs for some realizations of lagged Fibonacci generators with two or four feedback taps.

The detailed interfaces of the classes `trng::lagfib2xor`, `trng::lagfib4xor`, `trng::lagfib2plus`, `trng::lagfib4plus` are given as follows:

```
namespace trng {

  template<typename integer_type,
  unsigned int A, unsigned int B>
  class lagfib2xor {
  public:
    using result_type = integer_type;
    result_type operator()();
    static constexpr result_type min();
    static constexpr result_type max();

    class status_type;

    lagfib2xor();
    explicit lagfib2xor(unsigned long);
    template<typename gen>
    explicit lagfib2xor(gen &);

    void seed();
    void seed(unsigned long);
    template<typename gen>
    void seed(gen &);
```

```
    void discard(unsigned long long);
  };

  typedef lagfib2xor<unsigned long,       103,   250> r250_ul;
  typedef lagfib2xor<unsigned long long,  103,   250> r250_ull;
  typedef lagfib2xor<unsigned long,       168,   521> lagfib2xor_521_ul;
  typedef lagfib2xor<unsigned long long,  168,   521> lagfib2xor_521_ull;
  typedef lagfib2xor<unsigned long,       273,   607> lagfib2xor_607_ul;
  typedef lagfib2xor<unsigned long long,  273,   607> lagfib2xor_607_ull;
  typedef lagfib2xor<unsigned long,       418,  1279> lagfib2xor_1279_ul;
  typedef lagfib2xor<unsigned long long,  418,  1279> lagfib2xor_1279_ull;
  typedef lagfib2xor<unsigned long,      1029,  2281> lagfib2xor_2281_ul;
  typedef lagfib2xor<unsigned long long, 1029,  2281> lagfib2xor_2281_ull;
  typedef lagfib2xor<unsigned long,       576,  3217> lagfib2xor_3217_ul;
  typedef lagfib2xor<unsigned long long,  576,  3217> lagfib2xor_3217_ull;
  typedef lagfib2xor<unsigned long,      2098,  4423> lagfib2xor_4423_ul;
  typedef lagfib2xor<unsigned long long, 2098,  4423> lagfib2xor_4423_ull;
  typedef lagfib2xor<unsigned long,      4187,  9689> lagfib2xor_9689_ul;
  typedef lagfib2xor<unsigned long long, 4187,  9689> lagfib2xor_9689_ull;
  typedef lagfib2xor<unsigned long,      9842, 19937> lagfib2xor_19937_ul;
  typedef lagfib2xor<unsigned long long, 9842, 19937> lagfib2xor_19937_ull;

  typedef lagfib2xor<uint32_t,  103,   250> r250_32;
  typedef lagfib2xor<uint64_t,  103,   250> r250_64;
  typedef lagfib2xor<uint32_t,  168,   521> lagfib2xor_521_32;
  typedef lagfib2xor<uint64_t,  168,   521> lagfib2xor_521_64;
  typedef lagfib2xor<uint32_t,  273,   607> lagfib2xor_607_32;
  typedef lagfib2xor<uint64_t,  273,   607> lagfib2xor_607_64;
  typedef lagfib2xor<uint32_t,  418,  1279> lagfib2xor_1279_32;
  typedef lagfib2xor<uint64_t,  418,  1279> lagfib2xor_1279_64;
  typedef lagfib2xor<uint32_t, 1029,  2281> lagfib2xor_2281_32;
  typedef lagfib2xor<uint64_t, 1029,  2281> lagfib2xor_2281_64;
  typedef lagfib2xor<uint32_t,  576,  3217> lagfib2xor_3217_32;
  typedef lagfib2xor<uint64_t,  576,  3217> lagfib2xor_3217_64;
  typedef lagfib2xor<uint32_t, 2098,  4423> lagfib2xor_4423_32;
  typedef lagfib2xor<uint64_t, 2098,  4423> lagfib2xor_4423_64;
  typedef lagfib2xor<uint32_t, 4187,  9689> lagfib2xor_9689_32;
  typedef lagfib2xor<uint64_t, 4187,  9689> lagfib2xor_9689_64;
  typedef lagfib2xor<uint32_t, 9842, 19937> lagfib2xor_19937_32;
  typedef lagfib2xor<uint64_t, 9842, 19937> lagfib2xor_19937_64;

}
```

```
namespace trng {

  template<typename integer_type,
  unsigned int A, unsigned int B, unsigned int C, unsigned int D>
  class lagfib4xor {
  public:
    using result_type = integer_type;
    result_type operator()();
    static constexpr result_type min();
    static constexpr result_type max();
```

```
    class status_type;

    lagfib4xor();
    explicit lagfib4xor(unsigned long);
    template<typename gen>
    explicit lagfib4xor(gen &);

    void seed();
    void seed(unsigned long);
    template<typename gen>
    void seed(gen &);
    void discard(unsigned long long);
  };

  typedef lagfib4xor<unsigned long,       471, 1586,  6988,  9689> Ziff_ul;
  typedef lagfib4xor<unsigned long long,  471, 1586,  6988,  9689> Ziff_ull;
  typedef lagfib4xor<unsigned long,       168,  205,   242,   521> lagfib4xor_521_ul;
  typedef lagfib4xor<unsigned long long,  168,  205,   242,   521> lagfib4xor_521_ull;
  typedef lagfib4xor<unsigned long,       147,  239,   515,   607> lagfib4xor_607_ul;
  typedef lagfib4xor<unsigned long long,  147,  239,   515,   607> lagfib4xor_607_ull;
  typedef lagfib4xor<unsigned long,       418,  705,   992,  1279> lagfib4xor_1279_ul;
  typedef lagfib4xor<unsigned long long,  418,  705,   992,  1279> lagfib4xor_1279_ull;
  typedef lagfib4xor<unsigned long,       305,  610,   915,  2281> lagfib4xor_2281_ul;
  typedef lagfib4xor<unsigned long long,  305,  610,   915,  2281> lagfib4xor_2281_ull;
  typedef lagfib4xor<unsigned long,       576,  871,  1461,  3217> lagfib4xor_3217_ul;
  typedef lagfib4xor<unsigned long long,  576,  871,  1461,  3217> lagfib4xor_3217_ull;
  typedef lagfib4xor<unsigned long,      1419, 1736,  2053,  4423> lagfib4xor_4423_ul;
  typedef lagfib4xor<unsigned long long, 1419, 1736,  2053,  4423> lagfib4xor_4423_ull;
  typedef lagfib4xor<unsigned long,       471, 2032,  4064,  9689> lagfib4xor_9689_ul;
  typedef lagfib4xor<unsigned long long,  471, 2032,  4064,  9689> lagfib4xor_9689_ull;
  typedef lagfib4xor<unsigned long,      3860, 7083, 11580, 19937> lagfib4xor_19937_ul;
  typedef lagfib4xor<unsigned long long, 3860, 7083, 11580, 19937> lagfib4xor_19937_ull;

  typedef lagfib4xor<uint32_t,  471, 1586,  6988,  9689> Ziff_32;
  typedef lagfib4xor<uint64_t,  471, 1586,  6988,  9689> Ziff_64;
  typedef lagfib4xor<uint32_t,  168,  205,   242,   521> lagfib4xor_521_32;
  typedef lagfib4xor<uint64_t,  168,  205,   242,   521> lagfib4xor_521_64;
  typedef lagfib4xor<uint32_t,  147,  239,   515,   607> lagfib4xor_607_32;
  typedef lagfib4xor<uint64_t,  147,  239,   515,   607> lagfib4xor_607_64;
  typedef lagfib4xor<uint32_t,  418,  705,   992,  1279> lagfib4xor_1279_32;
  typedef lagfib4xor<uint64_t,  418,  705,   992,  1279> lagfib4xor_1279_64;
  typedef lagfib4xor<uint32_t,  305,  610,   915,  2281> lagfib4xor_2281_32;
  typedef lagfib4xor<uint64_t,  305,  610,   915,  2281> lagfib4xor_2281_64;
  typedef lagfib4xor<uint32_t,  576,  871,  1461,  3217> lagfib4xor_3217_32;
  typedef lagfib4xor<uint64_t,  576,  871,  1461,  3217> lagfib4xor_3217_64;
  typedef lagfib4xor<uint32_t, 1419, 1736,  2053,  4423> lagfib4xor_4423_32;
  typedef lagfib4xor<uint64_t, 1419, 1736,  2053,  4423> lagfib4xor_4423_64;
  typedef lagfib4xor<uint32_t,  471, 2032,  4064,  9689> lagfib4xor_9689_32;
  typedef lagfib4xor<uint64_t,  471, 2032,  4064,  9689> lagfib4xor_9689_64;
  typedef lagfib4xor<uint32_t, 3860, 7083, 11580, 19937> lagfib4xor_19937_32;
  typedef lagfib4xor<uint64_t, 3860, 7083, 11580, 19937> lagfib4xor_19937_64;

}
```

```
namespace trng {

  template<typename integer_type,
  unsigned int A, unsigned int B>
  class lagfib2plus {
  public:
    using result_type = integer_type;
    result_type operator()();
    static constexpr result_type min();
    static constexpr result_type max();

    class status_type;

    lagfib2plus();
    explicit lagfib2plus(unsigned long);
    template<typename gen>
    explicit lagfib2plus(gen &);

    void seed();
    void seed(unsigned long);
    template<typename gen>
    void seed(gen &);
    void discard(unsigned long long);
  };

  typedef lagfib2plus<unsigned long,       168,   521> lagfib2plus_521_ul;
  typedef lagfib2plus<unsigned long long,  168,   521> lagfib2plus_521_ull;
  typedef lagfib2plus<unsigned long,       273,   607> lagfib2plus_607_ul;
  typedef lagfib2plus<unsigned long long,  273,   607> lagfib2plus_607_ull;
  typedef lagfib2plus<unsigned long,       418,  1279> lagfib2plus_1279_ul;
  typedef lagfib2plus<unsigned long long,  418,  1279> lagfib2plus_1279_ull;
  typedef lagfib2plus<unsigned long,      1029,  2281> lagfib2plus_2281_ul;
  typedef lagfib2plus<unsigned long long, 1029,  2281> lagfib2plus_2281_ull;
  typedef lagfib2plus<unsigned long,       576,  3217> lagfib2plus_3217_ul;
  typedef lagfib2plus<unsigned long long,  576,  3217> lagfib2plus_3217_ull;
  typedef lagfib2plus<unsigned long,      2098,  4423> lagfib2plus_4423_ul;
  typedef lagfib2plus<unsigned long long, 2098,  4423> lagfib2plus_4423_ull;
  typedef lagfib2plus<unsigned long,      4187,  9689> lagfib2plus_9689_ul;
  typedef lagfib2plus<unsigned long long, 4187,  9689> lagfib2plus_9689_ull;
  typedef lagfib2plus<unsigned long,      9842, 19937> lagfib2plus_19937_ul;
  typedef lagfib2plus<unsigned long long, 9842, 19937> lagfib2plus_19937_ull;

  typedef lagfib2plus<uint32_t,  168,   521> lagfib2plus_521_32;
  typedef lagfib2plus<uint64_t,  168,   521> lagfib2plus_521_64;
  typedef lagfib2plus<uint32_t,  273,   607> lagfib2plus_607_32;
  typedef lagfib2plus<uint64_t,  273,   607> lagfib2plus_607_64;
  typedef lagfib2plus<uint32_t,  418,  1279> lagfib2plus_1279_32;
  typedef lagfib2plus<uint64_t,  418,  1279> lagfib2plus_1279_64;
  typedef lagfib2plus<uint32_t, 1029,  2281> lagfib2plus_2281_32;
  typedef lagfib2plus<uint64_t, 1029,  2281> lagfib2plus_2281_64;
  typedef lagfib2plus<uint32_t,  576,  3217> lagfib2plus_3217_32;
  typedef lagfib2plus<uint64_t,  576,  3217> lagfib2plus_3217_64;
  typedef lagfib2plus<uint32_t, 2098,  4423> lagfib2plus_4423_32;
```

```
  typedef lagfib2plus<uint64_t, 2098,  4423> lagfib2plus_4423_64;
  typedef lagfib2plus<uint32_t, 4187,  9689> lagfib2plus_9689_32;
  typedef lagfib2plus<uint64_t, 4187,  9689> lagfib2plus_9689_64;
  typedef lagfib2plus<uint32_t, 9842, 19937> lagfib2plus_19937_32;
  typedef lagfib2plus<uint64_t, 9842, 19937> lagfib2plus_19937_64;

}
```

```
namespace trng {

  template<typename integer_type,
  unsigned int A, unsigned int B, unsigned int C, unsigned int D>
  class lagfib4plus {
  public:
    using result_type = integer_type;
    result_type operator()();
    static constexpr result_type min();
    static constexpr result_type max();

    class status_type;

    lagfib4plus();
    explicit lagfib2plus(unsigned long);
    template<typename gen>
    explicit lagfib4plus(gen &);

    void seed();
    void seed(unsigned long);
    template<typename gen>
    void seed(gen &);
    void discard(unsigned long long);
  };

  typedef lagfib4plus<unsigned long,       168,  205,   242,   521> lagfib4plus_521_ul;
  typedef lagfib4plus<unsigned long long,  168,  205,   242,   521> lagfib4plus_521_ull;
  typedef lagfib4plus<unsigned long,       147,  239,   515,   607> lagfib4plus_607_ul;
  typedef lagfib4plus<unsigned long long,  147,  239,   515,   607> lagfib4plus_607_ull;
  typedef lagfib4plus<unsigned long,       418,  705,   992,  1279> lagfib4plus_1279_ul;
  typedef lagfib4plus<unsigned long long,  418,  705,   992,  1279> lagfib4plus_1279_ull;
  typedef lagfib4plus<unsigned long,       305,  610,   915,  2281> lagfib4plus_2281_ul;
  typedef lagfib4plus<unsigned long long,  305,  610,   915,  2281> lagfib4plus_2281_ull;
  typedef lagfib4plus<unsigned long,       576,  871,  1461,  3217> lagfib4plus_3217_ul;
  typedef lagfib4plus<unsigned long long,  576,  871,  1461,  3217> lagfib4plus_3217_ull;
  typedef lagfib4plus<unsigned long,      1419, 1736,  2053,  4423> lagfib4plus_4423_ul;
  typedef lagfib4plus<unsigned long long, 1419, 1736,  2053,  4423> lagfib4plus_4423_ull;
  typedef lagfib4plus<unsigned long,       471, 2032,  4064,  9689> lagfib4plus_9689_ul;
  typedef lagfib4plus<unsigned long long,  471, 2032,  4064,  9689> lagfib4plus_9689_ull;
  typedef lagfib4plus<unsigned long,      3860, 7083, 11580, 19937> lagfib4plus_19937_ul;
  typedef lagfib4plus<unsigned long long, 3860, 7083, 11580, 19937> lagfib4plus_19937_ull;

  typedef lagfib4plus<uint32_t,  168,  205,   242,   521> lagfib4plus_521_32;
  typedef lagfib4plus<uint64_t,  168,  205,   242,   521> lagfib4plus_521_64;
  typedef lagfib4plus<uint32_t,  147,  239,   515,   607> lagfib4plus_607_32;
  typedef lagfib4plus<uint64_t,  147,  239,   515,   607> lagfib4plus_607_64;
```

```
  typedef lagfib4plus<uint32_t,  418,  705,   992,  1279> lagfib4plus_1279_32;
  typedef lagfib4plus<uint64_t,  418,  705,   992,  1279> lagfib4plus_1279_64;
  typedef lagfib4plus<uint32_t,  305,  610,   915,  2281> lagfib4plus_2281_32;
  typedef lagfib4plus<uint64_t,  305,  610,   915,  2281> lagfib4plus_2281_64;
  typedef lagfib4plus<uint32_t,  576,  871,  1461,  3217> lagfib4plus_3217_32;
  typedef lagfib4plus<uint64_t,  576,  871,  1461,  3217> lagfib4plus_3217_64;
  typedef lagfib4plus<uint32_t, 1419, 1736,  2053,  4423> lagfib4plus_4423_32;
  typedef lagfib4plus<uint64_t, 1419, 1736,  2053,  4423> lagfib4plus_4423_64;
  typedef lagfib4plus<uint32_t,  471, 2032,  4064,  9689> lagfib4plus_9689_32;
  typedef lagfib4plus<uint64_t,  471, 2032,  4064,  9689> lagfib4plus_9689_64;
  typedef lagfib4plus<uint32_t, 3860, 7083, 11580, 19937> lagfib4plus_19937_32;
  typedef lagfib4plus<uint64_t, 3860, 7083, 11580, 19937> lagfib4plus_19937_64;

}
```

4.1.6 Xoshiro type generator

The xoshiro (xor/shift/rotate) type generators [8] are based on matrix linear congruential generators in $\mathbb{F}_2$. The matrix of the recurrence equation of xoshiro type generators is sparse and has a special form that allows an efficient implementation that uses xor, bit-shift and bit-rotation operations only, for example

$$\boldsymbol{A} = \begin{pmatrix} \boldsymbol{I} & \boldsymbol{I} & \boldsymbol{I} & \boldsymbol{0} \\ \boldsymbol{I} & \boldsymbol{I} & \boldsymbol{S}^a & \boldsymbol{R}^b \\ \boldsymbol{0} & \boldsymbol{I} & \boldsymbol{I} & \boldsymbol{0} \\ \boldsymbol{I} & \boldsymbol{0} & \boldsymbol{0} & \boldsymbol{R}^b \end{pmatrix} . \tag{4.1}$$

Here $\boldsymbol{I}$ denotes a $w \times w$ identity matrix, $\boldsymbol{S}$ is a $w \times w$ shift matrix and $\boldsymbol{R}$ is a $w \times w$ rotation matrix and a and b denote two integer parameters.

The class `trng::xoshiro256plus` in the header file `trng/xoshiro256plus.hpp` implements an xoshiro type generator with $w = 64$, $a = 17$ and $b = 45$. This means the generator has a 256 bit state vector. Its period equals $2^{256} - 1$. To output a pseudo random number this 256 bit state vector is transformed into a 64 bit integer by adding the lowest 64 bits to the highest 64 bit modulo 2^{64}. The detailed interfaces of the class `trng::xoshiro256plus` is given as follows:

```
namespace trng {

  class xoshiro256plus {
  public:
    using result_type = uint64_t;
    result_type operator()();

    static constexpr result_type min();
    static constexpr result_type max();

    class status_type;

    explicit xoshiro256plus();
```

```
    explicit xoshiro256plus(unsigned long);
    explicit xoshiro256plus(result_type s0, result_type s1, result_type s2, result_type s3);
    template<typename gen>
    explicit xoshiro256plus(gen &g);

    void seed();
    void seed(unsigned long);
    template<typename gen>
    void seed(gen &g);
    void seed(result_type, result_type, result_type, result_type);

    void jump2(unsigned int);
    void jump(unsigned long long);
    void discard(unsigned long long);

    static const char *name();
    long operator()(long);
  };

  bool operator==(const xoshiro256plus &, const xoshiro256plus &);
  bool operator!=(const xoshiro256plus &, const xoshiro256plus &);

  template<typename char_t, typename traits_t>
  std::basic_ostream<char_t, traits_t> &
  operator<<(std::basic_ostream<char_t, traits_t> &, const xoshiro256plus &);
  template<typename char_t, typename traits_t>
  std::basic_istream<char_t, traits_t> &
  operator>>(std::basic_istream<char_t, traits_t> &, xoshiro256plus &);
}
```

Note that the class `trng::xoshiro256plus` supports block splitting but not leapfrogging.

4.1.7 Scrambled multiply-with-carry generators

The classes `trng::mwc64x32`, `trng::mwc128x32`, `trng::mwc128x`, and `trng::mwc192x` implement scrambled multiply-with-carry (MWC) generators described by [76]. All four engines share the same structure and differ only in the word size, the number of data words, and the multiplier.

Recurrence relation. Let $b = 2^w$ denote the word base, where $w = 32$ for `mwc64x32` and `mwc128x32`, and $w = 64$ for `mwc128x` and `mwc192x`. Let k be the number of data words and let A be the multiplier of the respective engine, see Table 4.6. The engine generates a sequence of data words $x_n \in \{0, \ldots, b-1\}$ and a sequence of carries $c_n \in \{0, \ldots, A-1\}$ by

$$t_n = A\, x_n + c_n\,, \tag{4.2}$$

$$x_{n+k} = t_n \bmod b\,, \tag{4.3}$$

$$c_{n+1} = \left\lfloor \frac{t_n}{b} \right\rfloor . \tag{4.4}$$

class	b	k	state size	multiplier A	output u_n
`mwc64x32`	2^{32}	1	64 bit	`0xfff145f8`	$x_n \oplus c_n$
`mwc128x32`	2^{32}	3	128 bit	`0xffcccc66`	$x_{n+2} \oplus c_n$
`mwc128x`	2^{64}	1	128 bit	`0xff6da236274e252e`	$x_n \oplus c_n$
`mwc192x`	2^{64}	2	192 bit	`0xff4e24dd418f5433`	$x_{n+1} \oplus c_n$

Table 4.6: Parameters of the scrambled multiply-with-carry generators. The state size includes the carry.

Equivalently, $x_{n+k} = A\,x_n + c_n - b\,c_{n+1}$. The state of the engine at time n consists of the k data words $x_n, x_{n+1}, \ldots, x_{n+k-1}$ and the carry c_n, i.e., of $k+1$ words. In a transition x_{n+k} and c_{n+1} are computed from x_n and c_n, the data words are shifted by one position, and the oldest data word x_n is discarded. Explicitly, for the four engines (state variables are named as in the constructor arguments and written as the tuple before and after one transition):

$$\begin{aligned}
\texttt{mwc64x32}, \texttt{mwc128x} \quad (k=1): &\qquad (x_n, c_n) \mapsto (x_{n+1}, c_{n+1})\,, \\
\texttt{mwc192x} \quad (k=2): &\qquad (x_n, x_{n+1}, c_n) \mapsto (x_{n+1}, x_{n+2}, c_{n+1})\,, \\
\texttt{mwc128x32} \quad (k=3): &\qquad (x_n, x_{n+1}, x_{n+2}, c_n) \mapsto (x_{n+1}, x_{n+2}, x_{n+3}, c_{n+1})\,,
\end{aligned}$$

with x_{n+k} and c_{n+1} given by Eqs. (4.3) and (4.4). For example, for `mwc192x` with state (x, y, c) one transition reads $t = Ax + c$, $(x, y, c) \leftarrow (y,\ t \bmod 2^{64},\ \lfloor t/2^{64} \rfloor)$.

Output function. The generator is scrambled: the value returned at time n is the bitwise exclusive or of the newest data word in the state and the carry,

$$u_n = x_{n+k-1} \oplus c_n\,, \tag{4.5}$$

before the state is advanced. In terms of the state variables this is $x \oplus c$ for `mwc64x32` and `mwc128x`, $y \oplus c$ for `mwc192x`, and $z \oplus c$ for `mwc128x32`.

Relation to a modular recurrence. Combine the state into the integer

$$s_n = \sum_{j=0}^{k-1} x_{n+j}\, b^j + c_n\, b^k\,, \qquad 0 \le s_n < m = A\, b^k - 1\,. \tag{4.6}$$

Then one transition is equivalent to

$$s_{n+1} \equiv b^{-1}\, s_n \pmod{m}\,, \tag{4.7}$$

and, consequently, $s_{n+N} \equiv b^{-N} s_n \pmod{m}$. This relation is used to implement the jump-ahead functions `jump2`, `jump`, and `discard` without iterating the generator. The state size of each engine corresponds to the number of bits of s_n that are stored, i.e., $w(k+1)$ bits.

The mwc64x32 engine produces 32-bit values from a 64-bit state, and mwc128x32 produces 32-bit values from a 128-bit state. The mwc128x engine produces 64-bit values from a 128-bit state, while mwc192x produces 64-bit values from a 192-bit state. In each case the stated state width includes the carry. The classes are declared in trng/mwc64x32.hpp, trng/mwc128x32.hpp, trng/mwc128x.hpp, and trng/mwc192x.hpp, respectively.

Each class can be default-constructed or seeded with an unsigned long, with another generator, or with its individual state words. The state is saved and restored using the engine's stream insertion and extraction operators. The principal public interfaces are:

```
namespace trng {

  class mwc64x32 {
  public:
    using result_type = uint32_t;
    result_type operator()();
    static constexpr result_type min();
    static constexpr result_type max();
    explicit mwc64x32();
    explicit mwc64x32(unsigned long);
    explicit mwc64x32(result_type x, result_type c);
    template<typename gen> explicit mwc64x32(gen &g);
    void seed();
    void seed(unsigned long);
    template<typename gen> void seed(gen &g);
    void seed(result_type x, result_type c);
    void jump2(unsigned int);
    void jump(unsigned long long);
    void discard(unsigned long long);
  };

  class mwc128x32 {
  public:
    using result_type = uint32_t;
    result_type operator()();
    static constexpr result_type min();
    static constexpr result_type max();
    explicit mwc128x32();
    explicit mwc128x32(unsigned long);
    explicit mwc128x32(result_type x, result_type y,
                       result_type z, result_type c);
    template<typename gen> explicit mwc128x32(gen &g);
    void seed();
    void seed(unsigned long);
    template<typename gen> void seed(gen &g);
    void seed(result_type x, result_type y, result_type z, result_type c);
    void jump2(unsigned int);
    void jump(unsigned long long);
    void discard(unsigned long long);
  };

  class mwc128x {
  public:
    using result_type = uint64_t;
```

```
    result_type operator()();
    static constexpr result_type min();
    static constexpr result_type max();
    explicit mwc128x();
    explicit mwc128x(unsigned long);
    explicit mwc128x(result_type x, result_type c);
    template<typename gen> explicit mwc128x(gen &g);
    void seed();
    void seed(unsigned long);
    template<typename gen> void seed(gen &g);
    void seed(result_type x, result_type c);
    void jump2(unsigned int);
    void jump(unsigned long long);
    void discard(unsigned long long);
  };

  class mwc192x {
  public:
    using result_type = uint64_t;
    result_type operator()();
    static constexpr result_type min();
    static constexpr result_type max();
    explicit mwc192x();
    explicit mwc192x(unsigned long);
    explicit mwc192x(result_type x, result_type y, result_type c);
    template<typename gen> explicit mwc192x(gen &g);
    void seed();
    void seed(unsigned long);
    template<typename gen> void seed(gen &g);
    void seed(result_type x, result_type y, result_type c);
    void jump2(unsigned int);
    void jump(unsigned long long);
    void discard(unsigned long long);
  };
}
```

For all four engines, `jump2(n)` advances the state by 2^n outputs, `jump(n)` advances it by n outputs, and `discard(n)` has the same effect as `jump(n)`. These methods support block splitting: workers can start at offsets separated by the chosen block length, and each worker must stay within its assigned block. The MWC engines support block splitting but not leapfrogging.

4.1.8 Minimal standard generator

The class `trng::minstd` in the header file `trng/minstd.hpp` implements the Lehmer generator (multiplicative linear congruential generator)

$$x_{i+1} = 16807\, x_i \bmod (2^{31} - 1)\,, \tag{4.8}$$

which is also known as the "minimal standard" generator and corresponds to `std::minstd_rand0` of the C++ standard library. Its period equals $2^{31} - 2$ and it generates

integers in the range $1, \ldots, 2^{31} - 2$. Hence, `min()` returns 1 and `max()` returns $2^{31} - 2$. Due to its short period and its weak statistical properties this generator is not suited for serious Monte Carlo simulations. It is provided mainly for comparison and for reproducing results of other software. The class does not provide any splitting facilities, but it implements the member function `discard(s)`. The detailed interface of the class `trng::minstd` is given as follows:

```
namespace trng {

  class minstd {
  public:
    using result_type = std::uint32_t;
    result_type operator()();
    static constexpr result_type min();
    static constexpr result_type max();

    class status_type;

    minstd();
    explicit minstd(unsigned long);
    template<typename gen>
    explicit minstd(gen &);

    void seed();
    void seed(unsigned long);
    template<typename gen>
    void seed(gen &g);

    void discard(unsigned long long);

    static const char * name();
    long operator()(long);
  };

  bool operator==(const minstd &, const minstd &);
  bool operator!=(const minstd &, const minstd &);
  template<typename char_t, typename traits_t>
  std::basic_ostream<char_t, traits_t> &
  operator<<(std::basic_ostream<char_t, traits_t> &, const minstd &);
  template<typename char_t, typename traits_t>
  std::basic_istream<char_t, traits_t> &
  operator>>(std::basic_istream<char_t, traits_t> &, minstd &);
}
```

4.1.9 Mersenne twister generators

The Mersenne twister is a popular random number generator that has been introduced by Makoto Matsumoto and Takuji Nishimura [51]. In TRNG the Mersenne twister comes in two different flavors. The classical Mersenne twister implemented as `trng::mt19937` generates random integers of 32 bits, but there is also a version that generates integers of 64 bits as implemented by `trng::mt19937_64`. These classes are declared in the header

files `trng/mt19937.hpp` and `trng/mt19937_64.hpp`. The detailed interfaces of the classes `trng::mt19937` and `trng::mt19937_64` are given as follows:

```
namespace trng {

  class mt19937 {
  public:
    using result_type = unsigned long;
    result_type operator()();
    static constexpr result_type min();
    static constexpr result_type max();

    class parameter_type;
    class status_type;

    mt19937();
    explicit mt19937(unsigned long);
    template<typename gen>
    explicit mt19937(gen &);

    void seed();
    template<typename gen>
    void seed(gen &g);
    void seed(result_type);

    static const char * name();
    long operator()(long);
  };

  bool operator==(const mt19937 &, const mt19937 &);
  bool operator!=(const mt19937 &, const mt19937 &);
  template<typename char_t, typename traits_t>
  std::basic_ostream<char_t, traits_t> &
  operator<<(std::basic_ostream<char_t, traits_t> &, const mt19937 &);
  template<typename char_t, typename traits_t>
  std::basic_istream<char_t, traits_t> &
  operator>>(std::basic_istream<char_t, traits_t> &, mt19937 &);
}
```

```
namespace trng {

  class mt19937_64 {
  public:
    using result_type = unsigned long;
    result_type operator()();
    static constexpr result_type min();
    static constexpr result_type max();

    class parameter_type;
    class status_type;

    mt19937_64();
    explicit mt19937_64(unsigned long);
    template<typename gen>
```

```
    explicit mt19937_64(gen &);

    void seed();
    void seed(unsigned long);
    template<typename gen>
    void seed(gen &g);
    void seed(result_type);

    static const char * name();
    long operator()(long);
  };

  bool operator==(const mt19937_64 &, const mt19937_64 &);
  bool operator!=(const mt19937_64 &, const mt19937_64 &);
  template<typename char_t, typename traits_t>
  std::basic_ostream<char_t, traits_t> &
  operator<<(std::basic_ostream<char_t, traits_t> &, const mt19937_64 &);
  template<typename char_t, typename traits_t>
  std::basic_istream<char_t, traits_t> &
  operator>>(std::basic_istream<char_t, traits_t> &, mt19937_64 &);
}
```

4.2 Random number distributions

This section gives a detailed description of all random number distributions, that have been implemented by TRNG. Each subsection presents the public interface of one random number distribution. The part of the public interface, that is mandatory for a random number distribution, will not be discussed in detail, read section 3.2 instead.

Classes for continuous random number distributions are implemented as template classes. The template argument determines the `result_type` and might be either `float`, `double`, or `long double`, where `double` is the default.

Additionally to the requirements in section 3.2, each random number distribution class provides member functions that calculate its probability distribution function, its cumulative distribution function and in the case of continuous distributions its inverse cumulative distribution function as well. These member functions have the signatures

```
result_type pdf(result_type x) const;
result_type cdf(result_type x) const;
result_type icdf(result_type x) const;
```

and for discrete random variables

```
result_type pdf(int x) const;
result_type cdf(int x) const;
```

The concept of a random number distribution requires two functions that take a random number engine as its argument and generate a random variable with some specific distribution by calling `operator()` of the given random number engine. Note, the concept of a random number distribution does not specify how often `operator()`

is called. This allows the implementer of a random number distribution to choose between various algorithms [35] that transform uniform random numbers into non-uniform distributed numbers. Some of these algorithms transform exactly one uniform random number into one non-uniform number, while some other algorithms have to call `operator()` more than once. How often `operator()` is called may even vary at runtime. If not otherwise stated, all random number distributions in TRNG are implemented in such a way that `operator()` is called exactly once. Because of this special feature it is much more easy to write parallel Monte Carlo simulations that give the same result (and statistical error) independent of the number of parallel processes. We say such algorithms play fair, see section 2.3 and 6.

4.2.1 Uniform distributions

TRNG provides three different classes for generating uniformly distributed random numbers with distribution function

$$p(x|a,b) = \begin{cases} 1/(b-a) & \text{if } a \le x < b \\ 0 & \text{otherwise}. \end{cases}$$

parameters	$a, b \in \mathbb{R}$ with $a < b$
support	$[a, b)$
mean	$(a+b)/2$
variance	$(b-a)^2/12$

The class `uniform_dist` generates random numbers in the range $[a, b)$. Valid parameters for this distribution are $a, b \in \mathbb{R}$ with $a < b$.

Many Monte Carlo simulations consume random numbers uniformly distributed in $[0, 1)$ that can be generated using class `uniform_dist` with parameters $a = 0$ and $b = 1$. However, the uniform distribution in $[0, 1)$ is so common that TRNG has a specialized class `uniform01_dist` for this case. The class `uniform01_dist` might be faster than `uniform_dist` with parameters $a = 0$ and $b = 1$.

Class `uniform_int_dist` is a variant of `uniform_dist` for integer valued random variables. It provides random numbers with distribution function

$$p(x|a,b) = \begin{cases} 1/(b-a) & \text{if } a \le x < b \\ 0 & \text{otherwise} \end{cases} \qquad \text{for } x \in \mathbb{Z}.$$

Valid parameters for this distribution are $a, b \in \mathbb{Z}$ with $a < b$.

The class `uniform_dist` is declared in the header file `trng/uniform_dist.hpp` and its public interface is given as follows:

```
namespace trng {

  template<typename float_t = double>
  class uniform_dist {
  public:
    using result_type = float_t;
    class param_type {
    public:
      result_type a() const;
      void a(result_type);
```

```
      result_type b() const;
      void b(result_type);
      param_type(result_type a, result_type b);
    };
    uniform_dist(result_type a, result_type b);
    explicit uniform_dist(const param_type &);
    void reset();
    template<typename R>
    result_type operator()(R &);
    template<typename R>
    result_type operator()(R &, const param_type &)
    result_type min() const;
    result_type max() const;
    const param_type & param() const;
    void param(const param_type &);
    result_type a() const;
    void a(result_type);
    result_type b();
    void b(result_type);
    result_type pdf(result_type x) const;
    result_type cdf(result_type x) const;
    result_type icdf(result_type x) const;
  };

  template<typename float_t>
  bool operator==(const typename uniform_dist<float_t>::param_type &,
  const typename uniform_dist<float_t>::param_type &);
  template<typename float_t>
  bool operator!=(const typename uniform_dist<float_t>::param_type &,
  const typename uniform_dist<float_t>::param_type &);

  template<typename char_t, typename traits_t, typename float_t>
  std::basic_ostream<char_t, traits_t> &
  operator<<(std::basic_ostream<char_t, traits_t> &,
  const typename uniform_dist<float_t>::param_type &);
  template<typename char_t, typename traits_t, typename float_t>
  std::basic_istream<char_t, traits_t> &
  operator>>(std::basic_istream<char_t, traits_t> &,
  typename uniform_dist<float_t>::param_type &);

  template<typename float_t>
  bool operator==(const uniform_dist<float_t> &, const uniform_dist<float_t> &);
  template<typename float_t>
  bool operator!=(const uniform_dist<float_t> &, const uniform_dist<float_t> &);

  template<typename char_t, typename traits_t, typename float_t>
  std::basic_ostream<char_t, traits_t> &
  operator<<(std::basic_ostream<char_t, traits_t> &, const uniform_dist<float_t> &);
  template<typename char_t, typename traits_t, typename float_t>
  std::basic_istream<char_t, traits_t> &
  operator>>(std::basic_istream<char_t, traits_t> &, uniform_dist<float_t> &);
}
```

The class `uniform01_dist` is declared in the header file `trng/uniform01_dist.hpp`

and its public interface is given as follows:

```
namespace trng {

  template<typename float_t = double>
  class uniform01_dist {
  public:
    using result_type = float_t;
    class param_type;
    uniform01_dist();
    explicit uniform01_dist(const param_type &);
    void reset();
    template<typename R>
    result_type operator()(R &);
    template<typename R>
    result_type operator()(R &, const param_type &);
    result_type min() const;
    result_type max() const;
    const param_type & param() const;
    void param(const param_type &);
    result_type pdf(result_type x) const;
    result_type cdf(result_type x) const;
    result_type icdf(result_type x) const;
  };

  template<typename float_t>
  bool operator==(const typename uniform01_dist<float_t>::param_type &,
  const typename uniform01_dist<float_t>::param_type &);
  template<typename float_t>
  bool operator!=(const typename uniform01_dist<float_t>::param_type &,
  const typename uniform01_dist<float_t>::param_type &);

  template<typename char_t, typename traits_t, typename float_t>
  std::basic_ostream<char_t, traits_t> &
  operator<<(std::basic_ostream<char_t, traits_t> &,
  const typename uniform01_dist<float_t>::param_type &);
  template<typename char_t, typename traits_t, typename float_t>
  std::basic_istream<char_t, traits_t> &
  operator>>(std::basic_istream<char_t, traits_t> &,
  typename uniform01_dist<float_t>::param_type &);

  template<typename float_t>
  bool operator==(const uniform01_dist<float_t> &, const uniform01_dist<float_t> &);
  template<typename float_t>
  bool operator!=(const uniform01_dist<float_t> &, const uniform01_dist<float_t> &);

  template<typename char_t, typename traits_t, typename float_t>
  std::basic_ostream<char_t, traits_t> &
  operator<<(std::basic_ostream<char_t, traits_t> &, const uniform01_dist<float_t> &)
  template<typename char_t, typename traits_t, typename float_t>
  std::basic_istream<char_t, traits_t> &
  operator>>(std::basic_istream<char_t, traits_t> &, uniform01_dist<float_t> &);
}
```

The class uniform_int_dist is declared in the header file trng/uniform_int_dist.hpp and its public interface is given as follows:

```
namespace trng {

  class uniform_int_dist {
  public:
    typedef int result_type;
    class param_type {
    public:
      result_type a() const;
      void a(result_type);
      result_type b() const;
      void b(result_type);
      param_type(result_type a, result_type b);
    };
    uniform_int_dist(result_type a, result_type b);
    explicit uniform_int_dist(const param_type &)
    void reset();
    template<typename R>
    result_type operator()(R &);
    template<typename R>
    result_type operator()(R &, const param_type &);
    result_type min() const;
    result_type max() const;
    const param_type & param() const;
    void param(const param_type &);
    result_type a() const;
    void a(result_type);
    result_type b() const;
    void b(result_type);
    double pdf(result_type x) const;
    double cdf(result_type x) const;
  };

  bool operator==(const uniform_int_dist::param_type &, const uniform_int_dist::param_type &);
  bool operator!=(const uniform_int_dist::param_type &, const uniform_int_dist::param_type &);

  template<typename char_t, typename traits_t>
  std::basic_ostream<char_t, traits_t> &
  operator<<(std::basic_ostream<char_t, traits_t> &, const uniform_int_dist::param_type &);
  template<typename char_t, typename traits_t>
  std::basic_istream<char_t, traits_t> &
  operator>>(std::basic_istream<char_t, traits_t> &, uniform_int_dist::param_type &);

  bool operator==(const uniform_int_dist &, const uniform_int_dist &);
  bool operator!=(const uniform_int_dist &, const uniform_int_dist &);

  template<typename char_t, typename traits_t>
  std::basic_ostream<char_t, traits_t> &
  operator<<(std::basic_ostream<char_t, traits_t> &, const uniform_int_dist &);
  template<typename char_t, typename traits_t>
  std::basic_istream<char_t, traits_t> &
  operator>>(std::basic_istream<char_t, traits_t> &, uniform_int_dist &);
```

```
}
```

4.2.2 Exponential distribution

Class exponential_dist provides random numbers with exponential distribution with mean μ. The probability distribution function reads

$$p(x|\mu) = \begin{cases} \frac{1}{\mu} e^{-x/\mu} & \text{if } x \geq 0 \\ 0 & \text{otherwise}. \end{cases}$$

parameter	$\mu \in \mathbb{R}$ with $\mu > 0$
support	$[0, \infty)$
mean	μ
variance	μ^2

Valid parameter for this distribution is $\mu \in \mathbb{R}$ with $\mu > 0$.

The class exponential_dist is declared in the header file trng/exponential_dist.hpp and its public interface is given as follows:

```
namespace trng {

  template<typename float_t = double>
  class exponential_dist {
  public:
    using result_type = float_t;
    class param_type {
    public:
      result_type mu() const;
      void mu(result_type);
      explicit param_type(result_type mu);
    };
    explicit exponential_dist(result_type mu);
    explicit exponential_dist(const param_type &);
    void reset();
    template<typename R>
    result_type operator()(R &);
    template<typename R>
    result_type operator()(R &, const param_type &);
    result_type min() const;
    result_type max() const;
    const param_type & param() const;
    void param(const param_type &);
    result_type mu() const;
    void mu(result_type);
    result_type pdf(result_type) const;
    result_type cdf(result_type) const;
    result_type icdf(result_type) const;
  };

  template<typename float_t>
  bool operator==(const typename exponential_dist<float_t>::param_type &,
  const typename exponential_dist<float_t>::param_type &);
  template<typename float_t>
  bool operator!=(const typename exponential_dist<float_t>::param_type &,
  const typename exponential_dist<float_t>::param_type &);
```

```
  template<typename char_t, typename traits_t, typename float_t>
  std::basic_ostream<char_t, traits_t> &
  operator<<(std::basic_ostream<char_t, traits_t> &,
  const typename exponential_dist<float_t>::param_type &);
  template<typename char_t, typename traits_t, typename float_t>
  std::basic_istream<char_t, traits_t> &
  operator>>(std::basic_istream<char_t, traits_t> &,
  typename exponential_dist<float_t>::param_type &);

  template<typename float_t>
  bool operator==(const exponential_dist<float_t> &, const exponential_dist<float_t> &);
  template<typename float_t>
  bool operator!=(const exponential_dist<float_t> &, const exponential_dist<float_t> &);

  template<typename char_t, typename traits_t, typename float_t>
  std::basic_ostream<char_t, traits_t> &
  operator<<(std::basic_ostream<char_t, traits_t> &, const exponential_dist<float_t> &);
  template<typename char_t, typename traits_t, typename float_t>
  std::basic_istream<char_t, traits_t> &
  operator>>(std::basic_istream<char_t, traits_t> &, exponential_dist<float_t> &);
}
```

4.2.3 Two-sided exponential distribution

Class `twosided_exponential_dist` provides random numbers with two-sided exponential distribution with parameter μ. The probability distribution function reads

$$p(x|\mu) = \frac{1}{2\mu} e^{-|x|/\mu}$$

parameter	$\mu \in \mathbb{R}$ with $\mu > 0$
support	$(-\infty, \infty)$
mean	0
variance	$2\mu^2$

Valid parameter for this distribution is $\mu \in \mathbb{R}$ with $\mu > 0$.

The class `twosided_exponential_dist` is declared in the header file `trng/twosided_exponential_dist.hpp` and its public interface is given as follows:

```
namespace trng {

  template<typename float_t = double>
  class twosided_exponential_dist {
  public:
    using result_type = float_t;
    class param_type {
    public:
      result_type mu() const;
      void mu(result_type);
      explicit param_type(result_type mu);
    };
    explicit twosided_exponential_dist(result_type mu);
    explicit twosided_exponential_dist(const param_type &);
    void reset();
    template<typename R>
```

```
    result_type operator()(R &);
    template<typename R>
    result_type operator()(R &, const param_type &);
    result_type min() const;
    result_type max() const;
    const param_type & param() const;
    void param(const param_type &);
    result_type mu() const;
    void mu(result_type);
    result_type pdf(result_type) const;
    result_type cdf(result_type) const;
    result_type icdf(result_type) const;
  };

  template<typename float_t>
  bool operator==(const typename exponential_dist<float_t>::param_type &,
  const typename exponential_dist<float_t>::param_type &);
  template<typename float_t>
  bool operator!=(const typename exponential_dist<float_t>::param_type &,
  const typename exponential_dist<float_t>::param_type &);

  template<typename char_t, typename traits_t, typename float_t>
  std::basic_ostream<char_t, traits_t> &
  operator<<(std::basic_ostream<char_t, traits_t> &,
  const typename exponential_dist<float_t>::param_type &);
  template<typename char_t, typename traits_t, typename float_t>
  std::basic_istream<char_t, traits_t> &
  operator>>(std::basic_istream<char_t, traits_t> &,
  typename exponential_dist<float_t>::param_type &);

  template<typename float_t>
  bool operator==(const exponential_dist<float_t> &, const exponential_dist<float_t> &);
  template<typename float_t>
  bool operator!=(const exponential_dist<float_t> &, const exponential_dist<float_t> &);

  template<typename char_t, typename traits_t, typename float_t>
  std::basic_ostream<char_t, traits_t> &
  operator<<(std::basic_ostream<char_t, traits_t> &, const exponential_dist<float_t> &);
  template<typename char_t, typename traits_t, typename float_t>
  std::basic_istream<char_t, traits_t> &
  operator>>(std::basic_istream<char_t, traits_t> &, exponential_dist<float_t> &);
}
```

4.2.4 Normal distributions

There are two classes for producing random numbers with normal distribution, `normal_dist` and `correlated_normal_dist` . Class `normal_dist` provides uncorrelated random numbers with normal distribution with mean μ and standard devia-

parameters	$\mu, \sigma \in \mathbb{R}$, with $\sigma > 0$
support	$(-\infty, \infty)$
mean	μ
variance	σ^2

tion σ. The probability distribution function reads

$$p(x|\mu,\sigma) = \frac{1}{\sqrt{2\pi\sigma^2}} \mathrm{e}^{-(x-\mu)^2/(2\sigma^2)} .$$

Valid parameters for this distribution are $\mu, \sigma \in \mathbb{R}$ with $\sigma > 0$. The normal distribution is also known as Gaussian distribution.

The class `normal_dist` is declared in the header file `trng/normal_dist.hpp` and its public interface is given as follows:

```
namespace trng {

  template<typename float_t = double>
  class normal_dist {
  public:
    using result_type = float_t;
    class param_type {
    public:
      result_type mu() const;
      void mu(result_type);
      result_type sigma() const;
      void sigma(result_type);
      param_type(result_type mu, result_type sigma);
    };
    normal_dist(result_type mu, result_type sigma);
    explicit normal_dist(const param_type &);
    void reset();
    template<typename R>
    result_type operator()(R &);
    template<typename R>
    result_type operator()(R &, const param_type &);
    result_type min() const;
    result_type max() const;
    const param_type & param() const;
    void param(const param_type &);
    result_type mu() const;
    void mu(result_type);
    result_type sigma() const;
    void sigma(result_type);
    result_type pdf(result_type) const;
    result_type cdf(result_type) const;
    result_type icdf(result_type) const;
  };

  template<typename float_t>
  bool operator==(const typename normal_dist<float_t>::param_type &,
  const typename normal_dist<float_t>::param_type &);
  template<typename float_t>
  bool operator!=(const typename normal_dist<float_t>::param_type &,
  const typename normal_dist<float_t>::param_type &);

  template<typename char_t, typename traits_t, typename float_t>
  std::basic_ostream<char_t, traits_t> &
```

```
  operator<<(std::basic_ostream<char_t, traits_t> &,
  const typename normal_dist<float_t>::param_type &);
  template<typename char_t, typename traits_t, typename float_t>
  std::basic_istream<char_t, traits_t> &
  operator>>(std::basic_istream<char_t, traits_t> &,
  typename normal_dist<float_t>::param_type &);

  template<typename float_t>
  bool operator==(const normal_dist<float_t> &, const normal_dist<float_t> &);
  template<typename float_t>
  bool operator!=(const normal_dist<float_t> &, const normal_dist<float_t> &);

  template<typename char_t, typename traits_t, typename float_t>
  std::basic_ostream<char_t, traits_t> &
  operator<<(std::basic_ostream<char_t, traits_t> &, const normal_dist<float_t> &);
  template<typename char_t, typename traits_t, typename float_t>
  std::basic_istream<char_t, traits_t> &
  operator>>(std::basic_istream<char_t, traits_t> &, normal_dist<float_t> &);
}
```

If $\mathbf{x} = (x_1, x_2, \ldots x_d)$ are d random variables, then the multivariate normal density function for $\mathbf{x}$ is

$$p(\mathbf{x}|\mathbf{V}) = \frac{1}{\sqrt{(2\pi)^d \det \mathbf{V}}} \exp\left(-\frac{1}{2}\mathbf{x}^T\mathbf{V}^{-1}\mathbf{x}\right). \tag{4.9}$$

Each variable $x_1, x_2, \ldots x_d$ has mean zero and the the covariance matrix of $x_1, x_2, \ldots x_d$ is given by the symmetric positive definite $d \times d$ matrix $\mathbf{V}$. Class `correlated_normal_dist` provides correlated random numbers with normal distribution by the transformation of uncorrelated random numbers [18].

The class `correlated_normal_dist` is declared in the header file `trng/correlated_normal_dist.hpp` and its public interface is given as follows:

```
namespace trng {

  template<typename float_t = double>
  class correlated_normal_dist {
  public:
    using result_type = float_t;
    class param_type {
    public:
      template<typename iter>
      param_type(iter first, iter last);
    };
    template<typename iter>
    correlated_normal_dist(iter first, iter last);
    explicit correlated_normal_dist(const param_type &);
    void reset();
    template<typename R>
    result_type operator()(R &);
    template<typename R>
    result_type operator()(R &, const param_type &);
    result_type min() const;
    result_type max() const;
```

```
    const param_type & param() const;
    void param(const param_type &p_new);
  };

  template<typename float_t>
  bool operator==(const typename correlated_normal_dist<float_t>::param_type &,
  const typename correlated_normal_dist<float_t>::param_type &);
  template<typename float_t>
  bool operator!=(const typename correlated_normal_dist<float_t>::param_type &,
  const typename correlated_normal_dist<float_t>::param_type &);

  template<typename char_t, typename traits_t, template float_t>
  std::basic_ostream<char_t, traits_t> &
  operator<<(std::basic_ostream<char_t, traits_t> &,
  const typename correlated_normal_dist<float_t>::param_type &);
  template<typename char_t, typename traits_t, template float_t>
  std::basic_istream<char_t, traits_t> &
  operator>>(std::basic_istream<char_t, traits_t> &,
  typename correlated_normal_dist<float_t>::param_type &);

  template<typename float_t>
  bool operator==(const correlated_normal_dist<float_t> &,
  const correlated_normal_dist<float_t> &);
  template<typename float_t>
  bool operator!=(const correlated_normal_dist<float_t> &,
  const correlated_normal_dist<float_t> &);

  template<typename char_t, typename traits_t, template float_t>
  std::basic_ostream<char_t, traits_t> &
  operator<<(std::basic_ostream<char_t, traits_t> &,
  const correlated_normal_dist<float_t> &);
  template<typename char_t, typename traits_t, template float_t>
  std::basic_istream<char_t, traits_t> &
  operator>>(std::basic_istream<char_t, traits_t> &,
  correlated_normal_dist<float_t> &);
}
```

The covariance matrix $\mathbf{V}$ has to be passed to the constructor of `correlated_normal_dist` by two iterators. It is not checked, if the matrix is positive definite. The call operator `operator()` returns a single random number and has complexity $\mathcal{O}(d)$. As a consequence, the generation of a tuple of d correlated random numbers takes $\mathcal{O}(d^2)$ operations.

Successive calls return random numbers with variance $\mathbf{V}_{1,1}$, $\mathbf{V}_{2,2}$ and so on, until the `operator()` has been called d times, which returns a random number with variance $\mathbf{V}_{d,d}$. A sequence of further calls of `operator()` will return random numbers with the same sequences of variances. The method `reset` resets the internal state of the distribution such that, of further calls of `operator()` will return random numbers starting with a number with variance $\mathbf{V}_{1,1}$. Listing 4.1 illustrates the usage of class `correlated_normal_dist`.

Listing 4.1: Demonstration program illustrating the usage of `correlated_normal_dist`.

```
#include <cstdlib>
#include <iostream>
#include <iomanip>
#include <vector>
#include <trng/lcg64.hpp>
#include <trng/correlated_normal_dist.hpp>

double covariance(const std::vector<double>& v1, const std::vector<double>& v2);

double covariance(const std::vector<double>& v1, const std::vector<double>& v2) {
  const std::vector<double>::size_type n{v1.size()};
  double m1{0.0}, m2{0.0}, c{0.0};
  for (std::vector<double>::size_type i{0}; i < n; ++i) {
    m1 += v1[i] / n;
    m2 += v2[i] / n;
  }
  for (std::vector<double>::size_type i{0}; i < n; ++i)
    c += (v1[i] - m1) * (v2[i] - m2) / n;
  return c;
}

int main() {
  const int d{4};
  // covariance matrix
  const double sig[d][d]{{2.0, -0.5, 0.3, -0.3},
                         {-0.5, 3.0, -0.3, 0.3},
                         {0.3, -0.3, 1.0, -0.3},
                         {-0.3, 0.3, -0.3, 1.0}};
  trng::correlated_normal_dist<> D(&sig[0][0], &sig[d - 1][d - 1] + 1);
  trng::lcg64 R;

  std::vector<double> x1, x2, x3, x4;
  // generate 4-tuples of correlated normal variables
  for (int i{0}; i < 1000000; ++i) {
    x1.push_back(D(R));
    x2.push_back(D(R));
    x3.push_back(D(R));
    x4.push_back(D(R));
  }
  // print (empirical) covariance matrix
  std::cout << std::setprecision(4) << covariance(x1, x1) << '\t' << covariance(x1, x2) << '\t'
            << covariance(x1, x3) << '\t' << covariance(x1, x4) << '\n'
            << covariance(x2, x1) << '\t' << covariance(x2, x2) << '\t' << covariance(x2, x3)
            << '\t' << covariance(x2, x4) << '\n'
            << covariance(x3, x1) << '\t' << covariance(x3, x2) << '\t' << covariance(x3, x3)
            << '\t' << covariance(x3, x4) << '\n'
            << covariance(x4, x1) << '\t' << covariance(x4, x2) << '\t' << covariance(x4, x3)
            << '\t' << covariance(x4, x4) << '\n';
  return EXIT_SUCCESS;
}
```

4.2.5 Truncated normal distribution

parameters	$\mu, \sigma, a, b \in \mathbb{R}$, with $\sigma > 0$, $a < b$
support	$[a, b]$
mean	$\mu + \frac{\phi(\frac{a-\mu}{\sigma}) - \phi(\frac{b-\mu}{\sigma})}{\Phi(\frac{b-\mu}{\sigma}) - \Phi(\frac{a-\mu}{\sigma})}\sigma$
variance	$\sigma^2 \left[1 + \frac{\frac{a-\mu}{\sigma}\phi(\frac{a-\mu}{\sigma}) - \frac{b-\mu}{\sigma}\phi(\frac{b-\mu}{\sigma})}{\Phi(\frac{b-\mu}{\sigma}) - \Phi(\frac{a-\mu}{\sigma})} - \left(\frac{\phi(\frac{a-\mu}{\sigma}) - \phi(\frac{b-\mu}{\sigma})}{\Phi(\frac{b-\mu}{\sigma}) - \Phi(\frac{a-\mu}{\sigma})} \right)^2 \right]$

The class `truncated_normal_dist` provides random numbers with a truncated normal distribution with parameters μ, σ, a and b. The probability distribution function reads

$$p(x|\mu, \sigma, a, b) = \frac{\frac{1}{\sigma}\phi\left(\frac{x-\mu}{\sigma}\right)}{\Phi\left(\frac{b-\mu}{\sigma}\right) - \Phi\left(\frac{a-\mu}{\sigma}\right)}$$

where $\phi(x)$ denotes the probability density function of the standard normal distribution and $\Phi(x)$ its cumulative distribution function. Valid parameters for this distribution are $\mu, \sigma, a, b, \in \mathbb{R}$ with $\sigma > 0$ and $a < b$.

The class `truncated_normal_dist` is declared in the header file `trng/truncated_normal_dist.hpp` and its public interface is given as follows:

```
namespace trng {

  template<typename float_t = double>
  class truncated_normal_dist {
  public:
    using result_type = float_t;
    class param_type {
    public:
      result_type mu() const;
      void mu(result_type);
      result_type sigma() const;
      void sigma(result_type);
      result_type a() const;
      void a(result_type);
      result_type b() const;
      void b(result_type);
      param_type(result_type mu, result_type sigma, result_type a, result_type b);
    };
    truncated_normal_dist(result_type mu, result_type sigma,
    result_type a, result_type b);
    explicit truncated_normal_dist(const param_type &);
    void reset();
    template<typename R>
    result_type operator()(R &);
    template<typename R>
    result_type operator()(R &, const param_type &);
    result_type min() const;
    result_type max() const;
    const param_type & param() const;
```

```
    void param(const param_type &);
    result_type mu() const;
    void mu(result_type);
    result_type sigma() const;
    void sigma(result_type);
    result_type a() const;
    void a(result_type);
    result_type b() const;
    void b(result_type);
    result_type pdf(result_type) const;
    result_type cdf(result_type) const;
    result_type icdf(result_type) const;
  };

  template<typename float_t>
  bool operator==(const typename truncated_normal_dist<float_t>::param_type &,
  const typename truncated_normal_dist<float_t>::param_type &);
  template<typename float_t>
  bool operator!=(const typename truncated_normal_dist<float_t>::param_type &,
  const typename truncated_normal_dist<float_t>::param_type &);

  template<typename char_t, typename traits_t, typename float_t>
  std::basic_ostream<char_t, traits_t> &
  operator<<(std::basic_ostream<char_t, traits_t> &,
  const typename truncated_normal_dist<float_t>::param_type &);
  template<typename char_t, typename traits_t, typename float_t>
  std::basic_istream<char_t, traits_t> &
  operator>>(std::basic_istream<char_t, traits_t> &,
  typename truncated_normal_dist<float_t>::param_type &);

  template<typename float_t>
  bool operator==(const truncated_normal_dist<float_t> &, const truncated_normal_dist<float_t
> &);
  template<typename float_t>
  bool operator!=(const truncated_normal_dist<float_t> &, const truncated_normal_dist<float_t
> &);

  template<typename char_t, typename traits_t, typename float_t>
  std::basic_ostream<char_t, traits_t> &
  operator<<(std::basic_ostream<char_t, traits_t> &, const truncated_normal_dist<float_t> &);
  template<typename char_t, typename traits_t, typename float_t>
  std::basic_istream<char_t, traits_t> &
  operator>>(std::basic_istream<char_t, traits_t> &, truncated_normal_dist<float_t> &);

}
```

4.2.6 Maxwell distribution

The class `maxwell_dist` provides random numbers with Maxwell distribution with the parameter θ. The probability distribution function reads

$$p(x|\theta) = \sqrt{\frac{2}{\pi}} \frac{x^2 \mathrm{e}^{-x^2/(2\theta^2)}}{\theta^3} .$$

parameters	$\theta \in \mathbb{R}$, with $\theta > 0$
support	$(0, \infty)$
mean	$2\theta\sqrt{2/\pi}$
variance	$\theta^2(3\pi - 8)/\pi$

Valid parameters for this distribution are $\theta \in \mathbb{R}$ with $\theta > 0$. The Maxwell distribution is also know as Maxwell-Boltzmann distribution.

The class `maxwell_dist` is declared in the header file `trng/maxwell_dist.hpp` and its public interface is given as follows:

```
namespace trng {

  template<typename float_t = double>
  class maxwell_dist {
  public:
    using result_type = float_t;
    class param_type {
    public:
      result_type theta() const;
      void theta(result_type);
      param_type(result_type theta);
    };
    maxwell_dist(result_type theta);
    explicit maxwell_dist(const param_type &);
    void reset();
    template<typename R>
    result_type operator()(R &);
    template<typename R>
    result_type operator()(R &, const param_type &);
    result_type min() const;
    result_type max() const;
    const param_type & param() const;
    void param(const param_type &);
    result_type theta() const;
    void theta(result_type);
    result_type pdf(result_type) const;
    result_type cdf(result_type) const;
    result_type icdf(result_type) const;
  };

  template<typename float_t>
  bool operator==(const typename maxwell_dist<float_t>::param_type &,
  const typename maxwell_dist<float_t>::param_type &);
  template<typename float_t>
  bool operator!=(const typename maxwell_dist<float_t>::param_type &,
  const typename maxwell_dist<float_t>::param_type &);

  template<typename char_t, typename traits_t, typename float_t>
  std::basic_ostream<char_t, traits_t> &
```

```
  operator<<(std::basic_ostream<char_t, traits_t> &,
  const typename maxwell_dist<float_t>::param_type &);
  template<typename char_t, typename traits_t, typename float_t>
  std::basic_istream<char_t, traits_t> &
  operator>>(std::basic_istream<char_t, traits_t> &,
  typename maxwell_dist<float_t>::param_type &);

  template<typename float_t>
  bool operator==(const maxwell_dist<float_t> &, const maxwell_dist<float_t> &);
  template<typename float_t>
  bool operator!=(const maxwell_dist<float_t> &, const maxwell_dist<float_t> &);

  template<typename char_t, typename traits_t, typename float_t>
  std::basic_ostream<char_t, traits_t> &
  operator<<(std::basic_ostream<char_t, traits_t> &, const maxwell_dist<float_t> &);
  template<typename char_t, typename traits_t, typename float_t>
  std::basic_istream<char_t, traits_t> &
  operator>>(std::basic_istream<char_t, traits_t> &, maxwell_dist<float_t> &);
}
```

4.2.7 Cauchy distribution

The class `cauchy_dist` provides random numbers with Cauchy distribution with parameters θ and η. The probability distribution function reads

$$p(x|\theta,\eta) = \frac{1}{\theta\pi\left(1+\left(\frac{x-\eta}{\theta}\right)^2\right)}.$$

parameters	$\theta, \eta \in \mathbb{R}$, with $\theta > 0$
support	$(-\infty, \infty)$
mean	not defined
variance	not defined

Valid parameters for this distribution are $\theta, \eta \in \mathbb{R}$ with $\theta > 0$. The Cauchy distribution is also known as Lorentz distribution or Breit-Wigner distribution.

The class `cauchy_dist` is declared in the header file `trng/cauchy_dist.hpp` and its public interface is given as follows:

```
namespace trng {

  template<typename float_t = double>
  class cauchy_dist {
  public:
    using result_type = float_t;
    class param_type {
    public:
      result_type theta() const;
      void theta(result_type);
      result_type eta() const;
      void eta(result_type);
      param_type(result_type theta, result_type eta);
    };
    cauchy_dist(result_type theta, result_type eta);
    explicit cauchy_dist(const param_type &);
```

```
    void reset();
    template<typename R>
    result_type operator()(R &);
    template<typename R>
    result_type operator()(R &, const param_type &);
    result_type min() const;
    result_type max() const;
    const param_type & param() const;
    void param(const param_type &);
    result_type theta() const;
    void theta(result_type);
    result_type eta() const;
    void eta(result_type);
    result_type pdf(result_type) const;
    result_type cdf(result_type) const;
    result_type icdf(result_type) const;
  };

  template<typename float_t>
  bool operator==(const typename cauchy_dist<float_t>::param_type &,
  const typename cauchy_dist<float_t>::param_type &);
  template<typename float_t>
  bool operator!=(const typename cauchy_dist<float_t>::param_type &,
  const typename cauchy_dist<float_t>::param_type &);

  template<typename char_t, typename traits_t, typename float_t>
  std::basic_ostream<char_t, traits_t> &
  operator<<(std::basic_ostream<char_t, traits_t> &,
  const typename cauchy_dist<float_t>::param_type &);
  template<typename char_t, typename traits_t, typename float_t>
  std::basic_istream<char_t, traits_t> &
  operator>>(std::basic_istream<char_t, traits_t> &,
  typename cauchy_dist<float_t>::param_type &);

  template<typename float_t>
  bool operator==(const cauchy_dist<float_t> &, const cauchy_dist<float_t> &);
  template<typename float_t>
  bool operator!=(const cauchy_dist<float_t> &, const cauchy_dist<float_t> &);

  template<typename char_t, typename traits_t, typename float_t>
  std::basic_ostream<char_t, traits_t> &
  operator<<(std::basic_ostream<char_t, traits_t> &, const cauchy_dist<float_t> &);
  template<typename char_t, typename traits_t, typename float_t>
  std::basic_istream<char_t, traits_t> &
  operator>>(std::basic_istream<char_t, traits_t> &, cauchy_dist<float_t> &);
}
```

4.2.8 Logistic distribution

Class `logistic_dist` provides random numbers with Logistic distribution with parameters θ and η. The probability distribution function reads

$$p(x|\theta,\eta) = \frac{e^{-(x-\eta)/\theta}}{\theta\left(1+e^{-(x-\eta)/\theta}\right)^2}\,.$$

parameters	$\theta, \eta \in \mathbb{R}$, with $\theta > 0$
support	$(-\infty, \infty)$
mean	η
variance	$\pi^2\theta^2/3$

Valid parameters for this distribution are $\theta, \eta \in \mathbb{R}$ with $\theta > 0$.

The class `logistic_dist` is declared in the header file `trng/logistic_dist.hpp` and its public interface is given as follows:

```
namespace trng {

  template<typename float_t = double>
  class logistic_dist {
  public:
    typedef double result_type;
    class param_type {
    public:
      result_type theta() const;
      void theta(result_type);
      result_type eta() const;
      void eta(result_type);
      param_type(result_type theta, result_type eta);
    };
    logistic_dist(result_type theta, result_type eta);
    explicit logistic_dist(const param_type &);
    void reset();
    template<typename R>
    result_type operator()(R &);
    template<typename R>
    result_type operator()(R &, const param_type &);
    result_type min() const;
    result_type max() const;
    const param_type & param() const;
    void param(const param_type &);
    result_type theta() const;
    void theta(result_type);
    result_type eta() const;
    void eta(result_type);
    result_type pdf(result_type) const;
    result_type cdf(result_type) const;
    result_type icdf(result_type) const;
  };

  template<typename float_t>
  bool operator==(const typename logistic_dist<float_t>::param_type &,
  const typename logistic_dist<float_t>::param_type &);
  template<typename float_t>
  bool operator!=(const typename logistic_dist<float_t>::param_type &,
  const typename logistic_dist<float_t>::param_type &);
```

```
  template<typename char_t, typename traits_t, typename float_t>
  std::basic_ostream<char_t, traits_t> &
  operator<<(std::basic_ostream<char_t, traits_t> &,
  const typename logistic_dist<float_t>::param_type &);
  template<typename char_t, typename traits_t, typename float_t>
  std::basic_istream<char_t, traits_t> &
  operator>>(std::basic_istream<char_t, traits_t> &,
  typename logistic_dist<float_t>::param_type &);

  template<typename float_t>
  bool operator==(const logistic_dist<float_t> &, const logistic_dist<float_t> &);
  template<typename float_t>
  bool operator!=(const logistic_dist<float_t> &, const logistic_dist<float_t> &);

  template<typename char_t, typename traits_t>
  std::basic_ostream<char_t, traits_t, typename float_t> &
  operator<<(std::basic_ostream<char_t, traits_t> &, const logistic_dist<float_t> &);
  template<typename char_t, typename traits_t, typename float_t>
  std::basic_istream<char_t, traits_t> &
  operator>>(std::basic_istream<char_t, traits_t> &, logistic_dist<float_t> &);
}
```

4.2.9 Lognormal distribution

Class `lognormal_dist` provides random numbers with lognormal distribution with parameters μ and σ. The probability distribution function reads

$$p(x|\mu,\sigma) = \begin{cases} 0 & \text{for } x \le 0 \\ \dfrac{1}{x\sqrt{2\pi\sigma^2}} e^{-(\ln x-\mu)^2/(2\sigma^2)} & \text{for } x > 0\,. \end{cases}$$

parameters	$\mu, \sigma \in \mathbb{R}$, with $\sigma > 0$
support	$(0, \infty)$
mean	$e^{\mu+\sigma^2/2}$
variance	$(e^{\sigma^2} - 1)e^{\mu/2+\sigma^2}$

Valid parameters for this distribution are $\mu, \sigma \in \mathbb{R}$ with $\sigma > 0$.

The class `lognormal_dist` is declared in the header file `trng/lognormal_dist.hpp` and its public interface is given as follows:

```
namespace trng {

  template<typename float_t = double>
  class lognormal_dist {
  public:
    using result_type = float_t;
    class param_type {
    public:
      result_type mu() const;
      void mu(result_type);
      result_type sigma() const;
      void sigma(result_type);
      param_type(result_type mu, result_type sigma);
    };
```

```
    lognormal_dist(result_type mu, result_type sigma);
    explicit lognormal_dist(const param_type &);
    void reset();
    template<typename R>
    result_type operator()(R &);
    template<typename R>
    result_type operator()(R &, const param_type &);
    result_type min() const;
    result_type max() const;
    const param_type & param() const;
    void param(const param_type &);
    result_type mu() const;
    void mu(result_type);
    result_type sigma() const;
    void sigma(result_type);
    result_type pdf(result_type) const;
    result_type cdf(result_type) const;
    result_type icdf(result_type) const;
  };

  template<typename float_t>
  bool operator==(const typename lognormal_dist<float_t>::param_type &,
  const typename lognormal_dist<float_t>::param_type &);
  template<typename float_t>
  bool operator!=(const typename lognormal_dist<float_t>::param_type &,
  const typename lognormal_dist<float_t>::param_type &);

  template<typename char_t, typename traits_t, typename float_t>
  std::basic_ostream<char_t, traits_t> &
  operator<<(std::basic_ostream<char_t, traits_t> &,
  const typename lognormal_dist<float_t>::param_type &);
  template<typename char_t, typename traits_t, typename float_t>
  std::basic_istream<char_t, traits_t> &
  operator>>(std::basic_istream<char_t, traits_t> &,
  typename lognormal_dist<float_t>::param_type &);

  template<typename float_t>
  bool operator==(const lognormal_dist<float_t> &, const lognormal_dist<float_t> &);
  template<typename float_t>
  bool operator!=(const lognormal_dist<float_t> &, const lognormal_dist<float_t> &);

  template<typename char_t, typename traits_t, typename float_t>
  std::basic_ostream<char_t, traits_t> &
  operator<<(std::basic_ostream<char_t, traits_t> &, const lognormal_dist<float_t> &);
  template<typename char_t, typename traits_t, typename float_t>
  std::basic_istream<char_t, traits_t> &
  operator>>(std::basic_istream<char_t, traits_t> &, lognormal_dist<float_t> &);
}
```

4.2.10 Pareto distribution

Class `pareto_dist` provides random numbers with Pareto distribution with parameters γ and θ. The probability distribution function reads

$$p(x|\gamma,\theta) = \begin{cases} 0 & \text{for } x < 0 \\ \frac{\gamma}{\theta}\left(1+\frac{x}{\theta}\right)^{-\gamma-1} & \text{for } x \geq 0\,. \end{cases}$$

parameters	$\theta, \gamma \in (0,\infty)$
support	$[0,\infty)$
mean	$\theta/(\gamma-1)$
variance	$\dfrac{\theta^2\gamma}{(\gamma-1)^2(\gamma-2)}$

The mean and the variance are defined only if $\gamma > 1$ and $\gamma > 2$, respectively.

Valid parameters for this distribution are $\gamma, \theta \in \mathbb{R}$ with $\gamma > 0$ and $\theta > 0$. In the mathematics literature, one can find two different kinds of probability distributions that are referred to as the Pareto distribution. Section 4.2.11 introduces another probability distribution that is also sometimes called the Pareto distribution.

The class `pareto_dist` is declared in the header file `trng/pareto_dist.hpp` and its public interface is given as follows:

```
namespace trng {

  template<typename float_t = double>
  class pareto_dist {
  public:
    using result_type = float_t;
    class param_type {
    public:
      result_type gamma() const;
      void gamma(result_type);
      result_type theta() const;
      void theta(result_type);
      param_type(result_type gamma, result_type theta);
    };
    pareto_dist(result_type gamma, result_type theta);
    explicit pareto_dist(const param_type &);
    void reset();
    template<typename R>
    result_type operator()(R &);
    template<typename R>
    result_type operator()(R &, const param_type &);
    result_type min() const;
    result_type max() const;
    const param_type & param() const;
    void param(const param_type &);
    result_type gamma() const;
    void gamma(result_type);
    result_type theta() const;
    void theta(result_type);
    result_type pdf(result_type) const;
    result_type cdf(result_type) const;
    result_type icdf(result_type) const;
  };
```

```
  template<typename float_t>
  bool operator==(const typename pareto_dist<float_t>::param_type &,
  const typename pareto_dist<float_t>::param_type &);
  template<typename float_t>
  bool operator!=(const typename pareto_dist<float_t>::param_type &,
  const typename pareto_dist<float_t>::param_type &);

  template<typename char_t, typename traits_t, typename float_t>
  std::basic_ostream<char_t, traits_t> &
  operator<<(std::basic_ostream<char_t, traits_t> &,
  const typename pareto_dist<float_t>::param_type &);
  template<typename char_t, typename traits_t, typename float_t>
  std::basic_istream<char_t, traits_t> &
  operator>>(std::basic_istream<char_t, traits_t> &,
  typename pareto_dist<float_t>::param_type &);

  template<typename float_t>
  bool operator==(const pareto_dist<float_t> &, const pareto_dist<float_t> &);
  template<typename float_t>
  bool operator!=(const pareto_dist<float_t> &, const pareto_dist<float_t> &);

  template<typename char_t, typename traits_t, typename float_t>
  std::basic_ostream<char_t, traits_t> &
  operator<<(std::basic_ostream<char_t, traits_t> &, const pareto_dist<float_t> &);
  template<typename char_t, typename traits_t, typename float_t>
  std::basic_istream<char_t, traits_t> &
  operator>>(std::basic_istream<char_t, traits_t> &, pareto_dist<float_t> &);
}
```

4.2.11 Power-law distribution

Class `powerlaw_dist` provides random numbers with power-law distribution with parameters γ and θ. This distribution is related to the Pareto distribution and its probability distribution function reads

$$p(x|\gamma,\theta) = \begin{cases} 0 & \text{for } x < \theta \\ \frac{\gamma}{\theta}\left(\frac{x}{\theta}\right)^{-\gamma-1} & \text{for } x \geq \theta\,. \end{cases}$$

parameters	$\theta, \gamma \in (0, \infty)$
support	$[\theta, \infty)$
mean	$\gamma\theta/(\gamma-1)$
variance	$\dfrac{\theta^2\gamma}{(\gamma-1)^2(\gamma-2)}$

The mean and the variance are defined only if $\gamma > 1$ and $\gamma > 2$, respectively.

Valid parameters for this distribution are $\gamma, \theta \in \mathbb{R}$ with $\gamma > 0$ and $\theta > 0$.

The class `powerlaw_dist` is declared in the header file `trng/powerlaw_dist.hpp` and its public interface is given as follows:

```
namespace trng {

  template<typename float_t = double>
  class powerlaw_dist {
```

```
public:
  using result_type = float_t;
  class param_type {
  public:
    result_type gamma() const;
    void gamma(result_type);
    result_type theta() const;
    void theta(result_type);
    param_type(result_type gamma, result_type theta);
  };
  powerlaw_dist(result_type gamma, result_type theta);
  explicit powerlaw_dist(const param_type &);
  void reset();
  template<typename R>
  result_type operator()(R &);
  template<typename R>
  result_type operator()(R &, const param_type &);
  result_type min() const;
  result_type max() const;
  const param_type & param() const;
  void param(const param_type &);
  result_type gamma() const;
  void gamma(result_type);
  result_type theta() const;
  void theta(result_type);
  result_type pdf(result_type) const;
  result_type cdf(result_type) const;
  result_type icdf(result_type) const;
};

template<typename float_t>
bool operator==(const typename powerlaw_dist::param_type &,
const typename powerlaw_dist::param_type &);
template<typename float_t>
bool operator!=(const typename powerlaw_dist::param_type &,
const typename powerlaw_dist::param_type &);

template<typename char_t, typename traits_t, typename float_t>
std::basic_ostream<char_t, traits_t> &
operator<<(std::basic_ostream<char_t, traits_t> &,
const typename powerlaw_dist<float_t>::param_type &);
template<typename char_t, typename traits_t, typename float_t>
std::basic_istream<char_t, traits_t> &
operator>>(std::basic_istream<char_t, traits_t> &,
typename powerlaw_dist<float_t>::param_type &);

template<typename float_t>
bool operator==(const powerlaw_dist<float_t> &, const powerlaw_dist<float_t> &);
template<typename float_t>
bool operator!=(const powerlaw_dist<float_t> &, const powerlaw_dist<float_t> &);

template<typename char_t, typename traits_t, typename float_t>
std::basic_ostream<char_t, traits_t> &
operator<<(std::basic_ostream<char_t, traits_t> &, const powerlaw_dist<float_t> &);
```

```
  template<typename char_t, typename traits_t, typename float_t>
  std::basic_istream<char_t, traits_t> &
  operator>>(std::basic_istream<char_t, traits_t> &, powerlaw_dist<float_t> &);
}
```

4.2.12 Tent distribution

Class `tent_dist` provides random numbers with tent distribution with parameters m and d. This distribution is symmetric around m and its support is the interval $(m-d, m+d)$. The probability distribution function reads

parameters	$m, d \in \mathbb{R}$, $d > 0$
support	$(m-d, m+d)$
mean	m
variance	$d^2/6$

$$p(x|m,d) = \begin{cases} \dfrac{1+(x-m)/d}{d} & \text{for } m-d \le x \le m \\ \dfrac{1-(x-m)/d}{d} & \text{for } m \le x \le m+d \\ 0 & \text{else}\,. \end{cases}$$

Valid parameters for this distribution are $m, d \in \mathbb{R}$ with $d > 0$.

The class `tent_dist` is declared in the header file `trng/tent_dist.hpp` and its public interface is given as follows:

```
namespace trng {

  template<typename float_t = double>
  class tent_dist {
  public:
    using result_type = float_t;
    class param_type {
    public:
      result_type m() const;
      void m(result_type);
      result_type d() const;
      void d(result_type);
      param_type(result_type m, result_type d);
    };
    tent_dist(result_type m, result_type d);
    explicit tent_dist(const param_type &);
    void reset();
    template<typename R>
    result_type operator()(R &);
    template<typename R>
    result_type operator()(R &, const param_type &);
    result_type min() const;
    result_type max() const;
    const param_type & param() const;
    void param(const param_type &);
    result_type m() const;
    void m(result_type);
```

```
    result_type d() const;
    void d(result_type);
    result_type pdf(result_type) const;
    result_type cdf(result_type) const;
    result_type icdf(result_type) const;
  };

  template<typename float_t>
  bool operator==(const typename tent_dist<float_t>::param_type &,
  const typename tent_dist<float_t>::param_type &);
  template<typename float_t>
  bool operator!=(const typename tent_dist<float_t>::param_type &,
  const typename tent_dist<float_t>::param_type &);

  template<typename char_t, typename traits_t, typename float_t>
  std::basic_ostream<char_t, traits_t> &
  operator<<(std::basic_ostream<char_t, traits_t> &,
  const typename tent_dist<float_t>::param_type &);
  template<typename char_t, typename traits_t, typename float_t>
  std::basic_istream<char_t, traits_t> &
  operator>>(std::basic_istream<char_t, traits_t> &,
  typename tent_dist<float_t>::param_type &);

  template<typename float_t>
  bool operator==(const tent_dist<float_t> &, const tent_dist<float_t> &);
  template<typename float_t>
  bool operator!=(const tent_dist<float_t> &, const tent_dist<float_t> &);

  template<typename char_t, typename traits_t, typename float_t>
  std::basic_ostream<char_t, traits_t> &
  operator<<(std::basic_ostream<char_t, traits_t> &, const tent_dis<float_t>t &);
  template<typename char_t, typename traits_t, typename float_t>
  std::basic_istream<char_t, traits_t> &
  operator>>(std::basic_istream<char_t, traits_t> &, tent_dist<float_t> &);
}
```

4.2.13 Weibull distribution

Class `weibull_dist` provides random numbers with Weibull distribution with parameters β and θ. The probability distribution function reads

$$p(x|\theta,\beta) = \begin{cases} 0 & \text{for } x < \theta \\ \frac{\beta}{\theta}\left(\frac{x}{\theta}\right)^{\beta-1} \mathrm{e}^{-(x/\theta)^{\beta}} & \text{for } x \geq \theta\,. \end{cases}$$

parameters	$\beta, \theta \in (0, \infty)$
support	$(0, \infty)$
mean	$\theta\Gamma\left(1+\frac{1}{\beta}\right)$
variance	$\theta^2\left[\Gamma\left(1+\frac{2}{\beta}\right) - \Gamma^2\left(1+\frac{1}{\beta}\right)\right]$

Valid parameters for this distribution are $\theta, \beta \in \mathbb{R}$ with $\theta > 0$ and $\beta > 0$. For $\beta = 1$ Weibull distribution degenerates to an exponential distribution and for $\beta = 2$ and $\theta = \sqrt{2} \cdot \sigma$ this distribution is also known as Rayleigh distribution with parameter σ.

The class `weibull_dist` is declared in the header file `trng/weibull_dist.hpp` and its public interface is given as follows:

```
namespace trng {

  template<typename float_t = double>
  class weibull_dist {
  public:
    using result_type = float_t;
    class param_type {
    public:
      result_type theta() const;
      void theta(result_type);
      result_type beta() const;
      void beta(result_type);
      param_type(result_type theta, result_type beta);
    };
    weibull_dist(result_type theta, result_type beta);
    explicit weibull_dist(const param_type &);
    void reset();
    template<typename R>
    result_type operator()(R &);
    template<typename R>
    result_type operator()(R &, const param_type &);
    result_type min() const;
    result_type max() const;
    const param_type & param() const;
    void param(const param_type &);
    result_type beta() const;
    void beta(result_type);
    result_type theta() const;
    void theta(result_type);
    result_type pdf(result_type) const;
    result_type cdf(result_type) const;
    result_type icdf(result_type) const;
  };

  template<typename float_t>
  bool operator==(const typename weibull_dist<float_t>::param_type &,
  const typename weibull_dist<float_t>::param_type &);
  template<typename float_t>
  bool operator!=(const typename weibull_dist<float_t>::param_type &,
  const typename weibull_dist<float_t>::param_type &);

  template<typename char_t, typename traits_t, typename float_t>
  std::basic_ostream<char_t, traits_t> &
  operator<<(std::basic_ostream<char_t, traits_t> &,
  const typename weibull_dist<float_t>::param_type &);
  template<typename char_t, typename traits_t, typename float_t>
  std::basic_istream<char_t, traits_t> &
  operator>>(std::basic_istream<char_t, traits_t> &,
  typename weibull_dist<float_t>::param_type &);

  template<typename float_t>
```

```
  bool operator==(const weibull_dist<float_t> &, const weibull_dist<float_t> &);
  template<typename float_t>
  bool operator!=(const weibull_dist<float_t> &, const weibull_dist<float_t> &);

  template<typename char_t, typename traits_t, typename float_t>
  std::basic_ostream<char_t, traits_t> &
  operator<<(std::basic_ostream<char_t, traits_t> &, const weibull_dist<float_t> &);
  template<typename char_t, typename traits_t, typename float_t>
  std::basic_istream<char_t, traits_t> &
  operator>>(std::basic_istream<char_t, traits_t> &, weibull_dist<float_t> &);
}
```

4.2.14 Extreme value distribution

Class `extreme_value_dist` provides random numbers with extreme value distribution (also known as Gumbel distribution) with parameters θ and η. The probability distribution function reads

$$p(x|\theta,\eta) = \frac{1}{\theta}\exp\left(\frac{\eta-x}{\theta} - \exp\frac{\eta-x}{\theta}\right).$$

Valid parameters for this distribution are $\theta, \eta \in \mathbb{R}$ with $\theta > 0$.

parameters	$\theta, \eta \in \mathbb{R}, \theta > 0$
support	$(-\infty, \infty)$
mean	$\eta - \gamma\theta$
variance	$\pi^2\theta^2/6$

The symbol γ denotes the Euler-Mascheroni constant $\gamma = 0.57721\ldots$.

The class `extreme_value_dist` is declared in the header file `trng/extreme_value_dist.hpp` and its public interface is given as follows:

```
namespace trng {

  template<typename float_t = double>
  class extreme_value_dist {
  public:
    using result_type = float_t;
    class param_type {
    public:
      result_type theta() const;
      void theta(result_type);
      result_type eta() const;
      void eta(result_type);
      param_type(result_type theta, result_type eta);
    };
    extreme_value_dist(result_type theta, result_type eta);
    explicit extreme_value_dist(const param_type &);
    void reset();
    template<typename R>
    result_type operator()(R &);
    template<typename R>
    result_type operator()(R &, const param_type &);
    result_type min() const;
    result_type max() const;
    const param_type & param() const;
```

```
    void param(const param_type &);
    result_type theta() const;
    void theta(result_type);
    result_type eta() const;
    void eta(result_type);
    result_type pdf(result_type) const;
    result_type cdf(result_type) const;
    result_type icdf(result_type) const;
  };

  template<typename float_t>
  bool operator==(const typename extreme_value_dist<float_t>::param_type &,
  const typename extreme_value_dist<float_t>::param_type &);
  template<typename float_t>
  bool operator!=(const typename extreme_value_dist<float_t>::param_type &,
  const typename extreme_value_dist<float_t>::param_type &);

  template<typename char_t, typename traits_t, typename float_t>
  std::basic_ostream<char_t, traits_t> &
  operator<<(std::basic_ostream<char_t, traits_t> &,
  const typename extreme_value_dist<float_t>::param_type &);
  template<typename char_t, typename traits_t, typename float_t>
  std::basic_istream<char_t, traits_t> &
  operator>>(std::basic_istream<char_t, traits_t> &,
  typename extreme_value_dist<float_t>::param_type &);

  template<typename float_t>
  bool operator==(const extreme_value_dist<float_t> &, const extreme_value_dist<float_t> &);
  template<typename float_t>
  bool operator!=(const extreme_value_dist<float_t> &, const extreme_value_dist<float_t> &);

  template<typename char_t, typename traits_t, typename float_t>
  std::basic_ostream<char_t, traits_t> &
  operator<<(std::basic_ostream<char_t, traits_t> &, const extreme_value_dist<float_t> &);
  template<typename char_t, typename traits_t, typename float_t>
  std::basic_istream<char_t, traits_t> &
  operator>>(std::basic_istream<char_t, traits_t> &, extreme_value_dist<float_t> &);
}
```

Note that the definition of the extreme value distribution differs slightly from the one that has been introduced in C++11, see also section 6.4 and [30]. However, it is not difficult to switch from the C++ standard library to TRNG and vice versa. More precisely

```
  trng::extreme_value_dist<> D1(theta, eta);
  std::extreme_value_distribution<> D2(eta, -theta);
```

yield two equivalent distributions.

4.2.15 Γ-distribution

Class `gamma_dist` provides random numbers with Γ-distribution with parameters θ and κ. The probability distribution function reads

$$p(x|\theta,\kappa) = \begin{cases} 0 & \text{if } x < 0 \\ \dfrac{1}{\theta\Gamma(\kappa)} \left(\dfrac{x}{\theta}\right)^{\kappa-1} \mathrm{e}^{-x/\theta} & \text{if } x \geq 0\,. \end{cases}$$

parameters	$\kappa, \theta \in (0, \infty)$
support	$[0, \infty)$
mean	$\kappa\theta$
variance	$\kappa\theta^2$

Valid parameters for this distribution are $\kappa, \theta \in \mathbb{R}$ with $\kappa \geq 1$ and $\theta > 0$. Note, Γ-distribution is defined for arbitrary $\kappa \geq 0$, but class `gamma_dist` can handle only Γ-distributions with $\kappa \geq 1$ correctly. For $\kappa = 1$ the Γ-distribution degenerates to an exponential distribution.

The class `gamma_dist` is declared in the header file `trng/gamma_dist.hpp` and its public interface is given as follows:

```
namespace trng {

  template<typename float_t = double>
  class gamma_dist {
  public:
    using result_type = float_t;
    class param_type {
    public:
      result_type kappa() const;
      void kappa(result_type);
      result_type theta() const;
      void theta(result_type);
      param_type(result_type kappa, result_type theta);
    };
    gamma_dist(result_type kappa, result_type theta);
    explicit gamma_dist(const param_type &);
    void reset();
    template<typename R>
    result_type operator()(R &);
    template<typename R>
    result_type operator()(R &, const param_type &);
    result_type min() const;
    result_type max() const;
    const param_type & param() const;
    void param(const param_type &);
    result_type kappa() const;
    void kappa(result_type);
    result_type theta() const;
    void theta(result_type);
    result_type pdf(result_type) const;
    result_type cdf(result_type) const;
    result_type icdf(result_type) const;
  };

  template<typename float_t>
```

```
  bool operator==(const typename gamma_dist<float_t>::param_type &,
  const typename gamma_dist<float_t>::param_type &);
  template<typename float_t>
  bool operator!=(const typename gamma_dist<float_t>::param_type &,
  const typename gamma_dist<float_t>::param_type &);

  template<typename char_t, typename traits_t, typename float_t>
  std::basic_ostream<char_t, traits_t> &
  operator<<(std::basic_ostream<char_t, traits_t> &,
  const typename gamma_dist<float_t>::param_type &);
  template<typename char_t, typename traits_t, typename float_t>
  std::basic_istream<char_t, traits_t> &
  operator>>(std::basic_istream<char_t, traits_t> &,
  typename gamma_dist<float_t>::param_type &);

  template<typename float_t>
  bool operator==(const gamma_dist<float_t> &, const gamma_dist<float_t> &);
  template<typename float_t>
  bool operator!=(const gamma_dist<float_t> &, const gamma_dist<float_t> &);

  template<typename char_t, typename traits_t, typename float_t>
  std::basic_ostream<char_t, traits_t> &
  operator<<(std::basic_ostream<char_t, traits_t> &, const gamma_dist<float_t> &);
  template<typename char_t, typename traits_t, typename float_t>
  std::basic_istream<char_t, traits_t> &
  operator>>(std::basic_istream<char_t, traits_t> &, gamma_dist<float_t> &);
}
```

4.2.16 B-distribution

Class `beta_dist` provides random numbers with B-distribution with parameters α and β. The probability distribution function reads with the Beta function $\mathrm{B}(\alpha,\beta)$

parameters	$\alpha,\beta \in (0,\infty)$
support	$[0,1]$
mean	$\alpha/(\alpha+\beta)$
variance	$\alpha\beta/(\alpha+\beta+1)/(\alpha+\beta)^2$

$$p(x|\alpha,\beta) = \begin{cases} \dfrac{1}{\mathrm{B}(\alpha,\beta)} x^{\alpha-1}(1-x)^{\beta-1} & \text{if } x \geq 0 \text{ and } x \leq 1 \\ 0 & \text{else}. \end{cases}$$

Valid parameters for this distribution are $\alpha,\beta \in \mathbb{R}$ with $\alpha > 0$ and $\beta > 0$.

The class `beta_dist` is declared in the header file `trng/beta_dist.hpp` and its public interface is given as follows:

```
namespace trng {

  template<typename float_t = double>
  class beta_dist {
  public:
    using result_type = float_t;
    class param_type {
    public:
```

```
      result_type alpha() const;
      void alpha(result_type);
      result_type beta() const;
      void beta(result_type);
      param_type(result_type alpha, result_type beta);
    };
    beta_dist(result_type alpha, result_type beta);
    explicit beta_dist(const param_type &);
    void reset();
    template<typename R>
    result_type operator()(R &);
    template<typename R>
    result_type operator()(R &, const param_type &);
    result_type min() const;
    result_type max() const;
    const param_type & param() const;
    void param(const param_type &);
    result_type alpha() const;
    void alpha(result_type);
    result_type beta() const;
    void beta(result_type);
    result_type pdf(result_type) const;
    result_type cdf(result_type) const;
    result_type icdf(result_type) const;
  };

  template<typename float_t>
  bool operator==(const typename beta_dist<float_t>::param_type &,
  const typename beta_dist<float_t>::param_type &);
  template<typename float_t>
  bool operator!=(const typename beta_dist<float_t>::param_type &,
  const typename beta_dist<float_t>::param_type &);

  template<typename char_t, typename traits_t, typename float_t>
  std::basic_ostream<char_t, traits_t> &
  operator<<(std::basic_ostream<char_t, traits_t> &,
  const typename beta_dist<float_t>::param_type &);
  template<typename char_t, typename traits_t, typename float_t>
  std::basic_istream<char_t, traits_t> &
  operator>>(std::basic_istream<char_t, traits_t> &,
  typename beta_dist<float_t>::param_type &);

  template<typename float_t>
  bool operator==(const beta_dist<float_t> &, const beta_dist<float_t> &);
  template<typename float_t>
  bool operator!=(const beta_dist<float_t> &, const beta_dist<float_t> &);

  template<typename char_t, typename traits_t, typename float_t>
  std::basic_ostream<char_t, traits_t> &
  operator<<(std::basic_ostream<char_t, traits_t> &, const beta_dist<float_t> &);
  template<typename char_t, typename traits_t, typename float_t>
  std::basic_istream<char_t, traits_t> &
  operator>>(std::basic_istream<char_t, traits_t> &, beta_dist<float_t> &);
}
```

4.2.17 χ^2-distribution

Class `chi_square_dist` provides random numbers with χ^2-distribution with ν degrees of freedom. The probability distribution function reads

$$p(x|\nu) = \begin{cases} \dfrac{x^{\nu/2-1}\mathrm{e}^{-x/2}}{2^{\nu/2}\,\Gamma(\nu/2)} & \text{if } x \geq 0 \\ 0 & \text{if } x < 0\,. \end{cases}$$

parameter	$\nu \in \mathbb{N}$
support	$(0, \infty)$
mean	ν
variance	2ν

A valid parameter for this distribution is $\nu \in \mathbb{N}$ with $\nu \geq 1$. Note, the χ^2-distribution is a special case of the Γ-distribution with $\kappa = \nu/2$ and $\theta = 2$.

The class `chi_square_dist` is declared in the header file `trng/chi_square_dist.hpp` and its public interface is given as follows:

```
namespace trng {

  template<typename float_t = double>
  class chi_square_dist {
  public:
    using result_type = float_t;
    class param_type {
    public:
      int nu() const;
      void nu(int);
      explicit param_type(int nu);
    };
    explicit chi_square_dist(int nu);
    explicit chi_square_dist(const param_type &);
    void reset();
    template<typename R>
    result_type operator()(R &);
    template<typename R>
    result_type operator()(R &, const param_type &);
    result_type min() const;
    result_type max() const;
    const param_type & param() const;
    void param(const param_type &);
    int nu() const;
    void nu(int);
    result_type pdf(result_type) const;
    result_type cdf(result_type) const;
    result_type icdf(result_type) const;
  };

  template<typename float_t>
  bool operator==(const typename chi_square_dist<float_t>::param_type &,
  const typename chi_square_dist<float_t>::param_type &);
  template<typename float_t>
```

```
  bool operator!=(const typename chi_square_dist<float_t>::param_type &,
  const typename chi_square_dist<float_t>::param_type &);

  template<typename char_t, typename traits_t, typename float_t>
  std::basic_ostream<char_t, traits_t> &
  operator<<(std::basic_ostream<char_t, traits_t> &,
  const typename chi_square_dist<float_t>::param_type &);
  template<typename char_t, typename traits_t, typename float_t>
  std::basic_istream<char_t, traits_t> &
  operator>>(std::basic_istream<char_t, traits_t> &,
  typename chi_square_dist<float_t>::param_type &);

  template<typename float_t>
  bool operator==(const chi_square_dist<float_t> &, const chi_square_dist<float_t> &);
  template<typename float_t>
  bool operator!=(const chi_square_dist<float_t> &, const chi_square_dist<float_t> &);

  template<typename char_t, typename traits_t, typename float_t>
  std::basic_ostream<char_t, traits_t> &
  operator<<(std::basic_ostream<char_t, traits_t> &, const chi_square_dist<float_t> &);
  template<typename char_t, typename traits_t, typename float_t>
  std::basic_istream<char_t, traits_t> &
  operator>>(std::basic_istream<char_t, traits_t> &, chi_square_dis<float_t>t &);
}
```

4.2.18 Student-t distribution

parameter	$\nu \in \mathbb{N}$
support	$(-\infty, \infty)$
mean	0
variance	$\frac{\nu-1}{\nu-3}$

Class `student_t_dist` provides random numbers with Student-t distribution with ν degrees of freedom. The probability distribution function reads

$$p(x|\nu) = \frac{\Gamma(\frac{\nu+1}{2})}{\sqrt{\nu\pi}\,\Gamma(\frac{\nu}{2})} \left(1 + \frac{x^2}{\nu}\right)^{-(\frac{\nu+1}{2})} .$$

A valid parameter for this distribution is $\nu \in \mathbb{N}$ with $\nu \geq 1$.

The class `student_t_dist` is declared in the header file `trng/student_t_dist.hpp` and its public interface is given as follows:

```
namespace trng {

  template<typename float_t = double>
  class student_t_dist {
  public:
    using result_type = float_t;
    class param_type {
    public:
      int nu() const;
      void nu(int);
      explicit param_type(int nu);
    };
    explicit student_t_dist(int nu);
```

```
    explicit student_t_dist(const param_type &);
    void reset();
    template<typename R>
    result_type operator()(R &);
    template<typename R>
    result_type operator()(R &, const param_type &);
    result_type min() const;
    result_type max() const;
    const param_type & param() const;
    void param(const param_type &);
    int nu() const;
    void nu(int);
    result_type pdf(result_type) const;
    result_type cdf(result_type) const;
    result_type icdf(result_type) const;
  };

  template<typename float_t>
  bool operator==(const typename student_t_dist<float_t>::param_type &,
  const typename student_t_dist<float_t>::param_type &);
  template<typename float_t>
  bool operator!=(const typename student_t_dist<float_t>::param_type &,
  const typename student_t_dist<float_t>::param_type &);

  template<typename char_t, typename traits_t, typename float_t>
  std::basic_ostream<char_t, traits_t> &
  operator<<(std::basic_ostream<char_t, traits_t> &,
  const typename student_t_dist<float_t>::param_type &);
  template<typename char_t, typename traits_t, typename float_t>
  std::basic_istream<char_t, traits_t> &
  operator>>(std::basic_istream<char_t, traits_t> &,
  typename student_t_dist<float_t>::param_type &);

  template<typename float_t>
  bool operator==(const student_t_dist &, const student_t_dist<float_t> &);
  template<typename float_t>
  bool operator!=(const student_t_dist &, const student_t_dist<float_t> &);

  template<typename char_t, typename traits_t, typename float_t>
  std::basic_ostream<char_t, traits_t> &
  operator<<(std::basic_ostream<char_t, traits_t> &, const student_t_dist<float_t> &);
  template<typename char_t, typename traits_t, typename float_t>
  std::basic_istream<char_t, traits_t> &
  operator>>(std::basic_istream<char_t, traits_t> &, student_t_dist<float_t> &);
}
```

4.2.19 Snedecor-F distribution

Class snedecor_fsnedecor_f_dist provides random numbers with Snedecor-F distribution (or Fisher-Snedecor distribution) with parameters n and m. The probability distribution function reads

parameter	$n, m \in \mathbb{N}$
support	$[0, \infty)$
mean	$\frac{m}{m-2}$
variance	$\frac{2m^2(m+n-2)}{n(m-2)^2(m-4)}$

$$p(x|n,m) = \begin{cases} 0 & \text{if } x < 0 \\ \dfrac{\Gamma((n+m)/2)}{\Gamma(n/2)\Gamma(m/2)} \dfrac{n^{n/2} m^{m/2} x^{n/2-1}}{(m+nx)^{(n+m)/2}} & \text{if } x \geq 0\,. \end{cases}$$

Valid parameters for this distribution are $n, m \in \mathbb{N}$ with $n, m \geq 1$.

The class snedecor_f_dist is declared in the header file trng/snedecor_f_dist.hpp and its public interface is given as follows:

```
namespace trng {

  template<typename float_t = double>
  class snedecor_f_dist {
  public:
    using result_type = float_t;
    class param_type {
    public:
      int n() const;
      void n(int);
      int m() const;
      void m(int);
      param_type(int n, int m);
    };
    snedecor_f_dist(int n, int m);
    explicit snedecor_f_dist(const param_type &);
    void reset();
    template<typename R>
    result_type operator()(R &);
    template<typename R>
    result_type operator()(R &, const param_type &);
    result_type min() const;
    result_type max() const;
    const param_type & param() const;
    void param(const param_type &);
    int n() const;
    void n(int);
    int m() const;
    void m(int);
    result_type pdf(result_type) const;
    result_type cdf(result_type) const;
    result_type icdf(result_type) const;
  };

  template<typename float_t>
  bool operator==(const typename snedecor_f_dist<float_t>::param_type &,
  const typename snedecor_f_dist<float_t>::param_type &);
  template<typename float_t>
```

```
  bool operator!=(const typename snedecor_f_dist<float_t>::param_type &,
  const typename snedecor_f_dist<float_t>::param_type &);

  template<typename char_t, typename traits_t, typename float_t>
  std::basic_ostream<char_t, traits_t> &
  operator<<(std::basic_ostream<char_t, traits_t> &,
  const typename snedecor_f_dist<float_t>::param_type &);
  template<typename char_t, typename traits_t, typename float_t>
  std::basic_istream<char_t, traits_t> &
  operator>>(std::basic_istream<char_t, traits_t> &,
  typename snedecor_f_dist<float_t>::param_type &);

  template<typename float_t>
  bool operator==(const snedecor_f_dist<float_t> &, const snedecor_f_dist<float_t> &);
  template<typename float_t>
  bool operator!=(const snedecor_f_dist<float_t> &, const snedecor_f_dist<float_t> &);

  template<typename char_t, typename traits_t, typename float_t>
  std::basic_ostream<char_t, traits_t> &
  operator<<(std::basic_ostream<char_t, traits_t> &, const snedecor_f_dist<float_t> &);
  template<typename char_t, typename traits_t, typename float_t>
  std::basic_istream<char_t, traits_t> &
  operator>>(std::basic_istream<char_t, traits_t> &, snedecor_f_dist<float_t> &);
}
```

4.2.20 Rayleigh distribution

Class `rayleigh_dist` provides random numbers with Rayleigh distribution with parameter ν. The probability distribution function reads

$$p(x|\nu) = \begin{cases} 0 & \text{if } x \le 0 \\ \dfrac{x}{\nu^2} \mathrm{e}^{-x^2/(2\nu^2)} & \text{if } x > 0\,. \end{cases}$$

parameter	$\nu \in (0, \infty)$
support	$(0, \infty)$
mean	$\nu\sqrt{\pi/2}$
variance	$(4-\pi)\nu^2/2$

A valid parameter for this distribution is $\nu > 0$.

The class `rayleigh_dist` is declared in the header file `trng/rayleigh_dist.hpp` and its public interface is given as follows:

```
namespace trng {

  template<typename float_t = double>
  class rayleigh_dist {
  public:
    using result_type = float_t;
    class param_type {
    public:
      result_type nu() const;
      void nu(result_type nu_new);
      explicit param_type(result_type nu);
    };
```

```
    explicit rayleigh_dist(result_type nu);
    explicit rayleigh_dist(const param_type &);
    void reset();
    template<typename R>
    result_type operator()(R &);
    template<typename R>
    result_type operator()(R &, const param_type &);
    result_type min() const;
    result_type max() const;
    param_type param() const { return p; }
    void param(const param_type &);
    result_type nu() const;
    void nu(result_type);
    result_type pdf(result_type x) const;
    result_type cdf(result_type x) const;
    result_type icdf(result_type x) const;
  };

  template<typename float_t>
  bool operator==(const typename rayleigh_dist<float_t>::param_type &,
  const typename rayleigh_dist<float_t>::param_type &);
  template<typename float_t>
  bool operator!=(const typename rayleigh_dist<float_t>::param_type &,
  const typename rayleigh_dist<float_t>::param_type &);

  template<typename char_t, typename traits_t, typename float_t>
  std::basic_ostream<char_t, traits_t> &
  operator<<(std::basic_ostream<char_t, traits_t> &,
  const typename rayleigh_dist<float_t>::param_type &);
  template<typename char_t, typename traits_t, typename float_t>
  std::basic_istream<char_t, traits_t> &
  operator>>(std::basic_istream<char_t, traits_t> &,
  typename rayleigh_dist<float_t>::param_type &);

  template<typename float_t>
  bool operator==(const rayleigh_dist<float_t> &, const rayleigh_dist<float_t> &);
  template<typename float_t>
  bool operator!=(const rayleigh_dist<float_t> &, const rayleigh_dist<float_t> &);

  template<typename char_t, typename traits_t, typename float_t>
  std::basic_ostream<char_t, traits_t> &
  operator<<(std::basic_ostream<char_t, traits_t> &, const rayleigh_dist<float_t> &);
  template<typename char_t, typename traits_t, typename float_t>
  std::basic_istream<char_t, traits_t> &
  operator>>(std::basic_istream<char_t, traits_t> &, rayleigh_dist<float_t> &);
}
```

4.2.21 Bernoulli distribution

The template class `bernoulli_dist` provides random objects with Bernoulli distribution with parameter p. The probability distribution function reads

$$P(x|p) = \begin{cases} p & \text{if } x = 0 \text{ (head)} \\ 1-p & \text{if } x = 1 \text{ (tail)} \\ 0 & \text{else}\,. \end{cases}$$

parameter	$p \in [0,1]$
support	$0, 1$
mean	$p/2$
variance	$p^2/12$

A valid parameter for this distribution is $p \in [0,1]$. In contrast to other random distribution classes any default-constructible type (not only floating point types) may be utilized for the template parameter `T`.

The class `bernoulli_dist` is declared in the header file `trng/bernoulli_dist.hpp` and its public interface is given as follows:

```
namespace trng {

  template<typename T>
  class bernoulli_dist {
  public:
    typedef T result_type;

    class param_type {
    public:
      double p() const;
      void p(double);
      T head() const;
      void head(const T &);
      T tail() const;
      void tail(const T &);
      param_type(double p, const T &head, const T &tail);
    };
```

The one-parameter constructor `bernoulli_dist(double p)` initializes "head" to 0 (or false if `T` is `bool`) and "tail" to 1 (or true if `T` is `bool`) if `T` is an arithmetic type, i. e., either a floating point type, an integer type or `bool`. Using the one-parameter constructor with a non-arithmetic type `T` leads to compile-time errors.

```
    explicit bernoulli_dist(double p);
    explicit bernoulli_dist(double p, const T &head, const T &tail);
    explicit bernoulli_dist(const param_type &);
    void reset();
    template<typename R>
    T operator()(R &);
    template<typename R>
    T operator()(R &, const param_type &);
```

Method `min` returns "head" and method `max` returns "tail".

```
    T min() const;
    T max() const;
```

```
    const param_type & param() const;
    void param(const param_type &);
    double p() const;
    void p(double);
    T head() const;
    void head(const T &);
    T tail() const;
    void tail(const T &);
```

Method pdf will return p if its argument is "head", $1 - p$ if its argument is "tail" and 0 otherwise.

```
    double pdf(const T &) const;
```

Method cdf will return p if its argument is "head", 1 if its argument is "tail" and 0 otherwise.

```
    double cdf(const T &) const;
  };

  template<typename T>
  bool operator==(const typename bernoulli_dist<T>::param_type &,
  const typename bernoulli_dist<T>::param_type &);
  template<typename T>
  bool operator!=(const typename bernoulli_dist<T>::param_type &,
  const typename bernoulli_dist<T>::param_type &);

  template<typename char_t, typename traits_t, typename T>
  std::basic_ostream<char_t, traits_t> &
  operator<<(std::basic_ostream<char_t, traits_t> &,
  const typename bernoulli_dist<T>::param_type &);
  template<typename char_t, typename traits_t, typename T>
  std::basic_istream<char_t, traits_t> &
  operator>>(std::basic_istream<char_t, traits_t> &,
  typename bernoulli_dist<T>::param_type &);

  template<typename T>
  bool operator==(const bernoulli_dist<T> &, const bernoulli_dist<T> &);
  template<typename T>
  bool operator!=(const bernoulli_dist<T> &, const bernoulli_dist<T> &);

  template<typename char_t, typename traits_t, typename T>
  std::basic_ostream<char_t, traits_t> &
  operator<<(std::basic_ostream<char_t, traits_t> &, const bernoulli_dist<T> &);
  template<typename char_t, typename traits_t, typename T>
  std::basic_istream<char_t, traits_t> &
  operator>>(std::basic_istream<char_t, traits_t> &, bernoulli_dist<T> &);
}
```

Listing 4.2: Class bernoulli_dist in action.

```
1 #include <cstdlib>
2 #include <iostream>
3 #include <iomanip>
```

```
4  #include <vector>
5  #include <trng/lcg64.hpp>
6  #include <trng/bernoulli_dist.hpp>
7
8  enum class coin { head = 0, tail = 1 };
9
10 int main() {
11   // discrete distribution object
12   trng::bernoulli_dist<coin> biased_coin(0.51, coin::head, coin::tail);
13   // random number generator
14   trng::lcg64 r;
15   // draw some random numbers
16   std::vector<int> count(2, 0);
17   const int samples{100000};
18   for (int i = 0; i < samples; ++i) {
19     const coin x{biased_coin(r)};      // draw a random number
20     ++count[x == coin::head ? 0 : 1];  // count
21   }
22   // print results
23   std::cout << "value\t\tprobability\tcount\t\tempirical probability\n"
24             << "=====\t\t==========\t=====\t\t====================\n";
25   for (std::vector<int>::size_type i = 0; i < count.size(); ++i)
26     std::cout << std::setprecision(3) << i << "\t\t" << biased_coin.pdf(static_cast<coin>(i))
27               << "\t\t" << count[i] << "\t\t" << static_cast<double>(count[i]) / samples
28               << '\n';
29   return EXIT_SUCCESS;
30 }
```

4.2.22 Binomial distribution

Class `binomial_dist` provides random integers with binomial distribution with parameters p and n. The probability distribution function reads

$$P(x|p,n) = \begin{cases} \binom{n}{x} p^x (1-p)^{n-x} & \text{if } x \in \{0,1,\ldots,n\} \\ 0 & \text{else}. \end{cases}$$

parameters	$p \in [0,1]$, $n \in \mathbb{N}$
support	$0, 1, \ldots, n$
mean	np
variance	$np(1-p)$

Valid parameters for this distribution are $p \in [0,1]$ and $n \in \mathbb{N}$.

The class `binomial_dist` is declared in the header file `trng/binomial_dist.hpp` and its public interface is given as follows:

```
namespace trng {

  class binomial_dist {
  public:
    typedef int result_type;

    class param_type {
    public:
      double p() const;
```

```
      void p(double);
      int n() const;
      void n(int);
      param_type(double p, int n);
    };

    binomial_dist(double p, int n);
    explicit binomial_dist(const param_type &);
    void reset();
    template<typename R>
    int operator()(R &);
    template<typename R>
    int operator()(R &, const param_type &);
    int min() const;
    int max() const;
    const param_type & param() const;
    void param(const param_type &);
    double p() const;
    void p(double);
    int n() const;
    void n(int);
    double pdf(int) const;
    double cdf(int) const;
  };

  bool operator==(const binomial_dist::param_type &, const binomial_dist::param_type &);
  bool operator!=(const binomial_dist::param_type &, const binomial_dist::param_type &);

  template<typename char_t, typename traits_t>
  std::basic_ostream<char_t, traits_t> &
  operator<<(std::basic_ostream<char_t, traits_t> &, const binomial_dist::param_type &);
  template<typename char_t, typename traits_t>
  std::basic_istream<char_t, traits_t> &
  operator>>(std::basic_istream<char_t, traits_t> &, binomial_dist::param_type &);

  bool operator==(const binomial_dist &, const binomial_dist &);
  bool operator!=(const binomial_dist &, const binomial_dist &);

  template<typename char_t, typename traits_t>
  std::basic_ostream<char_t, traits_t> &
  operator<<(std::basic_ostream<char_t, traits_t> &, const binomial_dist &);
  template<typename char_t, typename traits_t>
  std::basic_istream<char_t, traits_t> &
  operator>>(std::basic_istream<char_t, traits_t> &, binomial_dist &);
}
```

4.2.23 Negative binomial distribution

Class `negative_binomial_dist` provides random integers with negative binomial distribution with parameters p and r. This distribution is also known as gamma–Poisson (mixture) distribution. The probabil-

parameters	$p \in [0,1], r \in \mathbb{N}$
support	$0, 1, \ldots$
mean	$r(1-p)/p$
variance	$r(1-p)/p^2$

ity distribution function reads

$$P(x|p,r) = \begin{cases} \dfrac{\Gamma(r+x)}{x!\Gamma(r)} p^r (1-p)^x & \text{if } x \in \{0,1,\dots\} \\ 0 & \text{else}\,. \end{cases}$$

Valid parameters for this distribution are $p \in [0,1]$ and $r \in (0,\infty)$.

The class `negative_binomial_dist` is declared in the header file `trng/negative_binomial_dist.hpp` and its public interface is given as follows:

```
namespace trng {

  class negative_binomial_dist {
  public:
    typedef int result_type;

    class param_type {
    public:
      double p() const;
      void p(double);
      int r() const;
      void r(int);
      param_type(double p, double r);
    };

    negative_binomial_dist(double p, double r);
    explicit negative_binomial_dist(const param_type &);
    void reset();
    template<typename R>
    int operator()(R &);
    template<typename R>
    int operator()(R &, const param_type &);
    int min() const;
    int max() const;
    const param_type & param() const;
    void param(const param_type &);
    double p() const;
    void p(double);
    double r() const;
    void r(double);
    double pdf(int) const;
    double cdf(int) const;
  };

  bool operator==(const negative_binomial_dist::param_type &,
  const negative_binomial_dist::param_type &);
  bool operator!=(const negative_binomial_dist::param_type &,
  const negative_binomial_dist::param_type &);

  template<typename char_t, typename traits_t>
  std::basic_ostream<char_t, traits_t> &
  operator<<(std::basic_ostream<char_t, traits_t> &, const negative_binomial_dist::param_type
&);
```

```
  template<typename char_t, typename traits_t>
  std::basic_istream<char_t, traits_t> &
  operator>>(std::basic_istream<char_t, traits_t> &, negative_binomial_dist::param_type &);

  bool operator==(const negative_binomial_dist &, const negative_binomial_dist &);
  bool operator!=(const negative_binomial_dist &, const negative_binomial_dist &);

  template<typename char_t, typename traits_t>
  std::basic_ostream<char_t, traits_t> &
  operator<<(std::basic_ostream<char_t, traits_t> &, const negative_binomial_dist &);
  template<typename char_t, typename traits_t>
  std::basic_istream<char_t, traits_t> &
  operator>>(std::basic_istream<char_t, traits_t> &, negative_binomial_dist &);
}
```

4.2.24 Hypergeometric distribution

Class `hypergeometric_dist` provides random integers with hypergeometric distribution with parameters n, m and d. The probability distribution function reads

parameters	$n \in \mathbb{N}$, $m \in \{0,1,\ldots,n\}$, $d \in \{1,2,\ldots,n\}$
support	$\max(0, d-n+m), \ldots, \min(d,m)$
mean	dm/n
variance	$d\frac{m}{n}\left(1-\frac{m}{n}\right)\frac{n-d}{n-1}$

$$P(x|n,m,d) = \begin{cases} \dfrac{\binom{m}{x}\binom{n-m}{d-x}}{\binom{n}{d}} & \text{if } x \in \{\max(0,d-n+m),\ldots,\min(d,m)\}\,, \\ 0 & \text{else}\,. \end{cases}$$

Valid parameters for this distribution are $n \in \mathbb{N}$, $m \in \{0,1,\ldots,n\}$, and $d \in \{1,2,\ldots,n\}$,

The class `hypergeometric_dist` is declared in the header file `trng/hypergeometric_dist.hpp` and its public interface is given as follows:

```
namespace trng {

  class hypergeometric_dist {
  public:
    typedef int result_type;

    class param_type {
    public:
      int n() const;
      void n(int);
      int m() const;
      void m(int);
      int d() const;
      void d(int);
      param_type(int n, int m, int d);
    };
```

```
    hypergeometric_dist(double n, int m, int d);
    explicit hypergeometric_dist(const param_type &);
    void reset();
    template<typename R>
    int operator()(R &);
    template<typename R>
    int operator()(R &, const param_type &);
    int min() const;
    int max() const;
    const param_type & param() const;
    void param(const param_type &);
    int n() const;
    void n(int);
    int m() const;
    void m(int);
    int d() const;
    void d(int);
    double pdf(int) const;
    double cdf(int) const;
  };

  bool operator==(const hypergeometric_dist::param_type &,
  const hypergeometric_dist::param_type &);
  bool operator!=(const hypergeometric_dist::param_type &,
  const hypergeometric_dist::param_type &);

  template<typename char_t, typename traits_t>
  std::basic_ostream<char_t, traits_t> &
  operator<<(std::basic_ostream<char_t, traits_t> &, const hypergeometric_dist::param_type &);
  template<typename char_t, typename traits_t>
  std::basic_istream<char_t, traits_t> &
  operator>>(std::basic_istream<char_t, traits_t> &, hypergeometric_dist::param_type &);

  bool operator==(const hypergeometric_dist &, const hypergeometric_dist &);
  bool operator!=(const hypergeometric_dist &, const hypergeometric_dist &);

  template<typename char_t, typename traits_t>
  std::basic_ostream<char_t, traits_t> &
  operator<<(std::basic_ostream<char_t, traits_t> &, const hypergeometric_dist &);
  template<typename char_t, typename traits_t>
  std::basic_istream<char_t, traits_t> &
  operator>>(std::basic_istream<char_t, traits_t> &, hypergeometric_dist &);
}
```

4.2.25 Geometric distribution

Class `geometric_dist` provides random integers with geometric distribution with parameter p. The probability distribution function reads

$$P(x|p) = p(1-p)^x \qquad \text{for } x \in \{0, 1, \dots\}.$$

parameter	$p \in (0,1)$
support	$0, 1, \dots$
mean	$(1-p)/p$
variance	$(1-p)/p^2$

A valid parameter p is $p \in (0,1)$.

The class `geometric_dist` is declared in the header file `trng/geometric_dist.hpp` and its public interface is given as follows:

```
namespace trng {

  class geometric_dist {
  public:
    typedef int result_type;

    class param_type {
    public:
      double p() const;
      void p(double);
      explicit param_type(double p);
    };

    explicit geometric_dist(double p);
    explicit geometric_dist(const param_type &);
    void reset();
    template<typename R>
    int operator()(R &);
    template<typename R>
    int operator()(R &, const param_type &);
    int min() const;
    int max() const;
    const param_type & param() const;
    void param(const param_type &);
    double p() const;
    void p(double);
    double pdf(int) const;
    double cdf(int) const;
  };

  bool operator==(const geometric_dist::param_type &, const geometric_dist::param_type &);
  bool operator!=(const geometric_dist::param_type &, const geometric_dist::param_type &);

  template<typename char_t, typename traits_t>
  std::basic_ostream<char_t, traits_t> &
  operator<<(std::basic_ostream<char_t, traits_t> &, const geometric_dist::param_type &);
  template<typename char_t, typename traits_t>
  std::basic_istream<char_t, traits_t> &
  operator>>(std::basic_istream<char_t, traits_t> &, geometric_dist::param_type &);

  bool operator==(const geometric_dist &, const geometric_dist &);
```

```
  bool operator!=(const geometric_dist &, const geometric_dist &);

  template<typename char_t, typename traits_t>
  std::basic_ostream<char_t, traits_t> &
  operator<<(std::basic_ostream<char_t, traits_t> &, const geometric_dist &);
  template<typename char_t, typename traits_t>
  std::basic_istream<char_t, traits_t> &
  operator>>(std::basic_istream<char_t, traits_t> &, geometric_dist &);
}
```

4.2.26 Poisson distribution

Class `poisson_dist` provides random integers with Poisson distribution with mean μ. The probability distribution function reads

$$P(x|\mu) = \frac{\mathrm{e}^{-\mu}\mu^x}{x!} \qquad \text{for } x \in \{0, 1, \dots\}.$$

parameter	$\mu \in [0, \infty)$
support	$0, 1, \dots$
mean	μ
variance	μ

A valid parameter μ is $\mu \in [0, \infty)$.

The class `poisson_dist` is declared in the header file `trng/poisson_dist.hpp` and its public interface is given as follows:

```
namespace trng {

  class poisson_dist {
  public:
    typedef int result_type;

    class param_type {
    public:
      double mu() const;
      void mu(double);
      explicit param_type(double mu);
    };

    explicit poisson_dist(double mu);
    explicit poisson_dist(const param_type &);
    void reset();
    template<typename R>
    int operator()(R &);
    template<typename R>
    int operator()(R &, const param_type &);
    int min() const;
    int max() const;
    const param_type & param() const;
    void param(const param_type &);
    double mu() const;
    void mu(double);
    double pdf(int) const;
    double cdf(int) const;
  };
```

```
  bool operator==(const poisson_dist::param_type &, const poisson_dist::param_type &);
  bool operator!=(const poisson_dist::param_type &, const poisson_dist::param_type &);

  template<typename char_t, typename traits_t>
  std::basic_ostream<char_t, traits_t> &
  operator<<(std::basic_ostream<char_t, traits_t> &, const poisson_dist::param_type &);
  template<typename char_t, typename traits_t>
  std::basic_istream<char_t, traits_t> &
  operator>>(std::basic_istream<char_t, traits_t> &, poisson_dist::param_type &);

  bool operator==(const poisson_dist &, const poisson_dist &);
  bool operator!=(const poisson_dist &, const poisson_dist &);

  template<typename char_t, typename traits_t>
  std::basic_ostream<char_t, traits_t> &
  operator<<(std::basic_ostream<char_t, traits_t> &, const poisson_dist &);
  template<typename char_t, typename traits_t>
  std::basic_istream<char_t, traits_t> &
  operator>>(std::basic_istream<char_t, traits_t> &, poisson_dist &);
}
```

4.2.27 Zero-truncated Poisson distribution

parameter	$\mu \in [0, \infty)$
support	$1, 2, \ldots$
mean	$\frac{\mu e^{\mu}}{1-e^{\mu}}$
variance	$\frac{\mu e^{\mu}}{1-e^{\mu}} \left(1 - \frac{\mu}{1-e^{\mu}}\right)$

Class `zero_truncated_poisson_dist` provides random integers with zero-truncated Poisson distribution (also known as the conditional Poisson distribution or the positive Poisson distribution) with parameter μ. It is the conditional probability distribution of a Poisson-distributed random variable, given that the value of the random variable is not zero. The probability distribution function reads

$$P(x|\mu) = \frac{e^{-\mu}\mu^x}{x!(1-e^{-\mu})} \qquad \text{for } x \in \{1, 2, \ldots\}.$$

A valid parameter μ is $\mu \in [0, \infty)$.

The class `zero_truncated_poisson_dist` is declared in the header file `trng/zero_truncated_poisson_dist.hpp` and its public interface is given as follows:

```
namespace trng {

  class zero_truncated_poisson_dist {
  public:
    typedef int result_type;

    class param_type {
    public:
      double mu() const;
      void mu(double);
      explicit param_type(double mu);
```

```
    };

    explicit zero_truncated_poisson_dist(double mu);
    explicit zero_truncated_poisson_dist(const param_type &);
    void reset();
    template<typename R>
    int operator()(R &);
    template<typename R>
    int operator()(R &, const param_type &);
    int min() const;
    int max() const;
    const param_type & param() const;
    void param(const param_type &);
    double mu() const;
    void mu(double);
    double pdf(int) const;
    double cdf(int) const;
  };

  bool operator==(const zero_truncated_poisson_dist::param_type &,
  const zero_truncated_poisson_dist::param_type &);
  bool operator!=(const zero_truncated_poisson_dist::param_type &,
  const zero_truncated_poisson_dist::param_type &);

  template<typename char_t, typename traits_t>
  std::basic_ostream<char_t, traits_t> &
  operator<<(std::basic_ostream<char_t, traits_t> &, const zero_truncated_poisson_dist::param
_type &);
  template<typename char_t, typename traits_t>
  std::basic_istream<char_t, traits_t> &
  operator>>(std::basic_istream<char_t, traits_t> &, zero_truncated_poisson_dist::param_type &);

  bool operator==(const zero_truncated_poisson_dist &, const zero_truncated_poisson_dist &);
  bool operator!=(const zero_truncated_poisson_dist &, const zero_truncated_poisson_dist &);

  template<typename char_t, typename traits_t>
  std::basic_ostream<char_t, traits_t> &
  operator<<(std::basic_ostream<char_t, traits_t> &, const zero_truncated_poisson_dist &);
  template<typename char_t, typename traits_t>
  std::basic_istream<char_t, traits_t> &
  operator>>(std::basic_istream<char_t, traits_t> &, zero_truncated_poisson_dist &);
}
```

4.2.28 Discrete distribution

The general probability distribution function for integers in $[0, 1, \ldots, n-1]$ is determined by a set of n non-negative weights p_i ($i = 0, 1, \ldots, n-1$) and reads

$$P(x|\{p_i\}) = \frac{p_x}{\sum_{i=0}^{n-1} p_i} \qquad \text{for } x \in \{0, 1, \ldots, n-1\}.$$

TRNG provides two classes for the generation of random integers with a general discrete distribution, class `discrete_dist` and `fast_discrete_dist`. Both classes provide basically the same interface but they are implemented by different internal data structures and feature different performance characteristics.

The classes `discrete_dist` and `fast_discrete_dist` have several different constructors. The constructor `discrete_dist(int n)` (`fast_discrete_dist(int n)`) sets up a flat distribution of n integers, each integer has the same statistical weight. Another way to construct an object of the class `discrete_dist` (`fast_discrete_dist`) is to pass the weights p_i to the constructor `discrete_dist(iter first, iter last);` (`fast_discrete_dist(iter first, iter last);`) by some iterator range.

Drawing a random number from a general discrete distribution is a $\mathcal{O}(\log n)$ operation for `discrete_dist`, while `fast_discrete_dist` is able to carryout this operation in constant time. For small n the performance difference is negligible, but for large n ($n \gtrapprox 1\,000$) becomes more and more important and therefore `fast_discrete_dist` will be used in most cases.

The method `param(int, double)` allows to change relative probability of a single relative probability p_i after an object of the type `discrete_dist` has been constructed. This will cause an update of the internal data structures that costs $\mathcal{O}(\log n)$ operation. Note that `fast_discrete_dist` does not allow to change relative probabilities and does not provide a method `param(int, double)`. This is the price we have to pay for performance.

The class `discrete_dist` is declared in the header file `trng/discrete_dist.hpp` and its public interface is given as follows:

```
namespace trng {

  class discrete_dist {
  public:
    typedef int result_type;
    class param_type {
    public:
      template<typename iter>
      explicit param_type(iter first, iter last);
    };

    discrete_dist(int n);
    template<typename iter>
    discrete_dist(iter first, iter last);
    explicit discrete_dist(const param_type &);
    void reset();
    template<typename R>
    int operator()(R &);
    template<typename R>
    int operator()(R &, const param_type &);
    int min() const;
    int max() const;
    const param_type & param() const;
    void param(const param_type &);
    void param(int, double);
```

```
    double pdf(int) const;
    double cdf(int) const;
  };

  bool operator==(const discrete_dist::param_type &, const discrete_dist::param_type &);
  bool operator!=(const discrete_dist::param_type &, const discrete_dist::param_type &);

  template<typename char_t, typename traits_t>
  std::basic_ostream<char_t, traits_t> &
  operator<<(std::basic_ostream<char_t, traits_t> &, const discrete_dist::param_type &);
  template<typename char_t, typename traits_t>
  std::basic_istream<char_t, traits_t> &
  operator>>(std::basic_istream<char_t, traits_t> &, discrete_dist::param_type &);

  bool operator==(const discrete_dist &, const discrete_dist &);
  bool operator!=(const discrete_dist &, const discrete_dist &);

  template<typename char_t, typename traits_t>
  std::basic_ostream<char_t, traits_t> &
  operator<<(std::basic_ostream<char_t, traits_t> &, const discrete_dist &);
  template<typename char_t, typename traits_t>
  std::basic_istream<char_t, traits_t> &
  operator>>(std::basic_istream<char_t, traits_t> &, discrete_dist &);

}
```

The files `discrete_dist.cc` (see Listing 4.3) and `discrete_dist_c_style.cc` in the TRNG source distribution demonstrate the usage of the class `discrete_dist` in detail.

The class `fast_discrete_dist` is declared in the header file `trng/fast_discrete_dist.hpp` and its public interface is given as follows:

```
namespace trng {

  class fast_discrete_dist {
  public:
    typedef int result_type;
    class param_type {
    public:
      template<typename iter>
      explicit param_type(iter first, iter last);
    };

    fast_discrete_dist(int n);
    template<typename iter>
    fast_discrete_dist(iter first, iter last);
    explicit fast_discrete_dist(const param_type &);
    void reset();
    template<typename R>
    int operator()(R &);
    template<typename R>
    int operator()(R &, const param_type &);
    int min() const;
    int max() const;
    const param_type & param() const;
```

```
    void param(const param_type &);
    double pdf(int) const;
    double cdf(int) const;
  };

  bool operator==(const fast_discrete_dist::param_type &,
  const fast_discrete_dist::param_type &);
  bool operator!=(const fast_discrete_dist::param_type &,
  const fast_discrete_dist::param_type &);

  template<typename char_t, typename traits_t>
  std::basic_ostream<char_t, traits_t> &
  operator<<(std::basic_ostream<char_t, traits_t> &, const fast_discrete_dist::param_type &);
  template<typename char_t, typename traits_t>
  std::basic_istream<char_t, traits_t> &
  operator>>(std::basic_istream<char_t, traits_t> &, fast_discrete_dist::param_type &);

  bool operator==(const fast_discrete_dist &, const fast_discrete_dist &);
  bool operator!=(const fast_discrete_dist &, const fast_discrete_dist &);

  template<typename char_t, typename traits_t>
  std::basic_ostream<char_t, traits_t> &
  operator<<(std::basic_ostream<char_t, traits_t> &, const fast_discrete_dist &);
  template<typename char_t, typename traits_t>
  std::basic_istream<char_t, traits_t> &
  operator>>(std::basic_istream<char_t, traits_t> &, fast_discrete_dist &);

}
```

4.3 Function template `generate_canonical`

In this section we describe a function template introduced by [12]. Each function instantiated from the template `generate_canonical` maps the result of a single invocation of a supplied uniform random number generator to one member of the set $\mathcal{L}$ (described below) such that, if the values produced by the generator are uniformly distributed, the results of the instantiation are distributed as uniformly as possible according to the uniformity requirements described below.

Let $\mathcal{L}$ consist of all values t of type `result_type` such that:

- If `result_type` is a floating-point type, `result_type(0)` $< t <$ `result_type(1)`.
- If `result_type` is a signed or an unsigned integral type, then the value t lays in the range

  `numeric_limits<result_type>::min()` $\leq t \leq$ `numeric_limits<result_type>::max()`.

Obtaining a value in $\mathcal{L}$ can be a useful step in the process of transforming a value generated by a uniform random number generator into a value that can be delivered by a random number distribution. The function template

```
template<class result_type, class UniformRandomNumberGenerator>
result_type generate_canonical(UniformRandomNumberGenerator &g);
```

Listing 4.3: Class `discrete_dist` in action.

```
#include <cstdlib>
#include <iostream>
#include <iomanip>
#include <vector>
#include <trng/lcg64.hpp>
#include <trng/discrete_dist.hpp>

int main() {
  // stores relative probabilities
  const std::vector<double> p{1., 3.25, 5., 6.5, 7., 2.};
  // discrete distribution object
  trng::discrete_dist dist(p.begin(), p.end());
  // random number generator
  trng::lcg64 r;
  // draw some random numbers
  std::vector<int> count(p.size(), 0);
  const int samples{10000};
  for (int i{0}; i < samples; ++i) {
    const int x{dist(r)};  // draw a random number
    ++count[x];            // count
  }
  // print results
  std::cout << "value\t\tprobability\tcount\t\tempirical probability\n"
            << "=====\t\t==========\t=====\t\t====================\n";
  for (std::vector<int>::size_type i{0}; i < count.size(); ++i) {
    std::cout << std::setprecision(3) << i << "\t\t" << dist.pdf(static_cast<int>(i)) << "\t\t"
              << count[i] << "\t\t" << static_cast<double>(count[i]) / samples << '\n';
  }
  return EXIT_SUCCESS;
}
```

returns a value from $\mathcal{L}$ by exactly one invocation of g, see [12] for details.

4.4 CUDA and HIP support

TRNG may be utilized in parallel Monte Carlo simulations. It does not depend on a specific parallelization technique, e. g., POSIX threads, MPI or others. TRNG also supports CUDA. CUDA is a parallel architecture and programming model for general purpose computations on graphics processing units (GPUs). GPU computing is enabled by the CUDA programming model that provides a set of abstractions that enable to express data parallelism and task parallelism. This programming model is implemented by equipping the sequential C++ programming language with extensions for parallel execution of so-called kernel functions on a GPU and providing an application programming interface. GPU kernel functions are implemented by a subset of the C++ programming language. See the [3, 32] for details.

Besides CUDA, TRNG can also be used with the HIP programming model for AMD

GPUs. The header file `trng/cuda.hpp` defines the macro `TRNG_CUDA_ENABLE`, which expands to the appropriate `__host__ __device__` function qualifiers if the code is compiled by a CUDA or a HIP compiler and to nothing otherwise. Everything that is said about CUDA in this section applies to HIP, too. Listing 6.6 has a HIP counterpart in the source tree, see `examples/pi_block_hip.hip`. It is built if CMake finds a HIP compiler.

Because there are some C++ features that can not be used in GPU functions not all TRNG classes and functions can be utilized in GPU code. For example, only the engines `lcg64`, `lcg64_shift`, `lcg64_count_shift`, `count128_lcg_shift`, `mrg`*n*, `mrg`*n*`s`, `yarn`*n*, `yarn`*n*`s`, `xoshiro256plus`, `mwc64x32`, `mwc128x32`, `mwc128x` and `mwc192x` may be used in GPU code, see Table 4.1. The Mersenne twister engines and the lagged Fibonacci engines as well as `trng::minstd` have no CUDA support. One may call the methods `jump` and `jump2`, the method `split` (if the engine provides it, which is the case for parallel random number engines only) or one of the call-operators of a supported random number engine. Other random number engine methods are not callable from GPU code, not even the constructor. Thus, a random number engine instance has to be constructed in CPU code and later to be copied to the GPU before it may be used on the GPU, see Listing 6.6 for an example.

The function template `generate_canonical` and random number distributions may be used for GPU code in the same way as in CPU code without any restrictions. Except the following distributions: `correlated_normal_dist`, `binomial_dist`, `hypergeometric_dist`, `negative_binomial_dist`, `poisson_dist`, `zero_truncated_poisson_dist`, `discrete_dist` and `fast_discrete_dist`, they provide no CUDA support at all. These restrictions might be lifted in future TRNG releases.

5 Installation

5.1 Prerequisites

To make the installation procedure portable and comfortable, TRNG utilizes the CMake build configuration generator. For a proper installation you will need

- CMake version 3.30 or later,
- a recent C++ compiler that implements the C++11 language standard and
- a make tool or an integrated environment with cmake support, e. g., Microsoft Visual Studio, Clion, Xcode or Eclipse.

TRNG comes with numerous sample programs that illustrate the usage of the TRNG library. Some of these sample programs will use external libraries, i. e.:

- Boost C++ libraries [9],
- an implementation of the Message Passing Interface (MPI) standard (various open source implementations can be found at [61, 57]),
- Intel Threading Building Blocks [28],
- OpenMP,
- Nvidia CUDA [1] and
- the HIP programming environment for AMD GPUs.

If you want to compile all sample programs, you will have to install these libraries as well. But TRNG does not depend on any of the libraries listed above.

5.2 Compilation

CMake can generate configurations for various build systems, e. g., Makefiles, which are typically employed on Unix-like systems, Visual Studio project files on Windows, or project files for various other integrated development environments. For example, Clion and Visual Studio 2019 come with build-in CMake support [14] and CMake is included in most Linux distributions. After the sources have been extracted from the source archive or have been cloned via git, the build configuration needs to be generated by CMake. In the following, the installation procedure on a typical Unix-like environment (BSD, Linux, Cygwin, etc.) will be given. For compilation in an integrated development environment read the documentation of your preferred tool. For Microsoft Visual Studio this is described in the Visual Studio documentation [14].

On a Unix-like box, just call the `cmake` tool to find your C++ compiler and to generate a set of build configurations, e.g., Makefiles. It is good practice to setup an out-of source

build in a separate directory. For this purpose, Makefiles are generated by the following sequence of shell commands

```
bauke@hal:~/trng-4.29$ mkdir build
bauke@hal:~/trng-4.29$ cd build
bauke@hal:~/trng-4.29/build$ cmake ..
```

The `cmake` tool may be controlled by various options and shell variables, see [13] for details. If no options are provided to `cmake` TRNG will be installed in the `/usr/local` hierarchy. Call

```
bauke@hal:~/trng-4.29/build$ cmake --help
```

to get an overview about all options. Here a complex example: to compile TRNG with the Intel C++ compiler `icpc` and to install the library and the header files in `/opt/trng` call

```
bauke@hal:~/trng-4.29/build$ CXX=icpc cmake -DCMAKE_INSTALL_PREFIX=/opt/trng ..
```

The `cmake` options `-DBUILD_SHARED_LIBS=ON` and `-DBUILD_SHARED_LIBS=OFF` determine if TRNG will be build as a shared library or static library. On default or if `-DBUILD_SHARED_LIBS=OFF` is set, TRNG is built as a static library, otherwise as a dynamic library. Furthermore, unit test and examples are build by default. This may be switched off by options `-DTRNG_ENABLE_TESTS=Off` and `-DTRNG_ENABLE_EXAMPLES=Off`, respectively. The unit tests are based on the Catch2 test framework. A copy of Catch2 is shipped in the `external` directory and is used by default. If `-DUSE_EXTERNAL_CATCH2=On` is set, an already installed version of Catch2 is used instead. If no build type is specified, TRNG is built with `CMAKE_BUILD_TYPE=Release`. The library version and, for shared libraries, the library `SOVERSION` are derived from the project version in the top-level `CMakeLists.txt`.

After TRNG has been configured and build configurations have been generated by CMake, the library can be compiled and installed by employing the following two commands:

```
bauke@hal:~/trng-4.29/build$ cmake --build .
bauke@hal:~/trng-4.29/build$ cmake --build . --target install
```

When TRNG is build as a dynamic library, further steps might be necessary to make the TRNG shared library known to the dynamic linker. These steps depend on your system. On a Linux system, the system administrator has to call `ldconfig` or you might set the `LD_LIBRARY_PATH` environment variable. See also the ld.so man page for further information.

In the source directory `examples` you will find some example programs. These sources are compiled also during the compilation of the TRNG library provided that all required third party libraries mentioned above have been found by the CMake tool.

A distributable package can be generated as a last optional build step. Calling the `cmake` utility with the target `package` such as

```
bauke@hal:~/trng-4.29/build$ cmake --build . --target package
```

yields on a Linux host a Debian package, an RPM package and a zipped tar archive. (RPM packages are created only if the `rpmbuild` tool has been found by CMake.) On all other operating systems only a zip file is created.

5.3 Running unit tests

When the TRNG library is built also a set of unit tests is compiled. Run the CTest tool to perform these tests with verbose output:

```
bauke@hal:~/trng-4.29/build$ ctest -V --progress
```

6 Examples

6.1 Hello world!

In listing 6.1 we present the simplest nontrivial C++ program that produces pseudo-random numbers by TRNG. Whenever one generates random numbers with TRNG at least two header files have to be included, one for a random number engine and one for a distribution function, see lines 4 and 5 in listing 6.1. In lines 9 and 11 respectively a random number engine and a random number distribution are declared. The parameters of a random number distribution object have to be specified by its declaration. In our example random numbers with a normal distribution with mean 6 and standard deviation of 2 are generated. Distribution parameters can be changed at run-time, if necessary. In the loop in lines 13 and 14 the random number engine object `R` and the random number distribution object `normal` are used to generate 1000 random numbers.

The program `hello_world.cc` has to be linked to the TRNG library. Using the GNU C++ compiler we transform the sources by

```
bauke@hal:~$ g++ -o hello_world hello_world.cc -ltrng4
```

into an executable.

In a second example we want to calculate an approximate value for π by a parallel Monte Carlo calculation. The general idea of this calculation is to choose random points

Listing 6.1: A simple TRNG sample program `hello_world.cc` that generates 1000 random variables with normal distribution.

```
1  #include <cstdlib>
2  #include <iostream>
3  // include TRNG header files
4  #include <trng/yarn2.hpp>
5  #include <trng/normal_dist.hpp>
6
7  int main() {
8    // random number engine
9    trng::yarn2 R;
10   // normal distribution with mean 6 and standard deviation 2
11   trng::normal_dist<> normal(6.0, 2.0);
12   // generate 1000 normal distributed random numbers
13   for (int i{0}; i < 100000; ++i)
14     std::cout << normal(R) << '\n';
15   return EXIT_SUCCESS;
16 }
```

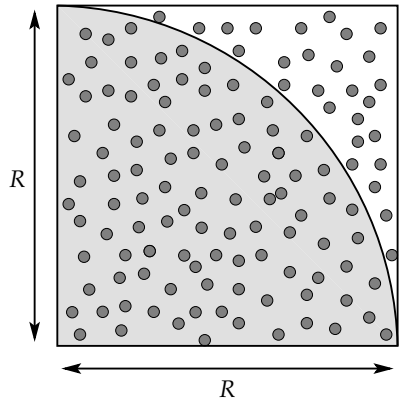


Figure 6.1: The numerical value of π can be estimated by throwing random points into a square.

in a square with edge length R. Some of these points fall into a sector of a circle in the square, see Figure 6.1. The value of π can be approximated by considering the fraction of points that fall into the circle. From the relation

$$\frac{\text{number of points in circle}}{\text{number of points in square}} \approx \frac{\pi R^2/4}{R^2} = \frac{\pi}{4}$$

we conclude

$$\pi \approx 4\frac{\text{number of points in circle}}{\text{number of points in square}}\,.$$

In listing 6.2 we use this equation to estimate π. In the for-loop in lines 12 to 16 a random x-coordinate and a random y-coordinate are chosen. Both coordinates are independently uniformly distributed in $[0,1)$. If $\sqrt{x^2+y^2} < 1$, or equivalently $x^2+y^2 < 1$, the point (x,y) lies within the circle. The program draws a huge number of points from the square and counts the number of points lying within the circle and at the end of the program the fraction $4 \cdot (\text{points in circle})/(\text{points in square})$ is shown as an estimate for π.

Listing 6.2: Sequential Monte Carlo calculation of π.

```
1  #include <cstdlib>
2  #include <iostream>
3  #include <trng/yarn2.hpp>
4  #include <trng/uniform01_dist.hpp>
5
6  int main() {
7    const long samples{10000001};  // total number of points in square
8    long in{0l};                   // no points in circle
9    trng::yarn2 r;                 // random number engine
10   trng::uniform01_dist<> u;      // random number distribution
11   // throw random points into square
12   for (long i{0}; i < samples; ++i) {
13     const double x{u(r)}, y{u(r)};  // choose random x- and y-coordinates
14     if (x * x + y * y <= 1.0)       // is point in circle?
15       ++in;                         // increase counter
```

```
16    }
17    std::cout << "pi = " << 4.0 * in / samples << std::endl;
18    return EXIT_SUCCESS;
19  }
```

6.2 Hello parallel world!

TRNG is a very flexible random number generator library. It allows for sequential as well as for parallel applications. The library does not depend on any particular communication library. It may be utilized with Message Passing Interface (MPI), OpenMP, and as well as with POSIX threads, or any other communication library. This section gives a short tutorial on writing parallel Monte Carlo applications with TRNG and various parallel programming models, e. g. MPI or OpenMP. Here we cannot give an introduction to MPI or OpenMP readers who are not familiar with parallel programming may consult [63, 6, 67, 68] instead.

How can we parallelize the Monte Carlo calculation of π? A striking feature of the Monte Carlo π calculation algorithm (from the previous section): the placement of some point in the square does not affect the placement of other points. In other words: throwing N points into a square is an embarrassingly parallel process. Everything that matters, is the fraction of points in the square that had been placed into the circle. Keeping this fact in mind the Monte Carlo calculation of π can be parallelized easily via the block splitting method or the leapfrog method.

6.2.1 Block splitting

Let us apply the block splitting parallelization technique as introduced in section 2. A total of N points has to be selected by p processes. We number the points from 0 to $N-1$ and the processes from 0 to $p-1$ respectively. The number of a process is called its rank. To distribute the workload equally, we split the entire set of N points into p consecutive blocks of about N/p points. To be specific, a process with rank r selects the points with numbers

$$\lfloor N \cdot r/p \rfloor \qquad \text{to} \qquad \lfloor N \cdot (r+1)/p \rfloor - 1\,,$$

where $\lfloor \cdot \rfloor$ denotes rounding to zero. Each point is determined by two coordinates and a process with rank r consumes

$$2\left(\lfloor N \cdot (r+1)/p \rfloor - \lfloor N \cdot r/p \rfloor\right)$$

random numbers, which are generated by the same random number engine.

All concurrent processes generate random points by their own local copy of the same random number engine. Of course, if all these engines start from the same initial state, they will produce the same sequence of random numbers. For that reason each process jumps $2\lfloor N \cdot r/p \rfloor$ steps ahead, before any random numbers are consumed. This ensures that sequences of random numbers of two different processes never overlap, and

Listing 6.3: Parallel Monte Carlo calculation of π using block splitting and MPI.

```
#include <trng/yarn2.hpp>
#include <trng/uniform01_dist.hpp>

int main(int argc, char *argv[]) {
  const long samples{10000001};  // total number of points in square
  MPI_Init(&argc, &argv);        // initialise MPI environment
  int size, rank;
  MPI_Comm_size(MPI_COMM_WORLD, &size);  // get total number of processes
  MPI_Comm_rank(MPI_COMM_WORLD, &rank);  // get rank of current process
  long in{0};                            // number of points in circle
  trng::yarn2 r;                         // random number engine
  trng::uniform01_dist<> u;              // random number distribution
  r.jump(2 * (rank * samples / size));   // jump ahead
  // throw random points into square and distribute workload over all processes
  for (long i{rank * samples / size}; i < (rank + 1) * samples / size; ++i) {
    const double x{u(r)}, y{u(r)};  // choose random x- and y-coordinates
    if (x * x + y * y <= 1.0)       // is point in circle?
      ++in;                         // increase counter
  }
  // calculate sum of all local variables 'in' and storre result in 'in_all' on process 0
  long in_all;
  MPI_Reduce(&in, &in_all, 1, MPI_LONG, MPI_SUM, 0, MPI_COMM_WORLD);
  if (rank == 0)  // print result
    std::cout << "pi = " << 4.0 * in_all / samples << std::endl;
  MPI_Finalize();  // quit MPI
  return EXIT_SUCCESS;
}
```

Listing 6.4: Parallel Monte Carlo calculation of π using block splitting and OpenMP.

```
#include <trng/yarn2.hpp>
#include <trng/uniform01_dist.hpp>

int main() {
  const long samples{10000001};  // total number of points in square
  long in{0l};                   // number of points in circle
  // distribute workload over all processes and make a global reduction
#pragma omp parallel reduction(+ : in) default(none)
  {
    trng::yarn2 r;                           // random number engine
    const int size{omp_get_num_threads()};   // get total number of processes
    const int rank{omp_get_thread_num()};    // get rank of current process
    trng::uniform01_dist<> u;                // random number distribution
    r.jump(2 * (rank * samples / size));     // jump ahead
    // throw random points into square
    for (long i{rank * samples / size}; i < (rank + 1) * samples / size; ++i) {
      const double x{u(r)}, y{u(r)};  // choose random x- and y-coordinates
      if (x * x + y * y <= 1.0)       // is point in circle?
        ++in;                         // increase thread-local counter
    }
  }
  // print result
```

```
23   std::cout << "pi = " << 4.0 * in / samples << std::endl;
24   return EXIT_SUCCESS;
25 }
```

Listing 6.5: Parallel Monte Carlo calculation of π using block splitting and Intel Threading Building Blocks.

```
1  #include <trng/uniform01_dist.hpp>
2  #include <tbb/blocked_range.h>
3  #include <tbb/parallel_reduce.h>
4
5  class parallel_pi {
6    trng::uniform01_dist<> u;  // random number distribution
7    const trng::yarn2 &r;
8    long in;
9
10 public:
11   void operator()(const tbb::blocked_range<long> &range) {
12     trng::yarn2 r_local(r);           // local copy of random number engine
13     r_local.jump(2 * range.begin());  // jump ahead
14     for (long i{range.begin()}; i != range.end(); ++i) {
15       const double x{u(r_local)}, y{u(r_local)};  // choose random x- and y-coordinates
16       if (x * x + y * y <= 1.0)                   // is point in circle?
17         ++in;                                     // increase thread-local counter
18     }
19   }
20   // join threds and counters
21   void join(const parallel_pi &other) { in += other.in; }
22   long in_circle() const { return in; }
23   explicit parallel_pi(const trng::yarn2 &r) : r{r}, in{0} {}
24   explicit parallel_pi(const parallel_pi &other, tbb::split) : r{other.r}, in{0} {}
25 };
26
27 int main() {
28   const long samples{10000001};   // total number of points in square
29   trng::yarn2 r;                  // random number engine
30   parallel_pi pi(r);              // functor for parallel reduce
31   // parallel MC computation of pi
32   tbb::parallel_reduce(tbb::blocked_range<long>(0, samples), pi, tbb::auto_partitioner());
33   // print result
34   std::cout << "pi = " << 4.0 * pi.in_circle() / samples << std::endl;
35   return EXIT_SUCCESS;
36 }
```

furthermore, the outcome of the parallelized program is the same as for the sequential in the previous section, even in its statistical errors.

Listing 6.3 presents an implementation of the parallel Monte Carlo computation of π by MPI, while in listing 6.4 an implementation presented that is based on OpenMP. Note the parenthesis within the argument of the `jump` method in lines 15 and 17 respectively. Together with the C++ rounding rules they are the C++ equivalent to the $\lfloor \cdot \rfloor$ function.

There is one important conceptual difference between the MPI version and the OpenMP implementation. While MPI is based on a distributed memory model, OpenMP

Listing 6.6: Parallel Monte Carlo calculation of π using block splitting and CUDA.

```
1  #include <cstdlib>
2  #include <iostream>
3  #include <vector>
4  #include <trng/yarn5s.hpp>
5  #include <trng/uniform01_dist.hpp>
6
7  __global__ void parallel_pi(long samples, long *in, trng::yarn5s r) {
8    long rank = threadIdx.x;
9    long size = blockDim.x;
10   r.jump(2 * (rank * samples / size));  // jump ahead
11   trng::uniform01_dist<float> u;        // random number distribution
12   in[rank] = 0;                         // local number of points in circle
13   for (long i = rank * samples / size; i < (rank + 1) * samples / size; ++i) {
14     const float x = u(r), y = u(r);  // choose random x- and y-coordinates
15     if (x * x + y * y <= 1)          // is point in circle?
16       ++in[rank];                    // increase thread-local counter
17   }
18 }
19
20 int main(int argc, char *argv[]) {
21   const long samples{10000001};  // total number of points in square
22   const int size{128};           // number of threads
23   long *in_device;
24   cudaMalloc(&in_device, size * sizeof(*in_device));
25   trng::yarn5s r;
26   // start parallel Monte Carlo
27   parallel_pi<<<1, size>>>(samples, in_device, r);
28   // gather results
29   std::vector<long> in(size);
30   cudaMemcpy(in.data(), in_device, size * sizeof(*in_device), cudaMemcpyDeviceToHost);
31   cudaFree(in_device);
32   long sum{0};
33   for (int rank{0}; rank < size; ++rank)
34     sum += in[rank];
35   // print result
36   std::cout << "pi = " << 4.0 * sum / samples << std::endl;
37   return EXIT_SUCCESS;
38 }
```

can utilize shared memory. For that reason the MPI program counts how many points lie in the circle for each process in a process local variable `in`. At the end of the computation the process local variables have to be summed up by `MPI::COMM_WORLD.Reduce` to the (process local) variable `in_all` on the process with rank zero. In a OpenMP program this global reduction can be avoided by using a shared memory variable. But here concurrent write accesses to `in` have to be prevented by the pragma `omp critical` in lines 23 to 24.

Listing 6.5 shows another block splitting Monte Carlo calculation of π that is based on the Intel Threading Building Blocks [28, 68]. To give a detailed introduction to this excellent C++ library is beyond the scope of the TRNG documentation. The reader

should note the following special features of the Intel Threading Building Blocks and listing 6.5. The (thread) parallel computation is based on the function `tbb::parallel_reduce`. This function requires a class object that implements the task that has to be parallelized. However, the programmer does not specify how the global task is divided into smaller subtasks. Work distribution, load balancing and reduction of the global result (number of points in the square) are handled by the Intel Threading Building Blocks library.

Listing 6.6 shows a block splitting Monte Carlo calculation of π using CUDA. For CUDA, we have to leap frog the random number engines in host memory and to copy random number engines to device memory before the parallel Monte Carlo calculation can be carried out.

6.2.2 Leapfrog

Leapfrog is a convenient approach to derive p non overlapping streams of pseudo-random numbers from a single base stream. As defined in section 3.1 each parallel random number engine provides a `split` method for leapfrog. If `split(p, s)` is called, the internal parameters of the random number engine are changed in such a way that future calls to `operator()` will generate the `s`th sub-stream of `p` sub-streams. Sub-streams are numbered from 0 to $\texttt{p} - 1$. Changing line 15 or line 17 in listing 6.3 or listing 6.4 respectively, which reads

```
r.jump(2*(rank*samples/size));  // jump ahead
```

into

```
r.split(size, rank);            // choose sub–stream no. rank out of size streams
```

provides different statistically independent sub-streams of pseudo-random numbers to each process.

But note, the pseudo-random numbers of the base stream are now utilized in a completely different fashion. The sequential program and also the two on block splitting based programs from section 6.2.1 determine the position of a point (its x- and y-coordinate) by two consecutive pseudo-random numbers of the base sequence. After calling `split(size, rank)` consecutive calls to `operator()` will return pseudo-random numbers that are no longer neighboring numbers of the base sequence. In fact, they have a distance of `size` with respect to the original sequence of pseudo-random numbers. For that reason the proposed replacement of the call of the `jump` method to a call to the `split` method will result in another value for the approximation of π with another statistical error.

To prevent this issue, we use the fact that the leapfrog method can be applied several times to a sequence of pseudo-random numbers by successive calls to `split`. Each time `split` is invoked the sequence is split into further sub-sequences. In listing 6.7 and listing 6.8 it is shown how this works. Both programs start with two random number engines of the same kind.

```
trng::yarn2 rx, ry;                 // random number engines for x- and y-coordinates
```

Listing 6.7: Parallel Monte Carlo calculation of π using leapfrog and MPI.

```
#include <trng/yarn2.hpp>
#include <trng/uniform01_dist.hpp>

int main(int argc, char *argv[]) {
  const long samples{10000000l};  // total number of points in square
  MPI_Init(&argc, &argv);         // initialize MPI environment
  int size, rank;
  MPI_Comm_size(MPI_COMM_WORLD, &size);  // get total number of processes
  MPI_Comm_rank(MPI_COMM_WORLD, &rank);  // get rank of current process
  trng::yarn2 rx, ry;                    // random number engines for x- and y-coordinates
  // split PRN sequences by leapfrog method
  rx.split(2, 0);            // choose sub-stream no. 0 out of 2 streams
  ry.split(2, 1);            // choose sub-stream no. 1 out of 2 streams
  rx.split(size, rank);      // choose sub-stream no. rank out of size streams
  ry.split(size, rank);      // choose sub-stream no. rank out of size streams
  long in{0l};               // number of points in circle
  trng::uniform01_dist<> u;  // random number distribution
  // throw random points into square and distribute workload over all processes
  for (long i{rank}; i < samples; i += size) {
    const double x{u(rx)}, y{u(ry)};  // choose random x- and y-coordinates
    if (x * x + y * y <= 1.0)         // is point in circle?
      ++in;                           // increase counter
  }
  // calculate sum of all local variables 'in' and storre result in 'in_all' on process 0
  long in_all;
  MPI_Reduce(&in, &in_all, 1, MPI_LONG, MPI_SUM, 0, MPI_COMM_WORLD);
  if (rank == 0)  // print result
    std::cout << "pi = " << 4.0 * in_all / samples << std::endl;
  MPI_Finalize();  // quit MPI
  return EXIT_SUCCESS;
}
```

Later all x- and y-coordinates will be determined exclusively by one of these random number engines. But without any manipulations of the internal status via `jump` or `split` method, both engines will return the same sequences of pseudo-random numbers. Therefore, if the coordinates of each point are chosen by calling `operator()` of `rx` and `ry` once, all points will lie on the diagonal of the square. For that reason the sequences are split by

```
rx.split(2, 0);                    // choose sub-stream no. 0 out of 2 streams
ry.split(2, 1);                    // choose sub-stream no. 1 out of 2 streams
```

into two non overlapping sequences. Now successive calls to `operator()` will return different sequences of pseudo-random numbers and the points are uniformly distributed over the square. But still each process consumes the same two sequences of random numbers. However, this can be solved by calling the `split` method a second time.

```
rx.split(size, rank);              // choose sub-stream no. rank out of size streams
ry.split(size, rank);              // choose sub-stream no. rank out of size streams
```

Listing 6.8: Parallel Monte Carlo calculation of π using leapfrog and OpenMP.

```
#include <trng/yarn2.hpp>
#include <trng/uniform01_dist.hpp>

int main() {
  const long samples{10000000l};  // total number of points in square
  long in{0l};                    // no points in circle
  // distribute workload over all processes and make a global reduction
#pragma omp parallel reduction(+ : in) default(none)
  {
    trng::yarn2 rx, ry;                     // random number engines for x- and y-coordinates
    const int size{omp_get_num_threads()};  // get total number of processes
    const int rank{omp_get_thread_num()};   // get rank of current process
    // split PRN sequences by leapfrog method
    rx.split(2, 0);            // choose sub-stream no. 0 out of 2 streams
    ry.split(2, 1);            // choose sub-stream no. 1 out of 2 streams
    rx.split(size, rank);      // choose sub-stream no. rank out of size streams
    ry.split(size, rank);      // choose sub-stream no. rank out of size streams
    trng::uniform01_dist<> u;  // random number distribution
    // throw random points into square
    for (long i{rank}; i < samples; i += size) {
      const double x{u(rx)}, y{u(ry)};  // choose random x- and y-coordinates
      if (x * x + y * y <= 1.0)         // is point in circle?
        ++in;                           // increase thread-local counter
    }
  }
  // print result
  std::cout << "pi = " << 4.0 * in / samples << std::endl;
  return EXIT_SUCCESS;
}
```

6.2.3 Block splitting or leapfrog?

TRNG provides two powerful techniques for parallelizing streams of pseudo-random numbers, namely block splitting and leapfrog. Which one to choose, depends highly on the structure of your Monte Carlo algorithm and your needs.

In the simplest case, each process of a parallel Monte Carlo application with a fixed number of processes p (that does not change at run time) has just to equipped with some source of pseudo-random numbers and the only requirement on the p streams of pseudo-random numbers is that they do not overlap with any stream of pseudo-random numbers on any other process. In this case it is sufficient to use a single random number engine of the same type for each of the p process. Different streams are deviated by the leapfrog method and calling the `split` method of a pseudo-random number engine object after these random number engines have been initialized with the same parameters and the same seed. Of course with this simple-minded approach the outcome of a Monte Carlo application (and the actual statistical errors) will depend on the number of processes.

On the other hand it is often desirable to design a parallel Monte Carlo algorithm in such a way that its outcome is independent of the number of processes. That means the Monte Carlo algorithm plays fair, see also section 2.3. Usually this additional constraint can be fulfilled by a creative combination of block splitting, leapfrog method and using more than one random number engine per processor. The previous sections gave already some elementary examples, how this can be achieved. But in general this can be quite intricate. Therefore, we give some general guidelines.

- Identify the inherently parallel parts of the Monte Carlo algorithm. Which steps of the Monte Carlo algorithm cannot be parallelized?
- Break the parallelizable tasks into p (p number of processes) smaller sub-parts of approximately equal size.
- Is the number of pseudo-random numbers consumed by a parallelizable task (before it is divided into subparts) constant or does it change at runtime? If it is constant, break up the sequence of a single pseudo-random number engine into sub-streams in such a way that mimics the way in which the parallelizable task is split into independent sub-problems. This can always be achieved by calling the `split` or the `jump` method of a random number engine object.
- If the number of pseudo-random numbers consumed by a parallelizable task is not constant, or cannot be determined a priori, e. g. because this number itself is a function of the random number sequence, an upper bound for this number may be estimated. With this number a Monte Carlo algorithm can often be parallelized as if the number of consumed random numbers was fixed.

To make this advice somewhat more clear, we give a further example. Imagine the simulation of a site percolation process [73] on a two-dimensional square lattice of size $N = N_x \times N_y$. In site percolation each site of the lattice is occupied with probability P independently of the other sites and clusters of neighboring occupied sites are constructed afterward. Once these clusters are known, one can answer for a particular realization of occupied sites a lot of questions that arise in percolation theory. Is there a spanning cluster that connects the lower line of the grid and its upper line? What is the size of the largest cluster? And so on. How can we parallelize such a Monte Carlo simulation for site percolation?

The easiest way is not to parallelize at all. At least not the analysis of a single realization of occupied sites itself. Usually one is not interested in the analysis of a single realization of occupied sites by itself, but one wants to know statistical properties of site percolation (or another problem) that arise after averaging over many, lets say M, realizations of systems of the same kind. It is quite natural to spread the workload over p processors in such a way that each process analyzes each pth lattice of the M lattices. If we number the processes by its rank from 0 to $p-1$ and the lattices form 0 to $M-1$, each process starts with a lattice which number equals the process' rank. Thereafter, each process can skip $p-1$ lattices, because these are handled by other processes, and continue with the next lattice. Of course each process has not only to skip the work that is done by other processes, but also the pseudo-random numbers that would be consumed by analyzing

Listing 6.9: Sketch of a coarse-grained parallel Monte Carlo simulation of site percolation via MPI. The program creates many realizations of lattices with randomly occupied sites. Each realization is generated by a single process.

```
#include <cstdlib>
#include <trng/yarn2.hpp>
#include <trng/uniform01_dist.hpp>
#include "mpi.h"

const int number_of_realizations{1000};
const int Nx{250}, Ny{200};  // grid size
const int number_of_PRNs_per_sweep{Nx * Ny};
int site[Nx][Ny];      // lattice
const double P{0.46};  // occupation probability

int main(int argc, char *argv[]) {
  MPI_Init(&argc, &argv);  // initialize MPI environment
  int size, rank;
  MPI_Comm_size(MPI_COMM_WORLD, &size);  // get total number of processes
  MPI_Comm_rank(MPI_COMM_WORLD, &rank);  // get rank of current process
  trng::yarn2 R;                         // random number engine
  trng::uniform01_dist<> u;              // random number distribution
  // skip random numbers that are consumed by other processes
  R.jump(rank * number_of_PRNs_per_sweep);
  for (int i{rank}; i < number_of_realizations; i += size) {
    // consume Nx * Ny pseudo–random numbers
    for (int x{0}; x < Nx; ++x)
      for (int y{0}; y < Ny; ++y)
        if (u(R) < P)
          site[x][y] = 1;  // site is occupied
        else
          site[x][y] = 0;  // site is not occupied
    // skip random numbers that are consumed by other processes
    R.jump((size - 1) * number_of_PRNs_per_sweep);
    // analyze lattice
    // ... source omitted
  }
  MPI_Finalize();  // quit MPI
  return EXIT_SUCCESS;
}
```

the skipped lattices. Listing 6.9 gives a sketch of such a parallelized site percolation program.

Unfortunately it is not always possible to parallelize a Monte Carlo simulation in such a coarse-grained fashion like in the last example. Sometimes (e. g. in the Swendson-Wang-cluster-algorithm [74, 59]) the generation and the analysis of a single lattice has to be parallelized by itself. For that reason we split the lattice into $p_x \times p_y$ sub-lattices in such a way that the number of parallel processes p equals $p_x \times p_y$ and $p_x \approx p_y$. Each process is responsible for one of the sub-lattices and uses the same random number engine. This generic parallelization paradigm is also known as domain decomposition.

Listing 6.10: Sketch of a fine-grained parallel Monte Carlo simulation of site percolation via MPI. The program creates many realizations of lattices with randomly occupied sites. Each realization is generated by all processes together, workload is distributed by domain decomposition.

```
#include <cstdlib>
#include <new>
#include <trng/yarn2.hpp>
#include <trng/uniform01_dist.hpp>
#include "mpi.h"

const int number_of_realizations{1000};
const int Nx{250}, Ny{200};  // grid size
const double P{0.46};        // occupation probability

int main(int argc, char *argv[]) {
  MPI_Init(&argc, &argv);  // initialize MPI environment
  int size;
  MPI_Comm_size(MPI_COMM_WORLD, &size);  // get total number of processes
  // create a two-dimensional Cartesian communicator
  int dims[2]{0, 0};            // number of processes in each domension
  int coords[2];                // coordinates of current process within the grid
  int periods[2]{false, false};  // no periodic boundary conditions
  // calculate a balanced grid partitioning such that  size = dims[0] * dims[1]
  MPI_Dims_create(size, 2, dims);
  MPI_Comm Comm;
  MPI_Cart_create(MPI_COMM_WORLD, 2, dims, periods, true, &Comm);
  int rank;
  MPI_Comm_rank(Comm, &rank);             // get rank of current process
  MPI_Cart_coords(Comm, rank, 2, coords);  // get coordinates of current process
  // determine section of current process
  int x0{coords[0] * Nx / dims[0]}, x1{(coords[0] + 1) * Nx / dims[0]}, Nxl{x1 - x0},
      y0{coords[1] * Ny / dims[1]}, y1{(coords[1] + 1) * Ny / dims[1]}, Nyl{y1 - y0};
  int *site{new int[Nxl * Nyl]};  // allocate memory to storre a sublattice
  trng::yarn2 R;                  // random number engine
  trng::uniform01_dist<> u;       // random number distribution
  // skip random numbers that are consumed by other processes
  R.jump(Nx * y0 + x0);
  for (int i{0}; i < number_of_realizations; ++i) {
    // consume Nxl * Nyl pseudo-random numbers
    int *s{site};
    for (int y{y0}; y < y1; ++y) {
      for (int x{x0}; x < x1; ++x) {
        if (u(R) < P)
          *s = 1;  // site is occupied
        else
          *s = 0;  // site is not occupied
        ++s;
      }
      // skip random numbers that are consumed by other processes
      R.jump(Nx - Nxl);
    }
    // skip random numbers that are consumed by other processes
    R.jump(Nx * (Ny - Nyl));
    // analyze lattice
    // ... source omitted
  }
  delete[] site;
  MPI_Finalize();  // quit MPI
  return EXIT_SUCCESS;
}
```

To make the site percolation lattice generation independent of the number processes and thus independent of the details of the lattice partition, some numbers within the stream of pseudo-random numbers of the random number engine have to be skipped by the `jump` method. If we determine the state (occupied or not occupied) of the sites in a row-major fashion, the `jump` method has to be called, whenever a process has filled a row of its sub-lattice. Of course each process has to skip a certain amount of pseudo-random numbers at the start of the simulation, too.

Listing 6.10 shows the outline of a fine-grained parallel Monte Carlo simulation of site percolation via MPI, where each single lattice generation is done in parallel via domain decomposition. This program shows two noteworthy implementation details. First the program uses a runtime generated Cartesian communicator rather than the standard communicator `MPI::COMM_WOLD` as seen in the MPI examples so far. Such a communicator reflects the special topology of the domain decomposition and eases its implementation significantly. The number of sub-lattices in each dimension, p_x and p_y respectively, is determined by `MPI::Compute_dims`, see [63, 6] for details. Its result (returned in the field `dims`) determines the topology of the Cartesian communicator `Comm`. Another nice feature of the example code in listing 6.10 is that it does not assume the number of sites in any dimension is a multiple of the number of sub-lattices in this dimension. So the sizes of the sub-lattices can vary slightly from process to process. The precise range of coordinates that each process is responsible for is calculated in lines 24 and 25.

Skipping numbers in a pseudo-random number sequence via `jump` is not for free. Of course, it is so smart that it can jump ahead without actually generating the numbers that have to be skipped. But the complexity of `jump` grows logarithmically in its argument. If the domain decomposition is coarse-grained enough, the overhead introduced by skipping numbers via `jump` can be neglected. But if the number of processes that generate a site percolation lattice becomes larger and larger, at a certain point this overhead can no longer be ignored, and it starts to limit the speedup achievable by parallelization. Finding the right level of granularity is a general problem in parallel computing. On one hand one wants to use a large number of processes to attain a large speedup, on the other hand, the relative portion of the inherent sequential part of a program and the overhead introduced by the parallelization grow with the number of processes as well. This fact is also known as Amdahl's law.

6.3 Using TRNG with STL and Boost

Whenever large scale Monte Carlo applications are written, they will not base on TRNG solely, but also on other libraries, e. g. the C++ Standard Template Library (STL) or Boost [9]. In this section we show, how to use TRNG in combination with the STL, especially its containers and algorithms. We assume you are familiar with the concepts of the C++ STL, otherwise we suggest to read [58].

Imagine a C++ array or an STL container like a vector or a list of integers that has to be populated by random numbers with a given distribution. This can be achieved by a

simple loop.

```
trng::yarn2 R;                          // random number engine
trng::uniform_int_dist U(0, 100);       // random number distribution
std::vector<long> v(10);                // vector of long with 10 elements
for (std::vector<long>::iterator i(v.begin()), end(v.end()); i!=end; ++i)
  *i=U(R);      // generate a random number form distribution U by engine R
```

This loop looks innocent, but it is not. It is error-prone and it is not obvious what is actually effected by the loop. The loop is error-prone because the programmer has to take care that the type of the iterator `i` fits to the container. Things become much more handy, if STL algorithms like `std::generate` are used.

The template function `std::generate` takes an iterator range and a function object that takes no arguments as its arguments. The prototype of this function reads

```
namespace std {

  template <class ForwardIterator, class Generator>
  void generate(ForwardIterator first, ForwardIterator last, Generator gen);

}
```

and it assigns the result of invoking `gen` to each element in the range $[\texttt{first}, \texttt{last})$. Random number distributions as introduced in section 3.2 do not meet the requirements of `std::generate`, because their overloaded call operator requires at least one argument, namely a random number engine, see Table 3.2. For that reason we need a function adapter that makes random number distributions compatible with `std::generate`, e. g., or `std::bind` or a lambda function. Employing the template class `std::bind`, an STL container `v` can be filled by

```
trng::yarn2 R;                          // random number engine
trng::uniform_int_dist U(0, 100);       // random number distribution
std::vector<long> v(10);                // vector of long with 10 elements
std::generate(v.begin(), v.end(), std::bind(U, std::ref(R));
```

The statement

```
std::bind(U, std::ref(R))
```

returns a temporary function object whose call operator requires no arguments. The function `std::ref` assures that the temporary function object holds a reference to the random number engine `R`, otherwise it would contain a copy of `R`. Omitting `std::ref` may have unexpected side effects, e. g. the loop

```
for (int i(0); i < 10; ++i)
  std::generate(v.begin(), v.end(), std::bind(U, R));
```

would fill the vector `v` ten times with random numbers, each time with the same set of random numbers. Because `std::bind` generates at each call to `std::generate` a copy of the random number engine `R` and this copy determines the random values in `v`, but not the random number engine `R` itself. As a consequence of this copy process

`std::generate` generates random numbers by a random number engine that starts with the same internal state in each cycle of the loop.

Listing 6.11 demonstrates all the techniques for binding function arguments that have been discussed in this section. Additionally, it shows that TRNG random number engine meet the requirements of the STL functions `std::random_shuffle` and `std::shuffle` directly, no function adaption via `std::bind` is needed.[1]

Listing 6.11: This demo program demonstrates the interplay of TRNG, the C++ STL.

```
1  #include <cstdlib>
2  #include <iostream>
3  #include <vector>
4  #include <algorithm>
5  #include <functional>
6  #include <trng/yarn2.hpp>
7  #include <trng/uniform_int_dist.hpp>
8
9  // print an iterator range to stdout
10 template<typename iter>
11 void print_range(iter i1, iter i2) {
12   while (i1 != i2)
13     std::cout << (*(i1++)) << '\t';
14   std::cout << "\n\n";
15 }
16
17 int main() {
18   trng::yarn2 R;
19   trng::uniform_int_dist U(0, 100);
20   std::vector<long> v(10);
21
22   std::cout << "random number generation by call operator\n";
23   for (auto &val : v)
24     val = U(R);
25   print_range(v.begin(), v.end());
26   std::vector<long> w(12);
27   std::cout << "random number generation by std::generate\n";
28   std::generate(w.begin(), w.end(), std::bind(U, std::ref(R)));
29   print_range(w.begin(), w.end());
30   std::cout << "random number generation by std::generate\n";
31   std::generate(w.begin(), w.end(), std::bind(U, std::ref(R)));
32   print_range(w.begin(), w.end());
33   std::cout << "same sequence as above, but in a random shuffled order\n";
34   std::shuffle(w.begin(), w.end(), R);
35   print_range(w.begin(), w.end());
36   return EXIT_SUCCESS;
37 }
```

[1] Note that `std::random_shuffle` has been removed from the C++ standard library in C++17.

6.4 Using TRNG with C++ standard library random number facility

Random number engines and distributions from TRNG and the C++11 (or later) standard library [29, 30] have the same interfaces and can therefore may be utilized in combination. This meas, for example, random numbers may be generated by using a random number distribution of the C++11 standard library and a TRNG random number engine, see listing 6.12.

There are some probability distributions that are implemented by TRNG random number distribution classes as well as by random number distribution classes from the C++11 standard library. There is, however, a crucial difference between TRNG distributions and C++11 distributions. TRNG distributions consume *exactly* one random number from a random number engine to generated a random number from a desired distribution. With C++11 distributions the number of consumed random numbers may be larger or may even vary. Thus, C++11 random number distributions should not be utilized in parallel Monte Carlo simulations. In particular, it is not possible to write parallel Monte Carlo simulations that play fair, see section 2.3.

Listing 6.12: TRNG random number generators and distributions may be mixed with C++11 random number generators and distributions.

```
#include <iostream>
#include <random>
#include <trng/lcg64.hpp>
#include <trng/normal_dist.hpp>

int main() {
  std::mt19937 R_cpp11;
  trng::lcg64 R_trng;
  std::normal_distribution<> N_cpp11;
  trng::normal_dist<> N_trng(0, 1);
  for (int i{0}; i < 10000; ++i) {
    std::cout << N_cpp11(R_cpp11) << '\t';
    std::cout << N_cpp11(R_trng) << '\t';
    std::cout << N_trng(R_cpp11) << '\t';
    std::cout << N_trng(R_trng) << '\n';
  }
  return EXIT_SUCCESS;
}
```

6.5 CMake package support

TRNG comes with built-in support for CMake's `find_package` function for an easy integration into CMake-based C++ projects. During installation, also a TRNG CMake package is installed. This CMake package provides dependency information to CMake and the underlying build system, including required compiler flags and linker flags. The following minimalistic CMake project illustrates its usage.

```
cmake_minimum_required(VERSION 3.28)
project(trng_example)

find_package(trng4)

add_executable(my_program main.cpp)
target_link_libraries(my_program trng4::trng4)
```

The `find_package(trng4)` statement loads the package information for TRNG. The next statement `add_executable(my_program main.cpp)` adds a new executable `my_program`, which is implemented by a single source file `main.cpp`. It may include TRNG header files and may use various TRNG classes. Thus, it requires linking against TRNG. The `target_link_libraries(my_program trng4::trng4)` statement tells the build system that the executable has some dependency to TRNG. As a consequence, all required compiler flags and linker flags are set. See the CMake documentation [13] for further details.

7 Implementation details and efficiency

Random number engines `trng::mrg`n, `trng::mrg`n`s`, `trng::yarn`n, and `trng::yarn`n`s` utilize LFSR sequences

$$r_i = a_1 \cdot r_{i-1} + a_2 \cdot r_{i-2} + \ldots + a_n \cdot r_{i-n} \bmod m \tag{7.1}$$

over a prime field $\mathbb{F}_m$. The modulus m may be any prime. But LFSR sequences over $\mathbb{F}_2$ have found much more proliferation in the random number generation business than LFSR sequences over other prime fields. LFSR sequences over general prime fields have been proposed in the literature [27, 38, 35] as PRNGs. But so far, they found less attention by practitioners because it is not straight forward to implement LFSR sequences over $\mathbb{F}_m$ efficiently, if m is a large prime, especially if m of the order of the largest in a single computer word representable integer. For that reason, we present some implementation techniques.

We assume that all integer arithmetic is done in w-bit registers and $m < 2^{w-1}$. Under this condition addition of modulo m can be done without overflow problems. But multiplying two $(w-1)$-bit integers modulo m is not straightforward because the intermediate product has $2(w-1)$ significant bits and cannot be stored in a w-bit register. For the special case $a_k < \sqrt{m}$ Schrage [70] showed how to calculate $a_k \cdot r_{i-k} \bmod m$ without overflow. Based on this technique a portable implementation of LFSR sequences with coefficients $a_k < \sqrt{m}$ is presented in [39]. For parallel PRNGs this methods do not apply because the leapfrog method may yield coefficients that violate this condition. Knuth [35, section 3.2.1.1] proposed a generalization of Schrage's method for arbitrary positive factors less than m, but this method requires up to twelve multiplications and divisions and is therefore not very efficient.

The only way to implement (2.9) without additional measures to circumvent overflow problems is to restrict m to $m < 2^{w/2}$. On machines with 32-bit registers, 16 random bits per number is not enough for some applications. Fortunately today's C compiler provide fast 64-bit-arithmetic even on 32-CPUs and genuine 64-CPUs become more and more common. This allows us to increase m to 32.

7.1 Efficient modular reduction

Since the modulo operation in (2.9) is usually slower than other integer operations like addition, multiplication, Boolean operations or shifting, it has a significant impact on the total performance of PRNGs based on LFSR sequences. If the modulus is a Mersenne Prime $m = 2^e - 1$, however, the modulo operation can be done using only a few additions, Boolean operations and shift operations [64].

A summand $s = a_k \cdot r_{i-k}$ in (2.9) will never exceed $(m-1)^2 = (2^e-2)^2$ and for each positive integer $s \in [0, (2^e-1)^2]$ there is a unique decomposition of s into

$$s = r \cdot 2^e + q \quad \text{with} \quad 0 \le q < 2^e \,. \tag{7.2}$$

From this decomposition we conclude

$$\begin{aligned} s - r \cdot 2^e &= q \\ s - r(2^e-1) &= q + r \\ s \bmod (2^e-1) &= q + r \bmod (2^e-1) \end{aligned}$$

and r and q are bounded form above by

$$q < 2^e \quad \text{and} \quad r \le \lfloor (2^e-2)^2/2^e \rfloor < 2^e - 2$$

respectively, and therefore

$$q + r < 2^e + 2^e - 2 = 2m \,.$$

So if $m = 2^e - 1$ and $s \le (m-1)^2$, $x = s \bmod m$ can be calculated solely by shift operations, Boolean operations and addition, viz

$$x = (s \bmod 2^e) + \lfloor s/2^e \rfloor \,. \tag{7.3}$$

If (7.3) yields a value $x \ge m$ we simply subtract m.

From a computational point of view Mersenne Prime moduli are optimal and we propose to choose the modulus $m = 2^{31} - 1$. This is the largest positive integer that can be represented by a signed 32-bit integer variable, and it is also a Mersenne Prime. On the other hand our theoretical considerations favor Sophie-Germain Prime moduli, for which (7.3) does not apply directly. But one can generalize (7.3) to moduli $2^e - k$ [49]. Again we start from a decomposition of s into

$$s = r \cdot 2^e + q \quad \text{with} \quad 0 \le q < 2^e \,, \tag{7.4}$$

and conclude

$$\begin{aligned} s - r \cdot 2^e &= q \\ s - r(2^e-k) &= q + kr \\ s \bmod (2^e-k) &= q + kr \bmod (2^e-k) \,. \end{aligned}$$

The sum $s' = q + kr$ exceeds the modulus at most by a factor $k+1$, because by applying

$$q < 2^e \quad \text{and} \quad r \le \lfloor (2^e-k-1)^2/2^e \rfloor < 2^e - k - 1$$

we get the bound

$$q + kr < 2^e + k(2^e - k - 1) = (k+1)m \,.$$

In addition by the decomposition of $s' = q + kr$

$$s' = r' \cdot 2^e + q' \quad \text{with} \quad 0 \leq q' < 2^e \,,$$

it follows

$$s \bmod (2^e - k) = s' \bmod (2^e - k) = q' + kr' \bmod (2^e - k) \,,$$

and this time the bounds

$$q' < 2^e \quad \text{and} \quad r' \leq \lfloor (k+1)(2^e - k)/2^e \rfloor < k+1$$

and

$$q' + kr' < 2^e + k(k+1) = m + k(k+2) \,.$$

hold. Therefore if $m = 2^e - k$, $s \leq (m-k)^2$ and $k(k+2) \leq m$, $x = s \bmod m$ can be calculated solely by shift operations, Boolean operations and addition, viz

$$\begin{aligned} s' &= (s \bmod 2^e) + k\lfloor s/2^e \rfloor \\ x &= (s' \bmod 2^e) + k\lfloor s'/2^e \rfloor \,. \end{aligned} \tag{7.5}$$

If (7.5) yields a value $x \geq m$, a single subtraction of m will complete the modular reduction. To carry out (7.5) twice as many operations as for (7.3) are needed. But (7.5) applies for all moduli $m = 2^e - k$ with $k(k+2) \leq m$.

7.2 Fast delinearization

YARN generators hide linear structures of LFSR sequences q_i by raising a generating element g to the power $g^{q_i} \bmod m$. This can be done efficiently by binary exponentiation, which takes $\mathcal{O}(\log m)$ steps. But considering LFSR sequences with only a few feedback taps ($n \leq 6$) and $m \approx 2^{31}$ even fast exponentiation is significantly more expensive than a single iteration of (2.9). Therefore we propose to implement exponentiation by table look up. If m is a $2e'$-bit number we apply the decomposition

$$\begin{aligned} q_i &= q_{i,1} \cdot 2^{e'} + q_{i,0} \quad \text{with} \\ q_{i,1} &= \lfloor q_i / 2^{e'} \rfloor \,, \quad q_{i,0} = q_i \bmod 2^{e'} \end{aligned} \tag{7.6}$$

and use the identity

$$r_i = g^{q_i} \bmod m = (g^{2^{e'}})^{q_{i,1}} \cdot g^{q_{i,0}} \bmod m \tag{7.7}$$

to calculate $g^{q_i} \bmod m$ by two table look-ups and one multiplication modulo m. If $m < 2^{31}$ the tables for $(g^{2^{e'}})^{q_{i,1}} \bmod m$ and $g^{q_{i,0}} \bmod m$ have 2^{16} and 2^{15} entries respectively and fit easily into the cache of modern CPUs.

7.3 Performance

By TRNG we provide an optimized PRNG library. The implementation uses 64-bit-arithmetic, fast modular reduction (7.3) and (7.5) and exponentiation by table look-up (7.7) to implement PRNGs based on LFSR sequences over prime fields, with Mersenne or Sophie-Germain Prime modulus. PRNGs of TRNG are able to compete with other sequential PRNGs in terms of speed and statistical properties but do support block splitting and leapfrog, too. Table 7.1 shows some benchmark results. For this benchmark 2^{26} PRNs were generated and the execution time was measured to compute how many PRNs each PRNG is able to generate per second. Apparently the performance of the PRNGs of TRNG compete quite well with popular PRNGs like the Mersenne Twister (`trng::mt19937`, `std::mt19937` and `boost::mt19937`) , lagged Fibonacci generators (LFSR sequences over $\mathbb{F}_2$) or RANLUX that can be found in the Boost library [9].

Table 7.1: Performance of various random number engines from TRNG, the C++ Standard Library and Boost. Test program was compiled and executed on a Intel Core i7-1051U 1.80 GHz in 64-bit mode using an Intel(R) oneAPI DPC++/C++ Compiler 2024.1.0 and the optimization option `-O3`.

generator	PRNs per second
TRNG	
`trng::lcg64`	$1050.7 \cdot 10^6$
`trng::lcg64_shift`	$874.3 \cdot 10^6$
`trng::lcg64_count_shift`	$640.4 \cdot 10^6$
`trng::count128_lcg_shift`	$760.6 \cdot 10^6$
`trng::mrg2`	$318.3 \cdot 10^6$
`trng::mrg3`	$287.7 \cdot 10^6$
`trng::mrg3s`	$217.1 \cdot 10^6$
`trng::mrg4`	$130.4 \cdot 10^6$
`trng::mrg5`	$177.4 \cdot 10^6$
`trng::mrg5s`	$168.9 \cdot 10^6$
`trng::yarn2`	$235.8 \cdot 10^6$
`trng::yarn3`	$215.1 \cdot 10^6$
`trng::yarn3s`	$170.5 \cdot 10^6$
`trng::yarn4`	$121.7 \cdot 10^6$
`trng::yarn5`	$143.9 \cdot 10^6$
`trng::yarn5s`	$129.7 \cdot 10^6$
`trng::mt19937`	$294.5 \cdot 10^6$
`trng::mt19937_64`	$265.2 \cdot 10^6$
`trng::lagfib2xor_19937_64`	$1298.8 \cdot 10^6$
`trng::lagfib4xor_19937_64`	$865.9 \cdot 10^6$
`trng::lagfib2plus_19937_64`	$1290.1 \cdot 10^6$
`trng::lagfib4plus_19937_64`	$876.0 \cdot 10^6$

generator	PRNs per second
`trng::xoshiro256plus`	$1230.8 \cdot 10^6$
C++ Standard Library	
`std::minstd_rand0`	$295.1 \cdot 10^6$
`std::minstd_rand`	$296.4 \cdot 10^6$
`std::mt19937`	$337.9 \cdot 10^6$
`std::mt19937_64`	$332.3 \cdot 10^6$
`std::ranlux24_base`	$460.2 \cdot 10^6$
`std::ranlux48_base`	$201.9 \cdot 10^6$
`std::ranlux24`	$52.7 \cdot 10^6$
`std::ranlux48`	$14.9 \cdot 10^6$
`std::knuth_b`	$80.7 \cdot 10^6$
Boost Library	
`boost::minstd_rand`	$291.6 \cdot 10^6$
`boost::ecuyer1988`	$267.6 \cdot 10^6$
`boost::kreutzer1986`	$253.5 \cdot 10^6$
`boost::hellekalek1995`	$7.6 \cdot 10^6$
`boost::mt11213b`	$493.5 \cdot 10^6$
`boost::mt19937`	$660.1 \cdot 10^6$
`boost::lagged_fibonacci607`	$812.0 \cdot 10^6$
`boost::lagged_fibonacci1279`	$814.6 \cdot 10^6$
`boost::lagged_fibonacci2281`	$856.9 \cdot 10^6$
`boost::lagged_fibonacci3217`	$806.4 \cdot 10^6$
`boost::lagged_fibonacci4423`	$812.7 \cdot 10^6$
`boost::lagged_fibonacci9689`	$794.7 \cdot 10^6$
`boost::lagged_fibonacci19937`	$760.8 \cdot 10^6$
`boost::lagged_fibonacci23209`	$754.1 \cdot 10^6$
`boost::lagged_fibonacci44497`	$726.7 \cdot 10^6$

8 Quality and statistical tests

Sequences of PRNs are sequences of deterministic numbers that try to mimic true random numbers and, one may wonder, how close sequences produced by a TRNG can come to sequences of real random numbers? This question can be answered (at least partly) by statistical tests. One can apply a battery of tests on a generator, and the more tests a generator can pass, the better its quality. One distinguishes empirical and theoretical test procedures.

Empirical tests take a finite sequence of PRNs and compute certain statistics, e. g. chi-square or Kolmogorov-Smirnov statistics, to judge the generator as "random" or not. The test statistic is a random variate with a probability distribution that can be calculated under the assumption that the test statistic is a function of true random numbers. This probability distribution is used to judge a finite sequence of PRNs as possibly random or non-random. For example in an actual test we may find a value of the test statistic that is so large (or small) that such a value or a larger (or smaller) value can be found by chance for true random numbers with a probability of 5 % only. In this case we assume the PRNG has failed the test and its sequence of PRNs behaves non-random. But note, we may be wrong, there is a 5 % probability that we have just seen normal statistical deviations. Therefore a statistical test should be applied several times. If the PRNG fails more often than it can be explained by normal statistical deviations, it has a serious flaw and should be rejected as non-random.

While empirical tests focus only on the statistical properties of a finite stream of PRNs and ignore all the details of the underlying PRNG algorithm, theoretical tests analyze the PRNG algorithm itself by number-theoretic methods and establish a priori characteristics of the PRN sequence. These a priori characteristics may be used to choose good parameter sets for a certain class of PRNGs, e. g. the coefficients of the LFSR sequences in the random number engines `trng::mrg`*n* and `trng::yarn`*n* (see section 4.1) have been found by an extensive computer search [39] and give good results in the spectral test [35], the most important theoretical test for this class of generators.

On one hand the more kinds of statistical test procedures a PRNG masters, the more we will trust its statistical properties. On the other hand statistical test can never prove that an finite sequence of numbers is "random" or not. Knuth writes in [35]:

> "In practice, we apply about half a dozen different kinds of statistical tests on a sequence, and if is passes them satisfactorily we consider it to be random—it is then presumed innocent until proven guilty."

All PRNGs of TRNG and sub-streams of them have been subject to different statistical tests as presented below. Empirical tests of the PRNGs of TNRG by other researchers have been carried out in [4] and [50]. In respect of these tests the generator you find in

TRNG are comparable to other well-known high-quality generators like the Mersenne twister generator [51]. The tables in this section present results of various statistical tests of streams of pseudo-random numbers that are generated by PRNGs of TRNG with default parameters and no leapfrog splitting. All statistical tests are implemented by an extended version [10] of the dieharder test suite [11] that incorporates the generators of the TRNG library. A detailed description of the statistical tests can be found on the Dieharder web site [11] or in [35] and [2]. Diehard offers many parameters to tweak the sensitivity of the statistical tests. In order to make it easier to compare test results for TRNG random number engines to results for other generators, the following tables are generated with the Dieharder's default settings. TRNG users may run their own tests with custom parameters if desired, see [10] for the source code of the applied tests.

There are a few things that are worth noting about the test results. The engine `trng::lcg64` fails in many tests which just illustrates the known weaknesses of linear congruential generators. The non-linear output mapping of `trng::lcg64_shift`, however, eliminates these issues very effectively. The engines `mrg`*n* and `yarn`*n* perform very well. They fail, however, all the test `diehard_dna`. This implementation of George Marsaglia's DNA test assumes that the PRNG generates pseudo random integers with at least 32 bits. Therefore the test is actually not applicable to the engines `mrg`*n* and `yarn`*n*, which yield only 31-bit integers due to their design.

Listing 8.1: Test results for random number engine `trng::lcg64`.

```
#=============================================================================#
#            dieharder version 3.31.1 Copyright 2003 Robert G. Brown          #
#=============================================================================#
   rng_name    |rands/second|   Seed   |
     trng_lcg64|  2.85e+08  |         1|
#=============================================================================#
        test_name   |ntup| tsamples |psamples|  p-value |Assessment
#=============================================================================#
   diehard_birthdays|   0|       100|     100|0.80299623|  PASSED
      diehard_operm5|   0|   1000000|     100|0.80349640|  PASSED
  diehard_rank_32x32|   0|     40000|     100|0.07704078|  PASSED
    diehard_rank_6x8|   0|    100000|     100|0.00000000|  FAILED
   diehard_bitstream|   0|   2097152|     100|0.00000000|  FAILED
        diehard_opso|   0|   2097152|     100|0.00000000|  FAILED
        diehard_oqso|   0|   2097152|     100|0.00000000|  FAILED
         diehard_dna|   0|   2097152|     100|0.00000000|  FAILED
diehard_count_1s_str|   0|    256000|     100|0.00000000|  FAILED
diehard_count_1s_byt|   0|    256000|     100|0.00000000|  FAILED
 diehard_parking_lot|   0|     12000|     100|0.86207866|  PASSED
    diehard_2dsphere|   2|      8000|     100|0.47907743|  PASSED
    diehard_3dsphere|   3|      4000|     100|0.68628195|  PASSED
     diehard_squeeze|   0|    100000|     100|0.06037378|  PASSED
        diehard_sums|   0|       100|     100|0.10674363|  PASSED
        diehard_runs|   0|    100000|     100|0.16639327|  PASSED
        diehard_runs|   0|    100000|     100|0.09538550|  PASSED
       diehard_craps|   0|    200000|     100|0.97441117|  PASSED
       diehard_craps|   0|    200000|     100|0.77206304|  PASSED
 marsaglia_tsang_gcd|   0|  10000000|     100|0.00000000|  FAILED
 marsaglia_tsang_gcd|   0|  10000000|     100|0.00000000|  FAILED
         sts_monobit|   1|    100000|     100|0.02985528|  PASSED
            sts_runs|   2|    100000|     100|0.06723456|  PASSED
          sts_serial|   1|    100000|     100|0.29091305|  PASSED
          sts_serial|   2|    100000|     100|0.00003924|   WEAK
          sts_serial|   3|    100000|     100|0.00000000|  FAILED
          sts_serial|   3|    100000|     100|0.00000000|  FAILED
```

```
          sts_serial|   4|    100000|     100|0.00000000|  FAILED
          sts_serial|   4|    100000|     100|0.00000007|  FAILED
          sts_serial|   5|    100000|     100|0.00000000|  FAILED
          sts_serial|   5|    100000|     100|0.00000000|  FAILED
          sts_serial|   6|    100000|     100|0.00000000|  FAILED
          sts_serial|   6|    100000|     100|0.00000000|  FAILED
          sts_serial|   7|    100000|     100|0.00000000|  FAILED
          sts_serial|   7|    100000|     100|0.00000000|  FAILED
          sts_serial|   8|    100000|     100|0.00000000|  FAILED
          sts_serial|   8|    100000|     100|0.00000000|  FAILED
          sts_serial|   9|    100000|     100|0.00000000|  FAILED
          sts_serial|   9|    100000|     100|0.00000000|  FAILED
          sts_serial|  10|    100000|     100|0.00000000|  FAILED
          sts_serial|  10|    100000|     100|0.00000000|  FAILED
          sts_serial|  11|    100000|     100|0.00000000|  FAILED
          sts_serial|  11|    100000|     100|0.00000000|  FAILED
          sts_serial|  12|    100000|     100|0.00000000|  FAILED
          sts_serial|  12|    100000|     100|0.00000000|  FAILED
          sts_serial|  13|    100000|     100|0.00000000|  FAILED
          sts_serial|  13|    100000|     100|0.00000000|  FAILED
          sts_serial|  14|    100000|     100|0.00000000|  FAILED
          sts_serial|  14|    100000|     100|0.00000000|  FAILED
          sts_serial|  15|    100000|     100|0.00000000|  FAILED
          sts_serial|  15|    100000|     100|0.00000000|  FAILED
          sts_serial|  16|    100000|     100|0.00000000|  FAILED
          sts_serial|  16|    100000|     100|0.00000000|  FAILED
         rgb_bitdist|   1|    100000|     100|0.00000000|  FAILED
         rgb_bitdist|   2|    100000|     100|0.00000000|  FAILED
         rgb_bitdist|   3|    100000|     100|0.00000000|  FAILED
         rgb_bitdist|   4|    100000|     100|0.00000000|  FAILED
         rgb_bitdist|   5|    100000|     100|0.00000000|  FAILED
         rgb_bitdist|   6|    100000|     100|0.00000000|  FAILED
         rgb_bitdist|   7|    100000|     100|0.00000000|  FAILED
         rgb_bitdist|   8|    100000|     100|0.00000000|  FAILED
         rgb_bitdist|   9|    100000|     100|0.00003686|   WEAK
         rgb_bitdist|  10|    100000|     100|0.00004693|   WEAK
         rgb_bitdist|  11|    100000|     100|0.71404066|  PASSED
         rgb_bitdist|  12|    100000|     100|0.01432920|  PASSED
rgb_minimum_distance|   2|     10000|    1000|0.67082062|  PASSED
rgb_minimum_distance|   3|     10000|    1000|0.84254750|  PASSED
rgb_minimum_distance|   4|     10000|    1000|0.93616796|  PASSED
rgb_minimum_distance|   5|     10000|    1000|0.28368500|  PASSED
    rgb_permutations|   2|    100000|     100|0.99455075|  PASSED
    rgb_permutations|   3|    100000|     100|0.39104838|  PASSED
    rgb_permutations|   4|    100000|     100|0.79069872|  PASSED
    rgb_permutations|   5|    100000|     100|0.59615656|  PASSED
      rgb_lagged_sum|   0|   1000000|     100|0.88292516|  PASSED
      rgb_lagged_sum|   1|   1000000|     100|0.79547848|  PASSED
      rgb_lagged_sum|   2|   1000000|     100|0.24253763|  PASSED
      rgb_lagged_sum|   3|   1000000|     100|0.43880277|  PASSED
      rgb_lagged_sum|   4|   1000000|     100|0.30370090|  PASSED
      rgb_lagged_sum|   5|   1000000|     100|0.68717239|  PASSED
      rgb_lagged_sum|   6|   1000000|     100|0.44081230|  PASSED
      rgb_lagged_sum|   7|   1000000|     100|0.15799889|  PASSED
      rgb_lagged_sum|   8|   1000000|     100|0.66270111|  PASSED
      rgb_lagged_sum|   9|   1000000|     100|0.66466163|  PASSED
      rgb_lagged_sum|  10|   1000000|     100|0.30013536|  PASSED
      rgb_lagged_sum|  11|   1000000|     100|0.77783230|  PASSED
      rgb_lagged_sum|  12|   1000000|     100|0.44055805|  PASSED
      rgb_lagged_sum|  13|   1000000|     100|0.14165199|  PASSED
      rgb_lagged_sum|  14|   1000000|     100|0.00528219|  PASSED
      rgb_lagged_sum|  15|   1000000|     100|0.53886328|  PASSED
      rgb_lagged_sum|  16|   1000000|     100|0.94514050|  PASSED
      rgb_lagged_sum|  17|   1000000|     100|0.13747644|  PASSED
      rgb_lagged_sum|  18|   1000000|     100|0.85654608|  PASSED
```

```
      rgb_lagged_sum|  19|   1000000|     100|0.33106483|  PASSED
      rgb_lagged_sum|  20|   1000000|     100|0.78389261|  PASSED
      rgb_lagged_sum|  21|   1000000|     100|0.69756340|  PASSED
      rgb_lagged_sum|  22|   1000000|     100|0.66876921|  PASSED
      rgb_lagged_sum|  23|   1000000|     100|0.43040437|  PASSED
      rgb_lagged_sum|  24|   1000000|     100|0.94431041|  PASSED
      rgb_lagged_sum|  25|   1000000|     100|0.06544563|  PASSED
      rgb_lagged_sum|  26|   1000000|     100|0.98939371|  PASSED
      rgb_lagged_sum|  27|   1000000|     100|0.83855975|  PASSED
      rgb_lagged_sum|  28|   1000000|     100|0.77420028|  PASSED
      rgb_lagged_sum|  29|   1000000|     100|0.07694399|  PASSED
      rgb_lagged_sum|  30|   1000000|     100|0.87410547|  PASSED
      rgb_lagged_sum|  31|   1000000|     100|0.82771228|  PASSED
      rgb_lagged_sum|  32|   1000000|     100|0.31870512|  PASSED
     rgb_kstest_test|   0|     10000|    1000|0.29920848|  PASSED
     dab_bytedistrib|   0|  51200000|       1|1.00000000|  FAILED
             dab_dct| 256|     50000|       1|0.00000000|  FAILED
Preparing to run test 207.  ntuple = 0
        dab_filltree|  32|  15000000|       1|0.07482477|  PASSED
        dab_filltree|  32|  15000000|       1|0.05899072|  PASSED
Preparing to run test 208.  ntuple = 0
       dab_filltree2|   0|   5000000|       1|0.00000000|  FAILED
       dab_filltree2|   1|   5000000|       1|0.00000000|  FAILED
Preparing to run test 209.  ntuple = 0
        dab_monobit2|  12|  65000000|       1|1.00000000|  FAILED
Preparing to run test 210.  ntuple = 0
      dab_birthdays1|  32|      2000|       1|0.00000000|  FAILED
Preparing to run test 211.  ntuple = 0
           dab_opso2|   0|  67108864|       1|1.00000000|  FAILED
           dab_opso2|   1|  67108864|       1|0.00000000|  FAILED
```

Listing 8.2: Test results for random number engine `trng::lcg64_shift`.

```
#=============================================================================#
#            dieharder version 3.31.1 Copyright 2003 Robert G. Brown          #
#=============================================================================#
   rng_name    |rands/second|   Seed   |
trng_lcg64_shift|  4.27e+08  |         1|
#=============================================================================#
        test_name   |ntup| tsamples |psamples|  p-value |Assessment
#=============================================================================#
   diehard_birthdays|   0|       100|     100|0.60775832|  PASSED
      diehard_operm5|   0|   1000000|     100|0.68451478|  PASSED
  diehard_rank_32x32|   0|     40000|     100|0.63984676|  PASSED
    diehard_rank_6x8|   0|    100000|     100|0.64634587|  PASSED
   diehard_bitstream|   0|   2097152|     100|0.39710020|  PASSED
        diehard_opso|   0|   2097152|     100|0.40065079|  PASSED
        diehard_oqso|   0|   2097152|     100|0.79036307|  PASSED
         diehard_dna|   0|   2097152|     100|0.66424077|  PASSED
diehard_count_1s_str|   0|    256000|     100|0.21331175|  PASSED
diehard_count_1s_byt|   0|    256000|     100|0.96307114|  PASSED
 diehard_parking_lot|   0|     12000|     100|0.69464262|  PASSED
    diehard_2dsphere|   2|      8000|     100|0.49034331|  PASSED
    diehard_3dsphere|   3|      4000|     100|0.46678093|  PASSED
     diehard_squeeze|   0|    100000|     100|0.95328482|  PASSED
        diehard_sums|   0|       100|     100|0.00870277|  PASSED
        diehard_runs|   0|    100000|     100|0.77247363|  PASSED
        diehard_runs|   0|    100000|     100|0.52767527|  PASSED
       diehard_craps|   0|    200000|     100|0.67262310|  PASSED
       diehard_craps|   0|    200000|     100|0.62218190|  PASSED
 marsaglia_tsang_gcd|   0|  10000000|     100|0.50441096|  PASSED
 marsaglia_tsang_gcd|   0|  10000000|     100|0.30701597|  PASSED
         sts_monobit|   1|    100000|     100|0.90623992|  PASSED
            sts_runs|   2|    100000|     100|0.10586020|  PASSED
          sts_serial|   1|    100000|     100|0.75980477|  PASSED
```

```
          sts_serial|   2|    100000|     100|0.32576495|  PASSED
          sts_serial|   3|    100000|     100|0.73498487|  PASSED
          sts_serial|   3|    100000|     100|0.65817437|  PASSED
          sts_serial|   4|    100000|     100|0.40185682|  PASSED
          sts_serial|   4|    100000|     100|0.64713612|  PASSED
          sts_serial|   5|    100000|     100|0.24664067|  PASSED
          sts_serial|   5|    100000|     100|0.26383150|  PASSED
          sts_serial|   6|    100000|     100|0.33837263|  PASSED
          sts_serial|   6|    100000|     100|0.90778127|  PASSED
          sts_serial|   7|    100000|     100|0.54110130|  PASSED
          sts_serial|   7|    100000|     100|0.73223696|  PASSED
          sts_serial|   8|    100000|     100|0.35099844|  PASSED
          sts_serial|   8|    100000|     100|0.31083764|  PASSED
          sts_serial|   9|    100000|     100|0.43029817|  PASSED
          sts_serial|   9|    100000|     100|0.60476060|  PASSED
          sts_serial|  10|    100000|     100|0.72270639|  PASSED
          sts_serial|  10|    100000|     100|0.39994886|  PASSED
          sts_serial|  11|    100000|     100|0.86474172|  PASSED
          sts_serial|  11|    100000|     100|0.86056201|  PASSED
          sts_serial|  12|    100000|     100|0.92480756|  PASSED
          sts_serial|  12|    100000|     100|0.20382018|  PASSED
          sts_serial|  13|    100000|     100|0.46624180|  PASSED
          sts_serial|  13|    100000|     100|0.92133334|  PASSED
          sts_serial|  14|    100000|     100|0.63908418|  PASSED
          sts_serial|  14|    100000|     100|0.17392074|  PASSED
          sts_serial|  15|    100000|     100|0.28046647|  PASSED
          sts_serial|  15|    100000|     100|0.15552492|  PASSED
          sts_serial|  16|    100000|     100|0.94038151|  PASSED
          sts_serial|  16|    100000|     100|0.97683002|  PASSED
         rgb_bitdist|   1|    100000|     100|0.38155844|  PASSED
         rgb_bitdist|   2|    100000|     100|0.99002749|  PASSED
         rgb_bitdist|   3|    100000|     100|0.99734390|   WEAK
         rgb_bitdist|   4|    100000|     100|0.21617322|  PASSED
         rgb_bitdist|   5|    100000|     100|0.67072848|  PASSED
         rgb_bitdist|   6|    100000|     100|0.66852710|  PASSED
         rgb_bitdist|   7|    100000|     100|0.28379729|  PASSED
         rgb_bitdist|   8|    100000|     100|0.97478742|  PASSED
         rgb_bitdist|   9|    100000|     100|0.31886144|  PASSED
         rgb_bitdist|  10|    100000|     100|0.97269569|  PASSED
         rgb_bitdist|  11|    100000|     100|0.89914860|  PASSED
         rgb_bitdist|  12|    100000|     100|0.65116306|  PASSED
rgb_minimum_distance|   2|     10000|    1000|0.84185864|  PASSED
rgb_minimum_distance|   3|     10000|    1000|0.98571072|  PASSED
rgb_minimum_distance|   4|     10000|    1000|0.01653319|  PASSED
rgb_minimum_distance|   5|     10000|    1000|0.50307706|  PASSED
    rgb_permutations|   2|    100000|     100|0.58277468|  PASSED
    rgb_permutations|   3|    100000|     100|0.47964822|  PASSED
    rgb_permutations|   4|    100000|     100|0.05421606|  PASSED
    rgb_permutations|   5|    100000|     100|0.83064525|  PASSED
      rgb_lagged_sum|   0|   1000000|     100|0.55814004|  PASSED
      rgb_lagged_sum|   1|   1000000|     100|0.65345641|  PASSED
      rgb_lagged_sum|   2|   1000000|     100|0.95608501|  PASSED
      rgb_lagged_sum|   3|   1000000|     100|0.11447882|  PASSED
      rgb_lagged_sum|   4|   1000000|     100|0.08981208|  PASSED
      rgb_lagged_sum|   5|   1000000|     100|0.46945897|  PASSED
      rgb_lagged_sum|   6|   1000000|     100|0.00516939|  PASSED
      rgb_lagged_sum|   7|   1000000|     100|0.98317886|  PASSED
      rgb_lagged_sum|   8|   1000000|     100|0.74455320|  PASSED
      rgb_lagged_sum|   9|   1000000|     100|0.25960114|  PASSED
      rgb_lagged_sum|  10|   1000000|     100|0.84816909|  PASSED
      rgb_lagged_sum|  11|   1000000|     100|0.55375859|  PASSED
      rgb_lagged_sum|  12|   1000000|     100|0.40585842|  PASSED
      rgb_lagged_sum|  13|   1000000|     100|0.99013929|  PASSED
      rgb_lagged_sum|  14|   1000000|     100|0.27124629|  PASSED
      rgb_lagged_sum|  15|   1000000|     100|0.48810620|  PASSED
```

```
      rgb_lagged_sum|  16|   1000000|     100|0.12691674|  PASSED
      rgb_lagged_sum|  17|   1000000|     100|0.65831804|  PASSED
      rgb_lagged_sum|  18|   1000000|     100|0.92442364|  PASSED
      rgb_lagged_sum|  19|   1000000|     100|0.97341425|  PASSED
      rgb_lagged_sum|  20|   1000000|     100|0.76636078|  PASSED
      rgb_lagged_sum|  21|   1000000|     100|0.99201418|  PASSED
      rgb_lagged_sum|  22|   1000000|     100|0.92690764|  PASSED
      rgb_lagged_sum|  23|   1000000|     100|0.39168222|  PASSED
      rgb_lagged_sum|  24|   1000000|     100|0.23974344|  PASSED
      rgb_lagged_sum|  25|   1000000|     100|0.22796606|  PASSED
      rgb_lagged_sum|  26|   1000000|     100|0.09420805|  PASSED
      rgb_lagged_sum|  27|   1000000|     100|0.42957084|  PASSED
      rgb_lagged_sum|  28|   1000000|     100|0.95314215|  PASSED
      rgb_lagged_sum|  29|   1000000|     100|0.81587586|  PASSED
      rgb_lagged_sum|  30|   1000000|     100|0.71489760|  PASSED
      rgb_lagged_sum|  31|   1000000|     100|0.81946703|  PASSED
      rgb_lagged_sum|  32|   1000000|     100|0.27887473|  PASSED
     rgb_kstest_test|   0|     10000|    1000|0.50464743|  PASSED
     dab_bytedistrib|   0|  51200000|       1|0.50551872|  PASSED
             dab_dct| 256|     50000|       1|0.98302578|  PASSED
Preparing to run test 207.  ntuple = 0
        dab_filltree|  32|  15000000|       1|0.44488737|  PASSED
        dab_filltree|  32|  15000000|       1|0.76851223|  PASSED
Preparing to run test 208.  ntuple = 0
       dab_filltree2|   0|   5000000|       1|0.68415737|  PASSED
       dab_filltree2|   1|   5000000|       1|0.98743334|  PASSED
Preparing to run test 209.  ntuple = 0
        dab_monobit2|  12|  65000000|       1|0.43718694|  PASSED
Preparing to run test 210.  ntuple = 0
      dab_birthdays1|  32|      2000|       1|0.00259076|   WEAK
Preparing to run test 211.  ntuple = 0
           dab_opso2|   0|  67108864|       1|0.60550294|  PASSED
           dab_opso2|   1|  67108864|       1|0.02623309|  PASSED
```

Listing 8.3: Test results for random number engine `trng::lcg64_count_shift`.

```
#=============================================================================#
#            dieharder version 3.31.1 Copyright 2003 Robert G. Brown          #
#=============================================================================#
   rng_name    |rands/second|   Seed   |
trng_lcg64_count_shift|  3.43e+08  |         1|
#=============================================================================#
        test_name   |ntup| tsamples |psamples|  p-value |Assessment
#=============================================================================#
   diehard_birthdays|   0|       100|     100|0.19220688|  PASSED
      diehard_operm5|   0|   1000000|     100|0.46112638|  PASSED
  diehard_rank_32x32|   0|     40000|     100|0.16090630|  PASSED
    diehard_rank_6x8|   0|    100000|     100|0.26485503|  PASSED
   diehard_bitstream|   0|   2097152|     100|0.41415375|  PASSED
        diehard_opso|   0|   2097152|     100|0.73517284|  PASSED
        diehard_oqso|   0|   2097152|     100|0.92539986|  PASSED
         diehard_dna|   0|   2097152|     100|0.22265898|  PASSED
diehard_count_1s_str|   0|    256000|     100|0.91293324|  PASSED
diehard_count_1s_byt|   0|    256000|     100|0.86629370|  PASSED
 diehard_parking_lot|   0|     12000|     100|0.38297064|  PASSED
    diehard_2dsphere|   2|      8000|     100|0.00546772|  PASSED
    diehard_3dsphere|   3|      4000|     100|0.37230275|  PASSED
     diehard_squeeze|   0|    100000|     100|0.99456179|  PASSED
        diehard_sums|   0|       100|     100|0.17393177|  PASSED
        diehard_runs|   0|    100000|     100|0.40323699|  PASSED
        diehard_runs|   0|    100000|     100|0.88106897|  PASSED
       diehard_craps|   0|    200000|     100|0.00830269|  PASSED
       diehard_craps|   0|    200000|     100|0.23812122|  PASSED
 marsaglia_tsang_gcd|   0|  10000000|     100|0.07524461|  PASSED
 marsaglia_tsang_gcd|   0|  10000000|     100|0.87068723|  PASSED
```

```
         sts_monobit|   1|    100000|     100|0.47207665|  PASSED
            sts_runs|   2|    100000|     100|0.58748082|  PASSED
          sts_serial|   1|    100000|     100|0.59849981|  PASSED
          sts_serial|   2|    100000|     100|0.56041324|  PASSED
          sts_serial|   3|    100000|     100|0.84120126|  PASSED
          sts_serial|   3|    100000|     100|0.95840066|  PASSED
          sts_serial|   4|    100000|     100|0.62179964|  PASSED
          sts_serial|   4|    100000|     100|0.96542775|  PASSED
          sts_serial|   5|    100000|     100|0.94685715|  PASSED
          sts_serial|   5|    100000|     100|0.82759413|  PASSED
          sts_serial|   6|    100000|     100|0.83985310|  PASSED
          sts_serial|   6|    100000|     100|0.72094509|  PASSED
          sts_serial|   7|    100000|     100|0.39817750|  PASSED
          sts_serial|   7|    100000|     100|0.93145178|  PASSED
          sts_serial|   8|    100000|     100|0.78348467|  PASSED
          sts_serial|   8|    100000|     100|0.62609447|  PASSED
          sts_serial|   9|    100000|     100|0.38939764|  PASSED
          sts_serial|   9|    100000|     100|0.44453468|  PASSED
          sts_serial|  10|    100000|     100|0.63789022|  PASSED
          sts_serial|  10|    100000|     100|0.62562069|  PASSED
          sts_serial|  11|    100000|     100|0.78316195|  PASSED
          sts_serial|  11|    100000|     100|0.00678674|  PASSED
          sts_serial|  12|    100000|     100|0.11241866|  PASSED
          sts_serial|  12|    100000|     100|0.24428284|  PASSED
          sts_serial|  13|    100000|     100|0.19265822|  PASSED
          sts_serial|  13|    100000|     100|0.75064285|  PASSED
          sts_serial|  14|    100000|     100|0.12624615|  PASSED
          sts_serial|  14|    100000|     100|0.05282448|  PASSED
          sts_serial|  15|    100000|     100|0.40455112|  PASSED
          sts_serial|  15|    100000|     100|0.69949640|  PASSED
          sts_serial|  16|    100000|     100|0.13934079|  PASSED
          sts_serial|  16|    100000|     100|0.33547853|  PASSED
         rgb_bitdist|   1|    100000|     100|0.62831487|  PASSED
         rgb_bitdist|   2|    100000|     100|0.73262984|  PASSED
         rgb_bitdist|   3|    100000|     100|0.82873012|  PASSED
         rgb_bitdist|   4|    100000|     100|0.34956534|  PASSED
         rgb_bitdist|   5|    100000|     100|0.30554084|  PASSED
         rgb_bitdist|   6|    100000|     100|0.88011037|  PASSED
         rgb_bitdist|   7|    100000|     100|0.55449984|  PASSED
         rgb_bitdist|   8|    100000|     100|0.70515162|  PASSED
         rgb_bitdist|   9|    100000|     100|0.56835238|  PASSED
         rgb_bitdist|  10|    100000|     100|0.52289310|  PASSED
         rgb_bitdist|  11|    100000|     100|0.54901452|  PASSED
         rgb_bitdist|  12|    100000|     100|0.28608797|  PASSED
rgb_minimum_distance|   2|     10000|    1000|0.00097648|   WEAK
rgb_minimum_distance|   3|     10000|    1000|0.60729080|  PASSED
rgb_minimum_distance|   4|     10000|    1000|0.80090823|  PASSED
rgb_minimum_distance|   5|     10000|    1000|0.47650063|  PASSED
    rgb_permutations|   2|    100000|     100|0.84511446|  PASSED
    rgb_permutations|   3|    100000|     100|0.86708009|  PASSED
    rgb_permutations|   4|    100000|     100|0.84021202|  PASSED
    rgb_permutations|   5|    100000|     100|0.15426971|  PASSED
      rgb_lagged_sum|   0|   1000000|     100|0.16694998|  PASSED
      rgb_lagged_sum|   1|   1000000|     100|0.66694583|  PASSED
      rgb_lagged_sum|   2|   1000000|     100|0.20380953|  PASSED
      rgb_lagged_sum|   3|   1000000|     100|0.68843657|  PASSED
      rgb_lagged_sum|   4|   1000000|     100|0.96539868|  PASSED
      rgb_lagged_sum|   5|   1000000|     100|0.64944161|  PASSED
      rgb_lagged_sum|   6|   1000000|     100|0.79857774|  PASSED
      rgb_lagged_sum|   7|   1000000|     100|0.16169283|  PASSED
      rgb_lagged_sum|   8|   1000000|     100|0.43661934|  PASSED
      rgb_lagged_sum|   9|   1000000|     100|0.51109469|  PASSED
      rgb_lagged_sum|  10|   1000000|     100|0.98006724|  PASSED
      rgb_lagged_sum|  11|   1000000|     100|0.76329879|  PASSED
      rgb_lagged_sum|  12|   1000000|     100|0.93133180|  PASSED
```

```
      rgb_lagged_sum|  13|   1000000|     100|0.77909043|  PASSED
      rgb_lagged_sum|  14|   1000000|     100|0.98914710|  PASSED
      rgb_lagged_sum|  15|   1000000|     100|0.66343125|  PASSED
      rgb_lagged_sum|  16|   1000000|     100|0.07634240|  PASSED
      rgb_lagged_sum|  17|   1000000|     100|0.46517081|  PASSED
      rgb_lagged_sum|  18|   1000000|     100|0.91920389|  PASSED
      rgb_lagged_sum|  19|   1000000|     100|0.48657310|  PASSED
      rgb_lagged_sum|  20|   1000000|     100|0.74389470|  PASSED
      rgb_lagged_sum|  21|   1000000|     100|0.82435324|  PASSED
      rgb_lagged_sum|  22|   1000000|     100|0.62307084|  PASSED
      rgb_lagged_sum|  23|   1000000|     100|0.63448949|  PASSED
      rgb_lagged_sum|  24|   1000000|     100|0.32025130|  PASSED
      rgb_lagged_sum|  25|   1000000|     100|0.35419288|  PASSED
      rgb_lagged_sum|  26|   1000000|     100|0.07054893|  PASSED
      rgb_lagged_sum|  27|   1000000|     100|0.80192218|  PASSED
      rgb_lagged_sum|  28|   1000000|     100|0.52190163|  PASSED
      rgb_lagged_sum|  29|   1000000|     100|0.61338984|  PASSED
      rgb_lagged_sum|  30|   1000000|     100|0.75635216|  PASSED
      rgb_lagged_sum|  31|   1000000|     100|0.67211299|  PASSED
      rgb_lagged_sum|  32|   1000000|     100|0.93261977|  PASSED
     rgb_kstest_test|   0|     10000|    1000|0.01374281|  PASSED
     dab_bytedistrib|   0|  51200000|       1|0.43241170|  PASSED
             dab_dct| 256|     50000|       1|0.13514590|  PASSED
Preparing to run test 207.  ntuple = 0
        dab_filltree|  32|  15000000|       1|0.00118349|   WEAK
        dab_filltree|  32|  15000000|       1|0.64522988|  PASSED
Preparing to run test 208.  ntuple = 0
       dab_filltree2|   0|   5000000|       1|0.22448093|  PASSED
       dab_filltree2|   1|   5000000|       1|0.28386684|  PASSED
Preparing to run test 209.  ntuple = 0
        dab_monobit2|  12|  65000000|       1|0.97336423|  PASSED
Preparing to run test 210.  ntuple = 0
      dab_birthdays1|  32|      2000|       1|0.79444497|  PASSED
Preparing to run test 211.  ntuple = 0
           dab_opso2|   0|  67108864|       1|0.06413007|  PASSED
           dab_opso2|   1|  67108864|       1|0.44086349|  PASSED
```

Listing 8.4: Test results for random number engine `trng::count128_lcg_shift`.

```
#=============================================================================#
#            dieharder version 3.31.1 Copyright 2003 Robert G. Brown          #
#=============================================================================#
   rng_name    |rands/second|   Seed   |
trng_count128_lcg_shift|  4.08e+08  |         1|
#=============================================================================#
        test_name   |ntup| tsamples |psamples|  p-value |Assessment
#=============================================================================#
   diehard_birthdays|   0|       100|     100|0.35801918|  PASSED
      diehard_operm5|   0|   1000000|     100|0.99185614|  PASSED
  diehard_rank_32x32|   0|     40000|     100|0.82993329|  PASSED
    diehard_rank_6x8|   0|    100000|     100|0.90077461|  PASSED
   diehard_bitstream|   0|   2097152|     100|0.87738122|  PASSED
        diehard_opso|   0|   2097152|     100|0.06701380|  PASSED
        diehard_oqso|   0|   2097152|     100|0.51450702|  PASSED
         diehard_dna|   0|   2097152|     100|0.21058128|  PASSED
diehard_count_1s_str|   0|    256000|     100|0.75289871|  PASSED
diehard_count_1s_byt|   0|    256000|     100|0.98172802|  PASSED
 diehard_parking_lot|   0|     12000|     100|0.26273081|  PASSED
    diehard_2dsphere|   2|      8000|     100|0.79947410|  PASSED
    diehard_3dsphere|   3|      4000|     100|0.59867542|  PASSED
     diehard_squeeze|   0|    100000|     100|0.93075842|  PASSED
        diehard_sums|   0|       100|     100|0.07475184|  PASSED
        diehard_runs|   0|    100000|     100|0.16392620|  PASSED
        diehard_runs|   0|    100000|     100|0.23038844|  PASSED
       diehard_craps|   0|    200000|     100|0.97449017|  PASSED
```

```
       diehard_craps|   0|    200000|     100|0.80505556|  PASSED
 marsaglia_tsang_gcd|   0|  10000000|     100|0.95591868|  PASSED
 marsaglia_tsang_gcd|   0|  10000000|     100|0.68171081|  PASSED
         sts_monobit|   1|    100000|     100|0.67507869|  PASSED
            sts_runs|   2|    100000|     100|0.42274116|  PASSED
          sts_serial|   1|    100000|     100|0.69456066|  PASSED
          sts_serial|   2|    100000|     100|0.63735742|  PASSED
          sts_serial|   3|    100000|     100|0.79683472|  PASSED
          sts_serial|   3|    100000|     100|0.87069444|  PASSED
          sts_serial|   4|    100000|     100|0.73738030|  PASSED
          sts_serial|   4|    100000|     100|0.96524252|  PASSED
          sts_serial|   5|    100000|     100|0.58678646|  PASSED
          sts_serial|   5|    100000|     100|0.45252408|  PASSED
          sts_serial|   6|    100000|     100|0.71307280|  PASSED
          sts_serial|   6|    100000|     100|0.98595803|  PASSED
          sts_serial|   7|    100000|     100|0.94613385|  PASSED
          sts_serial|   7|    100000|     100|0.48522574|  PASSED
          sts_serial|   8|    100000|     100|0.19202299|  PASSED
          sts_serial|   8|    100000|     100|0.14329665|  PASSED
          sts_serial|   9|    100000|     100|0.70987906|  PASSED
          sts_serial|   9|    100000|     100|0.19301131|  PASSED
          sts_serial|  10|    100000|     100|0.46477239|  PASSED
          sts_serial|  10|    100000|     100|0.36168578|  PASSED
          sts_serial|  11|    100000|     100|0.28853009|  PASSED
          sts_serial|  11|    100000|     100|0.31050009|  PASSED
          sts_serial|  12|    100000|     100|0.10101634|  PASSED
          sts_serial|  12|    100000|     100|0.69869673|  PASSED
          sts_serial|  13|    100000|     100|0.74489746|  PASSED
          sts_serial|  13|    100000|     100|0.79079765|  PASSED
          sts_serial|  14|    100000|     100|0.04626907|  PASSED
          sts_serial|  14|    100000|     100|0.43223176|  PASSED
          sts_serial|  15|    100000|     100|0.50337815|  PASSED
          sts_serial|  15|    100000|     100|0.55978493|  PASSED
          sts_serial|  16|    100000|     100|0.99396585|  PASSED
          sts_serial|  16|    100000|     100|0.61917512|  PASSED
         rgb_bitdist|   1|    100000|     100|0.05373182|  PASSED
         rgb_bitdist|   2|    100000|     100|0.51670052|  PASSED
         rgb_bitdist|   3|    100000|     100|0.97160047|  PASSED
         rgb_bitdist|   4|    100000|     100|0.27481918|  PASSED
         rgb_bitdist|   5|    100000|     100|0.46113333|  PASSED
         rgb_bitdist|   6|    100000|     100|0.89289505|  PASSED
         rgb_bitdist|   7|    100000|     100|0.89464258|  PASSED
         rgb_bitdist|   8|    100000|     100|0.13562437|  PASSED
         rgb_bitdist|   9|    100000|     100|0.26077860|  PASSED
         rgb_bitdist|  10|    100000|     100|0.84103181|  PASSED
         rgb_bitdist|  11|    100000|     100|0.36728353|  PASSED
         rgb_bitdist|  12|    100000|     100|0.78204833|  PASSED
rgb_minimum_distance|   2|     10000|    1000|0.97490100|  PASSED
rgb_minimum_distance|   3|     10000|    1000|0.55485573|  PASSED
rgb_minimum_distance|   4|     10000|    1000|0.12960262|  PASSED
rgb_minimum_distance|   5|     10000|    1000|0.75990390|  PASSED
    rgb_permutations|   2|    100000|     100|0.31797699|  PASSED
    rgb_permutations|   3|    100000|     100|0.87516224|  PASSED
    rgb_permutations|   4|    100000|     100|0.79355155|  PASSED
    rgb_permutations|   5|    100000|     100|0.96724561|  PASSED
      rgb_lagged_sum|   0|   1000000|     100|0.89106891|  PASSED
      rgb_lagged_sum|   1|   1000000|     100|0.15265973|  PASSED
      rgb_lagged_sum|   2|   1000000|     100|0.70501702|  PASSED
      rgb_lagged_sum|   3|   1000000|     100|0.08127639|  PASSED
      rgb_lagged_sum|   4|   1000000|     100|0.86480183|  PASSED
      rgb_lagged_sum|   5|   1000000|     100|0.19121893|  PASSED
      rgb_lagged_sum|   6|   1000000|     100|0.57583666|  PASSED
      rgb_lagged_sum|   7|   1000000|     100|0.02378204|  PASSED
      rgb_lagged_sum|   8|   1000000|     100|0.87657242|  PASSED
      rgb_lagged_sum|   9|   1000000|     100|0.89451603|  PASSED
```

```
      rgb_lagged_sum|  10|   1000000|     100|0.98849570|  PASSED
      rgb_lagged_sum|  11|   1000000|     100|0.84968520|  PASSED
      rgb_lagged_sum|  12|   1000000|     100|0.61614954|  PASSED
      rgb_lagged_sum|  13|   1000000|     100|0.51202054|  PASSED
      rgb_lagged_sum|  14|   1000000|     100|0.92943122|  PASSED
      rgb_lagged_sum|  15|   1000000|     100|0.99991348|   WEAK
      rgb_lagged_sum|  16|   1000000|     100|0.88098659|  PASSED
      rgb_lagged_sum|  17|   1000000|     100|0.53143797|  PASSED
      rgb_lagged_sum|  18|   1000000|     100|0.43015101|  PASSED
      rgb_lagged_sum|  19|   1000000|     100|0.14667026|  PASSED
      rgb_lagged_sum|  20|   1000000|     100|0.94098362|  PASSED
      rgb_lagged_sum|  21|   1000000|     100|0.55321935|  PASSED
      rgb_lagged_sum|  22|   1000000|     100|0.83700606|  PASSED
      rgb_lagged_sum|  23|   1000000|     100|0.99077930|  PASSED
      rgb_lagged_sum|  24|   1000000|     100|0.44546520|  PASSED
      rgb_lagged_sum|  25|   1000000|     100|0.72420217|  PASSED
      rgb_lagged_sum|  26|   1000000|     100|0.06459548|  PASSED
      rgb_lagged_sum|  27|   1000000|     100|0.39762782|  PASSED
      rgb_lagged_sum|  28|   1000000|     100|0.62454736|  PASSED
      rgb_lagged_sum|  29|   1000000|     100|0.30615025|  PASSED
      rgb_lagged_sum|  30|   1000000|     100|0.40489461|  PASSED
      rgb_lagged_sum|  31|   1000000|     100|0.44586634|  PASSED
      rgb_lagged_sum|  32|   1000000|     100|0.69089895|  PASSED
     rgb_kstest_test|   0|     10000|    1000|0.53123568|  PASSED
     dab_bytedistrib|   0|  51200000|       1|0.38110268|  PASSED
             dab_dct| 256|     50000|       1|0.63561465|  PASSED
Preparing to run test 207.  ntuple = 0
        dab_filltree|  32|  15000000|       1|0.13784139|  PASSED
        dab_filltree|  32|  15000000|       1|0.31720717|  PASSED
Preparing to run test 208.  ntuple = 0
       dab_filltree2|   0|   5000000|       1|0.04877886|  PASSED
       dab_filltree2|   1|   5000000|       1|0.83292891|  PASSED
Preparing to run test 209.  ntuple = 0
        dab_monobit2|  12|  65000000|       1|0.29374119|  PASSED
Preparing to run test 210.  ntuple = 0
      dab_birthdays1|  32|      2000|       1|0.11097829|  PASSED
Preparing to run test 211.  ntuple = 0
           dab_opso2|   0|  67108864|       1|0.16037172|  PASSED
           dab_opso2|   1|  67108864|       1|0.54250088|  PASSED
```

Listing 8.5: Test results for random number engine `trng::mrg2`.

```
#=============================================================================#
#            dieharder version 3.31.1 Copyright 2003 Robert G. Brown          #
#=============================================================================#
   rng_name    |rands/second|   Seed   |
      trng_mrg2|  2.71e+08  |         1|
#=============================================================================#
        test_name   |ntup| tsamples |psamples|  p-value |Assessment
#=============================================================================#
   diehard_birthdays|   0|       100|     100|0.97456085|  PASSED
      diehard_operm5|   0|   1000000|     100|0.42102987|  PASSED
  diehard_rank_32x32|   0|     40000|     100|0.25050460|  PASSED
    diehard_rank_6x8|   0|    100000|     100|0.82676859|  PASSED
   diehard_bitstream|   0|   2097152|     100|0.76888696|  PASSED
        diehard_opso|   0|   2097152|     100|0.70005356|  PASSED
        diehard_oqso|   0|   2097152|     100|0.28435715|  PASSED
         diehard_dna|   0|   2097152|     100|0.00000000|  FAILED
diehard_count_1s_str|   0|    256000|     100|0.24728747|  PASSED
diehard_count_1s_byt|   0|    256000|     100|0.09010313|  PASSED
 diehard_parking_lot|   0|     12000|     100|0.33862926|  PASSED
    diehard_2dsphere|   2|      8000|     100|0.73879334|  PASSED
    diehard_3dsphere|   3|      4000|     100|0.58485532|  PASSED
     diehard_squeeze|   0|    100000|     100|0.08506918|  PASSED
        diehard_sums|   0|       100|     100|0.09809123|  PASSED
```

```
        diehard_runs|   0|    100000|     100|0.83531916|  PASSED
        diehard_runs|   0|    100000|     100|0.01771539|  PASSED
       diehard_craps|   0|    200000|     100|0.23770609|  PASSED
       diehard_craps|   0|    200000|     100|0.96885120|  PASSED
 marsaglia_tsang_gcd|   0|  10000000|     100|0.58403329|  PASSED
 marsaglia_tsang_gcd|   0|  10000000|     100|0.22603711|  PASSED
         sts_monobit|   1|    100000|     100|0.84287051|  PASSED
            sts_runs|   2|    100000|     100|0.90657374|  PASSED
          sts_serial|   1|    100000|     100|0.98165491|  PASSED
          sts_serial|   2|    100000|     100|0.30198565|  PASSED
          sts_serial|   3|    100000|     100|0.82345328|  PASSED
          sts_serial|   3|    100000|     100|0.95486983|  PASSED
          sts_serial|   4|    100000|     100|0.82856291|  PASSED
          sts_serial|   4|    100000|     100|0.41428536|  PASSED
          sts_serial|   5|    100000|     100|0.49167397|  PASSED
          sts_serial|   5|    100000|     100|0.40095911|  PASSED
          sts_serial|   6|    100000|     100|0.36821502|  PASSED
          sts_serial|   6|    100000|     100|0.01184629|  PASSED
          sts_serial|   7|    100000|     100|0.37041881|  PASSED
          sts_serial|   7|    100000|     100|0.86860994|  PASSED
          sts_serial|   8|    100000|     100|0.92068431|  PASSED
          sts_serial|   8|    100000|     100|0.35370636|  PASSED
          sts_serial|   9|    100000|     100|0.78935493|  PASSED
          sts_serial|   9|    100000|     100|0.49510250|  PASSED
          sts_serial|  10|    100000|     100|0.82733778|  PASSED
          sts_serial|  10|    100000|     100|0.15763873|  PASSED
          sts_serial|  11|    100000|     100|0.55189214|  PASSED
          sts_serial|  11|    100000|     100|0.37427289|  PASSED
          sts_serial|  12|    100000|     100|0.41843196|  PASSED
          sts_serial|  12|    100000|     100|0.39004645|  PASSED
          sts_serial|  13|    100000|     100|0.56174150|  PASSED
          sts_serial|  13|    100000|     100|0.60077911|  PASSED
          sts_serial|  14|    100000|     100|0.41235514|  PASSED
          sts_serial|  14|    100000|     100|0.97075235|  PASSED
          sts_serial|  15|    100000|     100|0.81591354|  PASSED
          sts_serial|  15|    100000|     100|0.78431493|  PASSED
          sts_serial|  16|    100000|     100|0.99493283|  PASSED
          sts_serial|  16|    100000|     100|0.79504579|  PASSED
         rgb_bitdist|   1|    100000|     100|0.09585792|  PASSED
         rgb_bitdist|   2|    100000|     100|0.76676670|  PASSED
         rgb_bitdist|   3|    100000|     100|0.05679406|  PASSED
         rgb_bitdist|   4|    100000|     100|0.40410500|  PASSED
         rgb_bitdist|   5|    100000|     100|0.97925524|  PASSED
         rgb_bitdist|   6|    100000|     100|0.63998903|  PASSED
         rgb_bitdist|   7|    100000|     100|0.94135745|  PASSED
         rgb_bitdist|   8|    100000|     100|0.80578856|  PASSED
         rgb_bitdist|   9|    100000|     100|0.80366021|  PASSED
         rgb_bitdist|  10|    100000|     100|0.43410305|  PASSED
         rgb_bitdist|  11|    100000|     100|0.97147486|  PASSED
         rgb_bitdist|  12|    100000|     100|0.83499011|  PASSED
rgb_minimum_distance|   2|     10000|    1000|0.21541403|  PASSED
rgb_minimum_distance|   3|     10000|    1000|0.19054133|  PASSED
rgb_minimum_distance|   4|     10000|    1000|0.41723923|  PASSED
rgb_minimum_distance|   5|     10000|    1000|0.73844521|  PASSED
    rgb_permutations|   2|    100000|     100|0.92283778|  PASSED
    rgb_permutations|   3|    100000|     100|0.63768918|  PASSED
    rgb_permutations|   4|    100000|     100|0.04385192|  PASSED
    rgb_permutations|   5|    100000|     100|0.88176159|  PASSED
      rgb_lagged_sum|   0|   1000000|     100|0.10986190|  PASSED
      rgb_lagged_sum|   1|   1000000|     100|0.87609581|  PASSED
      rgb_lagged_sum|   2|   1000000|     100|0.47333172|  PASSED
      rgb_lagged_sum|   3|   1000000|     100|0.57005802|  PASSED
      rgb_lagged_sum|   4|   1000000|     100|0.33944061|  PASSED
      rgb_lagged_sum|   5|   1000000|     100|0.23549046|  PASSED
      rgb_lagged_sum|   6|   1000000|     100|0.85257616|  PASSED
```

```
      rgb_lagged_sum|   7|   1000000|     100|0.87179787|  PASSED
      rgb_lagged_sum|   8|   1000000|     100|0.68882448|  PASSED
      rgb_lagged_sum|   9|   1000000|     100|0.26446102|  PASSED
      rgb_lagged_sum|  10|   1000000|     100|0.73092010|  PASSED
      rgb_lagged_sum|  11|   1000000|     100|0.89656017|  PASSED
      rgb_lagged_sum|  12|   1000000|     100|0.71584657|  PASSED
      rgb_lagged_sum|  13|   1000000|     100|0.82650127|  PASSED
      rgb_lagged_sum|  14|   1000000|     100|0.96296122|  PASSED
      rgb_lagged_sum|  15|   1000000|     100|0.73040047|  PASSED
      rgb_lagged_sum|  16|   1000000|     100|0.93228283|  PASSED
      rgb_lagged_sum|  17|   1000000|     100|0.97149590|  PASSED
      rgb_lagged_sum|  18|   1000000|     100|0.97403740|  PASSED
      rgb_lagged_sum|  19|   1000000|     100|0.67916812|  PASSED
      rgb_lagged_sum|  20|   1000000|     100|0.22310353|  PASSED
      rgb_lagged_sum|  21|   1000000|     100|0.56079387|  PASSED
      rgb_lagged_sum|  22|   1000000|     100|0.08145667|  PASSED
      rgb_lagged_sum|  23|   1000000|     100|0.29937727|  PASSED
      rgb_lagged_sum|  24|   1000000|     100|0.78452209|  PASSED
      rgb_lagged_sum|  25|   1000000|     100|0.46168224|  PASSED
      rgb_lagged_sum|  26|   1000000|     100|0.27158576|  PASSED
      rgb_lagged_sum|  27|   1000000|     100|0.39502427|  PASSED
      rgb_lagged_sum|  28|   1000000|     100|0.18650499|  PASSED
      rgb_lagged_sum|  29|   1000000|     100|0.10807963|  PASSED
      rgb_lagged_sum|  30|   1000000|     100|0.09519386|  PASSED
      rgb_lagged_sum|  31|   1000000|     100|0.16943750|  PASSED
      rgb_lagged_sum|  32|   1000000|     100|0.66818656|  PASSED
     rgb_kstest_test|   0|     10000|    1000|0.20120409|  PASSED
     dab_bytedistrib|   0|  51200000|       1|0.60953965|  PASSED
             dab_dct| 256|     50000|       1|0.71847146|  PASSED
Preparing to run test 207.  ntuple = 0
        dab_filltree|  32|  15000000|       1|0.80335287|  PASSED
        dab_filltree|  32|  15000000|       1|0.49203626|  PASSED
Preparing to run test 208.  ntuple = 0
       dab_filltree2|   0|   5000000|       1|0.45772549|  PASSED
       dab_filltree2|   1|   5000000|       1|0.82131380|  PASSED
Preparing to run test 209.  ntuple = 0
        dab_monobit2|  12|  65000000|       1|0.09136968|  PASSED
Preparing to run test 210.  ntuple = 0
      dab_birthdays1|  31|      2000|       1|0.70998848|  PASSED
Preparing to run test 211.  ntuple = 0
           dab_opso2|   0|  67108864|       1|0.28404870|  PASSED
           dab_opso2|   1|  67108864|       1|0.47765606|  PASSED
```

Listing 8.6: Test results for random number engine `trng::mrg3`.

```
#=============================================================================#
#            dieharder version 3.31.1 Copyright 2003 Robert G. Brown          #
#=============================================================================#
   rng_name    |rands/second|   Seed   |
      trng_mrg3|  1.36e+08  |         1|
#=============================================================================#
        test_name   |ntup| tsamples |psamples|  p-value |Assessment
#=============================================================================#
   diehard_birthdays|   0|       100|     100|0.75972430|  PASSED
      diehard_operm5|   0|   1000000|     100|0.88738841|  PASSED
  diehard_rank_32x32|   0|     40000|     100|0.95524679|  PASSED
    diehard_rank_6x8|   0|    100000|     100|0.13099303|  PASSED
   diehard_bitstream|   0|   2097152|     100|0.34440501|  PASSED
        diehard_opso|   0|   2097152|     100|0.71862144|  PASSED
        diehard_oqso|   0|   2097152|     100|0.61438599|  PASSED
         diehard_dna|   0|   2097152|     100|0.00000000|  FAILED
diehard_count_1s_str|   0|    256000|     100|0.23768833|  PASSED
diehard_count_1s_byt|   0|    256000|     100|0.53048964|  PASSED
 diehard_parking_lot|   0|     12000|     100|0.69464262|  PASSED
    diehard_2dsphere|   2|      8000|     100|0.38326870|  PASSED
```

```
    diehard_3dsphere|   3|      4000|     100|0.54790464|  PASSED
     diehard_squeeze|   0|    100000|     100|0.49961532|  PASSED
        diehard_sums|   0|       100|     100|0.53094046|  PASSED
        diehard_runs|   0|    100000|     100|0.08622091|  PASSED
        diehard_runs|   0|    100000|     100|0.00813374|  PASSED
       diehard_craps|   0|    200000|     100|0.75047690|  PASSED
       diehard_craps|   0|    200000|     100|0.41044110|  PASSED
 marsaglia_tsang_gcd|   0|  10000000|     100|0.78162398|  PASSED
 marsaglia_tsang_gcd|   0|  10000000|     100|0.11045082|  PASSED
         sts_monobit|   1|    100000|     100|0.13941398|  PASSED
            sts_runs|   2|    100000|     100|0.64021240|  PASSED
          sts_serial|   1|    100000|     100|0.44874029|  PASSED
          sts_serial|   2|    100000|     100|0.13641927|  PASSED
          sts_serial|   3|    100000|     100|0.14831787|  PASSED
          sts_serial|   3|    100000|     100|0.82884228|  PASSED
          sts_serial|   4|    100000|     100|0.99403532|  PASSED
          sts_serial|   4|    100000|     100|0.27399104|  PASSED
          sts_serial|   5|    100000|     100|0.28881899|  PASSED
          sts_serial|   5|    100000|     100|0.16218286|  PASSED
          sts_serial|   6|    100000|     100|0.61859089|  PASSED
          sts_serial|   6|    100000|     100|0.58222305|  PASSED
          sts_serial|   7|    100000|     100|0.47067986|  PASSED
          sts_serial|   7|    100000|     100|0.68554261|  PASSED
          sts_serial|   8|    100000|     100|0.84904455|  PASSED
          sts_serial|   8|    100000|     100|0.75039320|  PASSED
          sts_serial|   9|    100000|     100|0.74130372|  PASSED
          sts_serial|   9|    100000|     100|0.78004719|  PASSED
          sts_serial|  10|    100000|     100|0.08404340|  PASSED
          sts_serial|  10|    100000|     100|0.22474504|  PASSED
          sts_serial|  11|    100000|     100|0.28609083|  PASSED
          sts_serial|  11|    100000|     100|0.98091180|  PASSED
          sts_serial|  12|    100000|     100|0.64928267|  PASSED
          sts_serial|  12|    100000|     100|0.92432108|  PASSED
          sts_serial|  13|    100000|     100|0.06539019|  PASSED
          sts_serial|  13|    100000|     100|0.67744144|  PASSED
          sts_serial|  14|    100000|     100|0.01661684|  PASSED
          sts_serial|  14|    100000|     100|0.02823713|  PASSED
          sts_serial|  15|    100000|     100|0.00042879|   WEAK
          sts_serial|  15|    100000|     100|0.69563132|  PASSED
          sts_serial|  16|    100000|     100|0.04086226|  PASSED
          sts_serial|  16|    100000|     100|0.83671404|  PASSED
         rgb_bitdist|   1|    100000|     100|0.89769723|  PASSED
         rgb_bitdist|   2|    100000|     100|0.36279211|  PASSED
         rgb_bitdist|   3|    100000|     100|0.46030717|  PASSED
         rgb_bitdist|   4|    100000|     100|0.41306369|  PASSED
         rgb_bitdist|   5|    100000|     100|0.83091374|  PASSED
         rgb_bitdist|   6|    100000|     100|0.67717850|  PASSED
         rgb_bitdist|   7|    100000|     100|0.15973390|  PASSED
         rgb_bitdist|   8|    100000|     100|0.78407223|  PASSED
         rgb_bitdist|   9|    100000|     100|0.57702417|  PASSED
         rgb_bitdist|  10|    100000|     100|0.96776432|  PASSED
         rgb_bitdist|  11|    100000|     100|0.73533852|  PASSED
         rgb_bitdist|  12|    100000|     100|0.66985201|  PASSED
rgb_minimum_distance|   2|     10000|    1000|0.19365712|  PASSED
rgb_minimum_distance|   3|     10000|    1000|0.85470859|  PASSED
rgb_minimum_distance|   4|     10000|    1000|0.74206172|  PASSED
rgb_minimum_distance|   5|     10000|    1000|0.42258041|  PASSED
    rgb_permutations|   2|    100000|     100|0.02162391|  PASSED
    rgb_permutations|   3|    100000|     100|0.43178285|  PASSED
    rgb_permutations|   4|    100000|     100|0.69314551|  PASSED
    rgb_permutations|   5|    100000|     100|0.93383525|  PASSED
      rgb_lagged_sum|   0|   1000000|     100|0.86135203|  PASSED
      rgb_lagged_sum|   1|   1000000|     100|0.94630170|  PASSED
      rgb_lagged_sum|   2|   1000000|     100|0.32044961|  PASSED
      rgb_lagged_sum|   3|   1000000|     100|0.14276871|  PASSED
```

```
      rgb_lagged_sum|   4|   1000000|     100|0.97209054|  PASSED
      rgb_lagged_sum|   5|   1000000|     100|0.49728571|  PASSED
      rgb_lagged_sum|   6|   1000000|     100|0.36068651|  PASSED
      rgb_lagged_sum|   7|   1000000|     100|0.83608707|  PASSED
      rgb_lagged_sum|   8|   1000000|     100|0.09209523|  PASSED
      rgb_lagged_sum|   9|   1000000|     100|0.48610107|  PASSED
      rgb_lagged_sum|  10|   1000000|     100|0.43271089|  PASSED
      rgb_lagged_sum|  11|   1000000|     100|0.99020121|  PASSED
      rgb_lagged_sum|  12|   1000000|     100|0.98929552|  PASSED
      rgb_lagged_sum|  13|   1000000|     100|0.88450255|  PASSED
      rgb_lagged_sum|  14|   1000000|     100|0.75810054|  PASSED
      rgb_lagged_sum|  15|   1000000|     100|0.02213710|  PASSED
      rgb_lagged_sum|  16|   1000000|     100|0.84294235|  PASSED
      rgb_lagged_sum|  17|   1000000|     100|0.80565737|  PASSED
      rgb_lagged_sum|  18|   1000000|     100|0.94299778|  PASSED
      rgb_lagged_sum|  19|   1000000|     100|0.60370921|  PASSED
      rgb_lagged_sum|  20|   1000000|     100|0.31426755|  PASSED
      rgb_lagged_sum|  21|   1000000|     100|0.25480358|  PASSED
      rgb_lagged_sum|  22|   1000000|     100|0.05406616|  PASSED
      rgb_lagged_sum|  23|   1000000|     100|0.61975504|  PASSED
      rgb_lagged_sum|  24|   1000000|     100|0.22619589|  PASSED
      rgb_lagged_sum|  25|   1000000|     100|0.22401870|  PASSED
      rgb_lagged_sum|  26|   1000000|     100|0.91469888|  PASSED
      rgb_lagged_sum|  27|   1000000|     100|0.95230815|  PASSED
      rgb_lagged_sum|  28|   1000000|     100|0.14240533|  PASSED
      rgb_lagged_sum|  29|   1000000|     100|0.90262109|  PASSED
      rgb_lagged_sum|  30|   1000000|     100|0.46638997|  PASSED
      rgb_lagged_sum|  31|   1000000|     100|0.25486766|  PASSED
      rgb_lagged_sum|  32|   1000000|     100|0.07400606|  PASSED
     rgb_kstest_test|   0|     10000|    1000|0.10561962|  PASSED
     dab_bytedistrib|   0|  51200000|       1|0.48899786|  PASSED
             dab_dct| 256|     50000|       1|0.52244899|  PASSED
Preparing to run test 207.  ntuple = 0
        dab_filltree|  32|  15000000|       1|0.46348975|  PASSED
        dab_filltree|  32|  15000000|       1|0.59051836|  PASSED
Preparing to run test 208.  ntuple = 0
       dab_filltree2|   0|   5000000|       1|0.66349846|  PASSED
       dab_filltree2|   1|   5000000|       1|0.17754684|  PASSED
Preparing to run test 209.  ntuple = 0
        dab_monobit2|  12|  65000000|       1|0.02442625|  PASSED
Preparing to run test 210.  ntuple = 0
      dab_birthdays1|  31|      2000|       1|0.07470451|  PASSED
Preparing to run test 211.  ntuple = 0
           dab_opso2|   0|  67108864|       1|0.37697072|  PASSED
           dab_opso2|   1|  67108864|       1|0.37625204|  PASSED
```

Listing 8.7: Test results for random number engine `trng::mrg3s`.

```
#=============================================================================#
#            dieharder version 3.31.1 Copyright 2003 Robert G. Brown          #
#=============================================================================#
   rng_name    |rands/second|   Seed   |
     trng_mrg3s|  1.74e+08  |         1|
#=============================================================================#
        test_name   |ntup| tsamples |psamples|  p-value |Assessment
#=============================================================================#
   diehard_birthdays|   0|       100|     100|0.90074972|  PASSED
      diehard_operm5|   0|   1000000|     100|0.13559371|  PASSED
  diehard_rank_32x32|   0|     40000|     100|0.98404994|  PASSED
    diehard_rank_6x8|   0|    100000|     100|0.45834406|  PASSED
   diehard_bitstream|   0|   2097152|     100|0.54246791|  PASSED
        diehard_opso|   0|   2097152|     100|0.75518067|  PASSED
        diehard_oqso|   0|   2097152|     100|0.97368197|  PASSED
         diehard_dna|   0|   2097152|     100|0.00000000|  FAILED
diehard_count_1s_str|   0|    256000|     100|0.95808746|  PASSED
```

```
diehard_count_1s_byt|   0|    256000|     100|0.64227546|  PASSED
 diehard_parking_lot|   0|     12000|     100|0.91403506|  PASSED
    diehard_2dsphere|   2|      8000|     100|0.89801903|  PASSED
    diehard_3dsphere|   3|      4000|     100|0.12427624|  PASSED
     diehard_squeeze|   0|    100000|     100|0.56533107|  PASSED
        diehard_sums|   0|       100|     100|0.00434120|   WEAK
        diehard_runs|   0|    100000|     100|0.33533206|  PASSED
        diehard_runs|   0|    100000|     100|0.89068208|  PASSED
       diehard_craps|   0|    200000|     100|0.90754055|  PASSED
       diehard_craps|   0|    200000|     100|0.98637153|  PASSED
 marsaglia_tsang_gcd|   0|  10000000|     100|0.94808779|  PASSED
 marsaglia_tsang_gcd|   0|  10000000|     100|0.63386351|  PASSED
         sts_monobit|   1|    100000|     100|0.98835251|  PASSED
            sts_runs|   2|    100000|     100|0.09697753|  PASSED
          sts_serial|   1|    100000|     100|0.50737056|  PASSED
          sts_serial|   2|    100000|     100|0.11970788|  PASSED
          sts_serial|   3|    100000|     100|0.99996640|   WEAK
          sts_serial|   3|    100000|     100|0.17032899|  PASSED
          sts_serial|   4|    100000|     100|0.49075750|  PASSED
          sts_serial|   4|    100000|     100|0.71152876|  PASSED
          sts_serial|   5|    100000|     100|0.18445227|  PASSED
          sts_serial|   5|    100000|     100|0.09776452|  PASSED
          sts_serial|   6|    100000|     100|0.39082068|  PASSED
          sts_serial|   6|    100000|     100|0.81557081|  PASSED
          sts_serial|   7|    100000|     100|0.70050907|  PASSED
          sts_serial|   7|    100000|     100|0.54297851|  PASSED
          sts_serial|   8|    100000|     100|0.56738270|  PASSED
          sts_serial|   8|    100000|     100|0.81730353|  PASSED
          sts_serial|   9|    100000|     100|0.52682187|  PASSED
          sts_serial|   9|    100000|     100|0.67033124|  PASSED
          sts_serial|  10|    100000|     100|0.60478620|  PASSED
          sts_serial|  10|    100000|     100|0.97876198|  PASSED
          sts_serial|  11|    100000|     100|0.50862813|  PASSED
          sts_serial|  11|    100000|     100|0.89637400|  PASSED
          sts_serial|  12|    100000|     100|0.64031994|  PASSED
          sts_serial|  12|    100000|     100|0.05676877|  PASSED
          sts_serial|  13|    100000|     100|0.39424980|  PASSED
          sts_serial|  13|    100000|     100|0.88848736|  PASSED
          sts_serial|  14|    100000|     100|0.38409990|  PASSED
          sts_serial|  14|    100000|     100|0.70078939|  PASSED
          sts_serial|  15|    100000|     100|0.94034709|  PASSED
          sts_serial|  15|    100000|     100|0.84334476|  PASSED
          sts_serial|  16|    100000|     100|0.76793805|  PASSED
          sts_serial|  16|    100000|     100|0.72837024|  PASSED
         rgb_bitdist|   1|    100000|     100|0.19568252|  PASSED
         rgb_bitdist|   2|    100000|     100|0.61131720|  PASSED
         rgb_bitdist|   3|    100000|     100|0.98660679|  PASSED
         rgb_bitdist|   4|    100000|     100|0.60412165|  PASSED
         rgb_bitdist|   5|    100000|     100|0.95989533|  PASSED
         rgb_bitdist|   6|    100000|     100|0.32607688|  PASSED
         rgb_bitdist|   7|    100000|     100|0.31205838|  PASSED
         rgb_bitdist|   8|    100000|     100|0.94180192|  PASSED
         rgb_bitdist|   9|    100000|     100|0.55916527|  PASSED
         rgb_bitdist|  10|    100000|     100|0.00069878|   WEAK
         rgb_bitdist|  11|    100000|     100|0.56578527|  PASSED
         rgb_bitdist|  12|    100000|     100|0.80604923|  PASSED
rgb_minimum_distance|   2|     10000|    1000|0.24586334|  PASSED
rgb_minimum_distance|   3|     10000|    1000|0.14923287|  PASSED
rgb_minimum_distance|   4|     10000|    1000|0.40639945|  PASSED
rgb_minimum_distance|   5|     10000|    1000|0.89350992|  PASSED
    rgb_permutations|   2|    100000|     100|0.56268068|  PASSED
    rgb_permutations|   3|    100000|     100|0.92155767|  PASSED
    rgb_permutations|   4|    100000|     100|0.26167463|  PASSED
    rgb_permutations|   5|    100000|     100|0.09277354|  PASSED
      rgb_lagged_sum|   0|   1000000|     100|0.60745264|  PASSED
```

```
      rgb_lagged_sum|   1|   1000000|     100|0.82680135|  PASSED
      rgb_lagged_sum|   2|   1000000|     100|0.45097207|  PASSED
      rgb_lagged_sum|   3|   1000000|     100|0.75546764|  PASSED
      rgb_lagged_sum|   4|   1000000|     100|0.45958155|  PASSED
      rgb_lagged_sum|   5|   1000000|     100|0.97443077|  PASSED
      rgb_lagged_sum|   6|   1000000|     100|0.54501424|  PASSED
      rgb_lagged_sum|   7|   1000000|     100|0.25160365|  PASSED
      rgb_lagged_sum|   8|   1000000|     100|0.88642085|  PASSED
      rgb_lagged_sum|   9|   1000000|     100|0.99142118|  PASSED
      rgb_lagged_sum|  10|   1000000|     100|0.82453171|  PASSED
      rgb_lagged_sum|  11|   1000000|     100|0.99055912|  PASSED
      rgb_lagged_sum|  12|   1000000|     100|0.11126477|  PASSED
      rgb_lagged_sum|  13|   1000000|     100|0.46757828|  PASSED
      rgb_lagged_sum|  14|   1000000|     100|0.81391938|  PASSED
      rgb_lagged_sum|  15|   1000000|     100|0.64247685|  PASSED
      rgb_lagged_sum|  16|   1000000|     100|0.48765733|  PASSED
      rgb_lagged_sum|  17|   1000000|     100|0.14238342|  PASSED
      rgb_lagged_sum|  18|   1000000|     100|0.41234328|  PASSED
      rgb_lagged_sum|  19|   1000000|     100|0.74305166|  PASSED
      rgb_lagged_sum|  20|   1000000|     100|0.94082851|  PASSED
      rgb_lagged_sum|  21|   1000000|     100|0.31681272|  PASSED
      rgb_lagged_sum|  22|   1000000|     100|0.38976103|  PASSED
      rgb_lagged_sum|  23|   1000000|     100|0.52445386|  PASSED
      rgb_lagged_sum|  24|   1000000|     100|0.56322953|  PASSED
      rgb_lagged_sum|  25|   1000000|     100|0.97327233|  PASSED
      rgb_lagged_sum|  26|   1000000|     100|0.72987813|  PASSED
      rgb_lagged_sum|  27|   1000000|     100|0.87622997|  PASSED
      rgb_lagged_sum|  28|   1000000|     100|0.74003500|  PASSED
      rgb_lagged_sum|  29|   1000000|     100|0.96431318|  PASSED
      rgb_lagged_sum|  30|   1000000|     100|0.72507398|  PASSED
      rgb_lagged_sum|  31|   1000000|     100|0.03751253|  PASSED
      rgb_lagged_sum|  32|   1000000|     100|0.38933346|  PASSED
     rgb_kstest_test|   0|     10000|    1000|0.60052287|  PASSED
     dab_bytedistrib|   0|  51200000|       1|0.17587305|  PASSED
             dab_dct| 256|     50000|       1|0.06477446|  PASSED
Preparing to run test 207.  ntuple = 0
        dab_filltree|  32|  15000000|       1|0.77648019|  PASSED
        dab_filltree|  32|  15000000|       1|0.97015679|  PASSED
Preparing to run test 208.  ntuple = 0
       dab_filltree2|   0|   5000000|       1|0.56241197|  PASSED
       dab_filltree2|   1|   5000000|       1|0.51301928|  PASSED
Preparing to run test 209.  ntuple = 0
        dab_monobit2|  12|  65000000|       1|0.92875375|  PASSED
Preparing to run test 210.  ntuple = 0
      dab_birthdays1|  31|      2000|       1|0.38497682|  PASSED
Preparing to run test 211.  ntuple = 0
           dab_opso2|   0|  67108864|       1|0.38997751|  PASSED
           dab_opso2|   1|  67108864|       1|0.12121112|  PASSED
```

Listing 8.8: Test results for random number engine `trng::mrg4`.

```
#=============================================================================#
#            dieharder version 3.31.1 Copyright 2003 Robert G. Brown          #
#=============================================================================#
   rng_name    |rands/second|   Seed   |
      trng_mrg4|  1.29e+08  |         1|
#=============================================================================#
        test_name   |ntup| tsamples |psamples|  p-value |Assessment
#=============================================================================#
   diehard_birthdays|   0|       100|     100|0.01941927|  PASSED
      diehard_operm5|   0|   1000000|     100|0.84237098|  PASSED
  diehard_rank_32x32|   0|     40000|     100|0.69824097|  PASSED
    diehard_rank_6x8|   0|    100000|     100|0.58164164|  PASSED
   diehard_bitstream|   0|   2097152|     100|0.85080295|  PASSED
        diehard_opso|   0|   2097152|     100|0.75964938|  PASSED
```

```
        diehard_oqso|   0|   2097152|     100|0.71388946|  PASSED
         diehard_dna|   0|   2097152|     100|0.00000000|  FAILED
diehard_count_1s_str|   0|    256000|     100|0.92212041|  PASSED
diehard_count_1s_byt|   0|    256000|     100|0.86791561|  PASSED
 diehard_parking_lot|   0|     12000|     100|0.55571802|  PASSED
    diehard_2dsphere|   2|      8000|     100|0.77287292|  PASSED
    diehard_3dsphere|   3|      4000|     100|0.93977876|  PASSED
     diehard_squeeze|   0|    100000|     100|0.98256927|  PASSED
        diehard_sums|   0|       100|     100|0.52491096|  PASSED
        diehard_runs|   0|    100000|     100|0.45662335|  PASSED
        diehard_runs|   0|    100000|     100|0.68684006|  PASSED
       diehard_craps|   0|    200000|     100|0.97840077|  PASSED
       diehard_craps|   0|    200000|     100|0.18829111|  PASSED
 marsaglia_tsang_gcd|   0|  10000000|     100|0.16570121|  PASSED
 marsaglia_tsang_gcd|   0|  10000000|     100|0.42742468|  PASSED
         sts_monobit|   1|    100000|     100|0.23496621|  PASSED
            sts_runs|   2|    100000|     100|0.01607208|  PASSED
          sts_serial|   1|    100000|     100|0.84062368|  PASSED
          sts_serial|   2|    100000|     100|0.64585519|  PASSED
          sts_serial|   3|    100000|     100|0.25054339|  PASSED
          sts_serial|   3|    100000|     100|0.45855011|  PASSED
          sts_serial|   4|    100000|     100|0.98503050|  PASSED
          sts_serial|   4|    100000|     100|0.74284138|  PASSED
          sts_serial|   5|    100000|     100|0.27846702|  PASSED
          sts_serial|   5|    100000|     100|0.78407086|  PASSED
          sts_serial|   6|    100000|     100|0.51205263|  PASSED
          sts_serial|   6|    100000|     100|0.21836078|  PASSED
          sts_serial|   7|    100000|     100|0.76514739|  PASSED
          sts_serial|   7|    100000|     100|0.78915555|  PASSED
          sts_serial|   8|    100000|     100|0.89785183|  PASSED
          sts_serial|   8|    100000|     100|0.68664011|  PASSED
          sts_serial|   9|    100000|     100|0.95427504|  PASSED
          sts_serial|   9|    100000|     100|0.55337718|  PASSED
          sts_serial|  10|    100000|     100|0.09790442|  PASSED
          sts_serial|  10|    100000|     100|0.04488362|  PASSED
          sts_serial|  11|    100000|     100|0.06035642|  PASSED
          sts_serial|  11|    100000|     100|0.26341142|  PASSED
          sts_serial|  12|    100000|     100|0.05740620|  PASSED
          sts_serial|  12|    100000|     100|0.95192690|  PASSED
          sts_serial|  13|    100000|     100|0.64896626|  PASSED
          sts_serial|  13|    100000|     100|0.83444406|  PASSED
          sts_serial|  14|    100000|     100|0.66693619|  PASSED
          sts_serial|  14|    100000|     100|0.58528618|  PASSED
          sts_serial|  15|    100000|     100|0.99190484|  PASSED
          sts_serial|  15|    100000|     100|0.94484775|  PASSED
          sts_serial|  16|    100000|     100|0.78187039|  PASSED
          sts_serial|  16|    100000|     100|0.27848388|  PASSED
         rgb_bitdist|   1|    100000|     100|0.75933958|  PASSED
         rgb_bitdist|   2|    100000|     100|0.98571631|  PASSED
         rgb_bitdist|   3|    100000|     100|0.93202487|  PASSED
         rgb_bitdist|   4|    100000|     100|0.32564629|  PASSED
         rgb_bitdist|   5|    100000|     100|0.94285515|  PASSED
         rgb_bitdist|   6|    100000|     100|0.52071880|  PASSED
         rgb_bitdist|   7|    100000|     100|0.91282463|  PASSED
         rgb_bitdist|   8|    100000|     100|0.99718745|   WEAK
         rgb_bitdist|   9|    100000|     100|0.35556465|  PASSED
         rgb_bitdist|  10|    100000|     100|0.92657492|  PASSED
         rgb_bitdist|  11|    100000|     100|0.70641139|  PASSED
         rgb_bitdist|  12|    100000|     100|0.19856863|  PASSED
rgb_minimum_distance|   2|     10000|    1000|0.63728980|  PASSED
rgb_minimum_distance|   3|     10000|    1000|0.77225483|  PASSED
rgb_minimum_distance|   4|     10000|    1000|0.35955852|  PASSED
rgb_minimum_distance|   5|     10000|    1000|0.08146534|  PASSED
    rgb_permutations|   2|    100000|     100|0.65783602|  PASSED
    rgb_permutations|   3|    100000|     100|0.48572960|  PASSED
```

```
    rgb_permutations|   4|    100000|     100|0.43321790|  PASSED
    rgb_permutations|   5|    100000|     100|0.50990441|  PASSED
      rgb_lagged_sum|   0|   1000000|     100|0.13297315|  PASSED
      rgb_lagged_sum|   1|   1000000|     100|0.18889906|  PASSED
      rgb_lagged_sum|   2|   1000000|     100|0.28959174|  PASSED
      rgb_lagged_sum|   3|   1000000|     100|0.66152254|  PASSED
      rgb_lagged_sum|   4|   1000000|     100|0.18922290|  PASSED
      rgb_lagged_sum|   5|   1000000|     100|0.31118731|  PASSED
      rgb_lagged_sum|   6|   1000000|     100|0.02388791|  PASSED
      rgb_lagged_sum|   7|   1000000|     100|0.79430977|  PASSED
      rgb_lagged_sum|   8|   1000000|     100|0.98784967|  PASSED
      rgb_lagged_sum|   9|   1000000|     100|0.36857769|  PASSED
      rgb_lagged_sum|  10|   1000000|     100|0.60103647|  PASSED
      rgb_lagged_sum|  11|   1000000|     100|0.69340293|  PASSED
      rgb_lagged_sum|  12|   1000000|     100|0.23984011|  PASSED
      rgb_lagged_sum|  13|   1000000|     100|0.10629317|  PASSED
      rgb_lagged_sum|  14|   1000000|     100|0.30542255|  PASSED
      rgb_lagged_sum|  15|   1000000|     100|0.23826975|  PASSED
      rgb_lagged_sum|  16|   1000000|     100|0.47171427|  PASSED
      rgb_lagged_sum|  17|   1000000|     100|0.35349025|  PASSED
      rgb_lagged_sum|  18|   1000000|     100|0.88708050|  PASSED
      rgb_lagged_sum|  19|   1000000|     100|0.93725109|  PASSED
      rgb_lagged_sum|  20|   1000000|     100|0.43652685|  PASSED
      rgb_lagged_sum|  21|   1000000|     100|0.10387515|  PASSED
      rgb_lagged_sum|  22|   1000000|     100|0.57215833|  PASSED
      rgb_lagged_sum|  23|   1000000|     100|0.83217269|  PASSED
      rgb_lagged_sum|  24|   1000000|     100|0.67119264|  PASSED
      rgb_lagged_sum|  25|   1000000|     100|0.33949967|  PASSED
      rgb_lagged_sum|  26|   1000000|     100|0.58801954|  PASSED
      rgb_lagged_sum|  27|   1000000|     100|0.98159394|  PASSED
      rgb_lagged_sum|  28|   1000000|     100|0.94554065|  PASSED
      rgb_lagged_sum|  29|   1000000|     100|0.66727327|  PASSED
      rgb_lagged_sum|  30|   1000000|     100|0.52421312|  PASSED
      rgb_lagged_sum|  31|   1000000|     100|0.11212031|  PASSED
      rgb_lagged_sum|  32|   1000000|     100|0.73059852|  PASSED
     rgb_kstest_test|   0|     10000|    1000|0.98676870|  PASSED
     dab_bytedistrib|   0|  51200000|       1|0.32357327|  PASSED
             dab_dct| 256|     50000|       1|0.04894318|  PASSED
Preparing to run test 207.  ntuple = 0
        dab_filltree|  32|  15000000|       1|0.51911137|  PASSED
        dab_filltree|  32|  15000000|       1|0.29948775|  PASSED
Preparing to run test 208.  ntuple = 0
       dab_filltree2|   0|   5000000|       1|0.35807939|  PASSED
       dab_filltree2|   1|   5000000|       1|0.90167084|  PASSED
Preparing to run test 209.  ntuple = 0
        dab_monobit2|  12|  65000000|       1|0.30236172|  PASSED
Preparing to run test 210.  ntuple = 0
      dab_birthdays1|  31|      2000|       1|0.81892331|  PASSED
Preparing to run test 211.  ntuple = 0
           dab_opso2|   0|  67108864|       1|0.33124678|  PASSED
           dab_opso2|   1|  67108864|       1|0.88210666|  PASSED
```

Listing 8.9: Test results for random number engine `trng::mrg5`.

```
#=============================================================================#
#            dieharder version 3.31.1 Copyright 2003 Robert G. Brown          #
#=============================================================================#
   rng_name    |rands/second|   Seed   |
      trng_mrg5|  2.17e+08  |         1|
#=============================================================================#
        test_name   |ntup| tsamples |psamples|  p-value |Assessment
#=============================================================================#
   diehard_birthdays|   0|       100|     100|0.68636854|  PASSED
      diehard_operm5|   0|   1000000|     100|0.99794155|   WEAK
  diehard_rank_32x32|   0|     40000|     100|0.25779131|  PASSED
```

```
    diehard_rank_6x8|   0|    100000|     100|0.64032963|  PASSED
   diehard_bitstream|   0|   2097152|     100|0.07704848|  PASSED
        diehard_opso|   0|   2097152|     100|0.77648977|  PASSED
        diehard_oqso|   0|   2097152|     100|0.77595522|  PASSED
         diehard_dna|   0|   2097152|     100|0.00000000|  FAILED
diehard_count_1s_str|   0|    256000|     100|0.65129749|  PASSED
diehard_count_1s_byt|   0|    256000|     100|0.86604168|  PASSED
 diehard_parking_lot|   0|     12000|     100|0.00647165|  PASSED
    diehard_2dsphere|   2|      8000|     100|0.71393479|  PASSED
    diehard_3dsphere|   3|      4000|     100|0.31490147|  PASSED
     diehard_squeeze|   0|    100000|     100|0.20585135|  PASSED
        diehard_sums|   0|       100|     100|0.43310903|  PASSED
        diehard_runs|   0|    100000|     100|0.00016876|   WEAK
        diehard_runs|   0|    100000|     100|0.97852769|  PASSED
       diehard_craps|   0|    200000|     100|0.64202117|  PASSED
       diehard_craps|   0|    200000|     100|0.45834367|  PASSED
 marsaglia_tsang_gcd|   0|  10000000|     100|0.26479812|  PASSED
 marsaglia_tsang_gcd|   0|  10000000|     100|0.63818507|  PASSED
         sts_monobit|   1|    100000|     100|0.46631215|  PASSED
            sts_runs|   2|    100000|     100|0.99676498|   WEAK
          sts_serial|   1|    100000|     100|0.64443830|  PASSED
          sts_serial|   2|    100000|     100|0.41406926|  PASSED
          sts_serial|   3|    100000|     100|0.75390338|  PASSED
          sts_serial|   3|    100000|     100|0.10916661|  PASSED
          sts_serial|   4|    100000|     100|0.56702657|  PASSED
          sts_serial|   4|    100000|     100|0.20119350|  PASSED
          sts_serial|   5|    100000|     100|0.81570164|  PASSED
          sts_serial|   5|    100000|     100|0.96172514|  PASSED
          sts_serial|   6|    100000|     100|0.92271107|  PASSED
          sts_serial|   6|    100000|     100|0.15933801|  PASSED
          sts_serial|   7|    100000|     100|0.56510138|  PASSED
          sts_serial|   7|    100000|     100|0.17045874|  PASSED
          sts_serial|   8|    100000|     100|0.77301596|  PASSED
          sts_serial|   8|    100000|     100|0.23926074|  PASSED
          sts_serial|   9|    100000|     100|0.57730304|  PASSED
          sts_serial|   9|    100000|     100|0.62064093|  PASSED
          sts_serial|  10|    100000|     100|0.42122834|  PASSED
          sts_serial|  10|    100000|     100|0.76717486|  PASSED
          sts_serial|  11|    100000|     100|0.50871929|  PASSED
          sts_serial|  11|    100000|     100|0.88469327|  PASSED
          sts_serial|  12|    100000|     100|0.25110996|  PASSED
          sts_serial|  12|    100000|     100|0.17137502|  PASSED
          sts_serial|  13|    100000|     100|0.57067205|  PASSED
          sts_serial|  13|    100000|     100|0.90747993|  PASSED
          sts_serial|  14|    100000|     100|0.09992227|  PASSED
          sts_serial|  14|    100000|     100|0.01676505|  PASSED
          sts_serial|  15|    100000|     100|0.00325663|   WEAK
          sts_serial|  15|    100000|     100|0.43235158|  PASSED
          sts_serial|  16|    100000|     100|0.22993050|  PASSED
          sts_serial|  16|    100000|     100|0.96119444|  PASSED
         rgb_bitdist|   1|    100000|     100|0.67563955|  PASSED
         rgb_bitdist|   2|    100000|     100|0.02008545|  PASSED
         rgb_bitdist|   3|    100000|     100|0.88704252|  PASSED
         rgb_bitdist|   4|    100000|     100|0.21912125|  PASSED
         rgb_bitdist|   5|    100000|     100|0.62615614|  PASSED
         rgb_bitdist|   6|    100000|     100|0.79242004|  PASSED
         rgb_bitdist|   7|    100000|     100|0.83920820|  PASSED
         rgb_bitdist|   8|    100000|     100|0.79357302|  PASSED
         rgb_bitdist|   9|    100000|     100|0.90758073|  PASSED
         rgb_bitdist|  10|    100000|     100|0.98640640|  PASSED
         rgb_bitdist|  11|    100000|     100|0.19289433|  PASSED
         rgb_bitdist|  12|    100000|     100|0.79304788|  PASSED
rgb_minimum_distance|   2|     10000|    1000|0.15579925|  PASSED
rgb_minimum_distance|   3|     10000|    1000|0.32191956|  PASSED
rgb_minimum_distance|   4|     10000|    1000|0.55212357|  PASSED
```

```
rgb_minimum_distance|   5|     10000|    1000|0.57298448|  PASSED
    rgb_permutations|   2|    100000|     100|0.66771848|  PASSED
    rgb_permutations|   3|    100000|     100|0.68570004|  PASSED
    rgb_permutations|   4|    100000|     100|0.22706864|  PASSED
    rgb_permutations|   5|    100000|     100|0.54379954|  PASSED
      rgb_lagged_sum|   0|   1000000|     100|0.99955814|   WEAK
      rgb_lagged_sum|   1|   1000000|     100|0.10898385|  PASSED
      rgb_lagged_sum|   2|   1000000|     100|0.77762743|  PASSED
      rgb_lagged_sum|   3|   1000000|     100|0.66408363|  PASSED
      rgb_lagged_sum|   4|   1000000|     100|0.51044528|  PASSED
      rgb_lagged_sum|   5|   1000000|     100|0.09718286|  PASSED
      rgb_lagged_sum|   6|   1000000|     100|0.08975347|  PASSED
      rgb_lagged_sum|   7|   1000000|     100|0.85295643|  PASSED
      rgb_lagged_sum|   8|   1000000|     100|0.94132270|  PASSED
      rgb_lagged_sum|   9|   1000000|     100|0.58259591|  PASSED
      rgb_lagged_sum|  10|   1000000|     100|0.74463088|  PASSED
      rgb_lagged_sum|  11|   1000000|     100|0.96764802|  PASSED
      rgb_lagged_sum|  12|   1000000|     100|0.95984755|  PASSED
      rgb_lagged_sum|  13|   1000000|     100|0.93023056|  PASSED
      rgb_lagged_sum|  14|   1000000|     100|0.01326266|  PASSED
      rgb_lagged_sum|  15|   1000000|     100|0.77975386|  PASSED
      rgb_lagged_sum|  16|   1000000|     100|0.98162414|  PASSED
      rgb_lagged_sum|  17|   1000000|     100|0.89990187|  PASSED
      rgb_lagged_sum|  18|   1000000|     100|0.50083553|  PASSED
      rgb_lagged_sum|  19|   1000000|     100|0.78633548|  PASSED
      rgb_lagged_sum|  20|   1000000|     100|0.66922415|  PASSED
      rgb_lagged_sum|  21|   1000000|     100|0.57367027|  PASSED
      rgb_lagged_sum|  22|   1000000|     100|0.91695976|  PASSED
      rgb_lagged_sum|  23|   1000000|     100|0.32084821|  PASSED
      rgb_lagged_sum|  24|   1000000|     100|0.94771511|  PASSED
      rgb_lagged_sum|  25|   1000000|     100|0.72469255|  PASSED
      rgb_lagged_sum|  26|   1000000|     100|0.24862550|  PASSED
      rgb_lagged_sum|  27|   1000000|     100|0.71808802|  PASSED
      rgb_lagged_sum|  28|   1000000|     100|0.09304221|  PASSED
      rgb_lagged_sum|  29|   1000000|     100|0.65749741|  PASSED
      rgb_lagged_sum|  30|   1000000|     100|0.22832290|  PASSED
      rgb_lagged_sum|  31|   1000000|     100|0.37112789|  PASSED
      rgb_lagged_sum|  32|   1000000|     100|0.08596216|  PASSED
     rgb_kstest_test|   0|     10000|    1000|0.75546106|  PASSED
     dab_bytedistrib|   0|  51200000|       1|0.62383360|  PASSED
             dab_dct| 256|     50000|       1|0.16713962|  PASSED
Preparing to run test 207.  ntuple = 0
        dab_filltree|  32|  15000000|       1|0.71629062|  PASSED
        dab_filltree|  32|  15000000|       1|0.64386830|  PASSED
Preparing to run test 208.  ntuple = 0
       dab_filltree2|   0|   5000000|       1|0.18819790|  PASSED
       dab_filltree2|   1|   5000000|       1|0.29052023|  PASSED
Preparing to run test 209.  ntuple = 0
        dab_monobit2|  12|  65000000|       1|0.78773629|  PASSED
Preparing to run test 210.  ntuple = 0
      dab_birthdays1|  31|      2000|       1|0.11665567|  PASSED
Preparing to run test 211.  ntuple = 0
           dab_opso2|   0|  67108864|       1|0.51765805|  PASSED
           dab_opso2|   1|  67108864|       1|0.09367179|  PASSED
```

Listing 8.10: Test results for random number engine `trng::mrg5s`.

```
#=============================================================================#
#            dieharder version 3.31.1 Copyright 2003 Robert G. Brown          #
#=============================================================================#
   rng_name    |rands/second|   Seed   |
     trng_mrg5s|  1.61e+08  |         1|
#=============================================================================#
        test_name   |ntup| tsamples |psamples|  p-value |Assessment
#=============================================================================#
```

```
   diehard_birthdays|   0|       100|     100|0.88834205|  PASSED
      diehard_operm5|   0|   1000000|     100|0.97948850|  PASSED
  diehard_rank_32x32|   0|     40000|     100|0.48391389|  PASSED
    diehard_rank_6x8|   0|    100000|     100|0.48195212|  PASSED
   diehard_bitstream|   0|   2097152|     100|0.93010217|  PASSED
        diehard_opso|   0|   2097152|     100|0.31976849|  PASSED
        diehard_oqso|   0|   2097152|     100|0.97425425|  PASSED
         diehard_dna|   0|   2097152|     100|0.00000000|  FAILED
diehard_count_1s_str|   0|    256000|     100|0.89614710|  PASSED
diehard_count_1s_byt|   0|    256000|     100|0.91944345|  PASSED
 diehard_parking_lot|   0|     12000|     100|0.89432206|  PASSED
    diehard_2dsphere|   2|      8000|     100|0.98547974|  PASSED
    diehard_3dsphere|   3|      4000|     100|0.38760633|  PASSED
     diehard_squeeze|   0|    100000|     100|0.32213851|  PASSED
        diehard_sums|   0|       100|     100|0.00001729|   WEAK
        diehard_runs|   0|    100000|     100|0.96307483|  PASSED
        diehard_runs|   0|    100000|     100|0.66819179|  PASSED
       diehard_craps|   0|    200000|     100|0.73195398|  PASSED
       diehard_craps|   0|    200000|     100|0.47638415|  PASSED
 marsaglia_tsang_gcd|   0|  10000000|     100|0.81328853|  PASSED
 marsaglia_tsang_gcd|   0|  10000000|     100|0.99353440|  PASSED
         sts_monobit|   1|    100000|     100|0.88586599|  PASSED
            sts_runs|   2|    100000|     100|0.24052596|  PASSED
          sts_serial|   1|    100000|     100|0.62189260|  PASSED
          sts_serial|   2|    100000|     100|0.20894901|  PASSED
          sts_serial|   3|    100000|     100|0.61029305|  PASSED
          sts_serial|   3|    100000|     100|0.88899626|  PASSED
          sts_serial|   4|    100000|     100|0.65147215|  PASSED
          sts_serial|   4|    100000|     100|0.90407887|  PASSED
          sts_serial|   5|    100000|     100|0.78064551|  PASSED
          sts_serial|   5|    100000|     100|0.19132633|  PASSED
          sts_serial|   6|    100000|     100|0.01290265|  PASSED
          sts_serial|   6|    100000|     100|0.00555266|  PASSED
          sts_serial|   7|    100000|     100|0.25582207|  PASSED
          sts_serial|   7|    100000|     100|0.61409383|  PASSED
          sts_serial|   8|    100000|     100|0.77034767|  PASSED
          sts_serial|   8|    100000|     100|0.96136511|  PASSED
          sts_serial|   9|    100000|     100|0.90429000|  PASSED
          sts_serial|   9|    100000|     100|0.83282868|  PASSED
          sts_serial|  10|    100000|     100|0.50499413|  PASSED
          sts_serial|  10|    100000|     100|0.77714361|  PASSED
          sts_serial|  11|    100000|     100|0.89550248|  PASSED
          sts_serial|  11|    100000|     100|0.92358553|  PASSED
          sts_serial|  12|    100000|     100|0.75615007|  PASSED
          sts_serial|  12|    100000|     100|0.77975984|  PASSED
          sts_serial|  13|    100000|     100|0.62781404|  PASSED
          sts_serial|  13|    100000|     100|0.84768823|  PASSED
          sts_serial|  14|    100000|     100|0.99651065|   WEAK
          sts_serial|  14|    100000|     100|0.60060365|  PASSED
          sts_serial|  15|    100000|     100|0.99771350|   WEAK
          sts_serial|  15|    100000|     100|0.96834361|  PASSED
          sts_serial|  16|    100000|     100|0.52520231|  PASSED
          sts_serial|  16|    100000|     100|0.93707915|  PASSED
         rgb_bitdist|   1|    100000|     100|0.99969412|   WEAK
         rgb_bitdist|   2|    100000|     100|0.97453616|  PASSED
         rgb_bitdist|   3|    100000|     100|0.75664600|  PASSED
         rgb_bitdist|   4|    100000|     100|0.99832458|   WEAK
         rgb_bitdist|   5|    100000|     100|0.77213654|  PASSED
         rgb_bitdist|   6|    100000|     100|0.09712792|  PASSED
         rgb_bitdist|   7|    100000|     100|0.29690240|  PASSED
         rgb_bitdist|   8|    100000|     100|0.85809861|  PASSED
         rgb_bitdist|   9|    100000|     100|0.29708079|  PASSED
         rgb_bitdist|  10|    100000|     100|0.00207606|   WEAK
         rgb_bitdist|  11|    100000|     100|0.64818242|  PASSED
         rgb_bitdist|  12|    100000|     100|0.35473994|  PASSED
```

```
rgb_minimum_distance|   2|     10000|    1000|0.94710896|  PASSED
rgb_minimum_distance|   3|     10000|    1000|0.71412978|  PASSED
rgb_minimum_distance|   4|     10000|    1000|0.41376911|  PASSED
rgb_minimum_distance|   5|     10000|    1000|0.00944658|  PASSED
    rgb_permutations|   2|    100000|     100|0.13173468|  PASSED
    rgb_permutations|   3|    100000|     100|0.35090405|  PASSED
    rgb_permutations|   4|    100000|     100|0.44643901|  PASSED
    rgb_permutations|   5|    100000|     100|0.21856419|  PASSED
      rgb_lagged_sum|   0|   1000000|     100|0.94542235|  PASSED
      rgb_lagged_sum|   1|   1000000|     100|0.82554349|  PASSED
      rgb_lagged_sum|   2|   1000000|     100|0.96410914|  PASSED
      rgb_lagged_sum|   3|   1000000|     100|0.70033325|  PASSED
      rgb_lagged_sum|   4|   1000000|     100|0.87575893|  PASSED
      rgb_lagged_sum|   5|   1000000|     100|0.24921917|  PASSED
      rgb_lagged_sum|   6|   1000000|     100|0.71007330|  PASSED
      rgb_lagged_sum|   7|   1000000|     100|0.18926732|  PASSED
      rgb_lagged_sum|   8|   1000000|     100|0.18620662|  PASSED
      rgb_lagged_sum|   9|   1000000|     100|0.57960522|  PASSED
      rgb_lagged_sum|  10|   1000000|     100|0.84543126|  PASSED
      rgb_lagged_sum|  11|   1000000|     100|0.63227463|  PASSED
      rgb_lagged_sum|  12|   1000000|     100|0.10470460|  PASSED
      rgb_lagged_sum|  13|   1000000|     100|0.79543358|  PASSED
      rgb_lagged_sum|  14|   1000000|     100|0.11747078|  PASSED
      rgb_lagged_sum|  15|   1000000|     100|0.87449307|  PASSED
      rgb_lagged_sum|  16|   1000000|     100|0.19203097|  PASSED
      rgb_lagged_sum|  17|   1000000|     100|0.25426108|  PASSED
      rgb_lagged_sum|  18|   1000000|     100|0.71197873|  PASSED
      rgb_lagged_sum|  19|   1000000|     100|0.17591690|  PASSED
      rgb_lagged_sum|  20|   1000000|     100|0.21572618|  PASSED
      rgb_lagged_sum|  21|   1000000|     100|0.91885545|  PASSED
      rgb_lagged_sum|  22|   1000000|     100|0.92354547|  PASSED
      rgb_lagged_sum|  23|   1000000|     100|0.98905939|  PASSED
      rgb_lagged_sum|  24|   1000000|     100|0.27960663|  PASSED
      rgb_lagged_sum|  25|   1000000|     100|0.97647338|  PASSED
      rgb_lagged_sum|  26|   1000000|     100|0.73712566|  PASSED
      rgb_lagged_sum|  27|   1000000|     100|0.97060786|  PASSED
      rgb_lagged_sum|  28|   1000000|     100|0.23351732|  PASSED
      rgb_lagged_sum|  29|   1000000|     100|0.39825339|  PASSED
      rgb_lagged_sum|  30|   1000000|     100|0.28305526|  PASSED
      rgb_lagged_sum|  31|   1000000|     100|0.80233080|  PASSED
      rgb_lagged_sum|  32|   1000000|     100|0.73155063|  PASSED
     rgb_kstest_test|   0|     10000|    1000|0.61100889|  PASSED
     dab_bytedistrib|   0|  51200000|       1|0.51525566|  PASSED
             dab_dct| 256|     50000|       1|0.61473601|  PASSED
Preparing to run test 207.  ntuple = 0
        dab_filltree|  32|  15000000|       1|0.32989510|  PASSED
        dab_filltree|  32|  15000000|       1|0.46692939|  PASSED
Preparing to run test 208.  ntuple = 0
       dab_filltree2|   0|   5000000|       1|0.88066484|  PASSED
       dab_filltree2|   1|   5000000|       1|0.60779829|  PASSED
Preparing to run test 209.  ntuple = 0
        dab_monobit2|  12|  65000000|       1|0.38275308|  PASSED
Preparing to run test 210.  ntuple = 0
      dab_birthdays1|  31|      2000|       1|0.22206655|  PASSED
Preparing to run test 211.  ntuple = 0
           dab_opso2|   0|  67108864|       1|0.20699028|  PASSED
           dab_opso2|   1|  67108864|       1|0.74029895|  PASSED
```

Listing 8.11: Test results for random number engine `trng::yarn2`.

```
#=============================================================================#
#            dieharder version 3.31.1 Copyright 2003 Robert G. Brown          #
#=============================================================================#
   rng_name    |rands/second|   Seed   |
     trng_yarn2|  1.84e+08  |         1|
```

```
#=============================================================================#
        test_name   |ntup| tsamples |psamples|  p-value |Assessment
#=============================================================================#
   diehard_birthdays|   0|       100|     100|0.68588093|  PASSED
      diehard_operm5|   0|   1000000|     100|0.91774112|  PASSED
  diehard_rank_32x32|   0|     40000|     100|0.76045963|  PASSED
    diehard_rank_6x8|   0|    100000|     100|0.70921389|  PASSED
   diehard_bitstream|   0|   2097152|     100|0.71387716|  PASSED
        diehard_opso|   0|   2097152|     100|0.36754326|  PASSED
        diehard_oqso|   0|   2097152|     100|0.52926874|  PASSED
         diehard_dna|   0|   2097152|     100|0.00000000|  FAILED
diehard_count_1s_str|   0|    256000|     100|0.96359612|  PASSED
diehard_count_1s_byt|   0|    256000|     100|0.27605152|  PASSED
 diehard_parking_lot|   0|     12000|     100|0.27377983|  PASSED
    diehard_2dsphere|   2|      8000|     100|0.07782703|  PASSED
    diehard_3dsphere|   3|      4000|     100|0.45431052|  PASSED
     diehard_squeeze|   0|    100000|     100|0.87711198|  PASSED
        diehard_sums|   0|       100|     100|0.42790868|  PASSED
        diehard_runs|   0|    100000|     100|0.68531520|  PASSED
        diehard_runs|   0|    100000|     100|0.99817606|   WEAK
       diehard_craps|   0|    200000|     100|0.50208209|  PASSED
       diehard_craps|   0|    200000|     100|0.78299025|  PASSED
 marsaglia_tsang_gcd|   0|  10000000|     100|0.24460650|  PASSED
 marsaglia_tsang_gcd|   0|  10000000|     100|0.89053278|  PASSED
         sts_monobit|   1|    100000|     100|0.99532676|   WEAK
            sts_runs|   2|    100000|     100|0.55908132|  PASSED
          sts_serial|   1|    100000|     100|0.57460105|  PASSED
          sts_serial|   2|    100000|     100|0.98709944|  PASSED
          sts_serial|   3|    100000|     100|0.96365746|  PASSED
          sts_serial|   3|    100000|     100|0.70527977|  PASSED
          sts_serial|   4|    100000|     100|0.33641326|  PASSED
          sts_serial|   4|    100000|     100|0.16370939|  PASSED
          sts_serial|   5|    100000|     100|0.27955843|  PASSED
          sts_serial|   5|    100000|     100|0.46509224|  PASSED
          sts_serial|   6|    100000|     100|0.98871142|  PASSED
          sts_serial|   6|    100000|     100|0.90560669|  PASSED
          sts_serial|   7|    100000|     100|0.99543018|   WEAK
          sts_serial|   7|    100000|     100|0.78575565|  PASSED
          sts_serial|   8|    100000|     100|0.84333889|  PASSED
          sts_serial|   8|    100000|     100|0.07720385|  PASSED
          sts_serial|   9|    100000|     100|0.95444588|  PASSED
          sts_serial|   9|    100000|     100|0.48328387|  PASSED
          sts_serial|  10|    100000|     100|0.50924344|  PASSED
          sts_serial|  10|    100000|     100|0.15020761|  PASSED
          sts_serial|  11|    100000|     100|0.39766693|  PASSED
          sts_serial|  11|    100000|     100|0.51590739|  PASSED
          sts_serial|  12|    100000|     100|0.61017999|  PASSED
          sts_serial|  12|    100000|     100|0.90759702|  PASSED
          sts_serial|  13|    100000|     100|0.86320116|  PASSED
          sts_serial|  13|    100000|     100|0.99223328|  PASSED
          sts_serial|  14|    100000|     100|0.86397452|  PASSED
          sts_serial|  14|    100000|     100|0.43296086|  PASSED
          sts_serial|  15|    100000|     100|0.86881585|  PASSED
          sts_serial|  15|    100000|     100|0.87737366|  PASSED
          sts_serial|  16|    100000|     100|0.55327024|  PASSED
          sts_serial|  16|    100000|     100|0.31373179|  PASSED
         rgb_bitdist|   1|    100000|     100|0.89034060|  PASSED
         rgb_bitdist|   2|    100000|     100|0.93136551|  PASSED
         rgb_bitdist|   3|    100000|     100|0.95413226|  PASSED
         rgb_bitdist|   4|    100000|     100|0.70554595|  PASSED
         rgb_bitdist|   5|    100000|     100|0.76952141|  PASSED
         rgb_bitdist|   6|    100000|     100|0.39559247|  PASSED
         rgb_bitdist|   7|    100000|     100|0.26713199|  PASSED
         rgb_bitdist|   8|    100000|     100|0.36688710|  PASSED
         rgb_bitdist|   9|    100000|     100|0.86469615|  PASSED
```

```
         rgb_bitdist|  10|    100000|     100|0.85015332|  PASSED
         rgb_bitdist|  11|    100000|     100|0.94382606|  PASSED
         rgb_bitdist|  12|    100000|     100|0.96182327|  PASSED
rgb_minimum_distance|   2|     10000|    1000|0.48936473|  PASSED
rgb_minimum_distance|   3|     10000|    1000|0.34640025|  PASSED
rgb_minimum_distance|   4|     10000|    1000|0.73917032|  PASSED
rgb_minimum_distance|   5|     10000|    1000|0.15532063|  PASSED
    rgb_permutations|   2|    100000|     100|0.40892086|  PASSED
    rgb_permutations|   3|    100000|     100|0.19126455|  PASSED
    rgb_permutations|   4|    100000|     100|0.25132061|  PASSED
    rgb_permutations|   5|    100000|     100|0.88997488|  PASSED
      rgb_lagged_sum|   0|   1000000|     100|0.28042300|  PASSED
      rgb_lagged_sum|   1|   1000000|     100|0.54539373|  PASSED
      rgb_lagged_sum|   2|   1000000|     100|0.87840738|  PASSED
      rgb_lagged_sum|   3|   1000000|     100|0.93357011|  PASSED
      rgb_lagged_sum|   4|   1000000|     100|0.57929529|  PASSED
      rgb_lagged_sum|   5|   1000000|     100|0.75083850|  PASSED
      rgb_lagged_sum|   6|   1000000|     100|0.68511679|  PASSED
      rgb_lagged_sum|   7|   1000000|     100|0.13589857|  PASSED
      rgb_lagged_sum|   8|   1000000|     100|0.92279224|  PASSED
      rgb_lagged_sum|   9|   1000000|     100|0.62651533|  PASSED
      rgb_lagged_sum|  10|   1000000|     100|0.17691895|  PASSED
      rgb_lagged_sum|  11|   1000000|     100|0.55201307|  PASSED
      rgb_lagged_sum|  12|   1000000|     100|0.89790390|  PASSED
      rgb_lagged_sum|  13|   1000000|     100|0.64631416|  PASSED
      rgb_lagged_sum|  14|   1000000|     100|0.64805958|  PASSED
      rgb_lagged_sum|  15|   1000000|     100|0.79082625|  PASSED
      rgb_lagged_sum|  16|   1000000|     100|0.83931151|  PASSED
      rgb_lagged_sum|  17|   1000000|     100|0.18500929|  PASSED
      rgb_lagged_sum|  18|   1000000|     100|0.46644186|  PASSED
      rgb_lagged_sum|  19|   1000000|     100|0.25983649|  PASSED
      rgb_lagged_sum|  20|   1000000|     100|0.39354580|  PASSED
      rgb_lagged_sum|  21|   1000000|     100|0.48429496|  PASSED
      rgb_lagged_sum|  22|   1000000|     100|0.54458168|  PASSED
      rgb_lagged_sum|  23|   1000000|     100|0.34368138|  PASSED
      rgb_lagged_sum|  24|   1000000|     100|0.52846593|  PASSED
      rgb_lagged_sum|  25|   1000000|     100|0.94257244|  PASSED
      rgb_lagged_sum|  26|   1000000|     100|0.59142221|  PASSED
      rgb_lagged_sum|  27|   1000000|     100|0.11437062|  PASSED
      rgb_lagged_sum|  28|   1000000|     100|0.96370737|  PASSED
      rgb_lagged_sum|  29|   1000000|     100|0.49746683|  PASSED
      rgb_lagged_sum|  30|   1000000|     100|0.72794402|  PASSED
      rgb_lagged_sum|  31|   1000000|     100|0.22777477|  PASSED
      rgb_lagged_sum|  32|   1000000|     100|0.30475107|  PASSED
     rgb_kstest_test|   0|     10000|    1000|0.47631390|  PASSED
     dab_bytedistrib|   0|  51200000|       1|0.61864991|  PASSED
             dab_dct| 256|     50000|       1|0.52335643|  PASSED
Preparing to run test 207.  ntuple = 0
        dab_filltree|  32|  15000000|       1|0.87563733|  PASSED
        dab_filltree|  32|  15000000|       1|0.60991977|  PASSED
Preparing to run test 208.  ntuple = 0
       dab_filltree2|   0|   5000000|       1|0.14032496|  PASSED
       dab_filltree2|   1|   5000000|       1|0.86052484|  PASSED
Preparing to run test 209.  ntuple = 0
        dab_monobit2|  12|  65000000|       1|0.99116596|  PASSED
Preparing to run test 210.  ntuple = 0
      dab_birthdays1|  31|      2000|       1|0.13668356|  PASSED
Preparing to run test 211.  ntuple = 0
           dab_opso2|   0|  67108864|       1|0.37194909|  PASSED
           dab_opso2|   1|  67108864|       1|0.70021754|  PASSED
```

Listing 8.12: Test results for random number engine `trng::yarn3`.

```
#=============================================================================#
#            dieharder version 3.31.1 Copyright 2003 Robert G. Brown          #
```

```
#=============================================================================#
   rng_name    |rands/second|   Seed   |
     trng_yarn3|  1.23e+08  |         1|
#=============================================================================#
        test_name   |ntup| tsamples |psamples|  p-value |Assessment
#=============================================================================#
   diehard_birthdays|   0|       100|     100|0.84832213|  PASSED
      diehard_operm5|   0|   1000000|     100|0.63955189|  PASSED
  diehard_rank_32x32|   0|     40000|     100|0.41729342|  PASSED
    diehard_rank_6x8|   0|    100000|     100|0.54634420|  PASSED
   diehard_bitstream|   0|   2097152|     100|0.06624475|  PASSED
        diehard_opso|   0|   2097152|     100|0.43910137|  PASSED
        diehard_oqso|   0|   2097152|     100|0.56708202|  PASSED
         diehard_dna|   0|   2097152|     100|0.00000000|  FAILED
diehard_count_1s_str|   0|    256000|     100|0.81638211|  PASSED
diehard_count_1s_byt|   0|    256000|     100|0.63735856|  PASSED
 diehard_parking_lot|   0|     12000|     100|0.15798710|  PASSED
    diehard_2dsphere|   2|      8000|     100|0.96514142|  PASSED
    diehard_3dsphere|   3|      4000|     100|0.82452413|  PASSED
     diehard_squeeze|   0|    100000|     100|0.50607435|  PASSED
        diehard_sums|   0|       100|     100|0.20918422|  PASSED
        diehard_runs|   0|    100000|     100|0.05652652|  PASSED
        diehard_runs|   0|    100000|     100|0.32022369|  PASSED
       diehard_craps|   0|    200000|     100|0.64454820|  PASSED
       diehard_craps|   0|    200000|     100|0.83203378|  PASSED
 marsaglia_tsang_gcd|   0|  10000000|     100|0.12442362|  PASSED
 marsaglia_tsang_gcd|   0|  10000000|     100|0.95149592|  PASSED
         sts_monobit|   1|    100000|     100|0.34889199|  PASSED
            sts_runs|   2|    100000|     100|0.40615469|  PASSED
          sts_serial|   1|    100000|     100|0.23940313|  PASSED
          sts_serial|   2|    100000|     100|0.99146975|  PASSED
          sts_serial|   3|    100000|     100|0.54570563|  PASSED
          sts_serial|   3|    100000|     100|0.94265138|  PASSED
          sts_serial|   4|    100000|     100|0.94988808|  PASSED
          sts_serial|   4|    100000|     100|0.69448340|  PASSED
          sts_serial|   5|    100000|     100|0.32946220|  PASSED
          sts_serial|   5|    100000|     100|0.78568361|  PASSED
          sts_serial|   6|    100000|     100|0.78025486|  PASSED
          sts_serial|   6|    100000|     100|0.94135630|  PASSED
          sts_serial|   7|    100000|     100|0.61064209|  PASSED
          sts_serial|   7|    100000|     100|0.90632525|  PASSED
          sts_serial|   8|    100000|     100|0.24010066|  PASSED
          sts_serial|   8|    100000|     100|0.99874381|   WEAK
          sts_serial|   9|    100000|     100|0.77688398|  PASSED
          sts_serial|   9|    100000|     100|0.30048897|  PASSED
          sts_serial|  10|    100000|     100|0.96008569|  PASSED
          sts_serial|  10|    100000|     100|0.30004892|  PASSED
          sts_serial|  11|    100000|     100|0.23090425|  PASSED
          sts_serial|  11|    100000|     100|0.67127486|  PASSED
          sts_serial|  12|    100000|     100|0.87732565|  PASSED
          sts_serial|  12|    100000|     100|0.97484566|  PASSED
          sts_serial|  13|    100000|     100|0.80630443|  PASSED
          sts_serial|  13|    100000|     100|0.17423957|  PASSED
          sts_serial|  14|    100000|     100|0.55659976|  PASSED
          sts_serial|  14|    100000|     100|0.19569932|  PASSED
          sts_serial|  15|    100000|     100|0.68156358|  PASSED
          sts_serial|  15|    100000|     100|0.74133358|  PASSED
          sts_serial|  16|    100000|     100|0.16851108|  PASSED
          sts_serial|  16|    100000|     100|0.41943935|  PASSED
         rgb_bitdist|   1|    100000|     100|0.99994417|   WEAK
         rgb_bitdist|   2|    100000|     100|0.77628396|  PASSED
         rgb_bitdist|   3|    100000|     100|0.45919289|  PASSED
         rgb_bitdist|   4|    100000|     100|0.41879972|  PASSED
         rgb_bitdist|   5|    100000|     100|0.86432839|  PASSED
         rgb_bitdist|   6|    100000|     100|0.97796871|  PASSED
```

```
         rgb_bitdist|   7|    100000|     100|0.13795926|  PASSED
         rgb_bitdist|   8|    100000|     100|0.25076583|  PASSED
         rgb_bitdist|   9|    100000|     100|0.82109867|  PASSED
         rgb_bitdist|  10|    100000|     100|0.73051740|  PASSED
         rgb_bitdist|  11|    100000|     100|0.42909407|  PASSED
         rgb_bitdist|  12|    100000|     100|0.68972694|  PASSED
rgb_minimum_distance|   2|     10000|    1000|0.18128249|  PASSED
rgb_minimum_distance|   3|     10000|    1000|0.09267862|  PASSED
rgb_minimum_distance|   4|     10000|    1000|0.45452309|  PASSED
rgb_minimum_distance|   5|     10000|    1000|0.58282383|  PASSED
    rgb_permutations|   2|    100000|     100|0.04698778|  PASSED
    rgb_permutations|   3|    100000|     100|0.96542815|  PASSED
    rgb_permutations|   4|    100000|     100|0.98229292|  PASSED
    rgb_permutations|   5|    100000|     100|0.31571926|  PASSED
      rgb_lagged_sum|   0|   1000000|     100|0.13920878|  PASSED
      rgb_lagged_sum|   1|   1000000|     100|0.40969425|  PASSED
      rgb_lagged_sum|   2|   1000000|     100|0.28921095|  PASSED
      rgb_lagged_sum|   3|   1000000|     100|0.14103663|  PASSED
      rgb_lagged_sum|   4|   1000000|     100|0.37074976|  PASSED
      rgb_lagged_sum|   5|   1000000|     100|0.57764945|  PASSED
      rgb_lagged_sum|   6|   1000000|     100|0.38445167|  PASSED
      rgb_lagged_sum|   7|   1000000|     100|0.46165760|  PASSED
      rgb_lagged_sum|   8|   1000000|     100|0.44041949|  PASSED
      rgb_lagged_sum|   9|   1000000|     100|0.57081991|  PASSED
      rgb_lagged_sum|  10|   1000000|     100|0.14791565|  PASSED
      rgb_lagged_sum|  11|   1000000|     100|0.18679017|  PASSED
      rgb_lagged_sum|  12|   1000000|     100|0.29092883|  PASSED
      rgb_lagged_sum|  13|   1000000|     100|0.44785830|  PASSED
      rgb_lagged_sum|  14|   1000000|     100|0.16002098|  PASSED
      rgb_lagged_sum|  15|   1000000|     100|0.60923009|  PASSED
      rgb_lagged_sum|  16|   1000000|     100|0.99220888|  PASSED
      rgb_lagged_sum|  17|   1000000|     100|0.77356193|  PASSED
      rgb_lagged_sum|  18|   1000000|     100|0.38249829|  PASSED
      rgb_lagged_sum|  19|   1000000|     100|0.08579750|  PASSED
      rgb_lagged_sum|  20|   1000000|     100|0.98085343|  PASSED
      rgb_lagged_sum|  21|   1000000|     100|0.40192287|  PASSED
      rgb_lagged_sum|  22|   1000000|     100|0.98482901|  PASSED
      rgb_lagged_sum|  23|   1000000|     100|0.17366462|  PASSED
      rgb_lagged_sum|  24|   1000000|     100|0.85192923|  PASSED
      rgb_lagged_sum|  25|   1000000|     100|0.51912559|  PASSED
      rgb_lagged_sum|  26|   1000000|     100|0.53403098|  PASSED
      rgb_lagged_sum|  27|   1000000|     100|0.79330973|  PASSED
      rgb_lagged_sum|  28|   1000000|     100|0.38656190|  PASSED
      rgb_lagged_sum|  29|   1000000|     100|0.66973948|  PASSED
      rgb_lagged_sum|  30|   1000000|     100|0.18317810|  PASSED
      rgb_lagged_sum|  31|   1000000|     100|0.23860696|  PASSED
      rgb_lagged_sum|  32|   1000000|     100|0.48279048|  PASSED
     rgb_kstest_test|   0|     10000|    1000|0.65762017|  PASSED
     dab_bytedistrib|   0|  51200000|       1|0.04570787|  PASSED
             dab_dct| 256|     50000|       1|0.27389092|  PASSED
Preparing to run test 207.  ntuple = 0
        dab_filltree|  32|  15000000|       1|0.74162665|  PASSED
        dab_filltree|  32|  15000000|       1|0.23718428|  PASSED
Preparing to run test 208.  ntuple = 0
       dab_filltree2|   0|   5000000|       1|0.28427178|  PASSED
       dab_filltree2|   1|   5000000|       1|0.61609616|  PASSED
Preparing to run test 209.  ntuple = 0
        dab_monobit2|  12|  65000000|       1|0.76655621|  PASSED
Preparing to run test 210.  ntuple = 0
      dab_birthdays1|  31|      2000|       1|0.78333166|  PASSED
Preparing to run test 211.  ntuple = 0
           dab_opso2|   0|  67108864|       1|0.22133326|  PASSED
           dab_opso2|   1|  67108864|       1|0.50784722|  PASSED
```

Listing 8.13: Test results for random number engine `trng::yarn3s`.

```
#=============================================================================#
#            dieharder version 3.31.1 Copyright 2003 Robert G. Brown          #
#=============================================================================#
   rng_name    |rands/second|   Seed   |
    trng_yarn3s|  1.36e+08  |         1|
#=============================================================================#
        test_name   |ntup| tsamples |psamples|  p-value |Assessment
#=============================================================================#
   diehard_birthdays|   0|       100|     100|0.55535722|  PASSED
      diehard_operm5|   0|   1000000|     100|0.90355894|  PASSED
  diehard_rank_32x32|   0|     40000|     100|0.53221309|  PASSED
    diehard_rank_6x8|   0|    100000|     100|0.06078260|  PASSED
   diehard_bitstream|   0|   2097152|     100|0.56285211|  PASSED
        diehard_opso|   0|   2097152|     100|0.72902528|  PASSED
        diehard_oqso|   0|   2097152|     100|0.48042439|  PASSED
         diehard_dna|   0|   2097152|     100|0.00000000|  FAILED
diehard_count_1s_str|   0|    256000|     100|0.09629158|  PASSED
diehard_count_1s_byt|   0|    256000|     100|0.94093441|  PASSED
 diehard_parking_lot|   0|     12000|     100|0.63285114|  PASSED
    diehard_2dsphere|   2|      8000|     100|0.16461324|  PASSED
    diehard_3dsphere|   3|      4000|     100|0.40227268|  PASSED
     diehard_squeeze|   0|    100000|     100|0.29737200|  PASSED
        diehard_sums|   0|       100|     100|0.59201475|  PASSED
        diehard_runs|   0|    100000|     100|0.94744869|  PASSED
        diehard_runs|   0|    100000|     100|0.21749573|  PASSED
       diehard_craps|   0|    200000|     100|0.99978942|   WEAK
       diehard_craps|   0|    200000|     100|0.38346937|  PASSED
 marsaglia_tsang_gcd|   0|  10000000|     100|0.78892127|  PASSED
 marsaglia_tsang_gcd|   0|  10000000|     100|0.98984928|  PASSED
         sts_monobit|   1|    100000|     100|0.65065405|  PASSED
            sts_runs|   2|    100000|     100|0.44611679|  PASSED
          sts_serial|   1|    100000|     100|0.30149462|  PASSED
          sts_serial|   2|    100000|     100|0.46360320|  PASSED
          sts_serial|   3|    100000|     100|0.96856625|  PASSED
          sts_serial|   3|    100000|     100|0.88274036|  PASSED
          sts_serial|   4|    100000|     100|0.90274851|  PASSED
          sts_serial|   4|    100000|     100|0.11324676|  PASSED
          sts_serial|   5|    100000|     100|0.77992207|  PASSED
          sts_serial|   5|    100000|     100|0.98811531|  PASSED
          sts_serial|   6|    100000|     100|0.99129257|  PASSED
          sts_serial|   6|    100000|     100|0.93159001|  PASSED
          sts_serial|   7|    100000|     100|0.14384384|  PASSED
          sts_serial|   7|    100000|     100|0.07991810|  PASSED
          sts_serial|   8|    100000|     100|0.05140297|  PASSED
          sts_serial|   8|    100000|     100|0.09489028|  PASSED
          sts_serial|   9|    100000|     100|0.05992247|  PASSED
          sts_serial|   9|    100000|     100|0.43892242|  PASSED
          sts_serial|  10|    100000|     100|0.05482796|  PASSED
          sts_serial|  10|    100000|     100|0.65134308|  PASSED
          sts_serial|  11|    100000|     100|0.23663494|  PASSED
          sts_serial|  11|    100000|     100|0.94019473|  PASSED
          sts_serial|  12|    100000|     100|0.61426102|  PASSED
          sts_serial|  12|    100000|     100|0.20869121|  PASSED
          sts_serial|  13|    100000|     100|0.11487232|  PASSED
          sts_serial|  13|    100000|     100|0.01698471|  PASSED
          sts_serial|  14|    100000|     100|0.79426422|  PASSED
          sts_serial|  14|    100000|     100|0.30834069|  PASSED
          sts_serial|  15|    100000|     100|0.72758194|  PASSED
          sts_serial|  15|    100000|     100|0.13758366|  PASSED
          sts_serial|  16|    100000|     100|0.36420047|  PASSED
          sts_serial|  16|    100000|     100|0.14702506|  PASSED
         rgb_bitdist|   1|    100000|     100|0.98038091|  PASSED
         rgb_bitdist|   2|    100000|     100|0.28414321|  PASSED
         rgb_bitdist|   3|    100000|     100|0.51750062|  PASSED
         rgb_bitdist|   4|    100000|     100|0.33310526|  PASSED
```

```
         rgb_bitdist|   5|    100000|     100|0.92678137|  PASSED
         rgb_bitdist|   6|    100000|     100|0.22078536|  PASSED
         rgb_bitdist|   7|    100000|     100|0.06586608|  PASSED
         rgb_bitdist|   8|    100000|     100|0.26444127|  PASSED
         rgb_bitdist|   9|    100000|     100|0.31618479|  PASSED
         rgb_bitdist|  10|    100000|     100|0.18755517|  PASSED
         rgb_bitdist|  11|    100000|     100|0.73826488|  PASSED
         rgb_bitdist|  12|    100000|     100|0.68878170|  PASSED
rgb_minimum_distance|   2|     10000|    1000|0.36562136|  PASSED
rgb_minimum_distance|   3|     10000|    1000|0.66602665|  PASSED
rgb_minimum_distance|   4|     10000|    1000|0.73704741|  PASSED
rgb_minimum_distance|   5|     10000|    1000|0.74644757|  PASSED
    rgb_permutations|   2|    100000|     100|0.11727677|  PASSED
    rgb_permutations|   3|    100000|     100|0.81867484|  PASSED
    rgb_permutations|   4|    100000|     100|0.61117793|  PASSED
    rgb_permutations|   5|    100000|     100|0.43083833|  PASSED
      rgb_lagged_sum|   0|   1000000|     100|0.97059806|  PASSED
      rgb_lagged_sum|   1|   1000000|     100|0.98550049|  PASSED
      rgb_lagged_sum|   2|   1000000|     100|0.61079688|  PASSED
      rgb_lagged_sum|   3|   1000000|     100|0.93585765|  PASSED
      rgb_lagged_sum|   4|   1000000|     100|0.32454852|  PASSED
      rgb_lagged_sum|   5|   1000000|     100|0.22727895|  PASSED
      rgb_lagged_sum|   6|   1000000|     100|0.34343980|  PASSED
      rgb_lagged_sum|   7|   1000000|     100|0.96403375|  PASSED
      rgb_lagged_sum|   8|   1000000|     100|0.83912488|  PASSED
      rgb_lagged_sum|   9|   1000000|     100|0.08471771|  PASSED
      rgb_lagged_sum|  10|   1000000|     100|0.53345804|  PASSED
      rgb_lagged_sum|  11|   1000000|     100|0.23428911|  PASSED
      rgb_lagged_sum|  12|   1000000|     100|0.71519738|  PASSED
      rgb_lagged_sum|  13|   1000000|     100|0.67960208|  PASSED
      rgb_lagged_sum|  14|   1000000|     100|0.85160587|  PASSED
      rgb_lagged_sum|  15|   1000000|     100|0.83052628|  PASSED
      rgb_lagged_sum|  16|   1000000|     100|0.79931880|  PASSED
      rgb_lagged_sum|  17|   1000000|     100|0.43052321|  PASSED
      rgb_lagged_sum|  18|   1000000|     100|0.07175056|  PASSED
      rgb_lagged_sum|  19|   1000000|     100|0.46823127|  PASSED
      rgb_lagged_sum|  20|   1000000|     100|0.33618431|  PASSED
      rgb_lagged_sum|  21|   1000000|     100|0.67147810|  PASSED
      rgb_lagged_sum|  22|   1000000|     100|0.38193300|  PASSED
      rgb_lagged_sum|  23|   1000000|     100|0.12215801|  PASSED
      rgb_lagged_sum|  24|   1000000|     100|0.81525449|  PASSED
      rgb_lagged_sum|  25|   1000000|     100|0.16517839|  PASSED
      rgb_lagged_sum|  26|   1000000|     100|0.05539492|  PASSED
      rgb_lagged_sum|  27|   1000000|     100|0.10987424|  PASSED
      rgb_lagged_sum|  28|   1000000|     100|0.99388433|  PASSED
      rgb_lagged_sum|  29|   1000000|     100|0.37712109|  PASSED
      rgb_lagged_sum|  30|   1000000|     100|0.38008075|  PASSED
      rgb_lagged_sum|  31|   1000000|     100|0.34154227|  PASSED
      rgb_lagged_sum|  32|   1000000|     100|0.67899631|  PASSED
     rgb_kstest_test|   0|     10000|    1000|0.95436137|  PASSED
     dab_bytedistrib|   0|  51200000|       1|0.99820431|   WEAK
            dab_dct| 256|     50000|       1|0.83974676|  PASSED
Preparing to run test 207.  ntuple = 0
        dab_filltree|  32|  15000000|       1|0.00843221|  PASSED
        dab_filltree|  32|  15000000|       1|0.69236735|  PASSED
Preparing to run test 208.  ntuple = 0
       dab_filltree2|   0|   5000000|       1|0.10540018|  PASSED
       dab_filltree2|   1|   5000000|       1|0.59130630|  PASSED
Preparing to run test 209.  ntuple = 0
        dab_monobit2|  12|  65000000|       1|0.92185571|  PASSED
Preparing to run test 210.  ntuple = 0
      dab_birthdays1|  31|      2000|       1|0.91296086|  PASSED
Preparing to run test 211.  ntuple = 0
           dab_opso2|   0|  67108864|       1|0.02337824|  PASSED
           dab_opso2|   1|  67108864|       1|0.14605637|  PASSED
```

Listing 8.14: Test results for random number engine `trng::yarn4`.

```
#=============================================================================#
#            dieharder version 3.31.1 Copyright 2003 Robert G. Brown          #
#=============================================================================#
   rng_name    |rands/second|   Seed   |
     trng_yarn4|  9.53e+07  |         1|
#=============================================================================#
        test_name   |ntup| tsamples |psamples|  p-value |Assessment
#=============================================================================#
   diehard_birthdays|   0|       100|     100|0.96892788|  PASSED
      diehard_operm5|   0|   1000000|     100|0.05239334|  PASSED
  diehard_rank_32x32|   0|     40000|     100|0.51572983|  PASSED
    diehard_rank_6x8|   0|    100000|     100|0.37160635|  PASSED
   diehard_bitstream|   0|   2097152|     100|0.91606444|  PASSED
        diehard_opso|   0|   2097152|     100|0.56591942|  PASSED
        diehard_oqso|   0|   2097152|     100|0.33512422|  PASSED
         diehard_dna|   0|   2097152|     100|0.00000000|  FAILED
diehard_count_1s_str|   0|    256000|     100|0.32395987|  PASSED
diehard_count_1s_byt|   0|    256000|     100|0.50573614|  PASSED
 diehard_parking_lot|   0|     12000|     100|0.66754904|  PASSED
    diehard_2dsphere|   2|      8000|     100|0.18067297|  PASSED
    diehard_3dsphere|   3|      4000|     100|0.99902645|   WEAK
     diehard_squeeze|   0|    100000|     100|0.76602802|  PASSED
        diehard_sums|   0|       100|     100|0.04929231|  PASSED
        diehard_runs|   0|    100000|     100|0.99090395|  PASSED
        diehard_runs|   0|    100000|     100|0.91188311|  PASSED
       diehard_craps|   0|    200000|     100|0.89577979|  PASSED
       diehard_craps|   0|    200000|     100|0.66152704|  PASSED
 marsaglia_tsang_gcd|   0|  10000000|     100|0.43155532|  PASSED
 marsaglia_tsang_gcd|   0|  10000000|     100|0.00603950|  PASSED
         sts_monobit|   1|    100000|     100|0.57186166|  PASSED
            sts_runs|   2|    100000|     100|0.20581550|  PASSED
          sts_serial|   1|    100000|     100|0.52272641|  PASSED
          sts_serial|   2|    100000|     100|0.93190844|  PASSED
          sts_serial|   3|    100000|     100|0.93069119|  PASSED
          sts_serial|   3|    100000|     100|0.45816912|  PASSED
          sts_serial|   4|    100000|     100|0.94904283|  PASSED
          sts_serial|   4|    100000|     100|0.49780211|  PASSED
          sts_serial|   5|    100000|     100|0.30257357|  PASSED
          sts_serial|   5|    100000|     100|0.35711847|  PASSED
          sts_serial|   6|    100000|     100|0.41602423|  PASSED
          sts_serial|   6|    100000|     100|0.75305005|  PASSED
          sts_serial|   7|    100000|     100|0.11123693|  PASSED
          sts_serial|   7|    100000|     100|0.05569341|  PASSED
          sts_serial|   8|    100000|     100|0.94333393|  PASSED
          sts_serial|   8|    100000|     100|0.55155018|  PASSED
          sts_serial|   9|    100000|     100|0.64074774|  PASSED
          sts_serial|   9|    100000|     100|0.88562902|  PASSED
          sts_serial|  10|    100000|     100|0.05110677|  PASSED
          sts_serial|  10|    100000|     100|0.00287746|   WEAK
          sts_serial|  11|    100000|     100|0.38701733|  PASSED
          sts_serial|  11|    100000|     100|0.57410679|  PASSED
          sts_serial|  12|    100000|     100|0.96969252|  PASSED
          sts_serial|  12|    100000|     100|0.53404310|  PASSED
          sts_serial|  13|    100000|     100|0.99527972|   WEAK
          sts_serial|  13|    100000|     100|0.61842507|  PASSED
          sts_serial|  14|    100000|     100|0.61164503|  PASSED
          sts_serial|  14|    100000|     100|0.06946110|  PASSED
          sts_serial|  15|    100000|     100|0.71190409|  PASSED
          sts_serial|  15|    100000|     100|0.98692703|  PASSED
          sts_serial|  16|    100000|     100|0.67333313|  PASSED
          sts_serial|  16|    100000|     100|0.90702719|  PASSED
         rgb_bitdist|   1|    100000|     100|0.46010829|  PASSED
         rgb_bitdist|   2|    100000|     100|0.67911813|  PASSED
         rgb_bitdist|   3|    100000|     100|0.42270708|  PASSED
```

```
         rgb_bitdist|   4|    100000|     100|0.51076850|  PASSED
         rgb_bitdist|   5|    100000|     100|0.55656731|  PASSED
         rgb_bitdist|   6|    100000|     100|0.76134597|  PASSED
         rgb_bitdist|   7|    100000|     100|0.99853833|   WEAK
         rgb_bitdist|   8|    100000|     100|0.22257456|  PASSED
         rgb_bitdist|   9|    100000|     100|0.51413862|  PASSED
         rgb_bitdist|  10|    100000|     100|0.60957606|  PASSED
         rgb_bitdist|  11|    100000|     100|0.24198655|  PASSED
         rgb_bitdist|  12|    100000|     100|0.48377941|  PASSED
rgb_minimum_distance|   2|     10000|    1000|0.15780950|  PASSED
rgb_minimum_distance|   3|     10000|    1000|0.43906012|  PASSED
rgb_minimum_distance|   4|     10000|    1000|0.24515347|  PASSED
rgb_minimum_distance|   5|     10000|    1000|0.49404449|  PASSED
    rgb_permutations|   2|    100000|     100|0.52963157|  PASSED
    rgb_permutations|   3|    100000|     100|0.56986564|  PASSED
    rgb_permutations|   4|    100000|     100|0.97198474|  PASSED
    rgb_permutations|   5|    100000|     100|0.73165382|  PASSED
      rgb_lagged_sum|   0|   1000000|     100|0.56662321|  PASSED
      rgb_lagged_sum|   1|   1000000|     100|0.95526435|  PASSED
      rgb_lagged_sum|   2|   1000000|     100|0.91256075|  PASSED
      rgb_lagged_sum|   3|   1000000|     100|0.40817571|  PASSED
      rgb_lagged_sum|   4|   1000000|     100|0.81278138|  PASSED
      rgb_lagged_sum|   5|   1000000|     100|0.94954450|  PASSED
      rgb_lagged_sum|   6|   1000000|     100|0.58857582|  PASSED
      rgb_lagged_sum|   7|   1000000|     100|0.63887905|  PASSED
      rgb_lagged_sum|   8|   1000000|     100|0.83293966|  PASSED
      rgb_lagged_sum|   9|   1000000|     100|0.10887930|  PASSED
      rgb_lagged_sum|  10|   1000000|     100|0.81232779|  PASSED
      rgb_lagged_sum|  11|   1000000|     100|0.99280020|  PASSED
      rgb_lagged_sum|  12|   1000000|     100|0.95559576|  PASSED
      rgb_lagged_sum|  13|   1000000|     100|0.65455293|  PASSED
      rgb_lagged_sum|  14|   1000000|     100|0.91387636|  PASSED
      rgb_lagged_sum|  15|   1000000|     100|0.45965540|  PASSED
      rgb_lagged_sum|  16|   1000000|     100|0.30662570|  PASSED
      rgb_lagged_sum|  17|   1000000|     100|0.70321484|  PASSED
      rgb_lagged_sum|  18|   1000000|     100|0.44282257|  PASSED
      rgb_lagged_sum|  19|   1000000|     100|0.98933123|  PASSED
      rgb_lagged_sum|  20|   1000000|     100|0.46575412|  PASSED
      rgb_lagged_sum|  21|   1000000|     100|0.40365950|  PASSED
      rgb_lagged_sum|  22|   1000000|     100|0.80416449|  PASSED
      rgb_lagged_sum|  23|   1000000|     100|0.43315960|  PASSED
      rgb_lagged_sum|  24|   1000000|     100|0.00374065|   WEAK
      rgb_lagged_sum|  25|   1000000|     100|0.49442909|  PASSED
      rgb_lagged_sum|  26|   1000000|     100|0.40423443|  PASSED
      rgb_lagged_sum|  27|   1000000|     100|0.21985897|  PASSED
      rgb_lagged_sum|  28|   1000000|     100|0.60479876|  PASSED
      rgb_lagged_sum|  29|   1000000|     100|0.46155846|  PASSED
      rgb_lagged_sum|  30|   1000000|     100|0.75714557|  PASSED
      rgb_lagged_sum|  31|   1000000|     100|0.56497252|  PASSED
      rgb_lagged_sum|  32|   1000000|     100|0.99895517|   WEAK
     rgb_kstest_test|   0|     10000|    1000|0.10883112|  PASSED
     dab_bytedistrib|   0|  51200000|       1|0.77451078|  PASSED
             dab_dct| 256|     50000|       1|0.30406855|  PASSED
Preparing to run test 207.  ntuple = 0
        dab_filltree|  32|  15000000|       1|0.04298924|  PASSED
        dab_filltree|  32|  15000000|       1|0.06873128|  PASSED
Preparing to run test 208.  ntuple = 0
       dab_filltree2|   0|   5000000|       1|0.90368433|  PASSED
       dab_filltree2|   1|   5000000|       1|0.86361357|  PASSED
Preparing to run test 209.  ntuple = 0
        dab_monobit2|  12|  65000000|       1|0.09610030|  PASSED
Preparing to run test 210.  ntuple = 0
      dab_birthdays1|  31|      2000|       1|0.23403610|  PASSED
Preparing to run test 211.  ntuple = 0
           dab_opso2|   0|  67108864|       1|0.93316360|  PASSED
```

```
          dab_opso2|   1|  67108864|       1|0.33399721|  PASSED
```

Listing 8.15: Test results for random number engine `trng::yarn5`.

```
#=============================================================================#
#            dieharder version 3.31.1 Copyright 2003 Robert G. Brown          #
#=============================================================================#
   rng_name    |rands/second|   Seed   |
     trng_yarn5|  1.39e+08  |         1|
#=============================================================================#
        test_name   |ntup| tsamples |psamples|  p-value |Assessment
#=============================================================================#
   diehard_birthdays|   0|       100|     100|0.22442829|  PASSED
      diehard_operm5|   0|   1000000|     100|0.05416545|  PASSED
  diehard_rank_32x32|   0|     40000|     100|0.66439430|  PASSED
    diehard_rank_6x8|   0|    100000|     100|0.43055512|  PASSED
   diehard_bitstream|   0|   2097152|     100|0.69356593|  PASSED
        diehard_opso|   0|   2097152|     100|0.01861672|  PASSED
        diehard_oqso|   0|   2097152|     100|0.97321075|  PASSED
         diehard_dna|   0|   2097152|     100|0.00000000|  FAILED
diehard_count_1s_str|   0|    256000|     100|0.13217726|  PASSED
diehard_count_1s_byt|   0|    256000|     100|0.71795791|  PASSED
 diehard_parking_lot|   0|     12000|     100|0.33164281|  PASSED
    diehard_2dsphere|   2|      8000|     100|0.19008769|  PASSED
    diehard_3dsphere|   3|      4000|     100|0.03480181|  PASSED
     diehard_squeeze|   0|    100000|     100|0.50362235|  PASSED
        diehard_sums|   0|       100|     100|0.04224400|  PASSED
        diehard_runs|   0|    100000|     100|0.73103203|  PASSED
        diehard_runs|   0|    100000|     100|0.92757418|  PASSED
       diehard_craps|   0|    200000|     100|0.93909063|  PASSED
       diehard_craps|   0|    200000|     100|0.96924487|  PASSED
 marsaglia_tsang_gcd|   0|  10000000|     100|0.59921757|  PASSED
 marsaglia_tsang_gcd|   0|  10000000|     100|0.88879228|  PASSED
         sts_monobit|   1|    100000|     100|0.87555412|  PASSED
            sts_runs|   2|    100000|     100|0.78713288|  PASSED
          sts_serial|   1|    100000|     100|0.49517091|  PASSED
          sts_serial|   2|    100000|     100|0.72582588|  PASSED
          sts_serial|   3|    100000|     100|0.29001825|  PASSED
          sts_serial|   3|    100000|     100|0.03636110|  PASSED
          sts_serial|   4|    100000|     100|0.74647661|  PASSED
          sts_serial|   4|    100000|     100|0.74843334|  PASSED
          sts_serial|   5|    100000|     100|0.99993167|   WEAK
          sts_serial|   5|    100000|     100|0.42335871|  PASSED
          sts_serial|   6|    100000|     100|0.61000299|  PASSED
          sts_serial|   6|    100000|     100|0.22445150|  PASSED
          sts_serial|   7|    100000|     100|0.31556366|  PASSED
          sts_serial|   7|    100000|     100|0.16331016|  PASSED
          sts_serial|   8|    100000|     100|0.42036798|  PASSED
          sts_serial|   8|    100000|     100|0.36142373|  PASSED
          sts_serial|   9|    100000|     100|0.61947332|  PASSED
          sts_serial|   9|    100000|     100|0.86005620|  PASSED
          sts_serial|  10|    100000|     100|0.60894461|  PASSED
          sts_serial|  10|    100000|     100|0.65926941|  PASSED
          sts_serial|  11|    100000|     100|0.87915570|  PASSED
          sts_serial|  11|    100000|     100|0.99536098|   WEAK
          sts_serial|  12|    100000|     100|0.97716422|  PASSED
          sts_serial|  12|    100000|     100|0.94465987|  PASSED
          sts_serial|  13|    100000|     100|0.92038489|  PASSED
          sts_serial|  13|    100000|     100|0.91348415|  PASSED
          sts_serial|  14|    100000|     100|0.92542180|  PASSED
          sts_serial|  14|    100000|     100|0.66654939|  PASSED
          sts_serial|  15|    100000|     100|0.77590487|  PASSED
          sts_serial|  15|    100000|     100|0.24664004|  PASSED
          sts_serial|  16|    100000|     100|0.73888099|  PASSED
          sts_serial|  16|    100000|     100|0.52936952|  PASSED
```

```
         rgb_bitdist|   1|    100000|     100|0.74072790|  PASSED
         rgb_bitdist|   2|    100000|     100|0.44356748|  PASSED
         rgb_bitdist|   3|    100000|     100|0.57393871|  PASSED
         rgb_bitdist|   4|    100000|     100|0.29293851|  PASSED
         rgb_bitdist|   5|    100000|     100|0.77098771|  PASSED
         rgb_bitdist|   6|    100000|     100|0.01548556|  PASSED
         rgb_bitdist|   7|    100000|     100|0.11801616|  PASSED
         rgb_bitdist|   8|    100000|     100|0.76781211|  PASSED
         rgb_bitdist|   9|    100000|     100|0.18615702|  PASSED
         rgb_bitdist|  10|    100000|     100|0.21097973|  PASSED
         rgb_bitdist|  11|    100000|     100|0.48502700|  PASSED
         rgb_bitdist|  12|    100000|     100|0.14871629|  PASSED
rgb_minimum_distance|   2|     10000|    1000|0.06402668|  PASSED
rgb_minimum_distance|   3|     10000|    1000|0.31876563|  PASSED
rgb_minimum_distance|   4|     10000|    1000|0.26853624|  PASSED
rgb_minimum_distance|   5|     10000|    1000|0.92568206|  PASSED
    rgb_permutations|   2|    100000|     100|0.16761677|  PASSED
    rgb_permutations|   3|    100000|     100|0.88435817|  PASSED
    rgb_permutations|   4|    100000|     100|0.64614344|  PASSED
    rgb_permutations|   5|    100000|     100|0.86030749|  PASSED
      rgb_lagged_sum|   0|   1000000|     100|0.51500629|  PASSED
      rgb_lagged_sum|   1|   1000000|     100|0.30328049|  PASSED
      rgb_lagged_sum|   2|   1000000|     100|0.91574672|  PASSED
      rgb_lagged_sum|   3|   1000000|     100|0.67344148|  PASSED
      rgb_lagged_sum|   4|   1000000|     100|0.95026090|  PASSED
      rgb_lagged_sum|   5|   1000000|     100|0.82362741|  PASSED
      rgb_lagged_sum|   6|   1000000|     100|0.54161495|  PASSED
      rgb_lagged_sum|   7|   1000000|     100|0.87919629|  PASSED
      rgb_lagged_sum|   8|   1000000|     100|0.41793170|  PASSED
      rgb_lagged_sum|   9|   1000000|     100|0.20707734|  PASSED
      rgb_lagged_sum|  10|   1000000|     100|0.92536453|  PASSED
      rgb_lagged_sum|  11|   1000000|     100|0.70463733|  PASSED
      rgb_lagged_sum|  12|   1000000|     100|0.90002294|  PASSED
      rgb_lagged_sum|  13|   1000000|     100|0.23389774|  PASSED
      rgb_lagged_sum|  14|   1000000|     100|0.92985228|  PASSED
      rgb_lagged_sum|  15|   1000000|     100|0.91152097|  PASSED
      rgb_lagged_sum|  16|   1000000|     100|0.89757923|  PASSED
      rgb_lagged_sum|  17|   1000000|     100|0.69152618|  PASSED
      rgb_lagged_sum|  18|   1000000|     100|0.31107682|  PASSED
      rgb_lagged_sum|  19|   1000000|     100|0.93567005|  PASSED
      rgb_lagged_sum|  20|   1000000|     100|0.17381132|  PASSED
      rgb_lagged_sum|  21|   1000000|     100|0.23660547|  PASSED
      rgb_lagged_sum|  22|   1000000|     100|0.74566794|  PASSED
      rgb_lagged_sum|  23|   1000000|     100|0.98864646|  PASSED
      rgb_lagged_sum|  24|   1000000|     100|0.05528174|  PASSED
      rgb_lagged_sum|  25|   1000000|     100|0.26742162|  PASSED
      rgb_lagged_sum|  26|   1000000|     100|0.97719994|  PASSED
      rgb_lagged_sum|  27|   1000000|     100|0.05199441|  PASSED
      rgb_lagged_sum|  28|   1000000|     100|0.62287482|  PASSED
      rgb_lagged_sum|  29|   1000000|     100|0.76150236|  PASSED
      rgb_lagged_sum|  30|   1000000|     100|0.56890612|  PASSED
      rgb_lagged_sum|  31|   1000000|     100|0.39975331|  PASSED
      rgb_lagged_sum|  32|   1000000|     100|0.62290923|  PASSED
     rgb_kstest_test|   0|     10000|    1000|0.03908195|  PASSED
     dab_bytedistrib|   0|  51200000|       1|0.62743908|  PASSED
             dab_dct| 256|     50000|       1|0.21365466|  PASSED
Preparing to run test 207.  ntuple = 0
        dab_filltree|  32|  15000000|       1|0.19293095|  PASSED
        dab_filltree|  32|  15000000|       1|0.41488954|  PASSED
Preparing to run test 208.  ntuple = 0
       dab_filltree2|   0|   5000000|       1|0.57291900|  PASSED
       dab_filltree2|   1|   5000000|       1|0.11100355|  PASSED
Preparing to run test 209.  ntuple = 0
        dab_monobit2|  12|  65000000|       1|0.83270804|  PASSED
Preparing to run test 210.  ntuple = 0
```

```
      dab_birthdays1|  31|      2000|       1|0.73865569|  PASSED
Preparing to run test 211.  ntuple = 0
           dab_opso2|   0|  67108864|       1|0.85983801|  PASSED
           dab_opso2|   1|  67108864|       1|0.03323881|  PASSED
```

Listing 8.16: Test results for random number engine `trng::yarn5s`.

```
#=============================================================================#
#            dieharder version 3.31.1 Copyright 2003 Robert G. Brown          #
#=============================================================================#
   rng_name    |rands/second|   Seed   |
    trng_yarn5s|  1.20e+08  |         1|
#=============================================================================#
        test_name   |ntup| tsamples |psamples|  p-value |Assessment
#=============================================================================#
   diehard_birthdays|   0|       100|     100|0.58621631|  PASSED
      diehard_operm5|   0|   1000000|     100|0.95834803|  PASSED
  diehard_rank_32x32|   0|     40000|     100|0.08580886|  PASSED
    diehard_rank_6x8|   0|    100000|     100|0.71009872|  PASSED
   diehard_bitstream|   0|   2097152|     100|0.98372427|  PASSED
        diehard_opso|   0|   2097152|     100|0.25776764|  PASSED
        diehard_oqso|   0|   2097152|     100|0.13143083|  PASSED
         diehard_dna|   0|   2097152|     100|0.00000000|  FAILED
diehard_count_1s_str|   0|    256000|     100|0.61551985|  PASSED
diehard_count_1s_byt|   0|    256000|     100|0.51746118|  PASSED
 diehard_parking_lot|   0|     12000|     100|0.87420580|  PASSED
    diehard_2dsphere|   2|      8000|     100|0.31836025|  PASSED
    diehard_3dsphere|   3|      4000|     100|0.82075186|  PASSED
     diehard_squeeze|   0|    100000|     100|0.30522573|  PASSED
        diehard_sums|   0|       100|     100|0.04350587|  PASSED
        diehard_runs|   0|    100000|     100|0.99704301|   WEAK
        diehard_runs|   0|    100000|     100|0.28193843|  PASSED
       diehard_craps|   0|    200000|     100|0.93447351|  PASSED
       diehard_craps|   0|    200000|     100|0.24944539|  PASSED
 marsaglia_tsang_gcd|   0|  10000000|     100|0.16862229|  PASSED
 marsaglia_tsang_gcd|   0|  10000000|     100|0.45723160|  PASSED
         sts_monobit|   1|    100000|     100|0.82136482|  PASSED
            sts_runs|   2|    100000|     100|0.71769029|  PASSED
          sts_serial|   1|    100000|     100|0.55109213|  PASSED
          sts_serial|   2|    100000|     100|0.51757448|  PASSED
          sts_serial|   3|    100000|     100|0.22841944|  PASSED
          sts_serial|   3|    100000|     100|0.46360219|  PASSED
          sts_serial|   4|    100000|     100|0.84452261|  PASSED
          sts_serial|   4|    100000|     100|0.79761422|  PASSED
          sts_serial|   5|    100000|     100|0.39068263|  PASSED
          sts_serial|   5|    100000|     100|0.97737230|  PASSED
          sts_serial|   6|    100000|     100|0.12746574|  PASSED
          sts_serial|   6|    100000|     100|0.41584348|  PASSED
          sts_serial|   7|    100000|     100|0.58699518|  PASSED
          sts_serial|   7|    100000|     100|0.59601365|  PASSED
          sts_serial|   8|    100000|     100|0.43560468|  PASSED
          sts_serial|   8|    100000|     100|0.17377947|  PASSED
          sts_serial|   9|    100000|     100|0.63298396|  PASSED
          sts_serial|   9|    100000|     100|0.34534343|  PASSED
          sts_serial|  10|    100000|     100|0.99059648|  PASSED
          sts_serial|  10|    100000|     100|0.40374850|  PASSED
          sts_serial|  11|    100000|     100|0.90922721|  PASSED
          sts_serial|  11|    100000|     100|0.95825005|  PASSED
          sts_serial|  12|    100000|     100|0.85636333|  PASSED
          sts_serial|  12|    100000|     100|0.25452124|  PASSED
          sts_serial|  13|    100000|     100|0.95401530|  PASSED
          sts_serial|  13|    100000|     100|0.91462678|  PASSED
          sts_serial|  14|    100000|     100|0.70519709|  PASSED
          sts_serial|  14|    100000|     100|0.99670345|   WEAK
          sts_serial|  15|    100000|     100|0.65540886|  PASSED
```

```
          sts_serial|  15|    100000|     100|0.90675541|  PASSED
          sts_serial|  16|    100000|     100|0.70565322|  PASSED
          sts_serial|  16|    100000|     100|0.56719374|  PASSED
         rgb_bitdist|   1|    100000|     100|0.95260762|  PASSED
         rgb_bitdist|   2|    100000|     100|0.85786191|  PASSED
         rgb_bitdist|   3|    100000|     100|0.46618404|  PASSED
         rgb_bitdist|   4|    100000|     100|0.11503157|  PASSED
         rgb_bitdist|   5|    100000|     100|0.43963490|  PASSED
         rgb_bitdist|   6|    100000|     100|0.52879301|  PASSED
         rgb_bitdist|   7|    100000|     100|0.87470972|  PASSED
         rgb_bitdist|   8|    100000|     100|0.49995093|  PASSED
         rgb_bitdist|   9|    100000|     100|0.35716816|  PASSED
         rgb_bitdist|  10|    100000|     100|0.84693371|  PASSED
         rgb_bitdist|  11|    100000|     100|0.17555266|  PASSED
         rgb_bitdist|  12|    100000|     100|0.34670994|  PASSED
rgb_minimum_distance|   2|     10000|    1000|0.66998658|  PASSED
rgb_minimum_distance|   3|     10000|    1000|0.28599854|  PASSED
rgb_minimum_distance|   4|     10000|    1000|0.24262766|  PASSED
rgb_minimum_distance|   5|     10000|    1000|0.93708675|  PASSED
    rgb_permutations|   2|    100000|     100|0.17390341|  PASSED
    rgb_permutations|   3|    100000|     100|0.75030855|  PASSED
    rgb_permutations|   4|    100000|     100|0.55943940|  PASSED
    rgb_permutations|   5|    100000|     100|0.01123060|  PASSED
      rgb_lagged_sum|   0|   1000000|     100|0.81472865|  PASSED
      rgb_lagged_sum|   1|   1000000|     100|0.14931598|  PASSED
      rgb_lagged_sum|   2|   1000000|     100|0.54977631|  PASSED
      rgb_lagged_sum|   3|   1000000|     100|0.85063469|  PASSED
      rgb_lagged_sum|   4|   1000000|     100|0.70993944|  PASSED
      rgb_lagged_sum|   5|   1000000|     100|0.98484029|  PASSED
      rgb_lagged_sum|   6|   1000000|     100|0.57072072|  PASSED
      rgb_lagged_sum|   7|   1000000|     100|0.06412675|  PASSED
      rgb_lagged_sum|   8|   1000000|     100|0.62098732|  PASSED
      rgb_lagged_sum|   9|   1000000|     100|0.61065344|  PASSED
      rgb_lagged_sum|  10|   1000000|     100|0.66639343|  PASSED
      rgb_lagged_sum|  11|   1000000|     100|0.11989029|  PASSED
      rgb_lagged_sum|  12|   1000000|     100|0.28902210|  PASSED
      rgb_lagged_sum|  13|   1000000|     100|0.60443461|  PASSED
      rgb_lagged_sum|  14|   1000000|     100|0.84445379|  PASSED
      rgb_lagged_sum|  15|   1000000|     100|0.50709916|  PASSED
      rgb_lagged_sum|  16|   1000000|     100|0.98702442|  PASSED
      rgb_lagged_sum|  17|   1000000|     100|0.54836560|  PASSED
      rgb_lagged_sum|  18|   1000000|     100|0.99362059|  PASSED
      rgb_lagged_sum|  19|   1000000|     100|0.19997619|  PASSED
      rgb_lagged_sum|  20|   1000000|     100|0.46234624|  PASSED
      rgb_lagged_sum|  21|   1000000|     100|0.03122258|  PASSED
      rgb_lagged_sum|  22|   1000000|     100|0.70975131|  PASSED
      rgb_lagged_sum|  23|   1000000|     100|0.49977288|  PASSED
      rgb_lagged_sum|  24|   1000000|     100|0.61976051|  PASSED
      rgb_lagged_sum|  25|   1000000|     100|0.20384014|  PASSED
      rgb_lagged_sum|  26|   1000000|     100|0.83548592|  PASSED
      rgb_lagged_sum|  27|   1000000|     100|0.61613127|  PASSED
      rgb_lagged_sum|  28|   1000000|     100|0.84646709|  PASSED
      rgb_lagged_sum|  29|   1000000|     100|0.66623511|  PASSED
      rgb_lagged_sum|  30|   1000000|     100|0.48246343|  PASSED
      rgb_lagged_sum|  31|   1000000|     100|0.97827345|  PASSED
      rgb_lagged_sum|  32|   1000000|     100|0.10855264|  PASSED
     rgb_kstest_test|   0|     10000|    1000|0.85895496|  PASSED
     dab_bytedistrib|   0|  51200000|       1|0.09862691|  PASSED
             dab_dct| 256|     50000|       1|0.22131228|  PASSED
Preparing to run test 207.  ntuple = 0
        dab_filltree|  32|  15000000|       1|0.53487030|  PASSED
        dab_filltree|  32|  15000000|       1|0.95950530|  PASSED
Preparing to run test 208.  ntuple = 0
       dab_filltree2|   0|   5000000|       1|0.10751426|  PASSED
       dab_filltree2|   1|   5000000|       1|0.75730264|  PASSED
```

```
Preparing to run test 209.  ntuple = 0
        dab_monobit2|  12|  65000000|       1|0.20068940|  PASSED
Preparing to run test 210.  ntuple = 0
      dab_birthdays1|  31|      2000|       1|0.57957920|  PASSED
Preparing to run test 211.  ntuple = 0
           dab_opso2|   0|  67108864|       1|0.02950052|  PASSED
           dab_opso2|   1|  67108864|       1|0.58791773|  PASSED
```

Listing 8.17: Test results for random number engine `trng::mt19937`.

```
#=============================================================================#
#            dieharder version 3.31.1 Copyright 2003 Robert G. Brown          #
#=============================================================================#
   rng_name    |rands/second|   Seed   |
   trng_mt19937|  2.60e+08  |         1|
#=============================================================================#
        test_name   |ntup| tsamples |psamples|  p-value |Assessment
#=============================================================================#
   diehard_birthdays|   0|       100|     100|0.11669084|  PASSED
      diehard_operm5|   0|   1000000|     100|0.71388126|  PASSED
  diehard_rank_32x32|   0|     40000|     100|0.97795008|  PASSED
    diehard_rank_6x8|   0|    100000|     100|0.90931986|  PASSED
   diehard_bitstream|   0|   2097152|     100|0.17781828|  PASSED
        diehard_opso|   0|   2097152|     100|0.60413306|  PASSED
        diehard_oqso|   0|   2097152|     100|0.56541214|  PASSED
         diehard_dna|   0|   2097152|     100|0.69733248|  PASSED
diehard_count_1s_str|   0|    256000|     100|0.40229561|  PASSED
diehard_count_1s_byt|   0|    256000|     100|0.00927025|  PASSED
 diehard_parking_lot|   0|     12000|     100|0.82963409|  PASSED
    diehard_2dsphere|   2|      8000|     100|0.35078979|  PASSED
    diehard_3dsphere|   3|      4000|     100|0.69156214|  PASSED
     diehard_squeeze|   0|    100000|     100|0.56187200|  PASSED
        diehard_sums|   0|       100|     100|0.01592407|  PASSED
        diehard_runs|   0|    100000|     100|0.81216558|  PASSED
        diehard_runs|   0|    100000|     100|0.33069282|  PASSED
       diehard_craps|   0|    200000|     100|0.98414542|  PASSED
       diehard_craps|   0|    200000|     100|0.90348993|  PASSED
 marsaglia_tsang_gcd|   0|  10000000|     100|0.93246961|  PASSED
 marsaglia_tsang_gcd|   0|  10000000|     100|0.21467140|  PASSED
         sts_monobit|   1|    100000|     100|0.22075149|  PASSED
            sts_runs|   2|    100000|     100|0.80797098|  PASSED
          sts_serial|   1|    100000|     100|0.18014195|  PASSED
          sts_serial|   2|    100000|     100|0.02231424|  PASSED
          sts_serial|   3|    100000|     100|0.30254611|  PASSED
          sts_serial|   3|    100000|     100|0.81215538|  PASSED
          sts_serial|   4|    100000|     100|0.87050595|  PASSED
          sts_serial|   4|    100000|     100|0.59473013|  PASSED
          sts_serial|   5|    100000|     100|0.76698020|  PASSED
          sts_serial|   5|    100000|     100|0.41797496|  PASSED
          sts_serial|   6|    100000|     100|0.18281188|  PASSED
          sts_serial|   6|    100000|     100|0.07476773|  PASSED
          sts_serial|   7|    100000|     100|0.72114056|  PASSED
          sts_serial|   7|    100000|     100|0.71023413|  PASSED
          sts_serial|   8|    100000|     100|0.24430871|  PASSED
          sts_serial|   8|    100000|     100|0.80149254|  PASSED
          sts_serial|   9|    100000|     100|0.90210505|  PASSED
          sts_serial|   9|    100000|     100|0.36681824|  PASSED
          sts_serial|  10|    100000|     100|0.84486201|  PASSED
          sts_serial|  10|    100000|     100|0.88230532|  PASSED
          sts_serial|  11|    100000|     100|0.98500178|  PASSED
          sts_serial|  11|    100000|     100|0.89248487|  PASSED
          sts_serial|  12|    100000|     100|0.74272637|  PASSED
          sts_serial|  12|    100000|     100|0.22530703|  PASSED
          sts_serial|  13|    100000|     100|0.46269406|  PASSED
          sts_serial|  13|    100000|     100|0.53222511|  PASSED
```

```
          sts_serial|  14|    100000|     100|0.32730402|  PASSED
          sts_serial|  14|    100000|     100|0.65652245|  PASSED
          sts_serial|  15|    100000|     100|0.24719593|  PASSED
          sts_serial|  15|    100000|     100|0.25606551|  PASSED
          sts_serial|  16|    100000|     100|0.43031251|  PASSED
          sts_serial|  16|    100000|     100|0.38983115|  PASSED
         rgb_bitdist|   1|    100000|     100|0.48930154|  PASSED
         rgb_bitdist|   2|    100000|     100|0.03973316|  PASSED
         rgb_bitdist|   3|    100000|     100|0.86665372|  PASSED
         rgb_bitdist|   4|    100000|     100|0.07097145|  PASSED
         rgb_bitdist|   5|    100000|     100|0.10386512|  PASSED
         rgb_bitdist|   6|    100000|     100|0.72205278|  PASSED
         rgb_bitdist|   7|    100000|     100|0.77521569|  PASSED
         rgb_bitdist|   8|    100000|     100|0.44520348|  PASSED
         rgb_bitdist|   9|    100000|     100|0.56411180|  PASSED
         rgb_bitdist|  10|    100000|     100|0.55720636|  PASSED
         rgb_bitdist|  11|    100000|     100|0.33155405|  PASSED
         rgb_bitdist|  12|    100000|     100|0.99548047|   WEAK
rgb_minimum_distance|   2|     10000|    1000|0.86320348|  PASSED
rgb_minimum_distance|   3|     10000|    1000|0.52633570|  PASSED
rgb_minimum_distance|   4|     10000|    1000|0.99923342|   WEAK
rgb_minimum_distance|   5|     10000|    1000|0.76766888|  PASSED
    rgb_permutations|   2|    100000|     100|0.12030777|  PASSED
    rgb_permutations|   3|    100000|     100|0.40097844|  PASSED
    rgb_permutations|   4|    100000|     100|0.90123154|  PASSED
    rgb_permutations|   5|    100000|     100|0.00726658|  PASSED
      rgb_lagged_sum|   0|   1000000|     100|0.37944717|  PASSED
      rgb_lagged_sum|   1|   1000000|     100|0.91651676|  PASSED
      rgb_lagged_sum|   2|   1000000|     100|0.34901795|  PASSED
      rgb_lagged_sum|   3|   1000000|     100|0.70731397|  PASSED
      rgb_lagged_sum|   4|   1000000|     100|0.26499795|  PASSED
      rgb_lagged_sum|   5|   1000000|     100|0.80156424|  PASSED
      rgb_lagged_sum|   6|   1000000|     100|0.13086153|  PASSED
      rgb_lagged_sum|   7|   1000000|     100|0.80496208|  PASSED
      rgb_lagged_sum|   8|   1000000|     100|0.45265262|  PASSED
      rgb_lagged_sum|   9|   1000000|     100|0.99982502|   WEAK
      rgb_lagged_sum|  10|   1000000|     100|0.14338611|  PASSED
      rgb_lagged_sum|  11|   1000000|     100|0.21514947|  PASSED
      rgb_lagged_sum|  12|   1000000|     100|0.32620166|  PASSED
      rgb_lagged_sum|  13|   1000000|     100|0.70781647|  PASSED
      rgb_lagged_sum|  14|   1000000|     100|0.90814316|  PASSED
      rgb_lagged_sum|  15|   1000000|     100|0.96343565|  PASSED
      rgb_lagged_sum|  16|   1000000|     100|0.49237641|  PASSED
      rgb_lagged_sum|  17|   1000000|     100|0.39746234|  PASSED
      rgb_lagged_sum|  18|   1000000|     100|0.86678756|  PASSED
      rgb_lagged_sum|  19|   1000000|     100|0.90850857|  PASSED
      rgb_lagged_sum|  20|   1000000|     100|0.60013304|  PASSED
      rgb_lagged_sum|  21|   1000000|     100|0.76258696|  PASSED
      rgb_lagged_sum|  22|   1000000|     100|0.23615019|  PASSED
      rgb_lagged_sum|  23|   1000000|     100|0.52967644|  PASSED
      rgb_lagged_sum|  24|   1000000|     100|0.81049592|  PASSED
      rgb_lagged_sum|  25|   1000000|     100|0.88790526|  PASSED
      rgb_lagged_sum|  26|   1000000|     100|0.73559207|  PASSED
      rgb_lagged_sum|  27|   1000000|     100|0.54861278|  PASSED
      rgb_lagged_sum|  28|   1000000|     100|0.82806027|  PASSED
      rgb_lagged_sum|  29|   1000000|     100|0.20406231|  PASSED
      rgb_lagged_sum|  30|   1000000|     100|0.15108355|  PASSED
      rgb_lagged_sum|  31|   1000000|     100|0.99423174|  PASSED
      rgb_lagged_sum|  32|   1000000|     100|0.99062048|  PASSED
     rgb_kstest_test|   0|     10000|    1000|0.08572642|  PASSED
     dab_bytedistrib|   0|  51200000|       1|0.68637756|  PASSED
             dab_dct| 256|     50000|       1|0.11431048|  PASSED
Preparing to run test 207.  ntuple = 0
        dab_filltree|  32|  15000000|       1|0.80452606|  PASSED
        dab_filltree|  32|  15000000|       1|0.14710854|  PASSED
```

```
Preparing to run test 208.  ntuple = 0
       dab_filltree2|   0|   5000000|       1|0.62190578|  PASSED
       dab_filltree2|   1|   5000000|       1|0.25852506|  PASSED
Preparing to run test 209.  ntuple = 0
        dab_monobit2|  12|  65000000|       1|0.62597999|  PASSED
Preparing to run test 210.  ntuple = 0
      dab_birthdays1|  32|      2000|       1|0.72814078|  PASSED
Preparing to run test 211.  ntuple = 0
           dab_opso2|   0|  67108864|       1|0.02862256|  PASSED
           dab_opso2|   1|  67108864|       1|0.77402020|  PASSED
```

Listing 8.18: Test results for random number engine `trng::mt19937_64`.

```
#=============================================================================#
#            dieharder version 3.31.1 Copyright 2003 Robert G. Brown          #
#=============================================================================#
   rng_name    |rands/second|   Seed   |
trng_mt19937_64|  2.03e+08  |         1|
#=============================================================================#
        test_name   |ntup| tsamples |psamples|  p-value |Assessment
#=============================================================================#
   diehard_birthdays|   0|       100|     100|0.76819383|  PASSED
      diehard_operm5|   0|   1000000|     100|0.96120593|  PASSED
  diehard_rank_32x32|   0|     40000|     100|0.90508511|  PASSED
    diehard_rank_6x8|   0|    100000|     100|0.85505342|  PASSED
   diehard_bitstream|   0|   2097152|     100|0.75118601|  PASSED
        diehard_opso|   0|   2097152|     100|0.48554373|  PASSED
        diehard_oqso|   0|   2097152|     100|0.98764042|  PASSED
         diehard_dna|   0|   2097152|     100|0.27413584|  PASSED
diehard_count_1s_str|   0|    256000|     100|0.99409784|  PASSED
diehard_count_1s_byt|   0|    256000|     100|0.39048263|  PASSED
 diehard_parking_lot|   0|     12000|     100|0.43595334|  PASSED
    diehard_2dsphere|   2|      8000|     100|0.40550621|  PASSED
    diehard_3dsphere|   3|      4000|     100|0.91684639|  PASSED
     diehard_squeeze|   0|    100000|     100|0.16558611|  PASSED
        diehard_sums|   0|       100|     100|0.04365877|  PASSED
        diehard_runs|   0|    100000|     100|0.56177104|  PASSED
        diehard_runs|   0|    100000|     100|0.25009794|  PASSED
       diehard_craps|   0|    200000|     100|0.40316320|  PASSED
       diehard_craps|   0|    200000|     100|0.89631489|  PASSED
 marsaglia_tsang_gcd|   0|  10000000|     100|0.99849264|   WEAK
 marsaglia_tsang_gcd|   0|  10000000|     100|0.54027677|  PASSED
         sts_monobit|   1|    100000|     100|0.15453928|  PASSED
            sts_runs|   2|    100000|     100|0.94836883|  PASSED
          sts_serial|   1|    100000|     100|0.72404987|  PASSED
          sts_serial|   2|    100000|     100|0.76818470|  PASSED
          sts_serial|   3|    100000|     100|0.79742148|  PASSED
          sts_serial|   3|    100000|     100|0.98892651|  PASSED
          sts_serial|   4|    100000|     100|0.45511052|  PASSED
          sts_serial|   4|    100000|     100|0.68338877|  PASSED
          sts_serial|   5|    100000|     100|0.69866594|  PASSED
          sts_serial|   5|    100000|     100|0.44549222|  PASSED
          sts_serial|   6|    100000|     100|0.73853758|  PASSED
          sts_serial|   6|    100000|     100|0.34670755|  PASSED
          sts_serial|   7|    100000|     100|0.87151299|  PASSED
          sts_serial|   7|    100000|     100|0.43284075|  PASSED
          sts_serial|   8|    100000|     100|0.73701469|  PASSED
          sts_serial|   8|    100000|     100|0.72151955|  PASSED
          sts_serial|   9|    100000|     100|0.16400262|  PASSED
          sts_serial|   9|    100000|     100|0.35508272|  PASSED
          sts_serial|  10|    100000|     100|0.28405262|  PASSED
          sts_serial|  10|    100000|     100|0.54696006|  PASSED
          sts_serial|  11|    100000|     100|0.98807801|  PASSED
          sts_serial|  11|    100000|     100|0.23529025|  PASSED
          sts_serial|  12|    100000|     100|0.97910769|  PASSED
```

```
          sts_serial|  12|    100000|     100|0.66202755|  PASSED
          sts_serial|  13|    100000|     100|0.89760264|  PASSED
          sts_serial|  13|    100000|     100|0.74550250|  PASSED
          sts_serial|  14|    100000|     100|0.92523135|  PASSED
          sts_serial|  14|    100000|     100|0.85701288|  PASSED
          sts_serial|  15|    100000|     100|0.78271260|  PASSED
          sts_serial|  15|    100000|     100|0.77702436|  PASSED
          sts_serial|  16|    100000|     100|0.69472097|  PASSED
          sts_serial|  16|    100000|     100|0.73629529|  PASSED
         rgb_bitdist|   1|    100000|     100|0.14003411|  PASSED
         rgb_bitdist|   2|    100000|     100|0.98855446|  PASSED
         rgb_bitdist|   3|    100000|     100|0.95926565|  PASSED
         rgb_bitdist|   4|    100000|     100|0.47209248|  PASSED
         rgb_bitdist|   5|    100000|     100|0.79625214|  PASSED
         rgb_bitdist|   6|    100000|     100|0.06046714|  PASSED
         rgb_bitdist|   7|    100000|     100|0.96874761|  PASSED
         rgb_bitdist|   8|    100000|     100|0.11315740|  PASSED
         rgb_bitdist|   9|    100000|     100|0.86130806|  PASSED
         rgb_bitdist|  10|    100000|     100|0.96784014|  PASSED
         rgb_bitdist|  11|    100000|     100|0.26089610|  PASSED
         rgb_bitdist|  12|    100000|     100|0.97645696|  PASSED
rgb_minimum_distance|   2|     10000|    1000|0.58575673|  PASSED
rgb_minimum_distance|   3|     10000|    1000|0.95840438|  PASSED
rgb_minimum_distance|   4|     10000|    1000|0.32084619|  PASSED
rgb_minimum_distance|   5|     10000|    1000|0.43743286|  PASSED
    rgb_permutations|   2|    100000|     100|0.99578787|   WEAK
    rgb_permutations|   3|    100000|     100|0.39558152|  PASSED
    rgb_permutations|   4|    100000|     100|0.98849084|  PASSED
    rgb_permutations|   5|    100000|     100|0.99695998|   WEAK
      rgb_lagged_sum|   0|   1000000|     100|0.45120841|  PASSED
      rgb_lagged_sum|   1|   1000000|     100|0.59561230|  PASSED
      rgb_lagged_sum|   2|   1000000|     100|0.98236275|  PASSED
      rgb_lagged_sum|   3|   1000000|     100|0.63415714|  PASSED
      rgb_lagged_sum|   4|   1000000|     100|0.60909032|  PASSED
      rgb_lagged_sum|   5|   1000000|     100|0.67328866|  PASSED
      rgb_lagged_sum|   6|   1000000|     100|0.61600020|  PASSED
      rgb_lagged_sum|   7|   1000000|     100|0.42392922|  PASSED
      rgb_lagged_sum|   8|   1000000|     100|0.50045631|  PASSED
      rgb_lagged_sum|   9|   1000000|     100|0.20383953|  PASSED
      rgb_lagged_sum|  10|   1000000|     100|0.52225580|  PASSED
      rgb_lagged_sum|  11|   1000000|     100|0.54080744|  PASSED
      rgb_lagged_sum|  12|   1000000|     100|0.93918547|  PASSED
      rgb_lagged_sum|  13|   1000000|     100|0.32555080|  PASSED
      rgb_lagged_sum|  14|   1000000|     100|0.97322048|  PASSED
      rgb_lagged_sum|  15|   1000000|     100|0.51676909|  PASSED
      rgb_lagged_sum|  16|   1000000|     100|0.33888569|  PASSED
      rgb_lagged_sum|  17|   1000000|     100|0.24254385|  PASSED
      rgb_lagged_sum|  18|   1000000|     100|0.41088623|  PASSED
      rgb_lagged_sum|  19|   1000000|     100|0.27483893|  PASSED
      rgb_lagged_sum|  20|   1000000|     100|0.40606883|  PASSED
      rgb_lagged_sum|  21|   1000000|     100|0.56091507|  PASSED
      rgb_lagged_sum|  22|   1000000|     100|0.78806672|  PASSED
      rgb_lagged_sum|  23|   1000000|     100|0.91496375|  PASSED
      rgb_lagged_sum|  24|   1000000|     100|0.01969797|  PASSED
      rgb_lagged_sum|  25|   1000000|     100|0.06705914|  PASSED
      rgb_lagged_sum|  26|   1000000|     100|0.88242187|  PASSED
      rgb_lagged_sum|  27|   1000000|     100|0.98374119|  PASSED
      rgb_lagged_sum|  28|   1000000|     100|0.42650300|  PASSED
      rgb_lagged_sum|  29|   1000000|     100|0.85451887|  PASSED
      rgb_lagged_sum|  30|   1000000|     100|0.88808635|  PASSED
      rgb_lagged_sum|  31|   1000000|     100|0.43503796|  PASSED
      rgb_lagged_sum|  32|   1000000|     100|0.48425096|  PASSED
     rgb_kstest_test|   0|     10000|    1000|0.88796246|  PASSED
     dab_bytedistrib|   0|  51200000|       1|0.83239755|  PASSED
             dab_dct| 256|     50000|       1|0.16563963|  PASSED
```

```
Preparing to run test 207.  ntuple = 0
        dab_filltree|  32|  15000000|       1|0.82612425|  PASSED
        dab_filltree|  32|  15000000|       1|0.64466529|  PASSED
Preparing to run test 208.  ntuple = 0
       dab_filltree2|   0|   5000000|       1|0.90433735|  PASSED
       dab_filltree2|   1|   5000000|       1|0.11643792|  PASSED
Preparing to run test 209.  ntuple = 0
        dab_monobit2|  12|  65000000|       1|0.66349705|  PASSED
Preparing to run test 210.  ntuple = 0
      dab_birthdays1|  32|      2000|       1|0.25110734|  PASSED
Preparing to run test 211.  ntuple = 0
           dab_opso2|   0|  67108864|       1|0.91744737|  PASSED
           dab_opso2|   1|  67108864|       1|0.92057122|  PASSED
```

Listing 8.19: Test results for random number engine `trng::xoshiro256plus`.

```
#=============================================================================#
#            dieharder version 3.31.1 Copyright 2003 Robert G. Brown          #
#=============================================================================#
   rng_name    |rands/second|   Seed   |
trng_xoshiro256plus|  4.24e+08  |         1|
#=============================================================================#
        test_name   |ntup| tsamples |psamples|  p-value |Assessment
#=============================================================================#
   diehard_birthdays|   0|       100|     100|0.57052056|  PASSED
      diehard_operm5|   0|   1000000|     100|0.53827481|  PASSED
  diehard_rank_32x32|   0|     40000|     100|0.62988782|  PASSED
    diehard_rank_6x8|   0|    100000|     100|0.32608518|  PASSED
   diehard_bitstream|   0|   2097152|     100|0.74933236|  PASSED
        diehard_opso|   0|   2097152|     100|0.64597348|  PASSED
        diehard_oqso|   0|   2097152|     100|0.83111279|  PASSED
         diehard_dna|   0|   2097152|     100|0.85425235|  PASSED
diehard_count_1s_str|   0|    256000|     100|0.02237928|  PASSED
diehard_count_1s_byt|   0|    256000|     100|0.87899346|  PASSED
 diehard_parking_lot|   0|     12000|     100|0.14887474|  PASSED
    diehard_2dsphere|   2|      8000|     100|0.87795109|  PASSED
    diehard_3dsphere|   3|      4000|     100|0.94587766|  PASSED
     diehard_squeeze|   0|    100000|     100|0.59827665|  PASSED
        diehard_sums|   0|       100|     100|0.81418321|  PASSED
        diehard_runs|   0|    100000|     100|0.66905970|  PASSED
        diehard_runs|   0|    100000|     100|0.77849084|  PASSED
       diehard_craps|   0|    200000|     100|0.46554025|  PASSED
       diehard_craps|   0|    200000|     100|0.78809502|  PASSED
 marsaglia_tsang_gcd|   0|  10000000|     100|0.72324698|  PASSED
 marsaglia_tsang_gcd|   0|  10000000|     100|0.52325598|  PASSED
         sts_monobit|   1|    100000|     100|0.69111635|  PASSED
            sts_runs|   2|    100000|     100|0.82017587|  PASSED
          sts_serial|   1|    100000|     100|0.18965761|  PASSED
          sts_serial|   2|    100000|     100|0.85952062|  PASSED
          sts_serial|   3|    100000|     100|0.57036482|  PASSED
          sts_serial|   3|    100000|     100|0.74696609|  PASSED
          sts_serial|   4|    100000|     100|0.85926064|  PASSED
          sts_serial|   4|    100000|     100|0.87580301|  PASSED
          sts_serial|   5|    100000|     100|0.87942202|  PASSED
          sts_serial|   5|    100000|     100|0.98286773|  PASSED
          sts_serial|   6|    100000|     100|0.76866974|  PASSED
          sts_serial|   6|    100000|     100|0.23119826|  PASSED
          sts_serial|   7|    100000|     100|0.90377619|  PASSED
          sts_serial|   7|    100000|     100|0.74957488|  PASSED
          sts_serial|   8|    100000|     100|0.48960372|  PASSED
          sts_serial|   8|    100000|     100|0.37829465|  PASSED
          sts_serial|   9|    100000|     100|0.36618966|  PASSED
          sts_serial|   9|    100000|     100|0.52428326|  PASSED
          sts_serial|  10|    100000|     100|0.94645914|  PASSED
          sts_serial|  10|    100000|     100|0.63261553|  PASSED
```

```
          sts_serial|  11|    100000|     100|0.98678213|  PASSED
          sts_serial|  11|    100000|     100|0.85173808|  PASSED
          sts_serial|  12|    100000|     100|0.42019178|  PASSED
          sts_serial|  12|    100000|     100|0.89718297|  PASSED
          sts_serial|  13|    100000|     100|0.66786677|  PASSED
          sts_serial|  13|    100000|     100|0.74506198|  PASSED
          sts_serial|  14|    100000|     100|0.62127864|  PASSED
          sts_serial|  14|    100000|     100|0.48825811|  PASSED
          sts_serial|  15|    100000|     100|0.36095829|  PASSED
          sts_serial|  15|    100000|     100|0.97867155|  PASSED
          sts_serial|  16|    100000|     100|0.79899363|  PASSED
          sts_serial|  16|    100000|     100|0.80570817|  PASSED
         rgb_bitdist|   1|    100000|     100|0.48430240|  PASSED
         rgb_bitdist|   2|    100000|     100|0.59740256|  PASSED
         rgb_bitdist|   3|    100000|     100|0.92543122|  PASSED
         rgb_bitdist|   4|    100000|     100|0.08418232|  PASSED
         rgb_bitdist|   5|    100000|     100|0.83056473|  PASSED
         rgb_bitdist|   6|    100000|     100|0.01919443|  PASSED
         rgb_bitdist|   7|    100000|     100|0.98039135|  PASSED
         rgb_bitdist|   8|    100000|     100|0.65531295|  PASSED
         rgb_bitdist|   9|    100000|     100|0.73393834|  PASSED
         rgb_bitdist|  10|    100000|     100|0.28415923|  PASSED
         rgb_bitdist|  11|    100000|     100|0.45451748|  PASSED
         rgb_bitdist|  12|    100000|     100|0.89221186|  PASSED
rgb_minimum_distance|   2|     10000|    1000|0.75318192|  PASSED
rgb_minimum_distance|   3|     10000|    1000|0.42879198|  PASSED
rgb_minimum_distance|   4|     10000|    1000|0.78590979|  PASSED
rgb_minimum_distance|   5|     10000|    1000|0.64642387|  PASSED
    rgb_permutations|   2|    100000|     100|0.22004884|  PASSED
    rgb_permutations|   3|    100000|     100|0.11958015|  PASSED
    rgb_permutations|   4|    100000|     100|0.06702182|  PASSED
    rgb_permutations|   5|    100000|     100|0.36309523|  PASSED
      rgb_lagged_sum|   0|   1000000|     100|0.28718711|  PASSED
      rgb_lagged_sum|   1|   1000000|     100|0.94817388|  PASSED
      rgb_lagged_sum|   2|   1000000|     100|0.24310773|  PASSED
      rgb_lagged_sum|   3|   1000000|     100|0.76034333|  PASSED
      rgb_lagged_sum|   4|   1000000|     100|0.94907058|  PASSED
      rgb_lagged_sum|   5|   1000000|     100|0.57822578|  PASSED
      rgb_lagged_sum|   6|   1000000|     100|0.96707148|  PASSED
      rgb_lagged_sum|   7|   1000000|     100|0.03959635|  PASSED
      rgb_lagged_sum|   8|   1000000|     100|0.98490111|  PASSED
      rgb_lagged_sum|   9|   1000000|     100|0.35090551|  PASSED
      rgb_lagged_sum|  10|   1000000|     100|0.65374806|  PASSED
      rgb_lagged_sum|  11|   1000000|     100|0.46356091|  PASSED
      rgb_lagged_sum|  12|   1000000|     100|0.17356961|  PASSED
      rgb_lagged_sum|  13|   1000000|     100|0.57799948|  PASSED
      rgb_lagged_sum|  14|   1000000|     100|0.39918242|  PASSED
      rgb_lagged_sum|  15|   1000000|     100|0.05971442|  PASSED
      rgb_lagged_sum|  16|   1000000|     100|0.00166822|   WEAK
      rgb_lagged_sum|  17|   1000000|     100|0.66815737|  PASSED
      rgb_lagged_sum|  18|   1000000|     100|0.08132020|  PASSED
      rgb_lagged_sum|  19|   1000000|     100|0.96034047|  PASSED
      rgb_lagged_sum|  20|   1000000|     100|0.69024037|  PASSED
      rgb_lagged_sum|  21|   1000000|     100|0.00276873|   WEAK
      rgb_lagged_sum|  22|   1000000|     100|0.62056795|  PASSED
      rgb_lagged_sum|  23|   1000000|     100|0.70746810|  PASSED
      rgb_lagged_sum|  24|   1000000|     100|0.15019399|  PASSED
      rgb_lagged_sum|  25|   1000000|     100|0.68445314|  PASSED
      rgb_lagged_sum|  26|   1000000|     100|0.96864253|  PASSED
      rgb_lagged_sum|  27|   1000000|     100|0.29760293|  PASSED
      rgb_lagged_sum|  28|   1000000|     100|0.95570996|  PASSED
      rgb_lagged_sum|  29|   1000000|     100|0.87569402|  PASSED
      rgb_lagged_sum|  30|   1000000|     100|0.08085499|  PASSED
      rgb_lagged_sum|  31|   1000000|     100|0.41290501|  PASSED
      rgb_lagged_sum|  32|   1000000|     100|0.99573357|   WEAK
```

```
     rgb_kstest_test|   0|     10000|    1000|0.01384423|  PASSED
     dab_bytedistrib|   0|  51200000|       1|0.79323932|  PASSED
             dab_dct| 256|     50000|       1|0.27170560|  PASSED
Preparing to run test 207.  ntuple = 0
        dab_filltree|  32|  15000000|       1|0.43307295|  PASSED
        dab_filltree|  32|  15000000|       1|0.37625843|  PASSED
Preparing to run test 208.  ntuple = 0
       dab_filltree2|   0|   5000000|       1|0.10128252|  PASSED
       dab_filltree2|   1|   5000000|       1|0.56459584|  PASSED
Preparing to run test 209.  ntuple = 0
        dab_monobit2|  12|  65000000|       1|0.79868199|  PASSED
Preparing to run test 210.  ntuple = 0
      dab_birthdays1|  32|      2000|       1|0.96785141|  PASSED
Preparing to run test 211.  ntuple = 0
           dab_opso2|   0|  67108864|       1|0.45133772|  PASSED
           dab_opso2|   1|  67108864|       1|0.31966021|  PASSED
```

9 Frequently asked questions

What are the license terms for using and distributing the TRNG library? TRNG is free software. Starting from version 4.9, the TRNG library is distributed under the terms of a BSD style license (3-clause license). Earlier TRNG versions are distributed under the GNU Public License (GPL) Version 2. See also page 184.

Why is the library called TRNG? Who is Tina? Tina is the name of a Linux cluster at the Institute of Theoretical Physics at the University Magdeburg in Germany. TRNG was written to carry out Monte Carlo simulations on this parallel computer. The name Tina is a self referring acronym for "Tina is no acronym". The abbreviation TRNG stands for "Tina's Random Number Generator Library". But sometimes it is used in the literature for "true random number generator" as well, which is a technical device that generates random numbers by a physical process (e. g. radioactive decay or noise in a electric circuit).

I am confused, there are so many different PRNGs in TRNG. Which one is the best? There is nothing like the best PRNG. If a generator behaves as a good source of randomness or not can depend on your Monte Carlo application, and there are trade-offs between speed and quality. In general, it is a good idea to test if the outcome of a Monte Carlo simulation is independent of the underlying PRNG. Therefore TRNG offers so many of them.

But generally speaking, YARN generators are a good choice (see section 4.1.4). If the PRNG is the bottleneck of your Monte Carlo simulation you might try the linear congruential generator (see section 4.1.1) or in the case of a sequential simulation a lagged Fibonacci generator with four feedback taps (see section 4.1.5).

Why is TRNG written in C++? C++ provides a lot of advanced features as inline functions and static polymorphism via templates. These language features give us the power to implement a fast, portable and easy to use library of PRNGs. Other languages (as FORTRAN or C) do no offer these (or comparable) features, are significantly slower (as Java or scripting languages), or are supported by fewer platforms.

How can I use TRNG in my FORTRAN programs? Unfortunately this is not possible. TRNG makes heavy use of special C++ language features as classes, inline functions, and templates. All theses concepts have no counterpart in the FORTRAN programming language. Large parts of TRNG even do not reside in the library that you link with `-ltrng4` to your object code. Template functions and inline functions are defined exclusively in the header files.

How can I use TRNG in my C programs? Unfortunately this is not possible. Here the same statements apply as for the last question. However, it is much more easy to port a C program to C++ than porting a FORTRAN program to C++. Just comply with the following recipe.

- Rename header files *foo*.h of the C standard library into c*foo* but let other header files untouched, i. e., change

```
#include <stdio.h>
#include <math.h>
#include <unistd.h>
```

  into

```
#include <cstdio>
#include <cmath>
#include <unistd.h>
```

  Note, the unistd.h header is not part of the C standard library.
- Insert the line

```
using namespace std;
```

  after the include directives of each source file.
- Do not use C++ function names that are C++ keywords, i. e., class, new, public or private.

This recipe will give you an ugly but valid C++ program, at least in the most cases. This modified "C" program has to be compiled by a C++ compiler now, but it is ready to benefit from the TRNG library.

How can I give feedback, report bugs, or make a feature request? Send bug reports and feature requests to the author of TRNG via e-mail to trng@mail.de or open an issue on Github [75].

I used TRNG in my research and want to give credit. How should I cite TRNG? The main concepts, which TRNG builds on, are published in Heiko Bauke and Stephan Mertens. Random numbers for large-scale distributed Monte Carlo simulations. *Physical Review E*, 75(6):066701, 2007. Please cite this publication.

License

Starting from version 4.9, the TRNG library is distributed under the terms of a BSD style license (3-clause license). Earlier TRNG versions are distributed under the GNU Public License (GPL) Version 2. The BSD license is a much more liberal license than the GPL but it is a GPL compatible license. Thus, TRNG 4.9 and later versions may be used in GPL software projects.


```
Copyright (c) 2000-2026, Heiko Bauke
All rights reserved.

Redistribution and use in source and binary forms, with or without
modification, are permitted provided that the following conditions
are met:

  * Redistributions of source code must retain the above copyright
    notice, this list of conditions and the following disclaimer.

  * Redistributions in binary form must reproduce the above
    copyright notice, this list of conditions and the following
    disclaimer in the documentation and/or other materials provided
    with the distribution.

  * Neither the name of the copyright holder nor the names of its
    contributors may be used to endorse or promote products derived
    from this software without specific prior written permission.

THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CONTRIBUTORS
"AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT
LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS
FOR A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL THE
COPYRIGHT HOLDERS OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT,
INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES
(INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR
SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION)
HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT,
STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE)
ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED
OF THE POSSIBILITY OF SUCH DAMAGE.
```

Bibliography

[1] CUDA. https://developer.nvidia.com/cuda-toolkit.

[2] Diehard tests. https://en.wikipedia.org/wiki/Diehard_tests.

[3] NVIDIA CUDA C Programming Guide, 2010.

[4] L. Yu. Barash and L. N. Shchur. PRAND: GPU accelerated parallel random number generation library: Using most reliable algorithms and applying rallelism of modern GPUs and CPUs. *Computer Physics Communications*, 185(4):1343–1353, 2014.

[5] Heiko Bauke and Stephan Mertens. Pseudo random coins show more heads than tails. *Journal of Statistical Physics*, 114(3):1149–1169, 2004.

[6] Heiko Bauke and Stephan Mertens. *Cluster Computing*. Springer, 2005.

[7] Heiko Bauke and Stephan Mertens. Random numbers for large-scale distributed Monte Carlo simulations. *Physical Review E*, 75(6):066701, 2007.

[8] David Blackman and Sebastiano Vigna. Scrambled linear pseudorandom number generators. https://arxiv.org/abs/1805.01407, 2019.

[9] Boost C++ libraries. http://www.boost.org.

[10] Robert G. Brown. Dieharder: A Random Number Test Suite (modified for TRNG). https://github.com/rabauke/dieharder/tree/trng.

[11] Robert G. Brown. Dieharder: A Random Number Test Suite. https://webhome.phy.duke.edu/~rgb/General/dieharder.php, 2021.

[12] Walter E. Brown, Mark Fischler, Jim Kowalkowski, and Marc Paterno. *Random Number Generation in C++0X: A Comprehensive Proposal, version 2*, 2006. http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2006/n2032.pdf.

[13] CMake documenation. https://cmake.org/documentation/.

[14] CMake projects in Visual Studio. https://docs.microsoft.com/en-us/cpp/build/cmake-projects-in-visual-studio.

[15] Aaldert Compagner. Definitions of randomness. *American Journal of Physics*, 59(8):700–705, August 1991.

[16] Aaldert Compagner. The hierarchy of correlations in random binary sequences. *Journal of Statistical Physics*, 63:883–896, 1991.

[17] Lih-Yuan Deng and Dale Bowman. Developments in pseudo-random number generators. *Wiley Interdisciplinary Reviews: Computational Statistics*, 9(5):e1404, Aug 2017.

[18] Luc Devroye. *Non-Uniform Random Variate Generation*. Springer, 1986.

[19] Jürgen Eichenauer-Herrmann and Holger Grothe. A remark on long-range correlations in multiplicative congruential pseudo random number generators. *Numerische Mathematik*, 56(6):609–611, 1989.

[20] Alan M. Ferrenberg and D. P. Landau. Monte carlo simulations: Hidden errors from "good" random number generators. *Physical Review Letters*, 69(23):3382–3384, 1992.

[21] Jay Fillmore and Morris Marx. Linear recursive sequences. *SIAM Review*, 10(3):342–353, 1968.

[22] George Fishman. *Monte Carlo*. Springer, 1996.

[23] Amparo Gil, Javier Segura, and Nico M. Temme. Efficient and accurate algorithms for the computation and inversion of the incomplete gamma function ratios. *SIAM Journal on Scientific Computing*, 34(6):A2965–A2981, 2012.

[24] S. W. Golomb. *Shift Register Sequences*. Aegan Park Press, Laguna Hills, CA, revised edition, 1982.

[25] Peter Grassberger. On correlations in "good" random number generators. *Physics Letters A*, 181(1):43–46, 1993.

[26] Holger Grothe. Matrix generators for pseudo-random vector generation. *Statistische Hefte*, 28(1):233–238, Dec 1987.

[27] A. Grube. Mehrfach rekursiv-erzeugte Pseudo-Zufallszahlen. *Zeitschrift für angewandte Mathematik und Mechanik*, 53:T223–T225, 1973.

[28] Intel Threading Building Blocks. http://www.threadingbuildingblocks.org.

[29] ISO. *ISO/IEC 14882:2011 Information technology – Programming languages – C++*. ISO.

[30] Nicolai M. Josuttis. *The C++ Standard Library*. Addison Wesley, 2nd edition, 2012.

[31] Dieter Jungnickel. *Finite Fields: Structure and Arithmetics*. Bibliographisches Institut, 1993.

[32] David Kirk and Wen mei Hwu. *Programming Massively Parallel Processors: A Hands-on Approach*. Morgan Kaufmann, 2010.

[33] Scott Kirkpatrick and Erich P. Stoll. A very fast shift-register sequence random number generator. *Journal of Computational Physics*, 40(2):517–526, 1981.

[34] Donald E. Knuth. *The Art of Computer Programming*, volume 2: Seminumerical Algorithms. Addison Wesley Professional, 1st edition, 1969.

[35] Donald E. Knuth. *The Art of Computer Programming*, volume 2: Seminumerical Algorithms. Addison Wesley Professional, 3rd edition, 1998.

[36] Werner Krauth. *Statistical Mechanics: Algorithms and Computations*. Oxford Master Series in Statistical, Computational, and Theoretical Physics. Oxford University Press, 2006.

[37] David P. Landau and Kurt Binder. *A Guide to Monte Carlo Simulations in Statistical Physics*. Cambridge University Press, 2nd edition, 2005.

[38] Pierre L'Ecuyer. Random numbers for simulation. *Communications of the ACM*, 33(10):85–97, 1990.

[39] Pierre L'Ecuyer. A search for good multiple recursive random number generators. *ACM Transactions on Modeling and Computer Simulation*, 3(2):87–98, 1993.

[40] Pierre L'Ecuyer. Tables of linear congruential generators of different sizes and good lattice structure. *Mathematics of Computation*, 68:249–260, 1999.

[41] Pierre L'Ecuyer. Software for uniform random number generation: Distinguishing the good and the bad. In *Proceedings of the 2001 Winter Simulation Conference*, pages 95–105. IEEE, IEEE Press, 2001.

[42] Pierre L'Ecuyer. Random number generation. In James E. Gentle, Wolfgang Härdle, and Yuichi Mori, editors, *Handbook of Computational Statistics*. Springer, 2004.

[43] Pierre L'Ecuyer and Peter Hellekalek. Random number generators: Selection criteria and testing. In *Random and Quasi-Random Point Sets*, volume 138 of *Lecture Notes in Statistics*, pages 223–266. Springer, 1998.

[44] D. H. Lehmer. Mathematical methods in large-scale computing units. In *Proc. 2nd Sympos. on Large-Scale Digital Calculating Machinery, Cambridge, MA, 1949*, pages 141–146. Harvard University Press, 1951.

[45] Rudolf Lidl and Harald Niederreiter. *Introduction to finite fields and their applications*. Cambridge University Press, 2nd edition, 1994.

[46] Rudolf Lidl and Harald Niederreiter. *Finite Fields*, volume 20 of *Encyclopedia of Mathematics and its Applications*. Cambridge University Press, 2nd edition, 1997.

[47] George Marsaglia. Random numbers fall mainly in the planes. *Proceedings of the National Academy of Sciences*, 61:25–28, 1968.

[48] Michael Mascagni. Parallel linear congruential generators with prime moduli. *Prallel Computing*, 24(5–6):923–936, 1998.

[49] Michael Mascagni and Hongmei Chi. Parallel linear congruential generators with Sophie-Germain moduli. *Parallel Computing*, 30(11):1217–1231, 2004.

[50] Michael Mascagni and Lin-Yee Hin. Parallel random number generators in monte carlo derivative pricing: An application-based test. *Monte Carlo Methods and Applications*, 18(2):161–179, Jan 2012.

[51] Makoto Matsumoto and Takuji Nishimura. Mersenne twister: a 623-dimensionally equidistributed uniform pseudo-random number generator. *ACM Transactions on Modeling and Computer Simulation*, 8(1):3–30, 1998.

[52] A. De Matteis and S. Pagnutti. A class of parallel random number generators. *Parallel Computing*, 13(2):193–198, 1990.

[53] A. De Matteis and S. Pagnutti. Long-range correlations in linear and non-linear random number generators. *Parallel Computing*, 14(2):207–210, 1990.

[54] Don L. McLeish. *Monte Carlo Simulation and Finance*. John Wiley & Sons, 2005.

[55] Stephan Mertens. Random number generators: A survival guide for large scale simulations. https://arxiv.org/abs/0905.4238, 2009.

[56] Stephan Mertens and Heiko Bauke. Entropy of pseudo-random-number generators. *Physical Review E*, 69:055702–1–055702–4, 2004.

[57] MPICH2. http://www-unix.mcs.anl.gov/mpi/mpich.

[58] David R. Musser, Gillmer J. Derge, and Atul Saini. *STL Tutorial and Reference Guide: C++ Programming with the Standard Template Library*. Addison-Wesley Professional, 2001.

[59] M. E. J. Newman and G. T. Barkema. *Monte Carlo Methods in Statistical Physics*. Oxford University Press, 1999.

[60] Random number generation with multiple streams for sequential and parallel computing. Pierre L'Ecuyer. In *2015 Winter Simulation Conference (WSC)*. IEEE.

[61] Open MPI. http://www.open-mpi.org.

[62] Melissa E. O'Neill. PCG: A family of simple fast space-efficient statistically good algorithms for random number generation. Technical Report HMC-CS-2014-0905, Harvey Mudd College, Claremont, CA, 2014. published as https://www.cs.hmc.edu/tr/hmc-cs-2014-0905.pdf.

[63] Peter Pacheco. *Parallel Programming with MPI*. Morgan Kaufmann Publishers Inc, 1996.

[64] W. H. Payne, J. R. Rabung, and T. P. Bogyo. Coding the Lehmer pseudo-random number generator. *Communications of the ACM*, 12(2):85–86, 1969.

[65] Ora E. Percus and Malvin H. Kalos. Random number generators for MIMD parallel processors. *Journal of Parallel and Distributed Computing*, 6:477–497, 1989.

[66] William H. Press, Saul A. Teukolsky, William T. Vetterling, and Brian P. Flannery. *Numerical Recipes*. Cambridge University Press, third edition, 2007.

[67] Michael J. Quinn. *Parallel Programming in C with MPI and OpenMP*. McGraw-Hill, 2003.

[68] James Reinders. *Intel Threading Building Blocks*. O'Reilly, 2007.

[69] Christian P. Robert and George Casella. *Monte Carlo Statistical Methods*. Springer Texts in Statistics. Springer, 2004.

[70] Linus Schrage. A more portable Fortran random number generator. *ACM Transactions on Mathematical Software*, 5(2):132–138, 1979.

[71] L. N. Shchur, J. R. Heringa, and H. W. J. Blöte. Simulation of a directed random-walk model the effect of pseudo-random-number correlations. *Physica A*, 241(3–4):579–592, 1997.

[72] Mirai Solutions. rTRNG and Valgrind: Docker & actions to the rescue. https://mirai-solutions.ch/news/2021/02/10/rtrng-4.23.1-1-valgrind-docker-actions/, 2021.

[73] Dietrich Stauffer and Ammon Aharony. *Introduction to Percolation Theory*. Taylor & Francis Ltd, 2nd edition, 1994.

[74] Robert H. Swendsen and Jian-Sheng Wang. Nonuniversal critical dynamics in monte carlo simulations. *Physical Review Letters*, 58:86–88, 1987.

[75] Tina's Random Number Generator Library. https://www.numbercrunch.de/trng/, https://github.com/rabauke/trng4/.

[76] Sebastiano Vigna. Scrambled Multiply-with-Carry Pseudorandom Number Generators. https://arxiv.org/abs/2610.27313, 2026. arXiv:2610.27313 [cs.CR].

[77] Zhe-Xian Wan. *Lectures on Finite Fields and Galois Rings*. World Scientific, 2003.

[78] Neal Zierler. Linear recurring sequences. *J. Soc. Indust. Appl. Math.*, 7(1):31–48, 1959.

[79] Robert M. Ziff. Four-tap shift-register-sequence random-number generators. *Computers in Physics*, 12(4), 1998.

Index

Bernoulli distribution, 100
B-distribution, 92
binomial distribution, 102
block splitting, 10, 121, 127
Breit-Wigner distribution, 78

C++11, 4, 134
Cauchy distribution, 78
χ^2-distribution, 94
CUDA, 114

delinearization, 18
discrete distribution, 111

exponential distribution, 68
extreme value distribution, 89

Fisher-Snedecor distribution, 97

Γ-distribution, 91
gamma–Poisson (mixture) distribution, 103
Gaussian distribution, 71
geometric distribution, 107
Gumbel distribution, 89

HIP, 114
hypergeometric distribution, 105

lagged Fibonacci generators, 51
leapfrog, 11, 125, 127
linear complexity, 19
linear congruential generators, 30
linear feedback shift register sequences, 13
linear recurrences, 12
logistic distribution, 80
lognormal distribution, 81

Lorentz distribution, 78

matrix linear congruential generators, 17
Maxwell distribution, 77
Mersenne twister generators, 61
minstd generator, 60
multiple recursive generators, 37
multiply-with-carry generators, 57

negative binomial distribution, 103
normal distribution, 70

parallelization, 10
parameterization, 10
Pareto distribution, 83
play fair, 11
Poisson distribution, 108
power-law distribution, 84
pseudo-noise sequence, 15
pseudo-random numbers, 9

random seesding, 10
Rayleigh distribution, 98

χ^2-distribution, 97
Student-t distribution, 95

tent distribution, 86
truncated normal distribution, 75
two-sided exponential distribution, 69

uniform distribution, 64
unit tests, 118

Weibull distribution, 87

xoshiro generators, 56

YARN generators, 43

yarn sequences, 18

zero-truncated Poisson distribution, 109